

# Fundamental Data Structures, Algorithms and Complexity, and Computer Architecture

---

What are CSC211 and its partner CSC212 about?

- CSC211 concentrates on *problem solving* that leads to correct programs whose timing and memory requirements are well understood. Students *develop programs* to prove they understand the theory. They *time* them and try to isolate what is slow, and *speed up their code*. Students learn how to validate and *prove that their code is correct*. Students learn many *problem solving approaches*.
- CSC212 has two parts— (1) It continues the themes of problem solving and programming in a high level language and understanding the time and space complexity of algorithms. Students continue to *broaden their problem solving skills*.  
(2) Hardware is covered and we learn some assembler programming to understand clearly what the underlying hardware does when it computes a loop or executes a function recursively. Different styles of computer architecture are compared and students gain an understanding of the interaction between high-level and low-level programming and hardware.

# Fundamental Data Structures, Algorithms and Complexity

---

- This course concentrates on *problem solving* that leads to correct efficient programs.
- We will often first try *formulating problems* in English or pseudo code and then proceed to *code* them in Python, Java, C++, or even C# or whatever language you like the most.
- Representing data on computers is done using binary digits—called bits. We learn how eight, sixteen or more bits can be used to represent individual characters, how 32 or 64 bits can be used to represent integers and how to represent floating point numbers, and how other data structures are stored and what their memory requirements are.
- We revisit logarithms and the summing of simple series for insights about numbers.
- We will revise some of the important ideas you should already have mastered. We learn new viewpoints about data structures we already know well, such as linear lists, queues, stacks, hash tables, binary search trees, priority queues.

# Fundamental Data Structures

---

- We learn algorithms to store and retrieve data in dictionaries, trees.
- We study efficiency of algorithms in terms of their time and space complexity.
- Algorithms are implemented using Java or Python.
- You are, however, expected to *become an expert in Java programming*.
- An integral part of this course consists of *your programming effort*.
- Most practicals must be completed on the same day they are issued.
- Marks contributing to your semester mark are awarded for original work submitted. **Unoriginal submitted work gets marked down.**

## How your 15 notional hours should be divided.

There are *three hours of practical programming* per week in the Sun lab. *Three lecture hours* and *three tutorial hours* need to be done. You need to spend at least *six notional hours on self study* per week for this course.

Your B.Sc. programme requires 1200 hours per year, 600 hours per semester, which boils down to 200 hours per semester module, i.e., about 15 hours per week.

# Fundamental Data Structures—prescribed books

---

## Class notes for 2016—core study material

These slides form the core of COS211/212.<sup>1</sup> Study the notes thoroughly.

## Prescribed book for 2016

*Algorithms* FOURTH EDITION, by Robert Sedgewick and Kevin Wayne, Addison-Wesley, 2011.

## Books for reference in 2016

*Data Structures and Algorithms in Java*, 4th Edition, by Michael T. Goodrich and Roberto Tamassia, John Wiley and Sons. Inc, 2006. There is a 5th edition.

*Introduction to Algorithms*, 3rd edition, Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, MIT Press, 2009. This is a daunting book that scares most people off but is the best.

*Data Structures and Algorithms in Java*, 2nd edition, Adam Drozdek, Thomson Course Technology, 2005. Prescribed in 2008.

---

<sup>1</sup>The class notes can be found on the Sun systems in the directory `/export/home/notes/ds/` in the files `ds-display.pdf`—or `ds-printable.pdf`.

# Fundamental Data Structures

---

For *Java programming* we recommend:  
Savitch: the book you used last year.

*Algorithms* FOURTH EDITION, by Robert Sedgewick and Kevin Wayne is packed with examples of objects and methods in Java.

Chapter 1 of *Algorithms* FOURTH EDITION, by Robert Sedgewick and Kevin Wayne is available free on the Internet.

It is essential to have your own copy of the book.

For *Python* there are many good, free books available on the Internet. Stick to Python 3 and learn the differences between 2 and 3.

# Fundamental Data Structures

---

Important *prerequisites* for this course are

- Proficiency in both Java and Python is expected before you enter this course. This is not a course about programming in Java or Python.
- Objects in Java.
- Translate pseudocode into runnable code
- Logarithms, summation of series, differential and integral calculus, linear algebra. Maths I is a prerequisite to this course.

At the conclusion of this course the aims are that you are expected:

- to program all the algorithms in this course proficiently in Java or in pseudo code.
- to explain any algorithm in this course coherently.
- to be able to understand, and discuss the time and space efficiency of each algorithm in this course.
- to know the problem-solving paradigms such as *divide-and-conquer, dynamic programming, brute force, breadth-first, depth-first or top-down design, recursion versus iteration, inductive evolution of programs, etc.*

## An introductory example—Shuffling Cards

- We start by considering how to shuffle a deck of cards.
- The abstraction for the *unshuffled* deck of cards is the set

$\{1, 2, \dots, 52\}$ , which is often written as  $[1..52]$ .

This can be built in Python using a list called **deck**.

```
1 deck = [0]*52
2 for card in range (1, 53):
3     deck[card-1] = card
```

or more stylishly with

```
1 deck = [card for card in range (1, 53)]
```

Notice that there is no extra unused zero at the start of the array.

**TRY IT NOW.**

## An introductory example—Shuffling Cards

- An alternate abstraction to  $\{1, 2, \dots, 52\}$ , or  $[1..52]$  is

$$\{0, 1, \dots, 51\}, \text{ or } [0..52) \equiv [0..51].$$

This is initialized very naturally in Python using

```
1 deck = [0]*52
2 for card in range (52):
3     deck[card] = card
```

or more stylishly with

```
1 deck = [card for card in range (52)]
```

It is hard to dictate which is preferable. The first way seems easy because we learn to count from 1 but the second way  $[0..52)$  comes naturally to programmers, mathematicians and computer scientists.

Learn to use both abstractions.



## Shuffling Cards—setting up the deck in Java

- In Java we need to declare the array `deck[]` with

```
1 int deck[] = new int [53];
```

and then initialize the array to  $\{1, 2, \dots, 52\}$  with:

```
1 for (card = 1; card <= 52; card++)  
2   deck[card] = card;
```

- The element `deck[0]` is unused.
- We need another array to put our “shuffled” cards into.
- Create the array `shuffledDeck[]` in Java.

```
1 int shuffledDeck[] = new int [53];
```

A shuffled deck can be built by drawing cards one-by-one randomly and stacking them onto the ‘shuffled’ deck.

## Shuffling Cards—in Java

- Set  $k = 1$ . A card is ‘drawn’ by randomly drawing from  $\{1, 2, \dots, 52\}$  and put into the  $k$ th position of the shuffled deck as follows:

```
1  r = (int)(Math.random()*52) + 1;  
2  shuffledDeck[k++] = r;
```

- This *does not work*—duplicates are put into the shuffled deck.

The solution is either:

- (1) to deal with the duplicates, otherwise
- (2) never generate a duplicate. *How do we do this?*

### Deal with the duplicates

- (1) If the new card is already in the list then discard it and try again.

### Never generate a duplicate

- (2) Maintain a list that contains numbers that have never been selected and then draw cards one-by-one from this list.

*We will attempt both approaches, and consider their accuracy and complexity, i.e., do they do what we expect them to do? and how long they take?*

## Shuffling—make bad method work

- We must insert  $r$  into `shuffledDeck` if and only if  $r$  is not there.

```
1 k = 0;
2 while (k < 52) {
3     r = (int)(Math.random()*52) + 1;
4     for (j=1; j<=k; j++)
5         if (shuffledDeck[j] == r) break;
6     if (j == k+1)
7         shuffledDeck[k++] = r; }
```

- When  $r$  never matches any of the  $k$  values, then  $j$  becomes  $k+1$ , the `for` loop is exited and Line 7 is executed.
- Notice that  $k < 52$ . Once all the values in the array `shuffledDeck[1..51]` are filled in, getting the 52nd one by letting the loop run to  $k = 52$  is very time consuming.

How long it takes on average?

## Shuffling—entering the last item.

---

To insert the *last* value we need to compare the generated random number with each of the 51 numbers already in the list. This must be done until the missing number is found. On average we need to generate 52 numbers before we find the missing one. There is a quicker way.

$$\text{Since, } 0 + 1 + 2 + 3 + \dots + 51 = \frac{51 \times 52}{2} = 1326,$$

the following strategy can be followed. Keep a running sum called **total** of each new card number as it is found. The value of the missing card is

$$1326 - \text{total}.$$

On average this calculation saves computing random numbers and comparing about 52 of them, 51 times.

There are some other ideas to improve matters.

## Shuffling—make bad method work

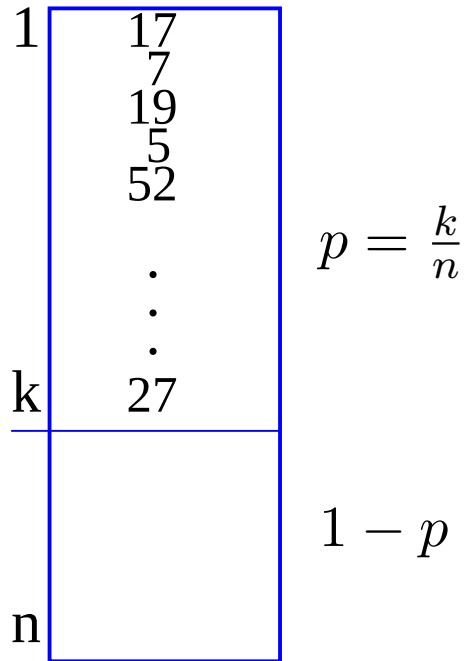
```
1 k = 1;
2 while (k < 52) {
3     r = (int)(Math.random()*52) + 1;
4     for (j=1; j<=k; j++)
5         if (shuffledDeck[j] == r) break;
6     if (j == k+1)
7         shuffledDeck[k++] = r; }
```

- At first the shuffled deck starts filling rapidly, but as it gets more cards—values of  $r$ —matters slow down.
- How fast is this method?
- —It is slow.
- We will see how slow after looking at the same method in Python.

## Shuffling—inserting into a shuffled deck in Python all the way

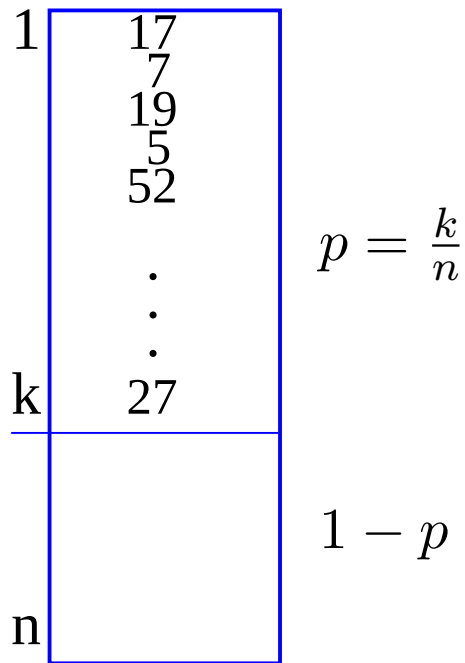
```
1 from random import random
2
3 def shuffle(N):
4     shuffledDeck = [0]*N
5     k = 0
6     while k < N:
7         r = int(random()*N)
8         for j in range (k+1):
9             if shuffledDeck[j] == r: break
10        if j == k:
11            shuffledDeck[k] = r
12            k += 1
13
14    return shuffledDeck
15
16 print (shuffle(52))
```

## Shuffling—average number of probes



- Assume that there are already  $k$  cards in the list. The fraction  $p = \frac{k}{n}$  contains the already shuffled cards in their final positions.
- The rest of the list, the fraction  $1 - p$  of the list is open. Here  $n = 52$ .
- The  $(k + 1)$ th card must now be randomly drawn by generating a random number  $\mathbf{r} \in [1..52]$ .
- How long does it take to draw a valid card  $= \mathbf{r}$  that can be added to the list?

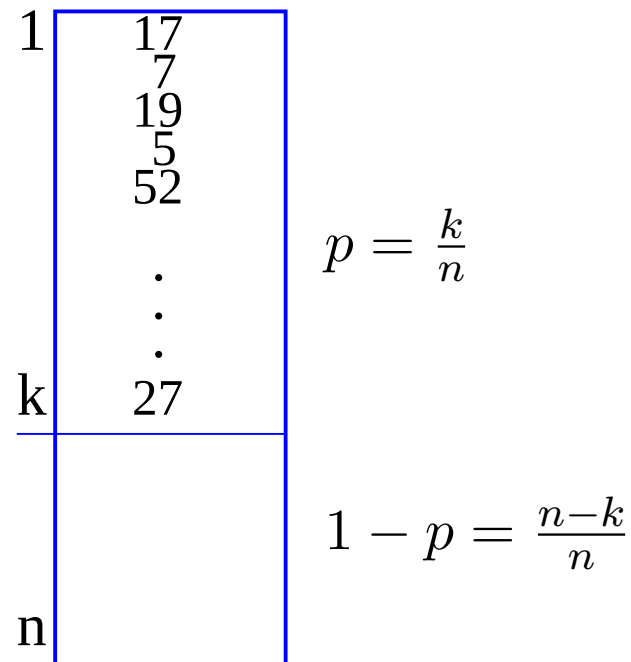
## Shuffling—average number of probes



- Card =  $r$  is only valid if it does *not* appear in the list `shuffledDeck[1..k]`
- For  $p = \frac{k}{n}$ , i.e., for  $p = \frac{k}{52}$ , of the time, card =  $r$  is already present in the first  $k$ .
- *How do we discover that a card has already been selected?*
- One method is to search through the list until it is found.



## Shuffling—checking for the presence of $r$



- Remember that we have assumed that we will find  $r$  in the list,
- In this case about half of the list of  $k$  elements must be searched to discover  $r$ , so  $\frac{k+1}{2}$  probes are done on average, for  $p$  of the time:

$$p \frac{k+1}{2}$$

## Shuffling—when $r$ is not in the list

|       |    |
|-------|----|
| 1     | 17 |
|       | 7  |
|       | 19 |
|       | 5  |
|       | 52 |
|       | .  |
|       | .  |
| k     | 27 |
| <hr/> |    |
| n     |    |

$$p = \frac{k}{n}$$

$$1 - p = \frac{n-k}{n}$$

- For the rest of the time the whole list—now of length  $k$ —is searched, i.e. the  $r$  is compared with every one of the  $k$  items in the list, and is unequal to any of them. So  $k$  comparisons are made.
- This occurs when a new  $r$  is drawn that is not yet in the list.
- How often is there a new  $r$ , i.e. an  $r$  that is not in the first  $k$  elements?

$$\left(1 - \frac{k}{52}\right)k = (1 - p)k.$$

## Shuffling—when $r$ is not in the list

---

- This must be summed for all 52 cards:  
Number of probes =

$$\sum_{k=1}^{52} p \frac{(k+1)}{2} + (1-p)k$$

- The first term is not quite correct, since if a card is already in the list we must search *again and again* until one is found that is not in the list, and that process may have to be repeated many times.
- Let us analyse this.

## Shuffling—total number of probes

- So, how long does it take to draw the  $k$ th card?
- For  $p = \frac{k}{52}$  of the time the card is already in the deck, so when a card *is not in the list*, then the entire list—currently of length  $k$ —must be searched, i.e.  $(1 - p)k$ .
- Suppose the generated card *is* in the list. It will be found on average after  $\frac{k+1}{2}$  probes.
- But, if it *is* in the list another card must be generated, and this process must be repeated until finding a card not in the list.
- 1st probe costs  $p\frac{k+1}{2}$ ,
- 2nd probe costs  $(p + p^2)\frac{k+1}{2}$ ,
- 3rd probe costs  $(p + p^2 + p^3)\frac{k+1}{2}$ ,

## Shuffling—total number of probes

- 4th probe costs  $(p + p^2 + p^3 + p^4) \frac{k+1}{2}$ , etc.
- $m$ th probe costs  $(p + p^2 + \dots + p^m) \frac{k+1}{2}$

Probing forever guarantees card is found, so

Probe time =  $\left[ \sum_{i=1}^{\infty} p^i \right] \frac{k+1}{2}$ , but putting

$$S_n = \sum_{i=1}^n p^i = p + p^2 + p^3 + \dots + p^n \text{ then}$$

$$pS_n = p^2 + p^3 + \dots + p^n + p^{n+1}$$

$$(1 - p)S_n = p - p^{n+1} \text{ giving } S_n = \frac{p - p^{n+1}}{1 - p}$$

and since  $p = \frac{k}{n} < 1$ , because  $k < n$ , it follows that

$\lim_{n \rightarrow \infty} p^{n+1} = 0$  and therefore

$$S_{\infty} = \sum_{i=1}^{\infty} p^i = \lim_{i \rightarrow \infty} S_n = \frac{p}{1-p}$$

- Probe time =  $\left[ \frac{p}{1-p} \right] \frac{k+1}{2}$  and

## Shuffling Cards—improving the method

- Probe time  $= \left\lceil \frac{p}{1-p} \right\rceil \frac{k+1}{2}$  and  
Total number of probes, or  $= \sum_{k=1}^{51} \frac{p}{1-p} \frac{(k+1)}{2} + (1-p)k$
- Why do we stop summing at 51?  
We already know what that card is. It is  $\sum_{k=1}^{52} -\text{total}$ .
- The time complexity is:  
 $= \sum_{k=1}^{51} \frac{p}{1-p} \frac{(k+1)}{2} + (1-p)k$
- Improve by avoiding  $\frac{p}{1-p} \frac{(k+1)}{2}$
- It mounts up to about 90% of the time.

## Shuffling Cards—improving the method

- The time complexity is:  
$$= \sum_{k=1}^{51} \frac{p}{1-p} \frac{(k+1)}{2} + (1-p)k. \text{ Improve by avoiding } \frac{p}{1-p} \frac{(k+1)}{2}$$
- It mounts up to about 90% of the time.
- The trick is to store a bit array—incorrectly called a bit vector—that indicates which cards have been generated:

```
1 boolean[]notInList = new boolean[53];
2   for (k=1; k<=52; k++)
3       notInList[k] = true;
4   while (k < 52) {
5       r = (int)(Math.random()*52) + 1;
6       if (notInList[r]) {
7           shuffledDeck[k++] = r;
8           notInList[k] = false; } }
```

## Shuffling Cards—improving the method

---

- The time complexity is:  
$$\sum_{k=1}^{51} (1 - p)k$$
- This is about 10% of the time taken by the original bad method.
- There is an efficient method that takes  $O(n)$  time.



## Shuffling Cards—juggle a sequence of numbers

- Start with a deck of  $N = 52$  cards, represented by an array of integers  $\{1, 2, \dots, N\}$ . Suppose the array is initialized using:

```
1 for (card = 1; card <= N; card++)  
2   deck[card] = card;
```

- Next, generate exactly  $N$  uniform random numbers drawn from  $\{1, 2, \dots, N\}$ .
- We hope that few duplicates do not affect the result significantly.

```
1 for (card = 1; card <= N; card++) {  
2   r = (int)(Math.random()*N) + 1;  
3   swap(deck[i], deck[r]); }
```

- There is an unpleasant surprise. As in card games the deck is unshuffled only when it is new. Starting with a new deck each time will yield a *biased* shuffle. This becomes obvious when the deck  $\{1, 2, 3\}$  is shuffled by hand.

## Shuffling three cards—an experiment

- Start with the cards in order:  $\{1, 2, 3\}$ . What are the outcomes?
- Suppose that **card** = 1. Get a random number  $r \in \{1, 2, 3\}$  and swap **deck[card]** with **deck[r]**. This yields three possibilities.

| $r$ | The deck      | Effect              |
|-----|---------------|---------------------|
| 1   | $\{1, 2, 3\}$ | 1 is swapped with 1 |
| 2   | $\{2, 1, 3\}$ | 2 is swapped with 1 |
| 3   | $\{3, 2, 1\}$ | 3 is swapped with 1 |

- Next, let **card** = 2. Get a random number  $r \in \{1, 2, 3\}$  and swap **deck[card]** with **deck[r]**. This yields three possibilities for each of the above configurations—a total of nine possibilities.

| The deck      | $r$ | Result of swap |
|---------------|-----|----------------|
| $\{1, 2, 3\}$ | 1   | $\{2, 1, 3\}$  |
| $\{1, 2, 3\}$ | 2   | $\{1, 2, 3\}$  |
| $\{1, 2, 3\}$ | 3   | $\{1, 3, 2\}$  |
| $\{2, 1, 3\}$ | 1   | $\{1, 2, 3\}$  |
| $\{2, 1, 3\}$ | 2   | $\{2, 1, 3\}$  |
| $\{2, 1, 3\}$ | 3   | $\{2, 3, 1\}$  |
| $\{3, 2, 1\}$ | 1   | $\{2, 3, 1\}$  |
| $\{3, 2, 1\}$ | 2   | $\{3, 2, 1\}$  |
| $\{3, 2, 1\}$ | 3   | $\{3, 1, 2\}$  |

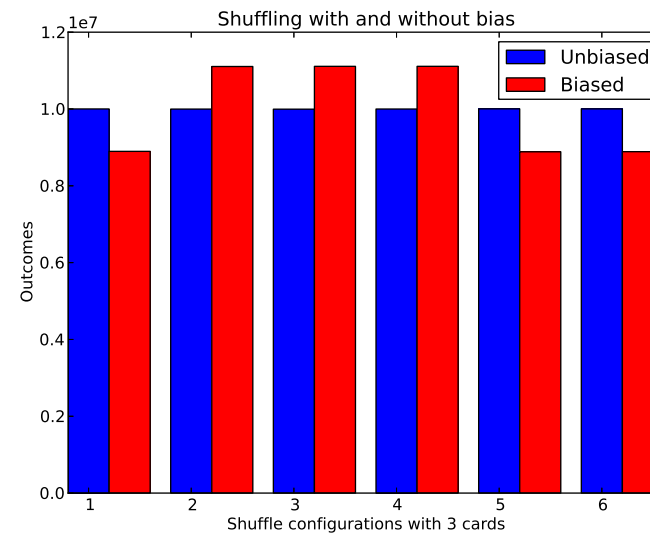
- Starting with these nine orderings we next consider swapping where **card** = 3 **deck[card]** with **deck[r]** for  $r \in \{1, 2, 3\}$ .

# Shuffling three cards—an experiment

| Deck      | $r$ | After final swap |
|-----------|-----|------------------|
| {2, 1, 3} | 1   | {3, 1, 2}        |
| {2, 1, 3} | 2   | {2, 3, 1}        |
| {2, 1, 3} | 3   | {2, 1, 3}        |
| {1, 2, 3} | 1   | {3, 2, 1}        |
| {1, 2, 3} | 2   | {1, 3, 2}        |
| {1, 2, 3} | 3   | {1, 2, 3}        |
| {1, 3, 2} | 1   | {2, 3, 1}        |
| {1, 3, 2} | 2   | {1, 2, 3}        |
| {1, 3, 2} | 3   | {1, 3, 2}        |
| {1, 2, 3} | 1   | {3, 2, 1}        |
| {1, 2, 3} | 2   | {1, 3, 2}        |
| {1, 2, 3} | 3   | {1, 2, 3}        |
| {2, 1, 3} | 1   | {3, 1, 2}        |
| {2, 1, 3} | 2   | {2, 3, 1}        |
| {2, 1, 3} | 3   | {2, 1, 3}        |
| {2, 3, 1} | 1   | {1, 3, 2}        |
| {2, 3, 1} | 2   | {2, 1, 3}        |
| {2, 3, 1} | 3   | {2, 3, 1}        |
| {2, 3, 1} | 1   | {1, 3, 2}        |
| {2, 3, 1} | 2   | {2, 1, 3}        |
| {2, 3, 1} | 3   | {2, 3, 1}        |
| {3, 2, 1} | 1   | {1, 2, 3}        |
| {3, 2, 1} | 2   | {3, 1, 2}        |
| {3, 2, 1} | 3   | {3, 2, 1}        |
| {3, 1, 2} | 1   | {2, 1, 3}        |
| {3, 1, 2} | 2   | {3, 2, 1}        |
| {3, 1, 2} | 3   | {3, 1, 2}        |

| Permutation | Number of occurrences | Random trial of $10^7$ <b>biased</b> shuffles | Random trial of $10^7$ <i>unbiased</i> shuffles |
|-------------|-----------------------|-----------------------------------------------|-------------------------------------------------|
| {1, 2, 3}   | 4                     | <b>8890048</b>                                | 10001750                                        |
| {1, 3, 2}   | 5                     | <b>11116476</b>                               | 10003651                                        |
| {2, 1, 3}   | 5                     | <b>11104844</b>                               | 9997698                                         |
| {2, 3, 1}   | 5                     | <b>11111553</b>                               | 9995874                                         |
| {3, 1, 2}   | 4                     | <b>8885890</b>                                | 10000476                                        |
| {3, 2, 1}   | 4                     | <b>8891189</b>                                | 10000551                                        |

The bias is quite clear and confirms the distribution of shuffles when  $r$  is chosen randomly. The histogram makes the bias very clear.



## Shuffling three cards—shuffling bias

---

It is not surprising that this bias occurs. We have just shown that when starting from the unshuffled deck  $[1, 2, 3]$  it turns out that there are  $3^3 = 27$  ways of reaching one of six possible outcomes. This means that each of the six possibilities can be reached in an average of 4.5 ways. so it is reached in either 4 or 5 possible ways.

If we randomly start with any other ordering a similar distribution ensues.

Since the pattern of outcomes is always 4, 5, 5, 5, 4, 4 the distribution quickly evens out when the shuffle starts at the current ordering of the deck. Let us call the probability of getting the ordering  $s_k$  when starting with  $s_1$ ,  $P_1(s_k)$   
 $P_1(s_1) = 4/25$ ,  $P_1(s_2) = 5/27$   $P_1(s_3) = 5/27$   $P_1(s_4) = 5/27$   $P_1(s_5) = 4/25$ ,  
 $P_1(s_6) = 4/25$ . Starting at ordering  $s_2$  yields the same probabilities tied to other orderings, namely  $P_2(s_1) = 5/25$ ,  $P_2(s_2) = 4/27$   $P_2(s_3) = 4/27$   
 $P_2(s_4) = 4/27$   $P_2(s_5) = 5/25$ ,  $P_2(s_6) = 5/25$ .

## Shuffling three cards—shuffling bias

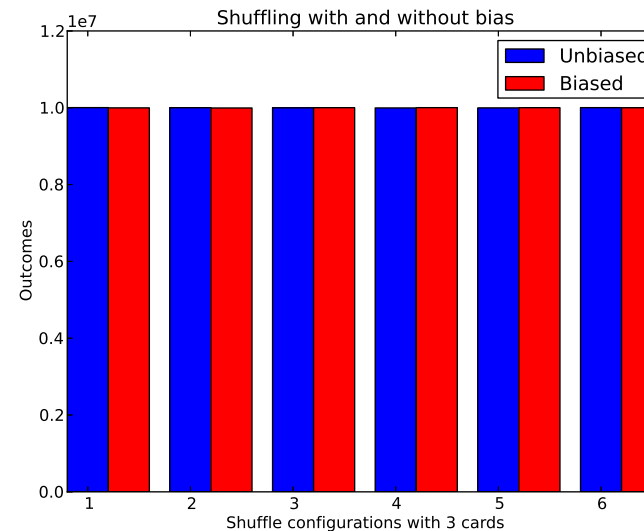
Call the orderings

$s_1 = [1, 2, 3]$ ,  $s_2 = [1, 3, 2]$ ,  $s_3 = [2, 1, 3]$ ,  $s_4 = [2, 3, 1]$ ,  $s_5 = [3, 1, 2]$ ,  $s_6 = [3, 2, 1]$ .

Let  $P(s_k)$  for  $k \in [1..6]$  be the probability of getting ordering  $s_k$ . The table below shows the distribution of the expected orderings.

| Starting<br>at ordering | $P(s_1)$ | $P(s_2)$ | $P(s_3)$ | $P(s_4)$ | $P(s_5)$ | $P(s_6)$ |
|-------------------------|----------|----------|----------|----------|----------|----------|
| $s_1$                   | 4/27     | 5/27     | 5/27     | 5/27     | 4/27     | 4/27     |
| $s_2$                   | 5/27     | 4/27     | 4/27     | 4/27     | 5/27     | 5/27     |
| $s_3$                   | 5/27     | 5/27     | 4/27     | 5/27     | 4/27     | 4/27     |
| $s_4$                   | 4/27     | 4/27     | 5/27     | 4/27     | 5/27     | 5/27     |
| $s_5$                   | 5/27     | 5/27     | 4/27     | 4/27     | 4/27     | 5/27     |
| $s_6$                   | 4/27     | 4/27     | 5/27     | 5/27     | 5/27     | 4/27     |

It can be seen that the expected probability of getting any ordering is equal to that of any other as is evidenced by the histogram.



## Shuffling Cards—much improved method—complexity

---

- The timing is simple: if there are  $n$  cards, there are  $n$  operations;
- each operation takes  $O(1) + O(1) = O(2)$ , but since 2 is a constant that is independent of  $n$ , we write  $O(2)$  as  $O(1)$ .
- But the expression  $n \times O(1)$  is dependent on  $n$ , so  $n \times O(1) = O(n)$
- so it is of the order of  $n$  and is written as  $O(n)$ .
- It is much faster than the previous two methods.
- The bad news is that **it does not generate the ideal shuffle.**
- If you use it in your online poker game you are going to lose money.
- We will first show why sorting three card cards with this method yields a biased result.
- We then illustrate experimentally how badly it is biased.

## Shuffling Cards—by drawing randomly from $\{1, 2, \dots, N\}$ .

---

How does the ideal shuffle work?

Suppose that  $N = 3$ . There are  $N! = 6$  ways to shuffle 3 cards.

The six possible orderings are

$[1, 2, 3]$ ,  $[1, 3, 2]$ ,  $[2, 1, 3]$ ,  $[2, 3, 1]$ ,  $[3, 2, 1]$  and  $[3, 1, 2]$ .

Randomly draw a card from the deck  $[1, 2, 3]$ , i.e., draw 1, 2 or 3—there are three possibilities—leaving two cards.

Next we draw another card—this is done in two possible ways.

When this card is drawn there is no further randomization.

There are exactly six— $3!$ —ways of doing the shuffle.

## Shuffling Cards—ideal shuffle

- Start with a deck cards, this time represented by an array of integers  $[0, 2, \dots, 51]$ . Initialize the array using:

```
1 for (card = 0; card < 52; card++)  
2     deck[card] = card;
```

- Next, generate exactly 51 uniform random numbers drawn from  $\{0, 1, 2, \dots, 51\}$ .
- Duplicates are never generated—there can be no bias.

```
1 for (card = 0; card < 52; card++) {  
2     r = (int)(Math.random()*(52 - card)) + card;  
3     swap(deck[card], deck[r]); }
```

- We prove that this method is not biased by running it a large enough number of times and counting the number of times each shuffle permutation appears. It turns out that the shuffle patterns appear to have an equal probability of appearing.



## Shuffling Cards—ideal shuffle

- Start by initializing the deck to  $\{c_1, c_2, \dots, c_n\}$ .
- Select an item  $c_j$  from the list  $\{c_1, c_2, \dots, c_{n-1}\}$  randomly.
- Swap  $c_j$  with  $c_n$ , turning the list into  $\{c_1, c_2, \dots, c_n, \dots, c_{n-1}, c_j\}$ .
- The final item in the list will not be moved again. So we will agree to call it  $c_n$ , and next we will consider the list ending at  $c_{n-1}$  :  $\{c_1, c_2, \dots, c_{n-1}\}$  and again select an item  $c_j$  from the list randomly.  
That  $c_j$  is swapped with  $c_{n-1}$  which will never be moved again.
- In the following round we consider the list  $\{c_1, c_2, \dots, c_{n-2}\}$  and choose an item called  $c_j$  that we swap with  $c_{n-2}$  that will never be moved again.
- We carry on like this until only one element is left, so there is no choice left, and it remains where it is.
- Next we try to program the selection and replacement.

## Shuffling Cards—ideal shuffle

- Next we generalize the selection steps.
- Considering the lists  $\{c_1, c_2, \dots, c_{n-i}\}$  for values of  $i = 0, 1, \dots, n - 1$ .
- We select an item called  $c_j$  randomly from each list and then swap  $c_j$  with  $c_{n-i}$ .

```
1 for (i=1; i < N; i++) {  
2     r = i + (int)(Math.random()*(N-i) + 1);  
3     swap(card[N-i+1], card[r]); }
```

## Shuffling Cards—try to riffle

- Start by initializing the deck

```
1 for (i=1; i <=n; i++)  
2   card[i] = i;
```

- Generate a random number  $r \in \{1, 2, \dots, n\}$ .
- Replace the old array of cards with the new array:  
 $card[r + 1], \dots, card[n - 1], card[n], card[1], \dots, card[r]$
- Repeated this  $m$  times.
- What is the time complexity?
- Each iteration costs  $O(n)$ , and this must be repeated  $n$  times, so the time complexity is  $mO(n) = O(mn)$ .
- Does this method work? Of course it merely rearranges the position of the first element and does not shuffle the deck.

## Shuffling Cards—riffing

- Start by initializing the deck

```
1 for (i=1; i <=n; i++)  
2   card[i] = i;
```

- Generate a random number  $r \in \{1, 2, \dots, n\}$ .
- Then cut the cards at  $card[r]$  to create two more or less half packs:  
 $\{card[1], \dots, card[r]\}$  and  $\{card[r + 1], \dots, card[n - 1], card[n]\}$
- Now *rifle* the cards together  $m$  times.
- How do you riffle programmatically? Think about it. It's almost like merging.
- Merging is when two sorted arrays are put together in sorted order.

## Shuffling Cards—riffing

---

- In our example the two arrays to merge would be  $card[1..r]$  and  $card[r + 1..n]$ , to be merged into  $target[1..n]$  in sorted order.
- An index  $i$  must be run over  $[1..r]$  and another index  $j$  must be run through  $[r + 1..n]$  and a third index  $k$  must be run over  $[1..n]$  to take the merged item.
- When MERGING  $card[i]$  is compared with  $card[j]$  and the smallest (biggest) one of these is moved to  $target[k]$ .  $i$  or  $j$  is updated by 1 and  $k$  is always updated by 1.
- When we RIFFLE instead of copying only 1 card a small positive random integer number of cards  $r$  are copied from  $card[i]$  or  $card[j]$  where the  $i$  or  $j$  is randomly chosen, and  $k$  is incremented by the  $r$  the riffing batch.

Riffing is not a really good method of shuffling because it does not disturb the cards at the end points much.

The cards must also be CUT more or less in the middle and ROTATED.

## Shuffling Cards—riffing

---

- It can be shown that if there are  $n$  cards then  $3/2 \log_2 n$  riffles will shuffle the cards randomly.
- So for a pack of 52 cards you need to riffle  $8.56 \approx 9$  times.
- What is the time complexity?
- Each iteration costs  $O(n)$ , and this must be repeated  $m = 3/2 \log_2 n$  times, so the time complexity is really  $mO(n) = O(n \log n)$ .
- Riffing is a  $O(n \log n)$  method.

## How long does linear search take?

- Linear search is applied in an  $N$  entry *unsorted* list.  
The call `int result = linearSearch(item, entry);` looks up `object item` in the array `object entry[N]`, and returns the `int` position of `item` or `-1` if it is missing.
- It compares each entry in the list with the `item` being looked up.
- The *best case* is when `item` lies at `entry[1]`. It needs only one probe.
- The *worst case* is when `item` lies at `entry[N]`—needs  $N$  probes, or when `item` being looked up is *missing* from the list.
- The *average case* may be estimated by looking up each `entry[i]` for  $i \in [1..N]$ : the 1st entry needs one probe, the 2nd entry needs two probes, ..., the  $N$ th entry needs  $N$  probes.
- In total there are  $\sum_{i=1}^N i = 1 + 2 + \dots + N - 1 + N = \frac{N(N+1)}{2}$  probes.
- So, the expected number of probes is  $\frac{N+1}{2}$ .
- We can instead say that that average number of probes is  $O(N)$ .
- This is also referred to as the *average search length* or *ASL* of linear search.

# The exact number of bits needed to store a digit

| Bits in N<br>$\lceil \log_2 N \rceil$ | N          | N as<br>$2^i - 1$ | digits exact<br>$\lfloor \log_{10} N \rfloor$ | $\log_{10} N$ |
|---------------------------------------|------------|-------------------|-----------------------------------------------|---------------|
| 1                                     | 1          | $2^1 - 1$         | 0                                             | 0.000         |
| 2                                     | 3          | $2^2 - 1$         | 0                                             | 0.301         |
| 3                                     | 7          | $2^3 - 1$         | 0                                             | 0.845         |
| 4                                     | 15         | $2^4 - 1$         | 1                                             | 1.176         |
| 5                                     | 31         | $2^5 - 1$         | 1                                             | 1.491         |
| 6                                     | 63         | $2^6 - 1$         | 1                                             | 1.799         |
| 7                                     | 127        | $2^7 - 1$         | 2                                             | 2.104         |
| 8                                     | 255        | $2^8 - 1$         | 2                                             | 2.407         |
| 9                                     | 511        | $2^9 - 1$         | 2                                             | 2.708         |
| 10                                    | 1023       | $2^{10} - 1$      | 3                                             | 3.010         |
| ...                                   | ...        | ...               | ...                                           | ...           |
| 13                                    | 8191       | $2^{13} - 1$      | 3                                             | 3.913         |
| 14                                    | 16383      | $2^{14} - 1$      | 4                                             | 4.214         |
| 15                                    | 32767      | $2^{15} - 1$      | 4                                             | 4.515         |
| 16                                    | 65535      | $2^{16} - 1$      | 4                                             | 4.816         |
| 17                                    | 131071     | $2^{17} - 1$      | 5                                             | 5.118         |
| 19                                    | 524287     | $2^{19} - 1$      | 5                                             | 5.720         |
| 20                                    | 1048575    | $2^{20} - 1$      | 6                                             | 6.021         |
| 21                                    | 2097151    | $2^{21} - 1$      | 6                                             | 6.322         |
| 22                                    | 4194303    | $2^{22} - 1$      | 6                                             | 6.623         |
| 23                                    | 8388607    | $2^{23} - 1$      | 6                                             | 6.924         |
| 24                                    | 16777215   | $2^{24} - 1$      | 7                                             | 7.225         |
| ...                                   | ...        | ...               | ...                                           | ...           |
| 29                                    | 536870911  | $2^{29} - 1$      | 8                                             | 8.730         |
| 30                                    | 1073741823 | $2^{30} - 1$      | 9                                             | 9.031         |
| 31                                    | 2147483647 | $2^{31} - 1$      | 9                                             | 9.332         |
| 32                                    | 4294967295 | $2^{32} - 1$      | 9                                             | 9.633         |

The decimal number  
 $1048575 = 11111111111111111111_2$  is  
represented exactly with 20  
binary digits, so *any* 6-digit  
decimal number also needs 20  
bits for exact representation.  
A 24-bit number has enough bits  
to store 7 digits—the numbers  
for doing geodesy require 7  
decimal digits.  
Why is the 32-bit number  
interesting?



## The bits needed to store a digit

---

- $\lceil \log_{10} n \rceil$  yields the number of exact decimal digits needed to store  $n$ .
- $\lceil \log_2 n \rceil$  gives the number of bits required to store any number  $n$ .
- Given a word with  $n$  bits, what is the number of *decimal* digits that it can store precisely?

Since the biggest  $n$ -bit binary number is  $2^n - 1$ :

$$\begin{array}{l} \text{Number of digits in } n\text{-bit} \\ \text{binary} \end{array} = \lceil \log_{10}(2^n - 1) \rceil$$

$$\doteq \lceil \log_{10} 2^n \rceil$$

$$= \lceil n \times \log_{10} 2 \rceil$$

$$\bullet \text{ i.e. number of } \textit{bits} \times 0.3, \quad \doteq \lceil n \times 0.3010299957 \rceil$$

i.e.  $n \times 0.3$  gives the accuracy in *digits*.

- To find the number of *bits* needed to store  $m$  *digits*, simply calculate  $\lceil m/0.3 \rceil$ , e.g. to store 7 digits, use  $\lceil 7/0.3 \rceil = \lceil 23.333333 \rceil = 24$  bits.

## The bits needed to store decimal number

---

- Another approach is to think of  $2^{10} = 1024$  as almost 1000 which can exactly store exactly up to 3 digits. So,  $2^{21}$  which is  $2 \times 2^{10} \times 2^{10}$  and since each 10-bits can store 3 digits, so 21 bits can store a 6-digit number precisely.
- In fact if the decimal value of the number starts with a 1 then the 21-bit binary can store 7 digit positive integers without any loss of precision.
- Note that 20 bits are necessary for 6 decimal digits.
- A 21-bit number can represent a sign and 6 decimal digits.

## How many times, $k$ , can $n$ be divided by $b$ so that $n/b^k < 1$ ?

- Let  $n$  and  $b$  be positive numbers. If the number  $n$  is represented in decimal digits, then each division by  $b = 10$  removes one digit.
- We are looking for the smallest  $k$  such that  $\frac{n}{b^k} < 1$ , i.e.  $n < b^k$ , taking logarithms to the base  $b$ ,  
 $\log_b n < \log_b b^k = k$ . So  $k > \log_b n$ ,  
but in turn  $k \leq \lceil \log_b n \rceil$ ,  
i.e. we have  $\lceil \log_b n \rceil \geq k > \log_b n$ .
- The number of times that  $n$  can be divided by 10 is thus  $\lceil \log_{10} n \rceil$ .
- The same holds for repeated halving—dividing by 2—of a binary number: each division by 2 removes one *bit* from the least significant end of the binary number, so it can only be done  $\lceil \log_2 n \rceil$  times before the result becomes less than 1.
- The number of times a list of length  $n$  can be halved until its length is 0 is thus  $\lceil \log_2 n \rceil$ .

## How long does binary search take?

- Suppose we are looking for *item* in the ordered, i.e. sorted, list  $d[1], d[2], \dots, d[n]$ .

```
1 def binarySearch(d, item):
2     left = 1; right = len(d) # which is n
3     while right >= left:
4         middle = (left + right)/2
5         if item == d[middle]:
6             return middle
7         else:
8             if item < d[middle]:
9                 right = middle-1
10            else:
11                left = middle+1
12    return -left # the item was not found
```

- Each iteration of the **while** loop halves the length of the list that is being searched. This can only happen  $\lceil \log_2 n \rceil$  times. So, the time complexity of binary search is thus  $O(\log_2 n)$ .

## Searching, Insertion and Deletion in lists

---

- Let there be  $n$  persons and a table  $d[m+1]$  of positions  $[1..m]$ . For linear and binary search  $n == m$
- If a list is not sorted then it takes  $\frac{n+1}{2} = O(n)$  to search for a value in the list.
- Inserting an item into an unsorted list is extremely fast—it takes  $O(1)$  time: `data[++n] = item;`
- For sorted lists, binary search takes  $O(\log n)$ , but inserting `item` in its correct position—maintaining the sorted order—requires: (1) a look up to find where the item must be inserted in  $O(\log n)$  time, and (2) on average half of the list must be moved down one place to make place for the new item, in  $\frac{n+1}{2} = O(n)$  time.

## Searching, Insertion and Deletion in lists

---

Repeating some of the previous slide:

- Inserting an item into an *unsorted* list is extremely fast—it takes  $O(1)$  time: `data[++n] = item;`
- For sorted lists, binary search takes  $O(\log n)$ , but inserting `item` in its correct position—maintaining the sorted order—requires: (1) a look up to find where the item must be inserted in  $O(\log n)$  time, and (2) on average half of the list must be moved down one place to make place for the new item, in  $\frac{n+1}{2} = O(n)$  time.
- So while *binary search* is excellent for looking items up, it is tardy when new items are inserted, as opposed to *linear search* which is slow for looking up, but extremely fast, i.e.  $O(1)$  for appending new items at the end of the list.

## $O(1)$ Searching, Insertion and Deletion— $\text{id} \in [1..m]$

---

- Assume we assign a unique identifier,  $\text{id} \in [1..m]$  to each of the  $n$  persons in the class—in the order in which persons arrive in the class, or any other order.
- If the data for person with identifier  $\text{id}$  is stored at  $d[\text{id}]$ , then any person's data may be retrieved given their  $\text{id}$  in  $O(1)$  time.
- Inserting a new arrival's data entails first assigning the next available  $\text{id}$  and by inserting his data into  $d[\text{id}]$ .
- Insertion and deletion can obviously be done in  $O(1)$  time, and  $n == m$ .
- Deletion is similarly efficient. To delete an item: (1) it must be marked as deleted, and (2) its  $\text{id}$  must be added to the list of available  $\text{ids}$ , so that this  $\text{id}$  is later available for re-use.
- The method is not very practical if the identifiers have already been allocated—we will *hash* the  $\text{ids}$ .

## $O(1)$ Searching, Insertion and Deletion— $\text{id} \in \mathbb{K}, |\mathbb{K}| \neq m$

- Suppose there are  $n$  persons or entries and we have a table with  $m$  positions or slots. Now  $n$  is at most  $m$ .
- A solution is to devise a function  $h(\text{id})$  where  $h(\ )$  is called a *scatter* function or more often a *hash* function.
- Insertion is done as follows—assuming  $\text{id}$  is not in list:

```
1  i = hash(id)
2  while d[i].isFilled:
3      i = (i + p) % m + 1
4  d[i].key = id
5  d[i].isFilled = true
```



## $O(1)$ Insertion in a hash table— $\text{id} \in \mathbb{K}, |\mathbb{K}| \neq m$

Insert an item into the hash table as follows:

Insert item directly when the *slot is open*

When the *slot has an entry* but is not filled with the item's key, try another index  $i = (i + p) \bmod m + 1$  where  $p$  is some prime number so that the new  $i$  lies in  $[1..m]$ .

If  $\text{id} = d[i].\text{key}$ , then the key is already present in the table, and the item most likely needs updating.

```
1  i = hash(id)
2  while d[i].isFilled and d[i].key != key:
3      i = (i + p) % m + 1
4  if d[i].key == id:
5      update(d[i], item)
6  else:
7      d[i].key = id
8      d[i].isFilled = true
```

## $O(1)$ Searching— $\text{id} \in \mathbb{K}, |\mathbb{K}| \neq m$

- Searching for  $\text{id}$  is similar:

```
1  i = hash(id)
2  while d[i].isFilled and d[i].key != id:
3      i = (i + p) % m + 1
4  if !d[i].isFilled:
5      return notFound
6  else:
7      return i
```

A *hash* function  $h : \mathbb{K} \rightarrow [1..m]$ , which maps every key  $\in \mathbb{K}$  into the set of slots  $[1..m]$  where  $n == m$ , is called *perfect* when every key maps uniquely onto its own slot.

## Hash functions— $\mathbf{id} \in \mathbb{K}, |\mathbb{K}| \neq m$

Let  $i_j = \text{hash}(id_j)$  and  $i_k = \text{hash}(id_k)$ . When  $id_j \neq id_k$  but  $i_j = i_k$ , this is known as a *collision*.

1. Design the hash function to minimize collisions.
  - Attempt to spread the range of  $h(id)$  uniformly over  $[1..m]$ .
  - $h(id)$ 's values must be as random as possible.
2. Since collisions are often unavoidable methods of handling, collisions must be devised.
  - Make use of linear probing: suppose  $id_j$  is already in the table and  $i_j = i_k$ , then map  $i_k$  onto some multiple  $r$  of a suitable prime  $p$ , i.e.  
 $i_k = (i_j + r \times p) \bmod m + 1$ .  
The  $\bmod m + 1$  at the end ensures that  $i_k \in [1..m]$ .

A hash function with no collisions is called a *perfect hash* function.

## The time complexity of *open* hashing

---

- In the literature this method has been given the confusing name of *open* hashing. We will consider *chained*—or ‘closed’—hashing later.
- The speed of the table depends on how “full” it is. The fullness of a table is defined in terms of its *load factor*,  $\alpha = \frac{n}{m}$ .
- For *open* hash tables  $\alpha = \frac{n}{m} \leq 1$ , i.e.,  $n \leq m$ .
- Assume that there are now  $k$  entries in the hash table, then the load factor is  $\alpha = \frac{k}{m}$ .
- When no entries have been made, then a single probe is enough to make the first entry.
- Suppose  $k$  entries have been made, then the probability of a collision is  $\alpha$ . The probability of a collision on the  $i$ th probe is  $\alpha^i$ . The probability of finding an entry in precisely  $i$  probes is  $\alpha^{i-1}(1 - \alpha)$ .

## Search length in *open* hash table

- Suppose  $k$  entries have been made, then the probability of a collision is  $\alpha$ . The probability of a collision on the  $i$ th probe is  $\alpha^i$ . The probability of finding an entry in precisely  $i$  probes is  $\alpha^{i-1}(1 - \alpha)$ .

*Average Search Length*<sub>hash</sub>

$$= \sum_{i=0}^n i\alpha^{i-1}(1 - \alpha)$$

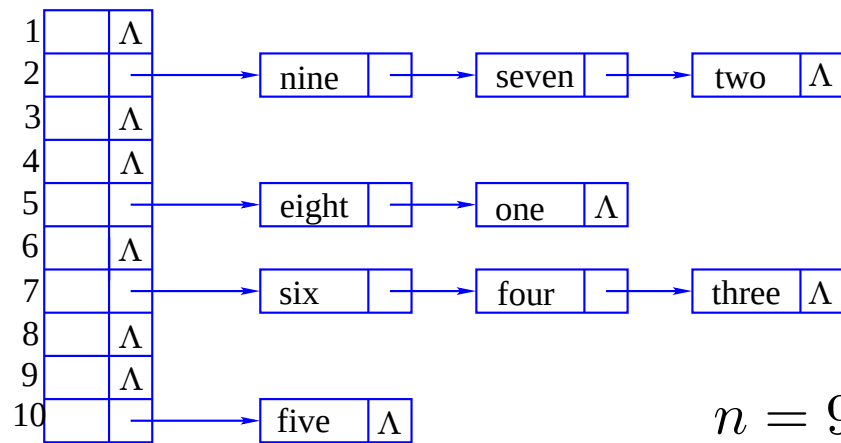
$$\leq (1 - \alpha) \sum_{i=0}^{\infty} i\alpha^{i-1}$$

$$= \frac{(1 - \alpha)}{(1 - \alpha)^2}$$

$$= \frac{1}{1 - \alpha}$$

| Loading<br>as a % | $\alpha$ | <i>Average Search Length</i><br>$\frac{1}{1-\alpha}$ |
|-------------------|----------|------------------------------------------------------|
| 50%               | 0.50     | $1/0.50 = 2.000$                                     |
| 75%               | 0.75     | $1/0.25 = 4.000$                                     |
| 85%               | 0.85     | $1/0.15 = 6.667$                                     |
| 95%               | 0.95     | $1/0.05 = 20$                                        |
| 99%               | 0.99     | $1/0.01 = 100$                                       |
| 99.9%             | 0.999    | $1/0.001 = 1000$                                     |
| 99.99%            | 0.9999   | $1/0.0001 = 10000$                                   |

## Chained hash tables



$n = 9$  and  $m = 10$

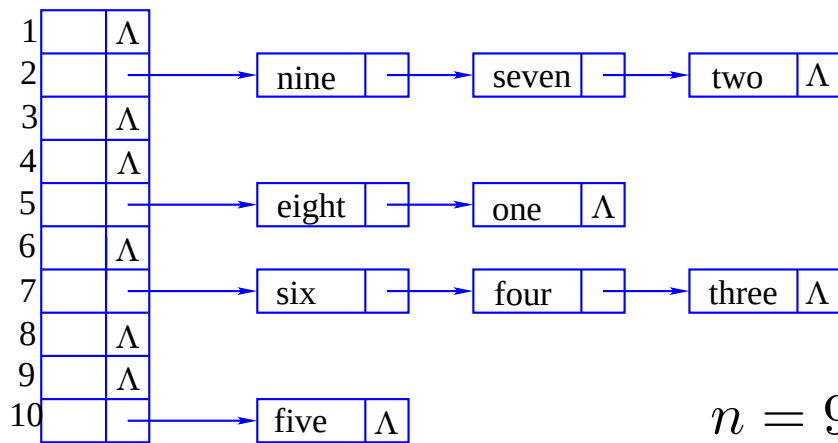
- The table  $d[m+1]$  has  $m$  positions and  $n$  entries. In chained tables  $n$  may exceed  $m$ .  
The performance of a *chained* hash table depends on the average length of the chains,  $\frac{n}{m}$ . So the performance may be tuned.
- One probe is required, to get to the head of the chain. This is followed by a linear search of average length  $\frac{n}{m} = \alpha$  is required.
- This amounts to an average of  $1 + \frac{\alpha+1}{2}$  probes to find any item.
- So  $ASL_{\text{chained hash}} = O(\alpha)$ .

## Chained hash tables can be tuned to $O(1)$

---

- The performance of a *chained* hash table depends on the average length of the chains,  $\frac{n}{m}$ . So the performance may be tuned.
- One probe is always required to get to the head of the chain, and then a linear search is done in a list of average length  $\frac{n}{m} = \alpha$  is required.
- This amounts to an average of  $1 + \frac{\alpha+1}{2}$  probes to find any item.
- So  $ASL_{\text{chained hash}} = O(\alpha)$ .
- But since  $\alpha$  can be made independent of the number of elements  $n$  in the list, e.g. putting  $6m = n$ , makes  $\alpha = \frac{n}{m} = 6$ ,
- i.e. hashing can be made to be  $O(1)$ .

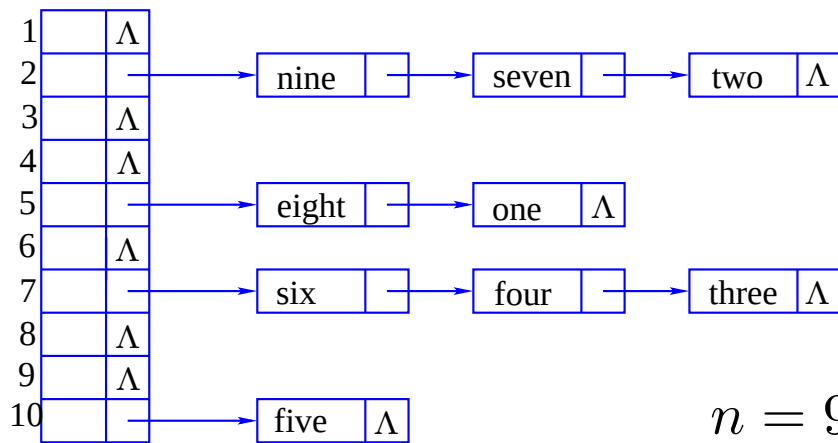
## Chained hashing—example



- Set up the hash table by entering keys: **two**, **five**, **three**, **four**, **six**, **one**, **seven**, **nine**, **eight** in this order. Other orders will give logically equivalent tables, but their links will be different.
- Initially there are  $n = 0$  items in the store. Let us enter **two**, assume **id** = "two", so, first hash it:  $i = \text{hash}(\text{id})$  giving  $i = 2$ . Enter **id** at  $++n$ , in the store, by:  $\text{store}[n] = \langle \text{id}, 0 \rangle$ . Check if the table entry 2 is open or filled by examining  $d[i]$ . If  $d[i] \neq 0$ , i.e.  $d[i]$  is already occupied, the link 0 in the node  $\langle \text{id}, 0 \rangle$  must be replaced by  $d[i]$  as in  $\langle \text{id}, d[i] \rangle$ , and  $d[i] == n$ .



## Chained hashing—example

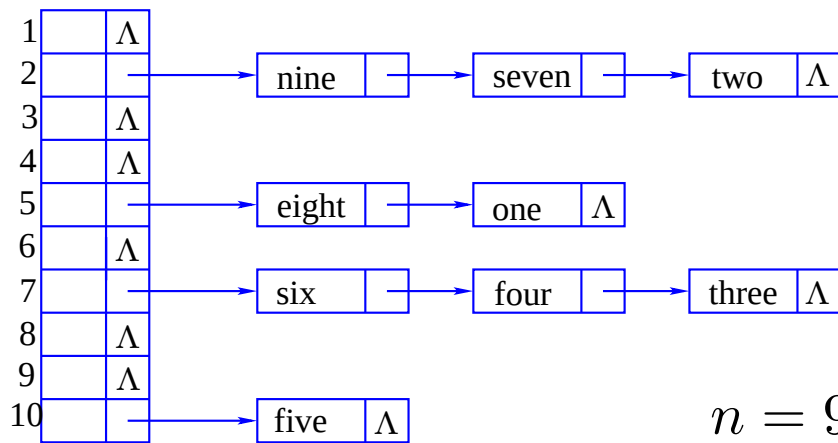


Set up the hash table by entering keys: **two**, **five**, **three**, **four**, **six**, **one**, **seven**, **nine**, **eight** in this order.

- Now enter **five**.

There is now  $n = 1$  item in the store. Entering **five**, assume  $id = \text{"five"}$ , so, first hash it:  $i = \text{hash}(id)$  For this table  $i = 10$ . The data,  $id$ , may be placed at the next open place,  $++n$ , making  $n = 2$  in the store, by:  $\text{store}[n] = \langle id, 0 \rangle$  Check if the table entry  $i$  is open or filled by examining  $d[i]$ . Since  $d[i] == 0$ , i.e.  $d[i]$  is *not* occupied, the link  $0$  in the node  $\langle id, 0 \rangle$  is used as in  $\langle id, 0 \rangle$ , and  $d[i] = n == 2$ .

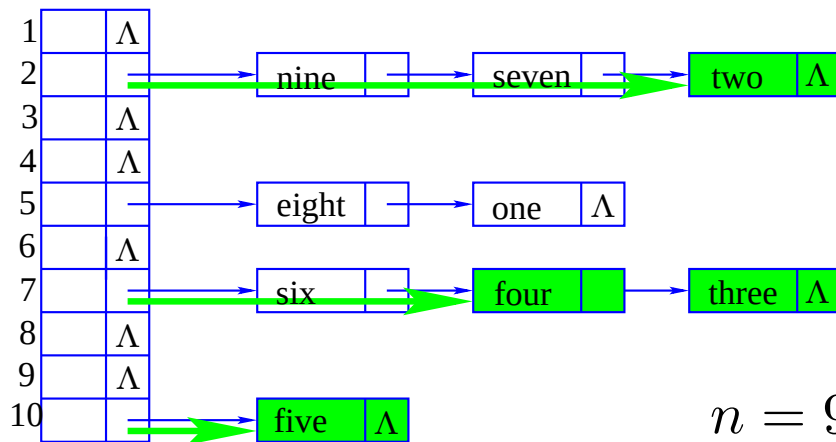
## Chained hashing—example



Set up the hash table by entering keys: **two**, **five**, **three**, **four**, **six**, **one**, **seven**, **nine**, **eight** in this order.

- Next enter **three**.
- There are  $n = 2$  items in the store. Entering **three**, put  $id = \text{"three"}$ , so, first hash it:  $i = \text{hash}(id)$ ; For this table  $i = 7$ . The data,  $id$ , may be placed at the next open place,  $++n$ , making  $n = 3$  in the store, by:  $store[n] = \langle id, 0 \rangle$ ; The table entry  $d[i]$  is still open, so the link  $0$  in the node  $\langle id, 0 \rangle$  is used and  $d[i] = n == 3$ ;

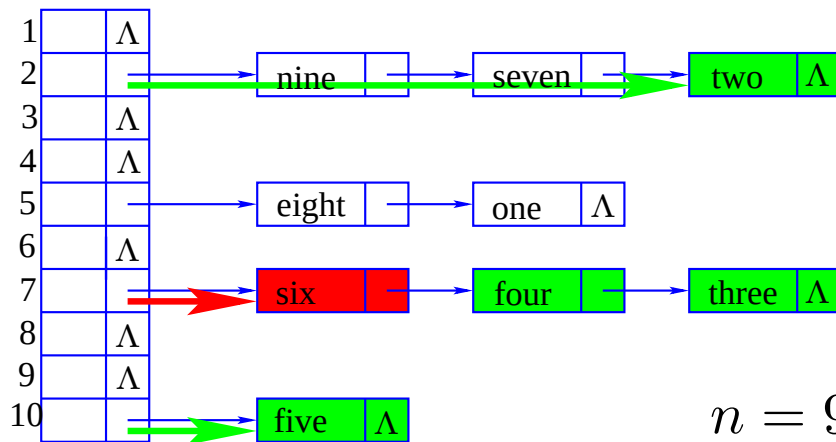
## Chained hashing—example: Enter **four**



Set up the hash table by entering keys: two, five, three, four, six, one, seven, nine, eight in this order.

- Now enter **four**.
- There are now  $n = 3$  item in the store. Entering **four**, assume  $id = \text{"four"}$ , so, first hash it:  $i = \text{hash}(id)$ ; For this table  $i = 7$ . The data,  $id$ , may be placed at the next open place,  $++n$ , makes  $n = 4$  in the store, by:  $\text{store}[n] = \langle id, \emptyset \rangle$ ; Check if the table entry  $i$  is open or filled by examining  $d[i]$ . Since  $d[i] \neq \emptyset$ , i.e.  $d[i]$  is *occupied*, the link  $\emptyset$  in the node  $\langle id, \emptyset \rangle$  must be replaced by  $d[i]$  as in  $\langle id, d[i] \rangle$ , and  $d[i] == n$ ;

## Chained hashing—example: Enter **six**



$n = 9$  and  $m = 10$

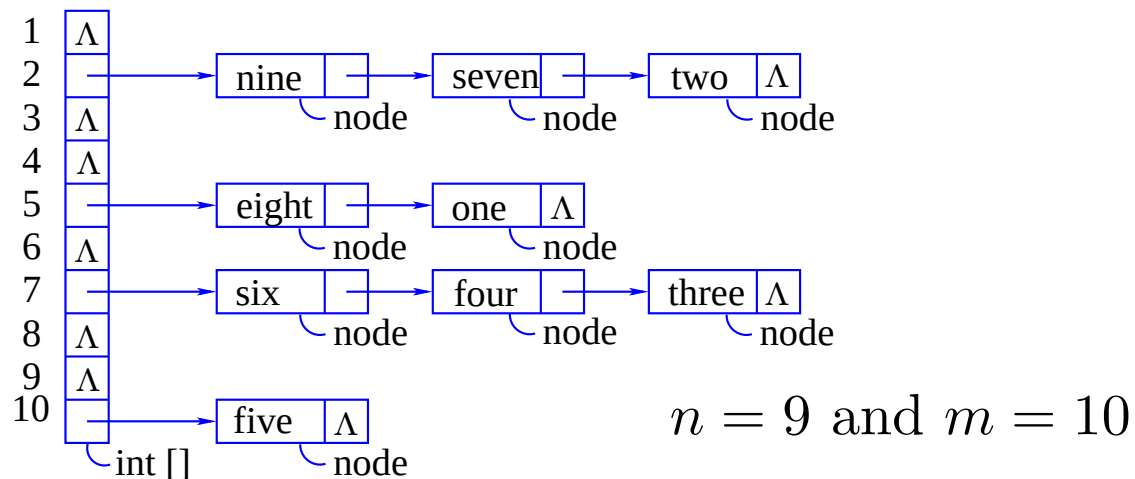
Set up the hash table by entering keys: two, five, three, four, six, one, seven, nine, eight in this order.

- Now enter **six**.
- There are now  $n = 4$  item in the store. Entering **six**, assume  $\text{id} = \text{"six"}$ , so, first hash it:  $i = \text{hash}(\text{id})$ ; For this table  $i = 7$ . The data,  $\text{id}$ , may be placed at the next open place,  $++n$ , makes  $n = 5$  in the store, by:  $\text{store}[n] = \langle \text{id}, \emptyset \rangle$ ; Check if the table entry  $i$  is open or filled by examining  $d[i]$ . Since  $d[i] \neq \emptyset$ , i.e.  $d[i]$  is *occupied*, the link  $\emptyset$  in the node  $\langle \text{id}, \emptyset \rangle$  must be replaced by  $d[i]$  as in  $\langle \text{id}, d[i] \rangle$ ; and  $d[i] == n$ ;

# Chained hashing—hashInsert—array implementation

The hash table has  $m$  entries or slots that each contains the index of the key of the data that has been hashed to that slot.

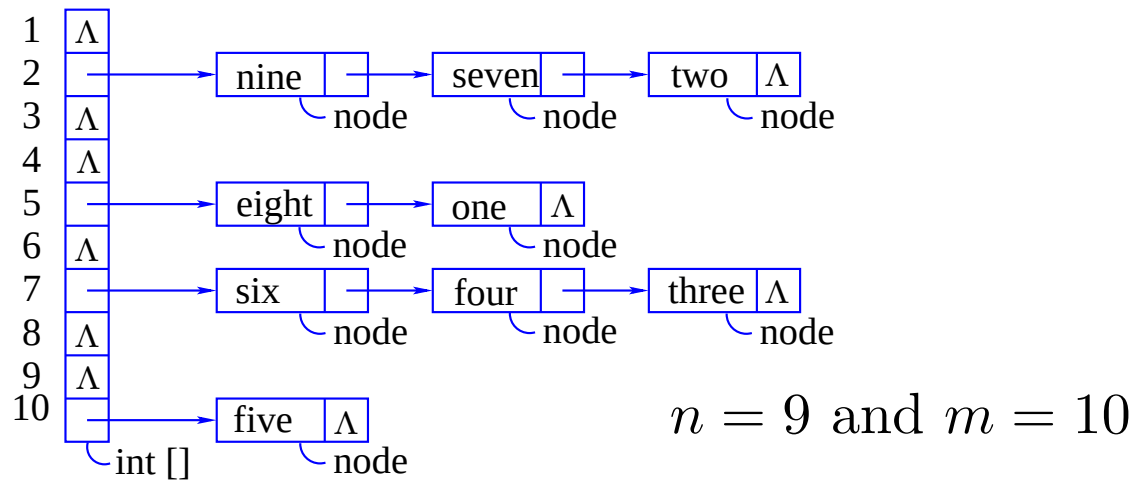
The data itself is stored in a storage pool of nodes.



Java-like pseudocode to create the table and the storage pool.

```
1 int d[] = new int [m+1];  
2 node store[] = new node [6*m +1];  
3 int n = 0;
```

# Chained hashing—hashInsert—array implementation



```
1 int d[] = new int [m+1]; // Java-like pseudocode
2 node store[] = new node [6*m +1];
3
4 int n = 0;
5
6 void hashInsert(string id) {
7     ++n;
8     store[n].key = id;
9     i = hash(id);
10    store[n].link = d[i];
11    d[i] = n; }
```

# Chained hashing—hashSearch

---

This method is called as in the example:

```
1 boolean found = hashSearch("ten");
```

The method follows

```
1 boolean hashSearch(string id) {  
2   int i = hash(id);  
3   i = d[i];  
4   while (i != 0 && store[i].key != id) {  
5     i = store[i].link; }  
6   return store[i].key == id; }
```

## Chained hashing into an array of nodes—Java for the main class

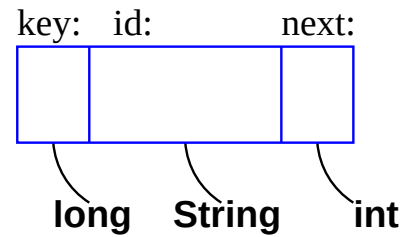
The coding of some of the methods clarify some of our doubtful pseudo code constructs.

```
1 public class tryHash {
2 public static void main (String args[]) {
3 String data [] =
4 {"zero", "one", "two", "three", "four", "five",
5   "six", "seven", "eight", "nine", "ten",
6   "eleven", "twelve", "thirteen", "fourteen", "fifteen",
7   "sixteen", "seventeen"};
8 int k;
9 tableHash H;
10
11   H = new tableHash();
12
13   for (k = 0; k < data.length; k++)
14     H.hashInsert(data[k]);
15   H.display(); }}
```

Here we have declared a reference or variable **H** to a hash table, and then instantiated or created it using the constructor **tableHash()**. Next, we insert the data items one-by-one into the **H**, and finally display the table.



# Chained hashing into an array of nodes—the **node** class in Java



```
1 public class node {  
2  
3     long key;  
4     String id;  
5     int next;  
6  
7     public node () {  
8         key = 0L;  
9         id = "";  
10        next = 0; }  
11  
12    public node (long k, String s) {  
13        key = k;  
14        id = s;  
15        next = 0; } }
```

## Chained hashing into an array—the `tableHash()` constructor

```
1 public class tableHash {
2     int m = 10;
3     int d[] = new int [m+1];
4     node store[] = new node [6*m + 1];
5     int n = 0;
6
7     public tableHash () { // the table constructor
8
9         for (int i=0; i < m + 1; i++)
10             d[i] = 0;
11
12         for (int j = 0; j < 6*m +1; j++)
13             store[j] = new node(); }
```

First `m` is defined as 10, next space is reserved for the hash table of integers `d[]`. Space is made for `store[]`, and `n` is set to zero.

In the `tableHash()` constructor the hash table is set to zeros and the pool of nodes, called `store[]`, is filled with instantiated nodes using the `node()` constructor.

## A hashing function $h(id)$ in Java

```
1 int h (String id) {  
2   int hash = 0;  
3  
4   for (int ch = 0; ch < id.length(); ch++)  
5     hash = (hash + (int)id.charAt(ch)) % m;  
6  
7   return hash + 1; }
```

The function  $h(id)$  simply sums the numerical or ASCII values of each character and gets  $\text{mod } m$  at each step. The value of the running hash will lie in the range  $[0..m-1]$ . The addition of 1 puts the result back into the range  $[1..m]$ .

## A method to display the entire hash table in Java

```
1
2 void display() {
3     int i, j;
4     System.out.printf("The_hash_table:\n");
5     for (i = 1; i <= m; i++) {
6         j = d[i];
7         System.out.printf("%d:_", j);
8         while (j > 0) {
9             System.out.printf("-->%s", store[j].id);
10            j = store[j].next; }
11         System.out.printf("-->%d\n", j); }
12     System.out.printf("-----\n"); }
```

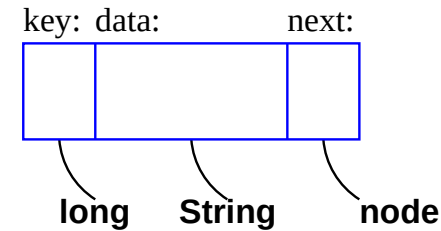
The `display()` method runs through the “pointers” in `d[]` one-by-one and follows the “pointers” until the end of the chain is reached.

## The `hashInsert(id)` and `hashSearch(id)` methods in Java

```
1 void hashInsert(String id) {
2     int i;
3     ++n;
4     store[n].key = (long)n;
5     i = h(id);
6     store[n].next = d[i];
7     store[n].id = id;
8     System.out.printf("insert_%s_at_%d\n" , id, n);
9     System.out.printf("%s\n" , store[n].id);
10    d[i] = n; }
11
12 boolean hashSearch(String id) {
13     int i = h(id);
14     i = d[i];
15     while (i != 0 && store[i].id != id) {
16         i = store[i].next; }
17     return store[i].id == id; } }
```

# Chained hashing—hashInsert—list implementation

Now we alter the **node** so that it has a **next** pointer that is a **node** reference instead of an **integer**.



```
1 node d[] = new node [m+1];
2 int n = 0;
3
4 void hashInsert(long key, String id) {
5     node n = new node (key, id);
6     i = hash(id);
7     n.next = d[i];
8     d[i] = n; }
```

Note that the array **store[]** is no longer needed.

The program creates new **nodes** on-the-fly as they are required.

The array **d[]** appears to be a list of nodes, but it is actually a list of references to nodes.

# Chained hashing—hashSearch—using lists

---

This method is called as in the example:

```
1 boolean found = hashSearch(10L);
```

The method follows

```
1 boolean hashSearch(long key, String id) {  
2   int i = hash(key);  
3   node here = d[i];  
4   while (here != null && here.key != key)  
5       here = here.next;  
6   return here != null && here.key == key; }
```

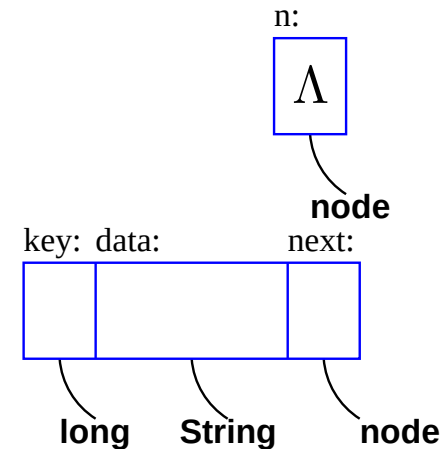
## A class for a node

Suppose we have a **node** class, the declaration:

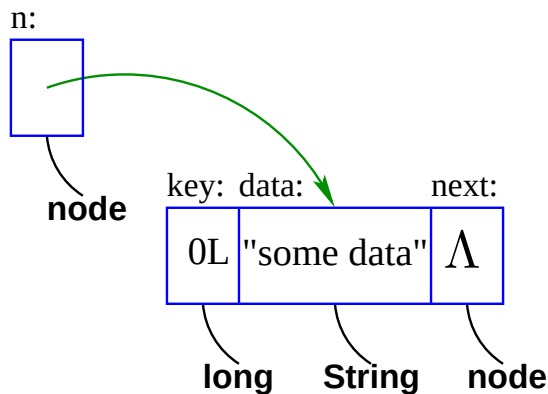
```
1 node n;
```

It creates the variable **n** of type **node**

We depict the instance of a **node** graphically using



When the method **n = new node();** is invoked an instance of the node is created and a pointer that points to that instance is placed into the variable **n**.





## class node

```
1 public class node{
2     long key;
3     String data;
4     node next;
5
6     public node(long k, String d){
7         key = k;
8         data = d;
9         next = null; }
10
11    public node(){
12        key = 0;
13        data = "";
14        next = null; }
15
16    public String toString() {
17        return "<" + key + ",_" + data +
18                ",_" + next + ">" } }
```

# class linkedList

---

```
1 public class linkedList{
2
3     protected static int length;
4     protected node head;
5
6     public linkedList(){
7         head = new node (0, "head_node");
8         length=0; }
9
10    public void insert(node n){
11        n.next = head.next;
12        head.next = n;
13        length++; }
14
15    public int length(){
16        return length; }
```

## class LinkedList continued

---

```
1 public boolean isEmpty(){
2     return length==0; }
3
4 public void clear(){
5     head.next = null;
6     length = 0; }
7
8 public void display(){
9     node here = head.next;
10    while (here != null) {
11        System.out.println(
12            "Key_=" + here.key +
13            "_Data_=" + here.data);
14        here = here.next; }
15    System.out.println("length=" + length); } }
```

## class tryList: Get going

```
1 import java.lang.Math.*;
2 import java.io.*;
3
4 public class tryList {
5     public int n = 2;
6     public static void main(String args[]) {
7         linkedList L = new linkedList();
8         System.out.println(
9             "L.Head.Key=_ " + L.head.key +
10            "_L.Head.Data=_ " + L.head.data +
11            "_L.Length=_ " + L.length +
12            "_L.Head.Next=_ " + L.head.next);
13         node item;
14         long id;
15         String studentName;
16         L.display();
17         L.head.next = null; // L.clear(); is better
```

## class tryList Check the arguments

---

```
1 String theFileName = args[0];
2 if (args.length == 1) {
3     try {
4         FileInputStream inFileName
5             = new FileInputStream (theFileName);
6         DataInputStream inFile
7             = new DataInputStream (inFileName);
8         System.out.println(
9             "File:_" + theFileName
10             + "_successfully_opened");
11         // proceed with processing
```

## class tryList Do the processing

```
1  int lineNumber = 0;
2  // Process first line
3  //---the header---differently
4  System.out.println(inFile.readLine());
5  // // readLine() deprecated
6  // Process rest of file
7  while (inFile.available() != 0){
8      String inString = inFile.readLine();
9      String outString; // readLine() deprecated
10     int commaPos = inString.indexOf(",");
11     // One space: handle Da Costa, Van Wyk, Le Roux, ...
12     // Not treated: van der Merwe, de la Querra, etc.
13     // Before processing
14     //2410832 VAN ZITTERS, ST ... 000000
15     // Final output for this record:
16     // Key = 2410832 Data = Van Zitters, ST
17     int spacePos = inString.substring
18         (10, commaPos).indexOf("_") + 10;
19     int minusPos = inString.substring
20         (10, commaPos).indexOf("-") + 10; }
```

## class tryList Continue processing

```
1 if (spacePos > 10)
2   outString = inString.substring(0, 10)
3   + inString.substring(10, spacePos).toLowerCase()
4   + inString.substring(spacePos, spacePos+2)
5   + inString.substring(spacePos+2, commaPos).toLowerCase()
6   + inString.substring(commaPos, inString.length());
7 else if (minusPos > 10) {
8   System.out.println(minusPos);
9   outString = inString.substring(0, 10)
10  + inString.substring(10, minusPos+1).toLowerCase()
11  + inString.substring(minusPos+1, minusPos+2)
12  + inString.substring(minusPos+2, commaPos).toLowerCase()
13  + inString.substring(commaPos, inString.length()); }
14 else outString = inString.substring(0, 10)
15   + inString.substring(10, commaPos).toLowerCase()
16   + inString.substring(commaPos, inString.length());
17 studentName = outString.substring(9, 71).trim();
18 System.out.println(outString + "␣" + outString.length());
```

## class tryList Build linked list

```
1  // Build the linked list:
2  id = (long)Integer.parseInt(
3      outString.substring(1,8));
4  // first 7 characters: the student number
5  item = new node (id, studentName);
6  L.insert (item);
7  }
8  inFile.close();
9  L.display(); }
10 catch (Exception e) {
11     System.err.println("File:_" + theFileName
12         + "_error_occurred"); } }
13 else {
14     System.err.println("There_are_" + args.length
15         + "parameters,_there_should_be_1"); } } }
```



## The `lookup` method

---

```
1 public node lookup (long key) { //tiny
2   node here = this.head.next,
3     prev = here;
4   while (here != null && here.key != key) {
5     prev = here;
6     here = here.next; }
7   return prev; }
```

# The Makefile

---

```
1 OBJ = tryList
2 LIST = linkedList
3 DEP = node
4 DATA = cos224.list
5
6 all:$(DEP).class, $(LIST).class
7     -rm errors
8     javac -deprecation $(OBJ).java 2> errors
9     -rm output
10    java $(OBJ) $(DATA) > output
11
12 $(DEP).class, $(LIST).class:
13     javac -deprecation $(LIST).java
14     javac -deprecation $(DEP).java
15
16 clean:
17     -rm *~ *.class errors output
```

# Improved `class linkedList`

---

```
1 public class linkedList{
2
3     protected static int length;
4     protected node head;
5
6     public linkedList(){
7         head = new node (0, "head_node");
8         length=0; }
9
10    public void insert(node n){
11        n.next = head.next;
12        head.next = n;
13        length++; }
14
15    public int length(){
16        return length; }
```

## Improved `class linkedList` continued

```
1 public boolean isEmpty(){
2     return length==0; }
3
4 public void clear(){
5     head.next = null;
6     this.length = 0; }
7
8 public void display(){
9     node here = this.head.next;
10    while (here != null) {
11        System.out.println(
12            "Key_=" + here.key +
13            "_Data_=" + here.data);
14        here = here.next; }
15    System.out.println(
16        "length=" + this.length); } }
```

## The `class node` revised

This version of `class node` is not properly encapsulated—one can access parts of the class that should be invisible outside of the `node` class.

```
1 public class node{
2     long key;
3     String data;
4     node next;
5
6     public node(long k, String d){
7         key = k;
8         data = d;
9         next = null; }
10
11    public node(){
12        key = 0;
13        data = "";
14        next = null; }
15
16    public String toString() {
17        return "<" + key + ",_" + data + ",_" + next + ">" } }
```

For example the `insert` method in the `linkedList` class can access the `next` field directly.

```
1 public void insert(node n){
2     n.next = head.next;
3     head.next = n;
4     length++;
5 }
```

## The `class node` revised

The fields inside `node` should be made `private` or `protected` and methods to `get` and `set` them should be supplied.

```
1 public class node{
2     private long key;
3     private String data;
4     private node next;
5
6     public node(long k, String d){
7         key = k;
8         data = d;
9         next = null; }
10
11     public node(){
12         key = 0;
13         data = "";
14         next = null; }
15
16     public void setKey(long k) {
17         key = k; }
18
19     public long getKey() {
20         return key; } . . . }
```

```
1     public long getKey() {
2         return key; }
3
4     public String getData() {
5         return data; }
6
7     public node getNext() {
8         return next; }
9
10    public void setKey(long k) {
11        key = k; }
12
13    public void setData(String d) {
14        data = d; }
15
16    public void setNext(node n) {
17        next = n; }
```

## The `class node` revised

---

Now the `linkedList` method `insert` becomes

```
1 public void insert(node n){
2     \ n.next = head.next;
3     n.setNext(head.getNext());
4
5     \ head.next = n;
6     head.setNext(n);
7
8     length++; }
```

## Linked list—insert (node n)

Start with an instantiated list

```
1 linkedList L;  
2 L = new linkedList();
```

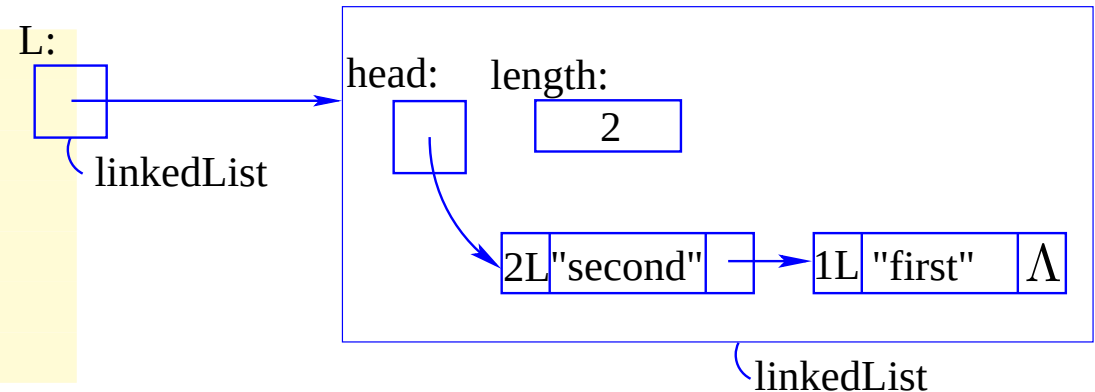
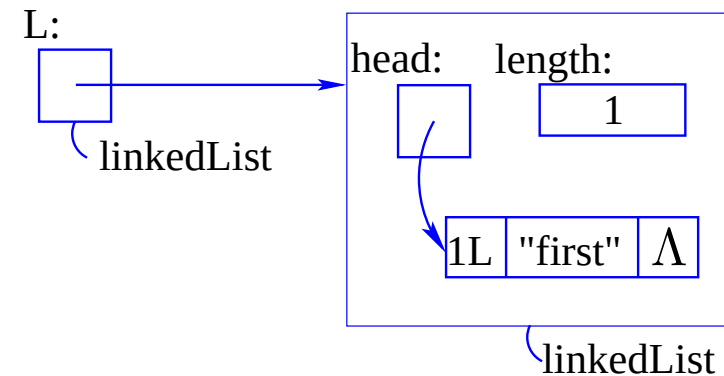
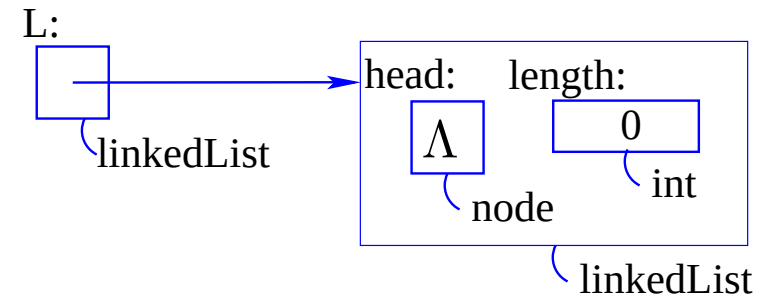
Insert a "first" node

```
1 node n = new node(1L, "first")  
2 L.insert(n);
```

Insert a "second" node

```
1 node n = new node(2L, "second")  
2 L.insert(n);
```

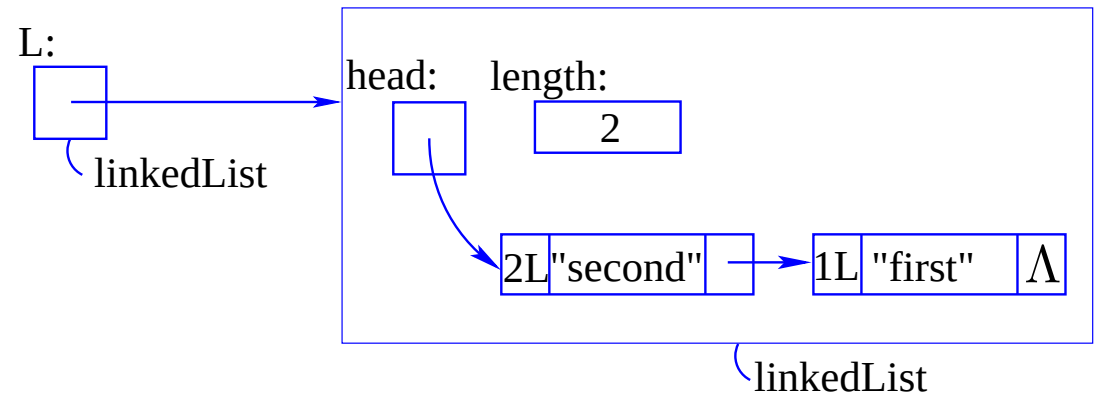
```
1 public void insert(node n){  
2   if (head == null)  
3     head == n;  
4   else {  
5     n.next = head;  
6     head = n;  
7     length++; }  
}
```





## Linked list—no dummy head: remove (long Key)

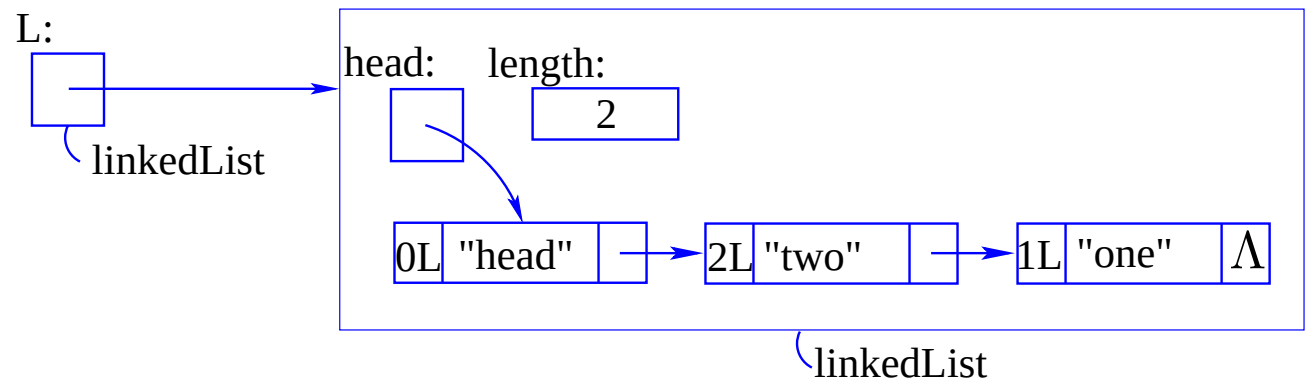
Start with a list of two elements. Consider removing either of the nodes.



```
1 public node remove(long Key){
2   node here = head, previous = here;
3   while (here != null && here.key != Key) {
4     previous = here;
5     here = here.next;}
6   if (here != null) {
7     if (here == head)
8       head = here.next;
9     else
10      previous.next = here.next;
11    length--; }
12   return here;}
```

## Linked list with dummy head—remove (long Key)

Start with a list of two elements. Consider removing either of the nodes.



```
1 public node remove(long Key){
2   node here = head.next, previous = here;
3   while (here != null && here.key != Key) {
4     previous = here;
5     here = here.next;}
6   if (here != null) {
7     previous.next = here.next;
8     length--; }
9   return here;}
```

The code is much simpler.

# Stacks

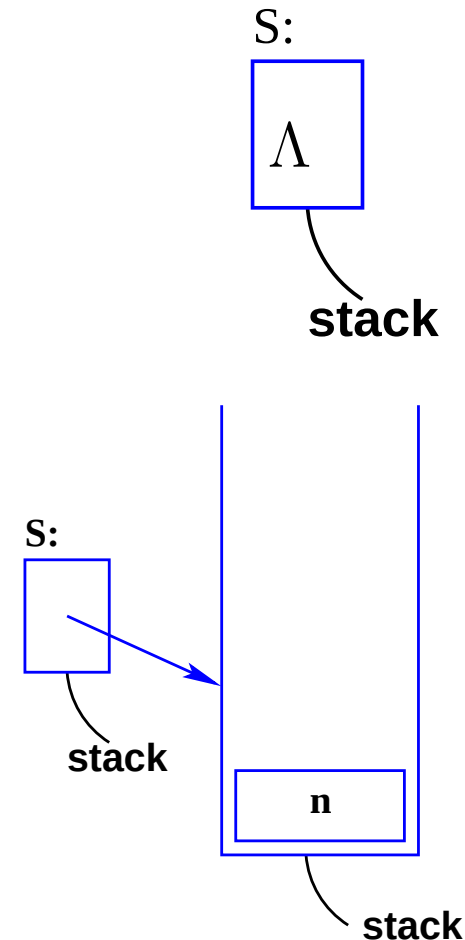
A stack behaves like a linked list where operations are allowed only at the head of the list.

Suppose that a **stack** variable called **S** is created by `public static stack S;`

Our stack will hold items of type **node**.

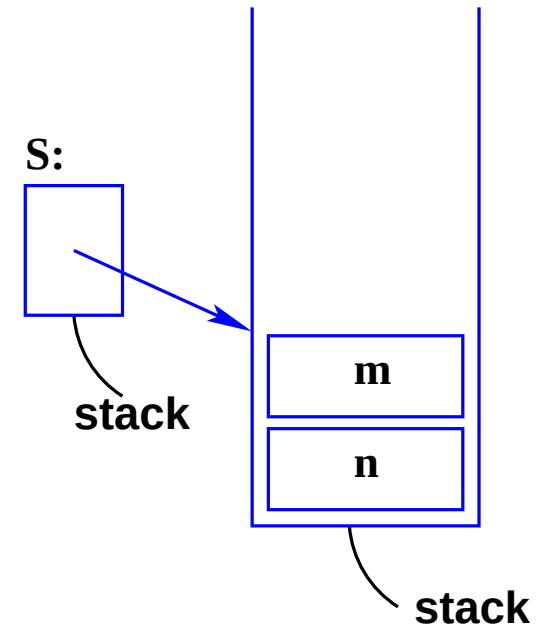
If **n** is of type **node**, then a new instance of a **stack** with **n** placed on the stack is created by

`S = new stack(n);`



## Stacks—a similar stack created differently

If **m** is a **node**, then it may be pushed onto the stack with `S.push(m);`



## Stacks—a similar stack created differently

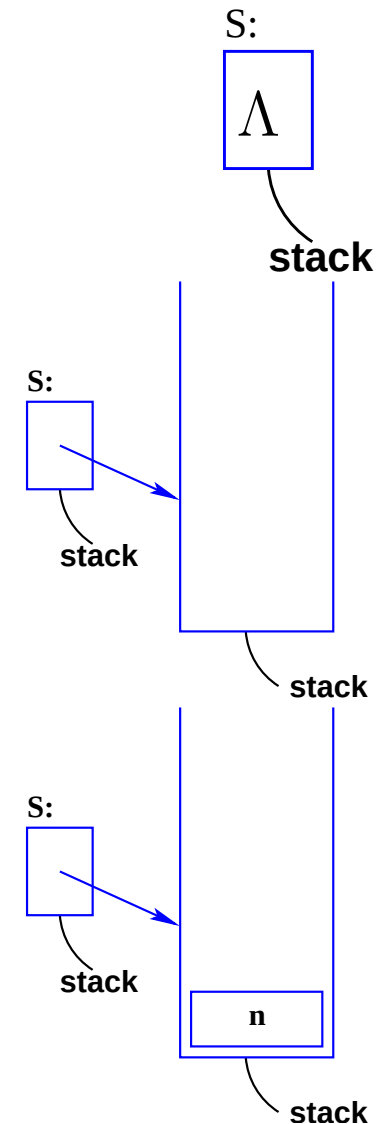
Suppose that a **stack** variable called **S** is created by

```
public static stack S;
```

If **n** is of type **node**, then a new instance of a stack with *nothing* placed on the stack is created by

```
S = new stack();
```

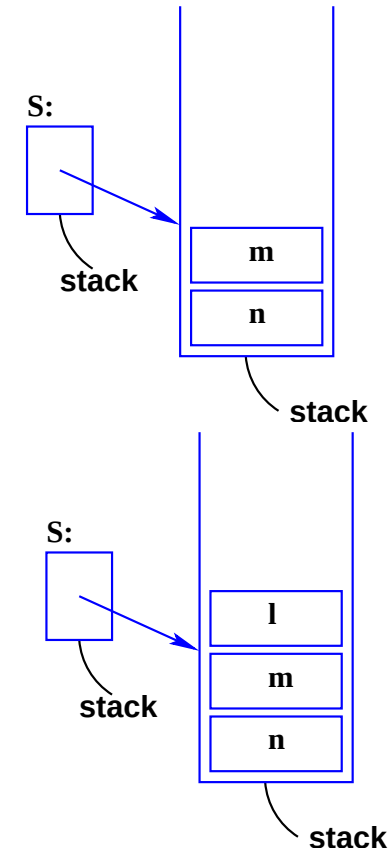
If **n** is a **node**, then it may be pushed onto the stack with **S.push(n)**;



## Stacks—`S.push(l)` and `item = S.pop()`;

If `m` is a **node**, then it may be pushed onto the stack with `S.push(m)`;

If `l` is a **node**, then it may be pushed onto the stack with `S.push(l)`;



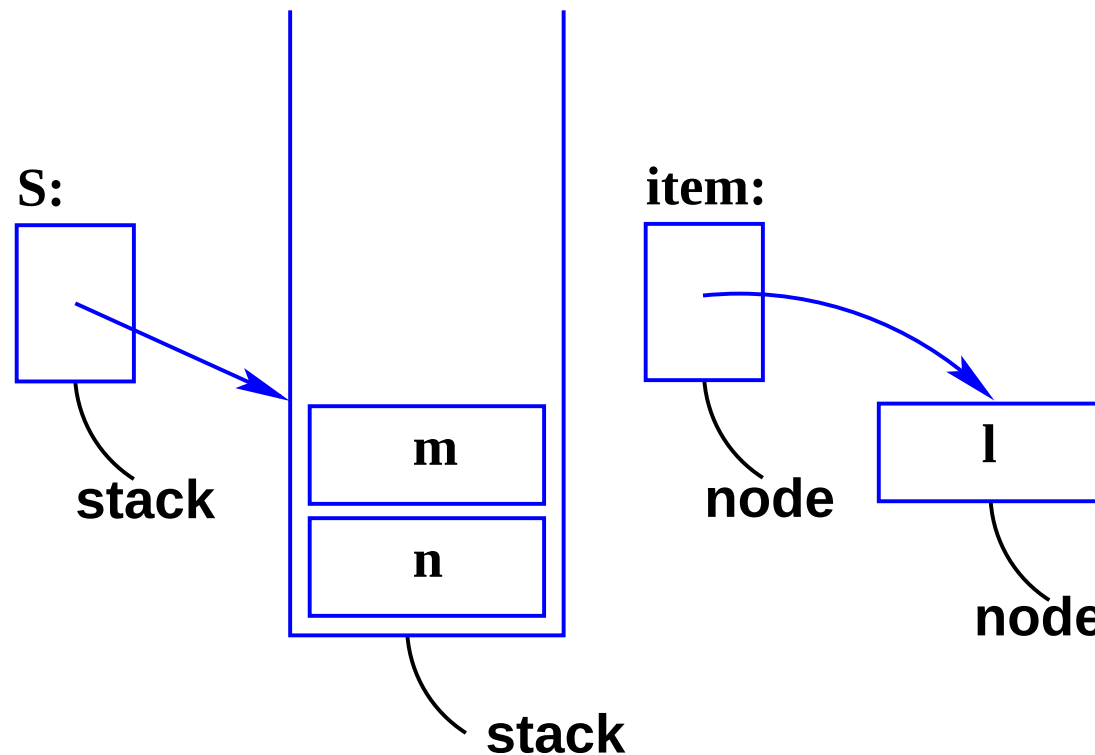
## Stacks—**S.push(l)** and **item = S.pop();**

If the stack is not empty, the function method

**node pop()**

returns the value of uppermost item on the stack and also deletes it off the stack. Now create a variable called **item** of type **node** that points to the node that was removed from the top of the stack.

**node item = S.pop();**



# Stacks

---

- Letting  
`S = new stack();`  
creates an instance of an empty stack to which **S** points.
- Further items of data, **data**, must first be put into a **node** before being pushed onto our stack by first creating an instance of a node  
`node n = new node(data);`
- **n**, may then be pushed onto the stack by  
`S.push(n);`
- The top item of stack **stack** is retrieved by:  
`item = S.pop();`  
which deletes it from the top of the stack.



## Stacks implemented with a fixed array

---

- A stack can also be made using an *array* of **nodes**.
- The integer **top** points to the top of the stack.
- Set the size of the array **static int m = 1<<20;**
- Create an array to hold the stack  
**static node [] d = new node [m+1];**
- Put methods **void push(node n)** and **node pop()** into the **class**. A useful method is **node top()** which undestructively reads the the uppermost element of the stack. The class would not be complete without the functions **int getLength()** and **boolean isEmpty()**.
- The **boolean isFull()** method is questionable because when a push is being done the node being stacked has already been allocated memory.

# A stack implemented with a fixed array

---

Build a stack of nodes.

```
1 public class stack {
2   public static node [] d = new node [1<<20+1];
3   public static int top;
4
5   public stack() {
6     top = 0; }
7
8   public void push (node n) {
9     d[++top] = n; }
10
11  public node pop() {
12    if (top > 0)
13      return d[top--];
14    else
15      return null; } }
```

## A stack implemented as linked nodes

Use the deprecated non Object Oriented **node** class.

```
1 public class stack {
2
3     public static node top;
4
5     public stack() {
6         top = null; }
7
8     public void push (node n) {
9         n.next = top;
10        top = n; }
11
12    public node pop() {
13        node value = top;
14        top = top.next;
15        return value; } }
```

## A stack implemented as linked nodes

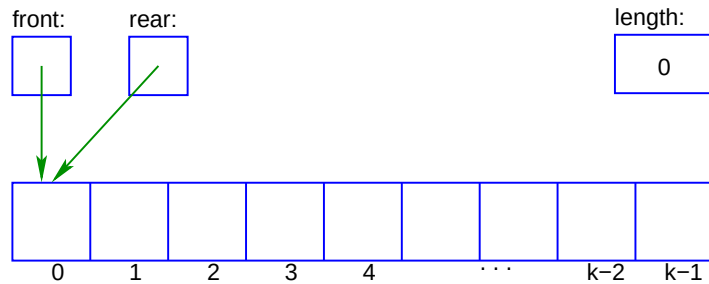
Use the Object Oriented **node** class.

```
1 public class stack {
2
3     public static node top;
4
5     public stack() {
6         top = null; }
7
8     public void push (node n) {
9         n.setNext(top); // n.next = top;
10        top = n; }
11
12    public node pop() {
13        node value = top;
14        top = top.getNext(); // top = top.next;
15        return value; } }
```

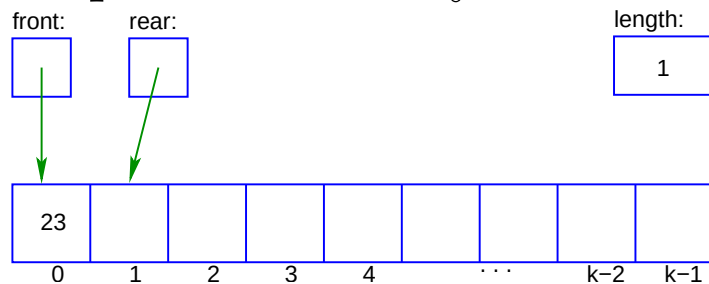
## Queues implemented with a fixed array

- A queue is a first-in-first-out (FIFO) data structure whose elements are served when they reach the front but they enter the queue at the rear.
- It may be implemented with a fixed length array with  $k$  elements.
- A queue is empty when its **length** is 0 (zero).

Note: We use the counter **length**.

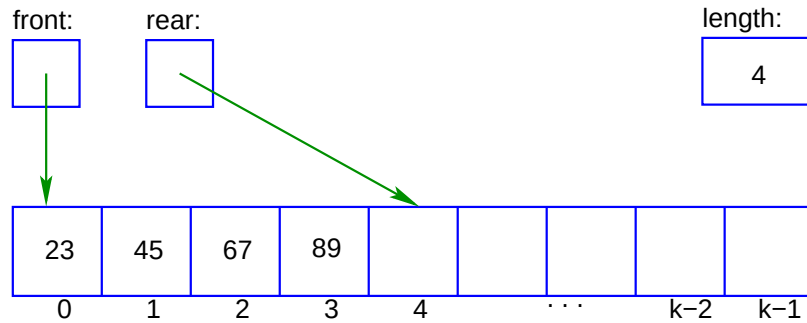


- A queue with only the node 23 entered.

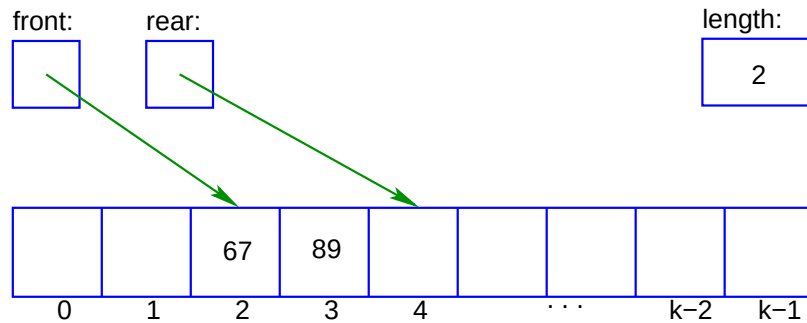


# Queues implemented with fixed arrays

- A queue with the nodes 23, 45, 67 and 89 entered in this order.

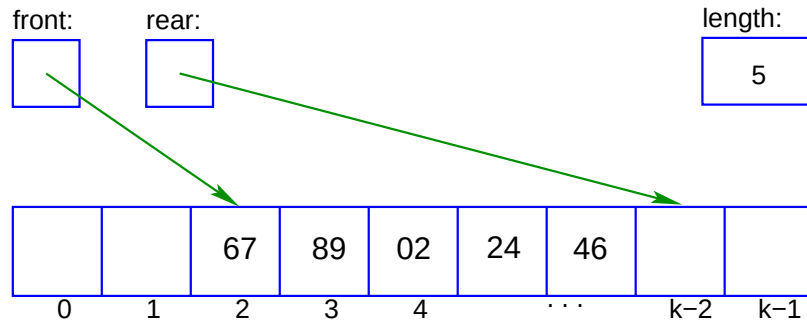


- The queue where the nodes 23, 45 have been dequeued or “served”.

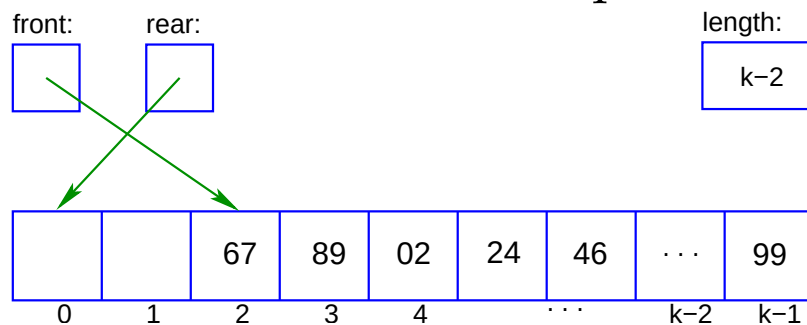


# Queues implemented with fixed arrays

- The previous queue where the nodes 02, 24 and 46 have also been entered.



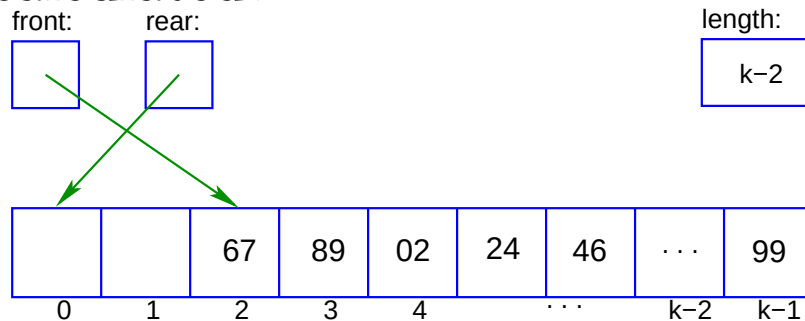
- When  $k$  nodes have been *entered* using—*enqueue*—the last node of the array becomes occupied and the next free place is at 0 (zero). There may be no elements in the queue. How is **rear** calculated?



## Fixed-array queues

### —computing **rear** and **front**

- When  $k$  nodes have been *entered*—the last node of the array becomes occupied and the next free place is at 0 (zero). How should **rear** be calculated?



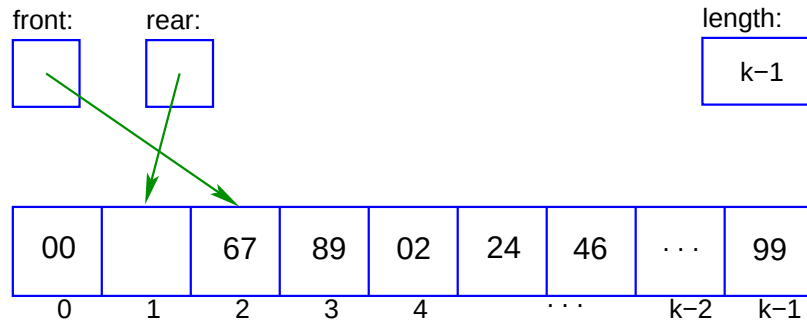
- Simply adding 1 as in **rear++** will crash when the value of **rear** reaches  $k$ —the maximum subscript of the array is  $k - 1$ . Use:  
**rear = (rear + 1) % k;** and also **length++;**
- Updating **front** when doing a *dequeue* is similar:  
**front = (front + 1) % k;** and also **length--;**
- Note that a 1 is *added* in both cases.



# Fixed-array queues

## —computing **rear** and **front**

- The queue after enqueueing the item 00 :

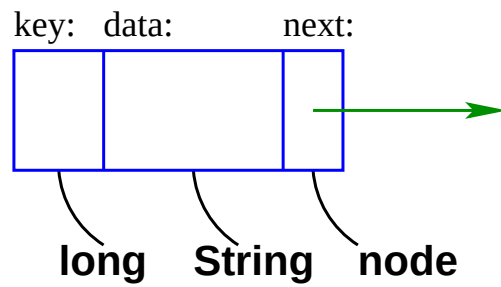


- If the “fullness” of the queue is implemented using a **length** counter and the relation, **length**  $\leq k$ , the array may be completely filled with **rear** pointing to the same place as **front**.
- Note: When the comparison **rear**  $==$  **front** is used to test if the array-implemented queue is filled without using the queue’s **length** counter then it is impossible to say whether the array is *full-up* or *empty*, so in this case the queue is regarded as being full when its length is  $k - 1$ .
- Here **length**  $= (\text{rear} - \text{front} + k) \% k$ ; calculates length when there is no counter.

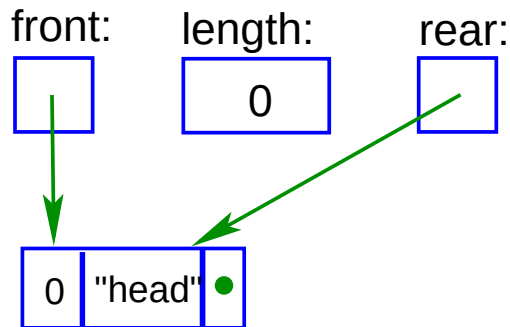
## A queue implemented with linked nodes

- Queues may be implemented with a linked list of nodes with a pointer to the serving or leaving end called **front**, and with another pointer to the joining end named **rear**.

- A queue's **node**:

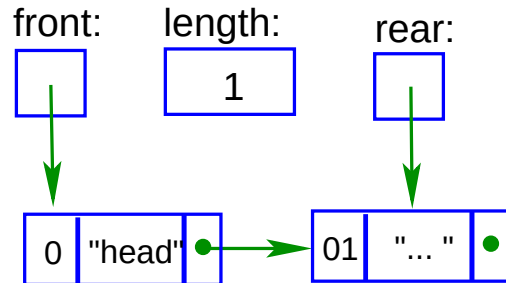


- An empty **queue**

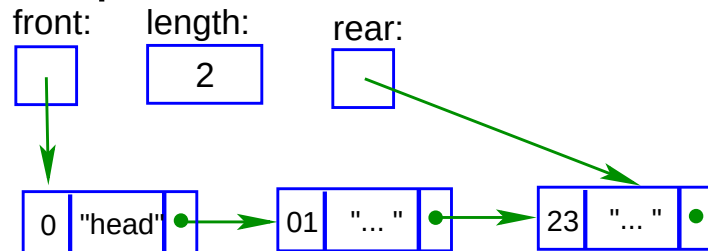


## A queue with linked nodes

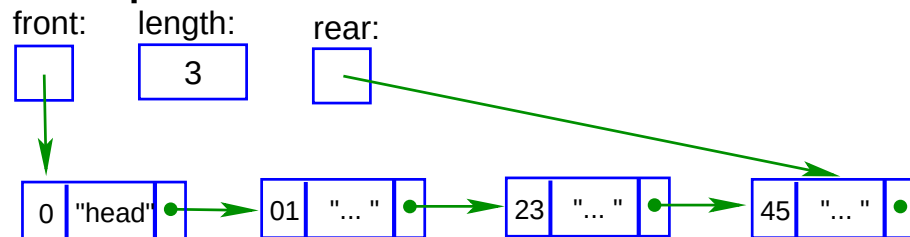
- An queue with one node:



- A queue with two linked nodes.

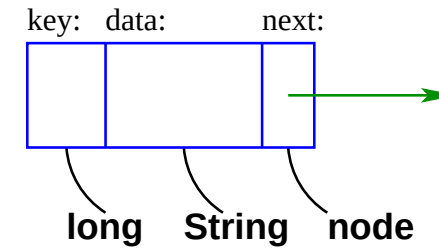


- An queue with three linked nodes.



# Queue's node class—elements private

A queue's node



```
1 public class node{
2
3 private long key;
4 private String data;
5 private node next;
6
7 public node(){
8     key = 0;
9     data = "";
10    next = null; }
11
12 public node(long k, String d){
13     key = k;
14     data = d;
15     next = null; }
16
17 public long getKey() {
18     return key }
19     ...
```

## Queue's node—methods public

```
1 public void setKey(long k) {  
2     key = k; }  
3  
4 public String getData() {  
5     return data; }  
6  
7 public void setData(String d) {  
8     data = d; }  
9  
10 public node getNext() {  
11     return next; }  
12  
13 public void setNext(node n) {  
14     next = n; }  
15  
16 public String toString() {  
17     return "<" + key + ", " + data + ", " + next + ">" } }
```

# The queue itself

```
1 public class queue{
2
3 private int length;
4 private node front;
5 private node rear;
6
7 public queue(){
8     rear = new node (0, "head_node");
9     front = rear;
10    length=0; }
11
12 public queue(long k, String d){
13     rear = new node (k, d);
14     front = rear;
15     length=0; }
16
17 public int size(){
18     return length; }
19
20 public boolean isEmpty(){
21     return length==0; }
```

# The basic manipulations of a queue

```
1 public boolean isFull(){
2     return length==N; }
3
4 public void enqueue(node n){
5     rear.setNext(n);
6     rear = n;
7     length++; }
8
9 public node dequeue(){
10 node n;
11     if (isEmpty())
12         return null;
13     else {
14         n = front;
15         front = front.getNext();
16         length--;
17         return n; } }
18
19 public node topQueue(){
20     if (isEmpty())
21         return null;
22     else {
23         return front; } }
```

## **void display()** is useful for debugging

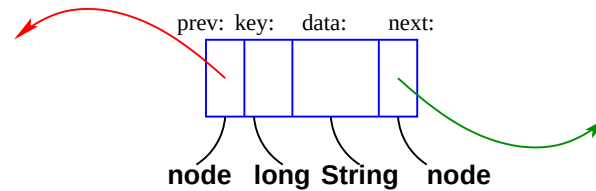
```
1 public void display(){
2     node here = front;
3     int listed = 0;
4     while (here != rear && listed++ < 6) {
5         System.out.println(here);
6         here = here.getNext(); }
7     if (length > 0)
8         System.out.println("Queue's_length=_ " + length);
9     else
10        System.out.println("Queue_is_Empty"); } }
```

The necessity of a **void display()** method is questionable, but we find it useful for debugging our **queues**.



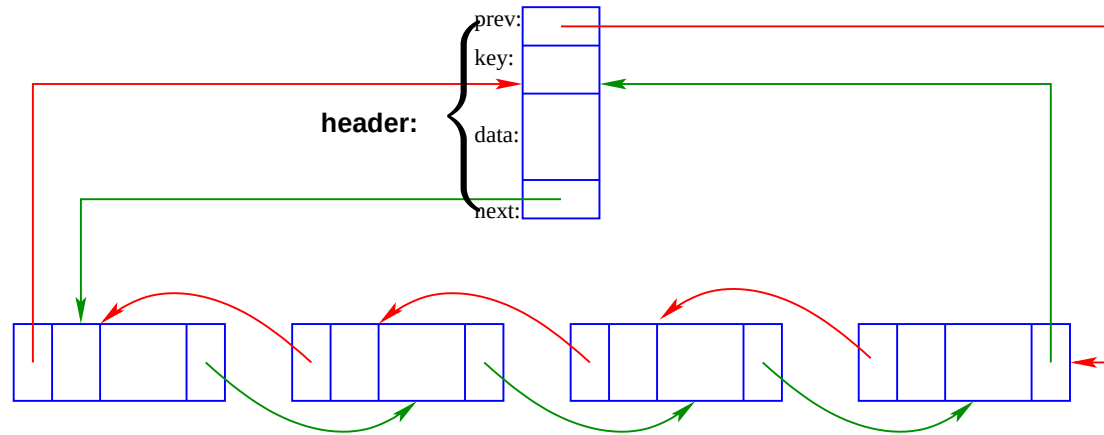
# The node of a deque

## A node for a deque



The node points backwards with its **prev** pointer and forwards with its **next** pointer.

## A deque

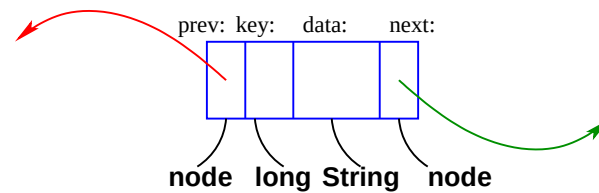


The “sentinel” node’s **next** pointer points to the front of the deque and the sentinel’s **prev** pointer points to the the rear of the deque.

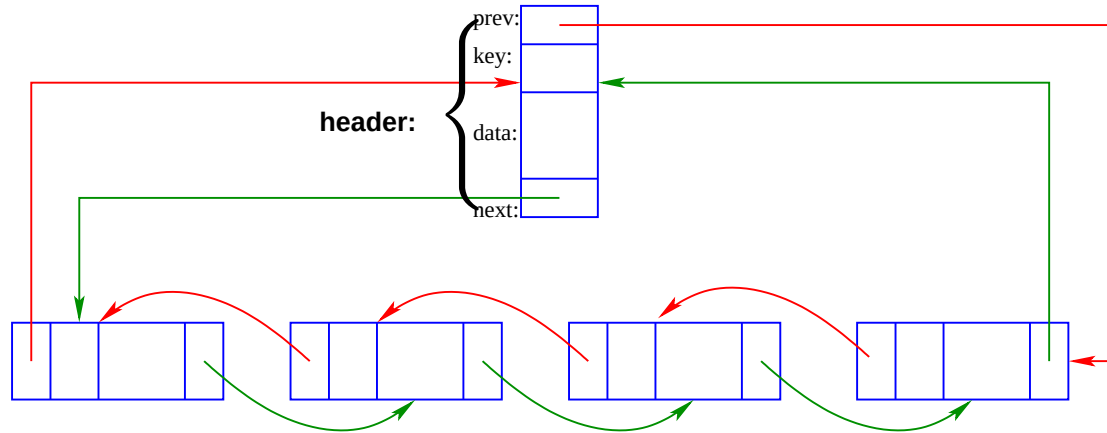
The deque has methods for adding and removing elements at both ends.

# Insert a **node** to the front of a deque

A deque node



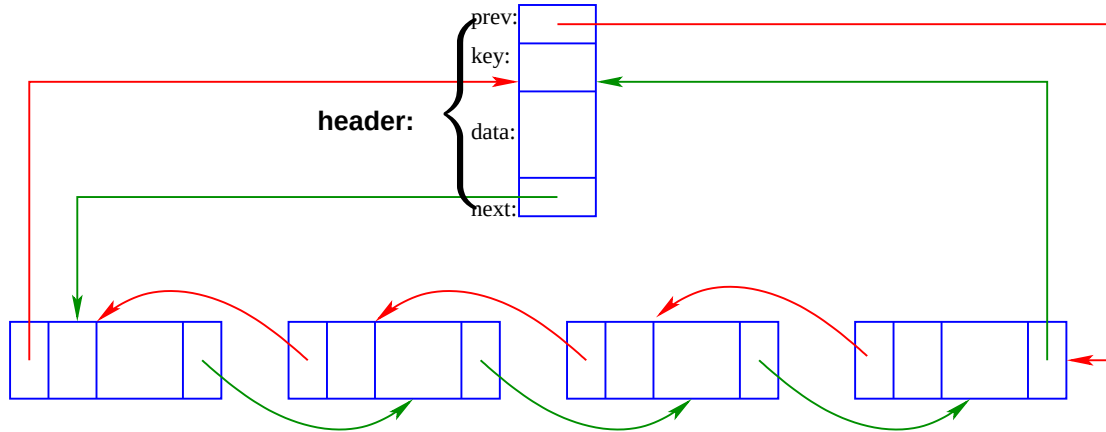
A deque



```
1 public void insertFirst(node n){  
2     n.setNext(header.getNext());  
3     n.setPrev(header);  
4     (header.getNext()).setPrev(n);  
5     header.setNext(n);  
6     length++; }
```

# Insert a **node** to the rear of a deque

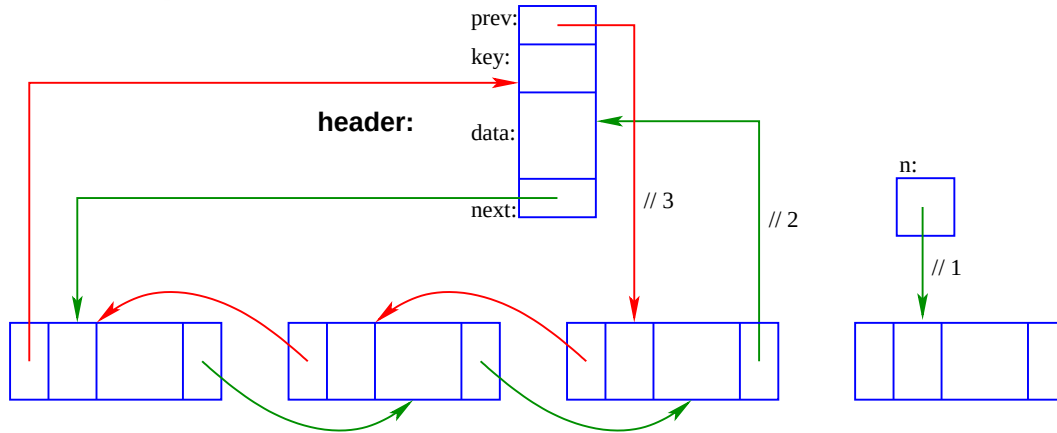
A deque



```
1 public void insertLast(node n){  
2     n.setPrev(header.getPrev());  
3     n.setNext(header);  
4     (header.getPrev()).setNext(n);  
5     header.setPrev(n);  
6     length++; }
```

# Remove a node from the rear of a deque

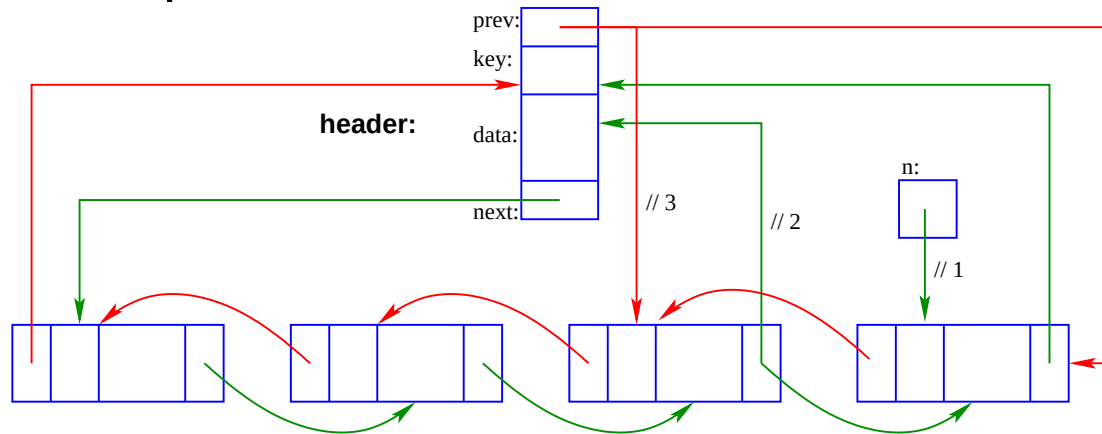
A deque



```
1 public node removeLast(){
2   node n = null;
3   if (!isEmpty()) {
4     n = header.getPrev(); // 1
5     header.setPrev(n.getPrev()); // 2
6     (n.getPrev()).setNext(header); // 3
7     n.setPrev(null);
8     n.setNext(null);
9     length--; }
10  return n; }
```

# Remove a **node** from the rear of a **deque**

- A deque



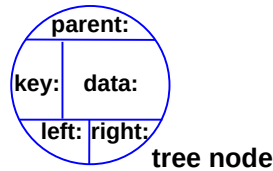
- ```
1 public node removeLast(){
2   node n = null;
3   if (!isEmpty()) {
4     n = header.getPrev(); // 1
5     header.getPrev(n.getPrev(n)); // 2
6     (n.getPrev()).setNext(header); // 3
7     n.setPrev(null);
8     n.setNext(null);
9     length--; }
10  return n; }
```

## Binary search trees (BST)

---

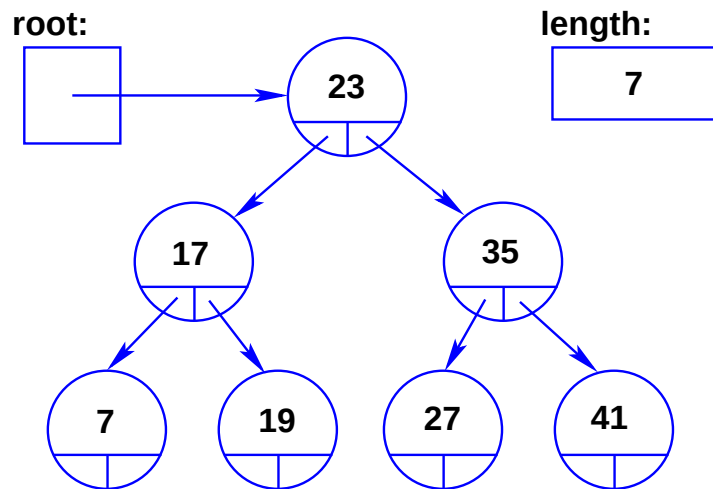
- A binary search tree (BST) is an efficient data structure for storing and retrieving data.
- A search in a balanced or proper BST is  $O(\log n)$  at worst.
- A binary tree is *proper* or *complete* when the nodes at the lowest level are all *leaves* and these leaves are the only nodes with null pointers.
- Numbering the level with only one node in it, *level-0*, there are  $n = 2^k$  elements in the  $k$ th level of a proper binary tree.
- The bottom level of a binary tree has one more node than the rest of the tree above it.
- The number of nodes in a complete binary tree up to but excluding *level-k* is  $n = 2^{k+1} - 1 - 2^k = 2^k - 1$ .
- Since the height of a proper binary tree is thus  $k$ , it follows that the maximum search length is  $k$ , but  $k = \log n + 1$ , i.e.  $O(\log n)$ .

## The nodes of a tree



- A tree node
- The nodes in a binary tree have a **left** and a **right** forward pointer, a back link to the **parent** is useful. The element also should contain a **key** and **data** field.
- In a *binary search tree* each node's **left** pointer points to a node such that  $\text{left.key} < \text{key}$  when  $\text{left} \neq \text{null}$  and **right** and a pointer that points to a node such that  $\text{right.key} \geq \text{key}$  when  $\text{right} \neq \text{null}$ .

# A proper binary search tree



```
1 public class tryTree{
2 public static void main(String args[]){
3 tNode tItem;
4 tree T = new tree ();
5
6 T.display ();
7 T.insert (new tNode (23, "twenty_three"));
8 T.insert (new tNode (17, "seventeen"));
9 T.insert (new tNode (7, "seven"));
10 T.insert (new tNode (35, "thirty_five"));
11 tItem = new tNode (41, "fourty_one");
12 T.insert (new tNode (27, "twenty_seven"));
13 T.insert (tItem);
14 T.insert (19, "nineteen");
15 T.display (); } }
```



## A class for a tNode

```
1 public class tNode{
2
3 private long key; private String data;
4 private tNode left; private tNode parent;
5 private tNode right;
6
7 public tNode() {
8     key = 0;
9     data = "";
10    parent = null;
11    left = null;
12    right = null; }
13
14 public tNode(long k, String d) {
15     key = k;
16     data = d;
17     parent = null;
18     left = null;
19     right = null; }
20
21 public long getKey() {
22     return key; }
23
24 public void setKey(long k) {
25     key = k; }
26 // continued on next slide
```

## The tNode class (continued)

```
1 public String getData() {
2     return data; }
3 public void setData(String d) {
4     data = d; }
5
6 public tNode getLeft() {
7     return left; }
8 public void setLeft(tNode n) {
9     left = n; }
10
11 public tNode getRight() {
12     return right; }
13 public void setRight(tNode n) {
14     right = n; }
15
16 public tNode getParent() {
17     return parent; }
18 public void setParent(tNode n) {
19     parent = n; } }
```

## Defining a tree

- An empty **tree** has a **root** set to **null** and a counter **length** set to 0 by the constructor.

root:

null

length:

0

```
1 public class tree {  
2   private int length;  
3   private tNode root;  
4  
5   public tree() {  
6     root = null;  
7     length=0; } }
```

- An instance of the tree is defined in the user program by:

```
1 tree T = new tree();
```

- The first **node** in a tree of type **tNode** is pointed to by **root**.
- A subsequent node **n** is added to the left subtree if **n.key < root.key** or otherwise to its right.

## Inserting a node into an empty tree

- An empty **tree** has a **root** set to **null** and a counter **length** set to 0 by the constructor.

```
1 tree T = new tree();
```

root:

null

length:

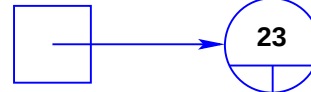
0

- The tree after inserting a node of type **tNode** with **key = 23** using:

```
1 T.insert(new tNode(23, "twenty_three"));
```

```
1 public class tree {  
2 ...  
3 public void insert(tNode n) {  
4     if (root == null) {  
5         root = n;  
6         root.setParent(null); }  
7     else  
8         root.insert(n); }  
}
```

root:



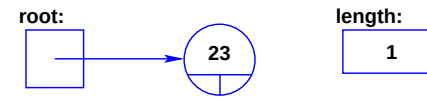
length:

1

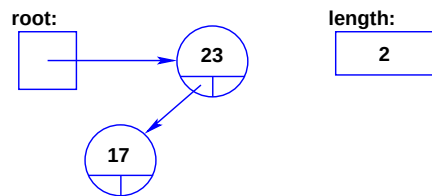
- **root.insert(tNode)** in the **tNode** class inserts the **tNode** **n** into the “tree” starting at **root**.

# Inserting a second node into the tree

- Starting with a tree with a single node at the root
- and after inserting a node with **key = 17** using:



```
1 T.insert(new tNode(17, "seventeen"));
```



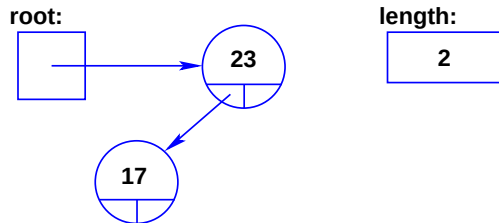
- The code that does this lies in the **tNode** class.

```
1 public void insert(tNode n) {  
2     if (n.key < key) // n into left subtree  
3         if (left == null) {  
4             n.parent = this;  
5             left = n; }  
6     else  
7         left.insert(n); ... }
```

- If **left != null**, then **left.insert(n)** is called, as happens when a node with **key = 7** is entered.

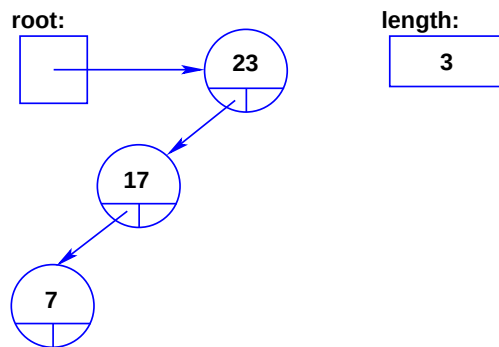
## Adding a node with key = 7

- Start with the two node tree:



- Insert a node with key = 7 using:

```
1 T.insert(new tNode(7, "seven"));
```



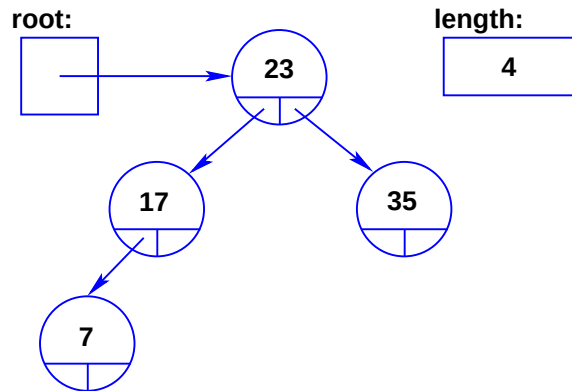
The pointers down the left are followed until a `null` is found.

```
1 public void insert(tNode n){
2     if (n.key < key) // n into left subtree
3         if (left == null){
4             n.parent = this;
5             left = n;
6         }
7     else
8         left.insert(n);
```

## Adding the node with key = 35

- Insert a node with key = 35 using:

```
1 T.insert(new tNode(35, "thirty_five"));
```



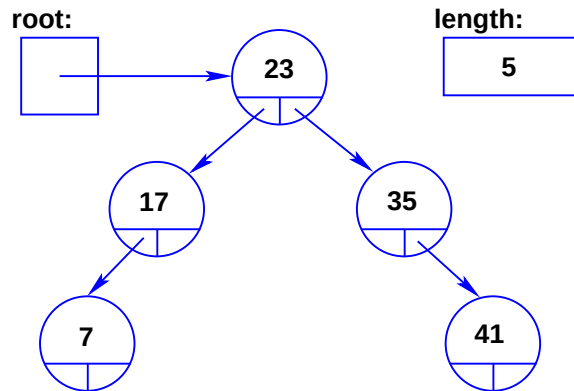
The **right** pointer of the root node is **null** and **n** is inserted there.

```
1 public void insert(tNode n) {
2     if (n.key < key) // n into left subtree
3         if (left == null) {
4             n.parent = this;
5             left = n; }
6     else
7         left.insert(n);
8     else // n into right subtree
9         if (right == null) {
10            n.parent = this;
11            right = n; }
12    else
13        right.insert(n); }
```

## Adding the node with key = 41

- Insert a node with key = 41 using:

```
1 T.insert(new tNode(41, "fourty_one"));
```



The **right** pointer of the root node is not **null** and the pointer is followed to **35** whose right pointer is **null**. where **n** is inserted.

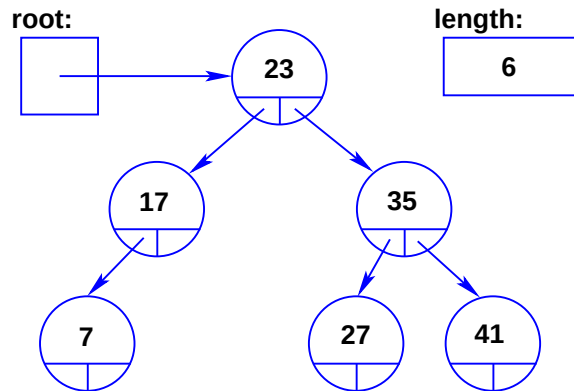
```
1 public void insert(tNode n) {
2     if (n.key < key) // n into left subtree
3         if (left == null) {
4             n.parent = this;
5             left = n; }
6     else
7         left.insert(n);
8 else // n into right subtree
9     if (right == null) {
10        n.parent = this;
11        right = n; }
12    else
13        right.insert(n); }
```



## Adding the node with key = 27

- Insert a node with key = 27 using:

```
1 T.insert(new tNode(27, "twenty_seven"));
```



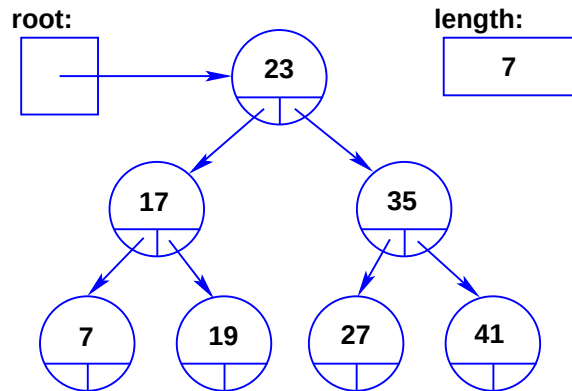
The **right** pointer of the root node is not **null** and the pointer is followed to **35** whose left pointer is **null**. where **n** is inserted.

```
1 public void insert(tNode n) {
2     if (n.key < key) // n into left subtree
3         if (left == null) {
4             n.parent = this;
5             left = n; }
6     else
7         left.insert(n);
8 else // n into right subtree
9     if (right == null) {
10        n.parent = this;
11        right = n; }
12    else
13        right.insert(n); }
```

## Adding the node with key = 19

- Insert a node with key = 19 using:

```
1 T.insert(new tNode(19, "nineteen"));
```



The **left** pointer of the root node is not **null** and is followed to **17** whose right pointer is **null** where **n** is inserted.

```
1 public void insert(tNode n) {
2     if (n.key < key) // n into left subtree
3         if (left == null) {
4             n.parent = this;
5             left = n; }
6     else
7         left.insert(n);
8     else // n into right subtree
9         if (right == null) {
10            n.parent = this;
11            right = n; }
12    else
13        right.insert(n); }
```

# The tree class

```
1 public class tree{
2 private static final int N = 100000;
3 private static int listed = 0;
4
5 private static int length;
6 private tNode root;
7
8 public tree() {
9     root = null;
10    length=0; }
11
12 public void setRoot(tNode n) {
13     root = n; }
14 public tNode getRoot() {
15     return root; }
16
17 public void insert(tNode n) {
18     if (root == null) {
19         root = n;
20         trace(n, "entered_at_root"); }
21     else
22         insert(root, n); }
23 // two more insert methods on next slide
```

## insert methods entirely in tree class

```
1 public void insert(long k, String d){
2     tNode n = new tNode(k, d);
3     if (root == null){
4         root = n;
5         trace(n, "entered_at_root"); }
6     else
7         insert(root, n);
8     length++; // Why not at the start? }
9
10 public void insert(tNode here, tNode n) {
11     if (n.getKey() < here.getKey()) // n goes left
12         // n must go into left subtree
13         if (here.getLeft() == null) {
14             n.setParent(here);
15             here.setLeft(n); }
16     else insert(here.getLeft(), n);
17     else if (here.getRight() == null) { // n goes right
18         n.setParent(here);
19         here.setRight(n); }
20     else insert(here.getRight(), n); }
```

# Methods in class tree

---

```
1 \begin{verbatim}
2 public void tDelete(long key){
3     . . . follows later
4     length--; }
5
6 public int size(){
7     return length; }
8
9 public boolean isEmpty(){
10    return length==0; }
11
12 public boolean isFull(){
13    return length==N; }
```

## The *lNr* display method for class tree

```
1 public void display() {
2     if (this == null)
3         System.out.println("Tree_is_empty.");
4     else
5         display(root); }
6
7 public void display(tNode n) { // left Node right
8                                 // i.e. lNr
9     if (n != null) {
10        display(n.getLeft()); // left
11        System.out.print( // Node
12            "In_Tree:_" + n.getNodeNumber() + " listed++ +
13            "_" + n.getKey() +
14            "_" + n.getData());
15        if (n.getParent() == null)
16            System.out.println("_Parent.key=_null");
17        else
18            System.out.println("_Parent.key=_
19                                + (n.getParent()).getKey());
20        display(n.getRight()); } } } // right
```

## A tree class where nodes are trees

```
1 public class tree {
2     private static final int N = 100000;
3     private static int length = 0;
4     private static tree root = null;
5     // tree nodes
6     private long key; private String data;
7     private tree parent;
8     private tree left; private tree right;
9
10    public tree() {
11        key = 0;
12        data = "";
13        parent = null;
14        left = null;
15        right = null; }
16
17    public tree(long k, String d) {
18        key = k;
19        data = d;
20        parent = null;
21        left = null;
22        right = null; } }
```

## A tree class where nodes are trees

```
1 public void insertNode(tree n) {
2     if (root == null){
3         root = n;
4         trace(n, "entered_at_root"); }
5     else
6         root.insert(n);
7     length++; }
8
9 public void insertNode(long key, String data) {
10    tree n = new tree(key, data);
11    if (root == null) {
12        root = n;
13        trace(n, "entered_at_root"); }
14    else
15        root.insert(n);
16    length++; }
```



## A tree class where nodes are trees

---

```
1 public void setRoot(tree n) {  
2     root = n; }  
3 public tree getRoot() {  
4     return root; }  
5  
6 public void tDelete(long key) {  
7     length--; }  
8  
9 public int size() {  
10    return length; }  
11  
12 public boolean isEmpty() {  
13    return length==0; }  
14 public boolean isFull() {  
15    return length==N; }
```

## A tree class where nodes are trees

---

```
1 public long getKey() {  
2     return key; }  
3 public void setKey(long k) {  
4     key = k; }  
5  
6 public String getData() {  
7     return data; }  
8 public void setData(String d) {  
9     data = d; }  
10  
11 public tree getLeft() {  
12     return left; }  
13 public void setLeft(tree n) {  
14     left = n; }
```

## A tree class where nodes are trees

---

```
1 public tree getRight() {  
2     return right; }  
3 public void setRight(tree n) {  
4     right = n; }  
5  
6 public tree getParent() {  
7     return parent; }  
8 public void setParent(tree n) {  
9     parent = n; }
```

## A tree class where nodes are trees

```
1 public void insert(tree n) {
2     if (n.key < key)
3         // n must go into left subtree
4         if (left == null){
5             n.parent = this;
6             trace(n, "entered_at_here.L");
7             left = n; }
8     else {
9         trace(n, "insert(here.L,_n)");
10        left.insert(n); }
11    else
12        // n must go into rightsubtree
13        if (right == null){
14            n.parent = this;
15            trace(n, "entered_at_here.R");
16            right = n; }
17        else {
18            trace(n, "insert(here.R,_n)");
19            right.insert(n); } }
```

## A tree class where nodes are trees

```
1 public void displayTree() {
2     if (root == null)
3         System.out.println("Tree_is_empty.");
4     else
5         root.display();
6     System.out.println(listed + "_nodes_in_tree."); }
7
8 public void display(){// Node left right
9     // i.e. Nlr
10    if (left != null) left.display(); // left
11    System.out.print("// N
12        "In_Tree:_Node_number_" + listed++ +
13        "_Key_=" + key +
14        "_Data_=" + data);
15    if (parent == null)
16        System.out.println("_Parent.key=_null");
17    else
18        System.out.println("_Parent.key_="
19            + parent.key);
20    if (right != null) right.display(); } // right
21
22 public void trace(tree n, String message) {
23     System.out.println("tree:_
24         + n.key
25         + "_" + message); } }
```

## A method for removing nodes from a tree

```
1 public void Remove(long k) {
2     tree n = find(k);
3     tree here;
4     if (n != null)
5         if (n.left == null) // one right child
6             if (n.parent == null){
7                 root = n.right;
8                 if (root != null)
9                     root.parent = null;
10            }
11            else if (n.key < (n.parent).key)
12                (n.parent).left = n.right;
13            else
14                (n.parent).right = n.right;
15        else if (n.right == null) // one left child
16            if (n.parent == null){
17                root = n.left;
18                if (root != null)
19                    root.parent = null;
20            }
21            else if (n.key < (n.parent).key)
22                (n.parent).left = n.left;
23            else
24                (n.parent).right = n.left;
```

## A method for removing nodes from a tree

```
1  else { // both children not null
2      here = n.left;
3      while (here.right != null)
4          here = here.right;
5      if (here.parent == null){
6          root = here.left;
7          if (root != null)
8              root.parent = null;
9      }
10     else if ( here.key < (here.parent).key)
11         (here.parent).left = here.left;
12     else
13         (here.parent).right = here.left;
14     n.key = here.key;
15     n.data = here.data; }
16 else
17     System.out.println("Node_" + k + "_not_in_tree"); }
```

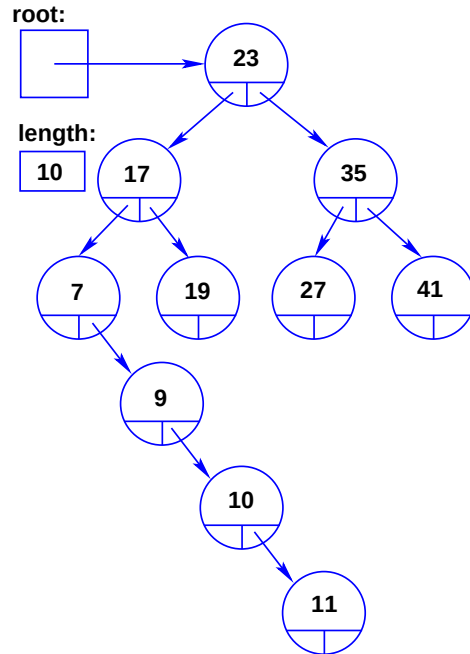
## Remove node from a tree—more readable

```
1 public void remove(long k) {
2     tree n = find(k);
3     tree here;
4     if (n != null)
5         if (n.left == null) // one right child
6             updateTree(n, n.right);
7         else if (n.right == null) // one left child
8             updateTree(n, n.left);
9         else { // both children not null
10            here = n.left;
11            while (here.right != null)
12                here = here.right;
13            updateTree(here, here.left);
14            n.key = here.key;
15            n.data = here.data; }
16     else
17         System.out.println("Node_" + k + "_not_in_tree"); }
18
19 public void updateTree (tree n, tree p){
20     if (n.parent == null){
21         root = p;
22         if (root != null)
23             root.parent = null; }
24     else if (n.key < (n.parent).key)
25         (n.parent).left = p;
26     else
27         (n.parent).right = p; }
```



# Explanation of removal of nodes from a tree

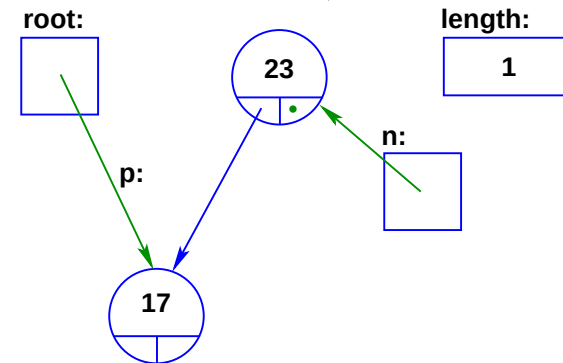
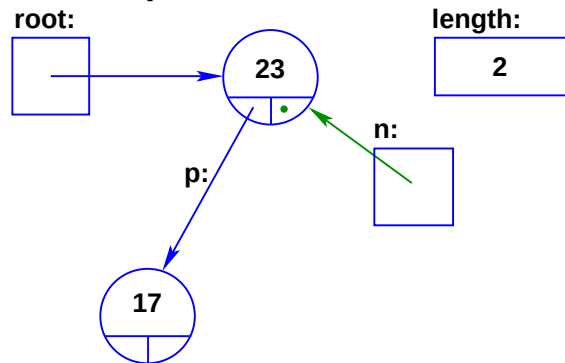
Consider the tree



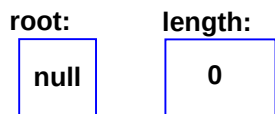
1. Removing elements with zero children is the easiest: Simply replace the pointer to the leaf with a null. We will see later that *this case can be ignored* because it is subsumed by the case where any one of the pointers is a null. Initially, the nodes with keys 11, 19, 27 and 41 are such nodes.
2. To remove a node with only one child: The non-null pointer emanating from the node to be deleted replaces the pointer pointing to the node that must be removed. Initially, the nodes with keys 7, 9, and 10 are such nodes.
3. To remove a node with two children: Find its biggest (smallest) child in its left (right) subtree, copy that biggest (smallest) child over the node to be deleted and then remove the node that was copied as usual. The copied node cannot have two children.

## Remove using `update(n, p)`

- The method `update(n, p)` is given a non-null node `n` to be removed and a pointer `p` to the node to replace it.
- If `n.parent == null` then `n` is the root node, i.e. it has no parent:



- Then `root = p`; suffices, but `p` could become `null`. This would make the `root == null`,



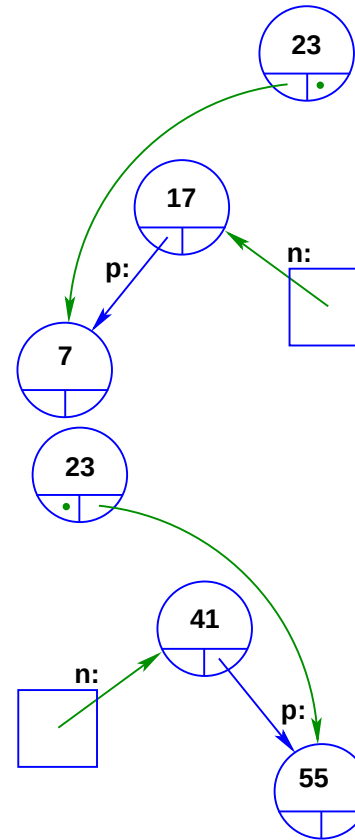
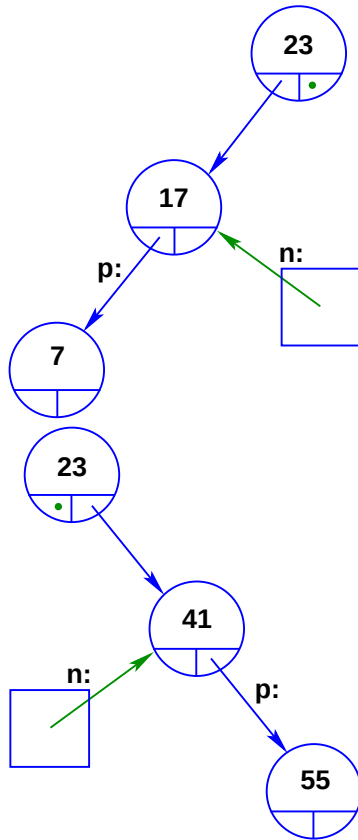
otherwise the new node, now pointed to by `root`, must have its `parent` set to `null`, i.e.

```
1 if (root != null)
2   root.parent = null;
```

## Remove using `update(n, p)`

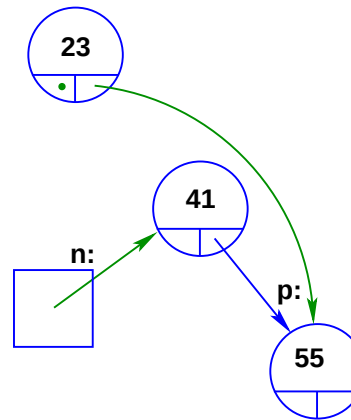
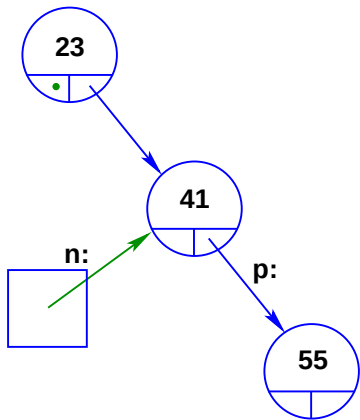
- The parent may have its `left` pointing to `n`. Test this using:

```
1  else if (n.key < (n.parent).key)
2      (n.parent).left = p;
3  else
4      (n.parent).right = p;
```



## Remove using update(n, p)

```
1 public void updateTree (tree n, tree p){
2     if (n.parent == null){
3         root = p;
4         if (root != null)
5             root.parent = null;
6     }
7     else if (n.key < (n.parent).key)
8         (n.parent).left = p;
9     else
10        (n.parent).right = p;
11 }
```



## Traversing trees—depth-first

- A tree may be traversed in  $\ell N r$  order or any of its other 5 combinations,  $\ell r N, N \ell r, N r \ell, r N \ell, r \ell N$ .

```
1 public void NlrTraverse (tree t) {  
2     visit(t);  
3     if (t.left != null)  
4         left.NlrTraverse();  
5     if (t.right != null)  
6         right.NlrTraverse(); }
```

- $N \ell r$  is also called ‘depth-first’ order.
- A **visit** to a node could entail printing its value or processing it somehow.

## Traversing trees—breadth-first

- A tree may be traversed in ‘breadth-first’ order, where all the nodes on one level are dealt with before going down to the next level.
- Start by queueing the root.
- Service the head of the queue and put the children of each node into the queue and repeat until the queue empties.

```
1 queue Q = new queue();  
2 public void BFS(tree t) {  
3     Q.enqueue(t);  
4     while (!Q.isEmpty()) {  
5         n = Q.dequeue();  
6         Q.enqueue(n.left);  
7         Q.enqueue(n.right);  
8         visit(n); } }
```

- A **visit** to a node could entail printing or processing it.

## Heaps—priority trees

- A *heap* has a tree structure without pointers,
- “most important” item is at the top of the tree,
- and is also called a *priority queue*.
- Store the *heap* in an array, **A**, of type **node** [ ].
- $\text{leftChild}(i) \equiv 2 \times i$
- $\text{rightChild}(i) \equiv 2 \times i + 1$
- $\text{parent}(i) \equiv \lfloor i/2 \rfloor$
- The *heap* property:  
 $A[\text{parent}(i)] \geq A[i]$  for a heap with its greatest element on top, *or*  
 $A[\text{parent}(i)] \leq A[i]$  with the minimum element on top.

## The *heap* property stated in words

---

- In a maximizing *heap*:  
For every node its key is greater or equal to the key of any of its descendants.
- In a minimizing *heap*:  
Every node's key is less than or equal to the key of any of its descendants.



## Heaps—the methods

---

- Constructors, gets and sets.
- *node removeMax(); or node removeMin();*
- *boolean isEmpty();*
- *boolean isFull();*
- *insert(node n);*
- *buildUp(node X[], int sizeX);*
- *heapify(int i);*
- *sort(node X[], int sizeX);*

## Heaps—Constructors, gets and sets

```
1 public class Heap {
2     private int final maxSize = 1 << 20;
3     private int n = 0; // n is the size of the heap
4     private node[] A = new node[maxSize+1];
5
6     public Heap(){ }
7
8     int getSize(){
9         return n; }
10    void setSize(int i){
11        n = i; }
12
13    ... }
```

## Heaps—*heapify(int i)*

```
1 public void heapify(int i) {  
2  
3     l = left(i); r = right(i);  
4     if (l <= n && A[l].key > A[i].key)  
5         top = l;  
6     else  
7         top = i;  
8  
9     if (r <= n && A[r].key > A[top].key)  
10         top = r;  
11  
12     if (top != i) {  
13         swap(A[i], A[top]); // this is not Java  
14         heapify(top); } }
```

## Heaps—*heapify(int i)*—a bit faster

```
1 public void heapify(int i) {  
2  
3      $\ell = 2 \times i$  ;  $r = \ell + 1$ ;  
4     if ( $\ell \leq n$  &&  $A[\ell].key > A[i].key$ )  
5         top =  $\ell$ ;  
6     else  
7         top = i;  
8  
9     if ( $r \leq n$   
10        &&  $A[r].key > A[top].key$ )  
11        top = r;  
12  
13     if (top != i) {  
14         swap( $A[i]$ ,  $A[top]$ );  
15         if (left(top) <= n)  
16             heapify(top); } }
```

## Heaps—*heapify(int i)*—iterative

First, we see if `child = left(i)` is within the heap. Next, make `child += 1` if the right child is bigger than the left child, then `break` if `A[i] >= A[child]` or carry on by swapping `A[i]` and `A[child]` and repeat another level.

```
1 public void heapify(int i) {
2     int child = left(i);
3     while (child <= n) {
4         if (child < n
5             && A[child] < A[child+1])
6             child++;
7         if (A[child] < A[i]){
8             swap(A[i], A[child]); // not Java
9             i = child;
10            child = left(i); }
11     else
12         break } }
```

## Heaps—*insert(node N)*

```
1 public void insert(node N) {  
2  
3     n++;  
4     i = n;  
5     while (i > 1 && A[parent(i)].key < N.key) {  
6         A[i] = A[parent(i)];  
7         i = parent(i); }  
8     A[i] = N; }
```

## Heaps—*node removeMax()*

```
1 public node remove() {  
2     if (n < 1)  
3         return null;  
4     else {  
5         top = A[1];  
6         A[1] = A[n];  
7         n--;  
8         heapify(1);  
9         return top; } }
```

## Heaps—*node buildUp(node X[], int sizeX)*

Pseudocode:

```
1 // pre: given an array X of sizeX nodes
2 // post: node X[] is copied into the
3 // heap array, node A[] which is
4 // turned into a heap of size n.
5 public void buildUp(node X[], int sizeX) {
6
7     for (i = 1; i <= sizeX; i++)
8         A[i] = X[i];
9
10    n = sizeX;
11    for (i = n/2; i > 0; i--)
12        heapify(i); }
```



## Heaps—*node [] sort(node X[], int sizeX)*

```
1 // pre: Given array X[] of sizeX elements
2 // n becomes the size of the heap.
3 // post: The array node A[] is sorted.
4
5 public node [] sort(node X[], int sizeX) {
6     buildUp(X, sizeX);
7     for (i = n; i>1; i--){
8         swap(A[1], A[i]);
9         n--;
10        heapify(1); }
11    n = sizeX;
12    return A;}
```

Note that the heap is both created and “destroyed” in the process of sorting using **buildUp** and **sort**.

The procedure invocation **swap(A[1], A[i])** needs to be adapted to your programming language. Try **swap(A, 1, i);** in Java.

## Heaps—*node sort*(*node X*[], *int sizeX*)

```
1 // pre: Given array X[] of sizeX elements
2 // n becomes the size of the heap.
3 // post: The array node A[] is sorted.
4
5 public void sort(node X[], int sizeX) {
6     buildUp(X, sizeX);
7     for (i = n; i>1; i--){
8         swap(A[1],A[i]);
9         n--;
10        heapify(1); }
11    copy A back to X;}
```

Note that the heap is both created and destroyed in the process of sorting using **buildUp** and **sort**.

The procedure invocation **swap(A[1], A[i])** needs to be adapted to your programming language.

## Heaps—complexity of `buildUp`

- In a heap of  $n$  nodes call the total time to run `buildUp`,  $T$ .
- The height of the heap tree,  $H = \log_2 n$ .
- For the purposes of this derivation the height of a vertex is the number of edges that separate the vertex from the bottom level of the tree.
- At most  $\lceil \frac{n}{2^{h+1}} \rceil$  vertices are of height  $h$ .
- The heights of vertices run from  $0, 1, 2, \dots, H$ .
- So, by running over all heights:

$$T \leq \sum_{h=0}^H h \frac{n}{2^{h+1}} = n \sum_{h=1}^H h 2^{-h-1}$$

## Heaps—complexity of `buildUp`

---

- The heights of vertices run from  $0, 1, 2, \dots, H$ .
- So, by running over all heights:

$$T \leq \sum_{h=0}^H h \frac{n}{2^{h+1}} = n \sum_{h=1}^H h 2^{-h-1}$$

- It is easily shown that  $\sum_{h=1}^H h 2^{-h-1} < 1$ ,
- and it follows that  $T = O(n)$ .

$$S_H = \sum_{h=1}^H h2^{-h-1} < 1$$

$$\begin{aligned} S_H &= \sum_{h=1}^H h2^{-h-1} \\ &= 1.2^{-2} + 2.2^{-3} + \dots + (H-1).2^{-H} + H.2^{-H-1} \\ 2S_H &= 1.2^{-1} + 2.2^{-2} + 3.2^{-3} + \dots + H.2^{-H} \end{aligned}$$

Subtracting  $S_H$  from  $2S_H$  gives

$$\begin{aligned} S_H &= \underbrace{2^{-1} + 2^{-2} + 2^{-3} + \dots + 2^{-H}}_{0.111\dots 11_2} - H.2^{-H-1} \\ &= 0.111\dots 11_2 - H.2^{-H-1} \\ &= 1 - 2^{-H} - H.2^{-H-1} \\ &< 1 \end{aligned}$$

## An $n$ -node heap has height $H = \lfloor \log_2 n \rfloor$

Suppose that the root of a heap has height 0.

| n    | height | $\lfloor \log_2 n \rfloor$ |
|------|--------|----------------------------|
| 1    | 0      | 0                          |
| 2    | 1      | 1                          |
| 3    | 1      | 1                          |
| 4    | 2      | 2                          |
| 5    | 2      | 2                          |
| 6    | 2      | 2                          |
| 7    | 2      | 2                          |
| 8    | 3      | 3                          |
| 9    | 3      | 3                          |
| 10   | 3      | 3                          |
| 11   | 3      | 3                          |
| 12   | 3      | 3                          |
| 13   | 3      | 3                          |
| 14   | 3      | 3                          |
| 15   | 3      | 3                          |
| 16   | 4      | 4                          |
| 31   | 4      | 4                          |
| 32   | 5      | 5                          |
| 64   | 6      | 6                          |
| 127  | 6      | 6                          |
| 255  | 7      | 7                          |
| 256  | 8      | 8                          |
| 511  | 8      | 8                          |
| 1023 | 9      | 9                          |
| 1024 | 10     | 10                         |
| 2048 | 11     | 11                         |

- A heap is *full* when it has exactly  $2^{h+1} - 1$  nodes, where  $h$  is the height of the heap.
- Adding a node to a full heap increases its height by 1.
- The height of the  $i^{th}$  node in a heap is  $\lfloor \log_2 i \rfloor$ .

At most  $\lceil \frac{n}{2^{h+1}} \rceil$  vertices are of height  $h$

Suppose that the root of a heap has height 0.

| n    | height | $\lceil \frac{n}{2^{h+1}} \rceil$ |
|------|--------|-----------------------------------|
| 1    | 0      | 1                                 |
| 2    | 1      | 1                                 |
| 3    | 1      | 2                                 |
| 4    | 1      | 2                                 |
| 5    | 1      | 3                                 |
| 6    | 1      | 3                                 |
| 7    | 1      | 4                                 |
| 8    | 1      | 4                                 |
| 9    | 1      | 5                                 |
| 10   | 1      | 5                                 |
| 11   | 1      | 6                                 |
| 12   | 1      | 6                                 |
| 13   | 1      | 7                                 |
| 14   | 1      | 7                                 |
| 15   | 1      | 8                                 |
| 16   | 1      | 8                                 |
| 31   | 1      | 16                                |
| 32   | 1      | 16                                |
| 64   | 1      | 32                                |
| 127  | 1      | 64                                |
| 255  | 1      | 128                               |
| 256  | 1      | 128                               |
| 511  | 1      | 256                               |
| 1023 | 1      | 512                               |
| 1024 | 1      | 512                               |
| 2048 | 1      | 1024                              |

- A heap is *full* when it has exactly  $2^{h+1} - 1$  nodes, where  $h$  is the height of the heap.
- Adding a node to a full heap increases its height by 1.
- The height of the  $i^{th}$  node in a heap is  $\lfloor \log_2 i \rfloor$ .
- The heights run from  $1, 2, \dots, H$ .
- At most  $\lceil \frac{n}{2^{h+1}} \rceil$  vertices are of height  $h$ .

## Heaps—applications

Heaps have many applications in other algorithms.<sup>2</sup>

|                          |                                                 |
|--------------------------|-------------------------------------------------|
| Sorting                  | Heap sort is optimal $\sim 2N \log N$           |
| Data compression         | Combine smallest two elements in Huffman coding |
| Event-driven simulation  | Put most important request first                |
| Physics simulation       | Next particle to collide                        |
| Dijkstra's shortest path | Finds closest node                              |
| Prim's MST algorithm     | Choosing edge with minimal weight               |
| Number theory            | Sum of powers                                   |
| Artificial intelligence  | $A^*$ search                                    |
| Statistics               | Maintain largest $K$ values in a sequence       |
| Finding the median       | Compare using quicksort for median              |
| Operating systems        | Prioritize interrupts, load balancing           |
| Discrete optimization    | Bin packing, scheduling                         |
| Bayesian spam filtering  | Filter messages with highest spam weight        |
| Reducing roundoff error  | How is heap used?                               |

---

<sup>2</sup>The list was inspired by [Sedgewick and Wayne, 2011]



At most  $\lceil \frac{n}{2^{h+1}} \rceil$  vertices are of height  $h$

Now we vary the heights for a fixed value of  $n$ .

Suppose that the root of a heap has height 0.

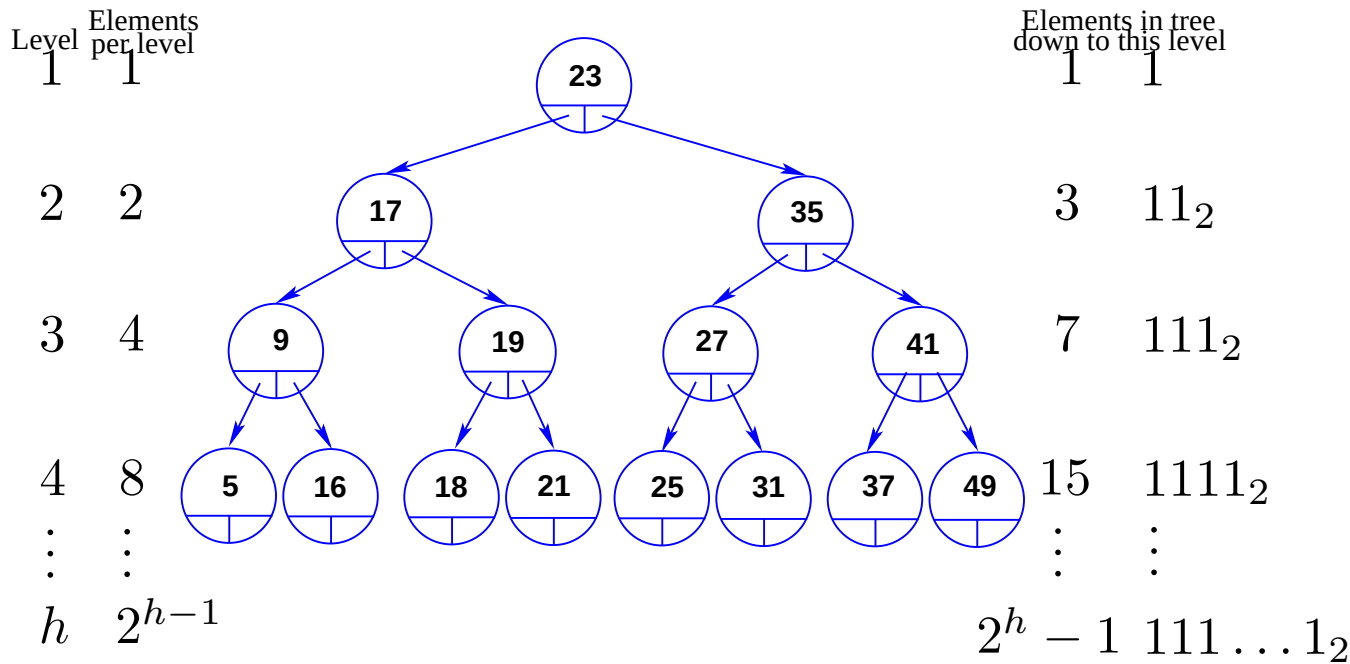
| n    | height | $\lceil \frac{n}{2^{h+1}} \rceil$ |
|------|--------|-----------------------------------|
| 4096 | 0      | 2048                              |
| 4096 | 1      | 1024                              |
| 4096 | 2      | 512                               |
| 4096 | 3      | 256                               |
| 4096 | 4      | 128                               |
| 4096 | 5      | 64                                |
| 4096 | 6      | 32                                |
| 4096 | 7      | 16                                |
| 4096 | 8      | 8                                 |
| 4096 | 9      | 4                                 |
| 4096 | 10     | 2                                 |
| 4096 | 11     | 1                                 |
| 4096 | 12     | 0                                 |

$$\sum_{h=1}^H h 2^{-h-1} = 4083.$$

Which is clearly *less* than 4096.

- A heap is *full* when it has exactly  $2^{h+1} - 1$  nodes, where  $h$  is the height of the heap.
- Adding a node to a full heap increases its height by 1.
- The height of the  $i^{th}$  node in a heap is  $\lfloor \log_2 i \rfloor$ .
- The heights run from  $1, 2, \dots, H$ .
- At most  $\lceil \frac{n}{2^{h+1}} \rceil$  vertices are of height  $h$ .

# A complete binary search tree



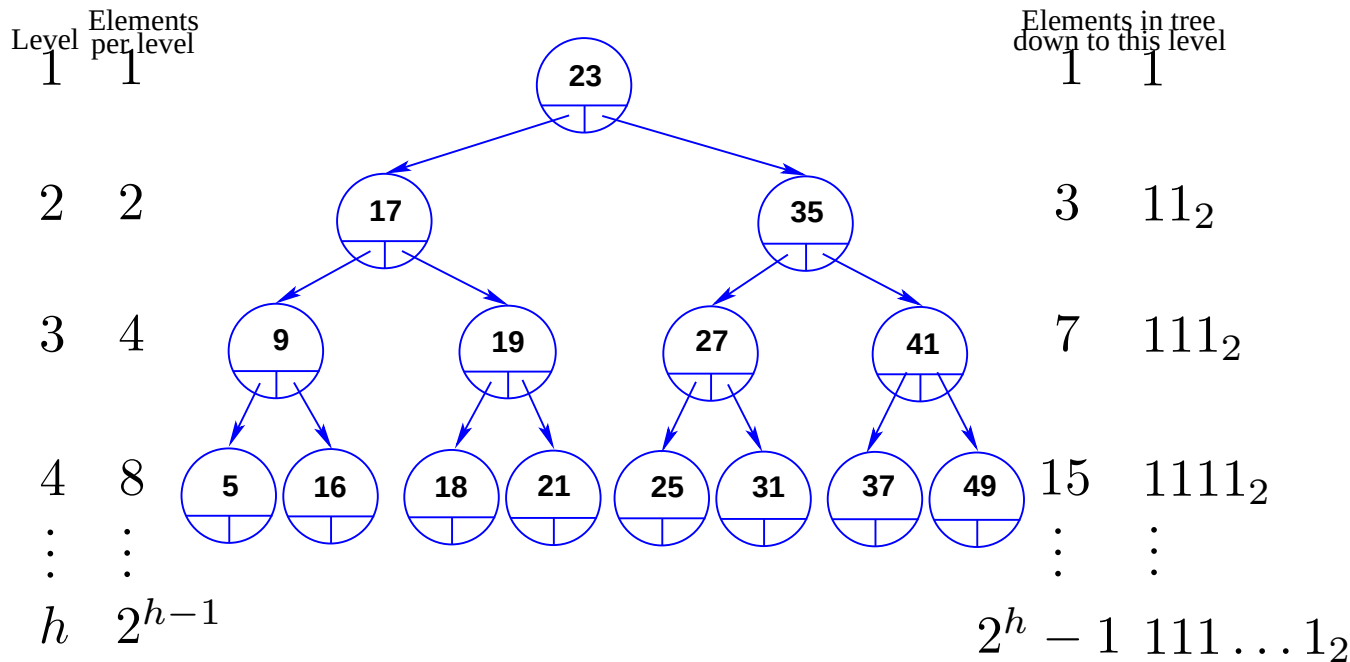
A tree with  $h$  levels, has  $N = 2^h - 1$  nodes, and  $h = \log_2(N + 1)$ .

The number of probes to visit each node once in a complete BST is

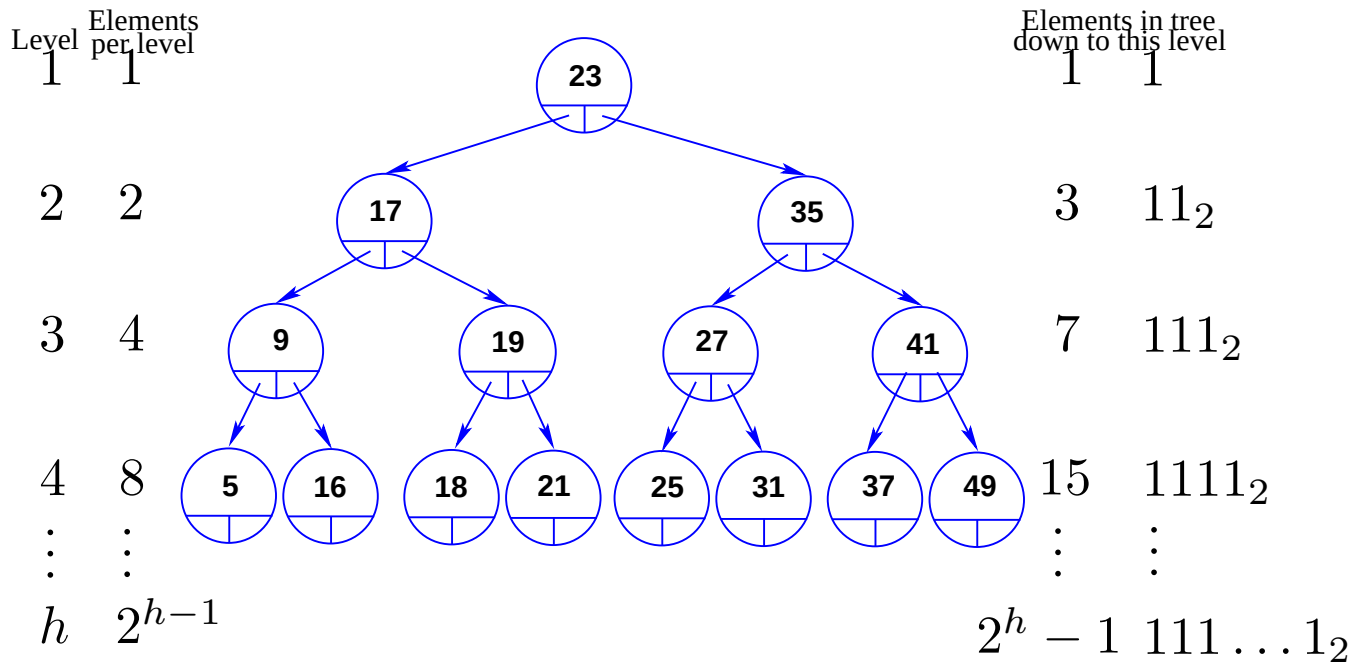
$$T = \sum_{r=1}^h r 2^{r-1}, \text{ and } 2T = \sum_{r=1}^h r 2^r$$

$$T = 2T - T = h 2^h - \sum_{r=0}^{h-1} 2^r = h 2^h - 1 + 2^h$$

# A complete binary search tree

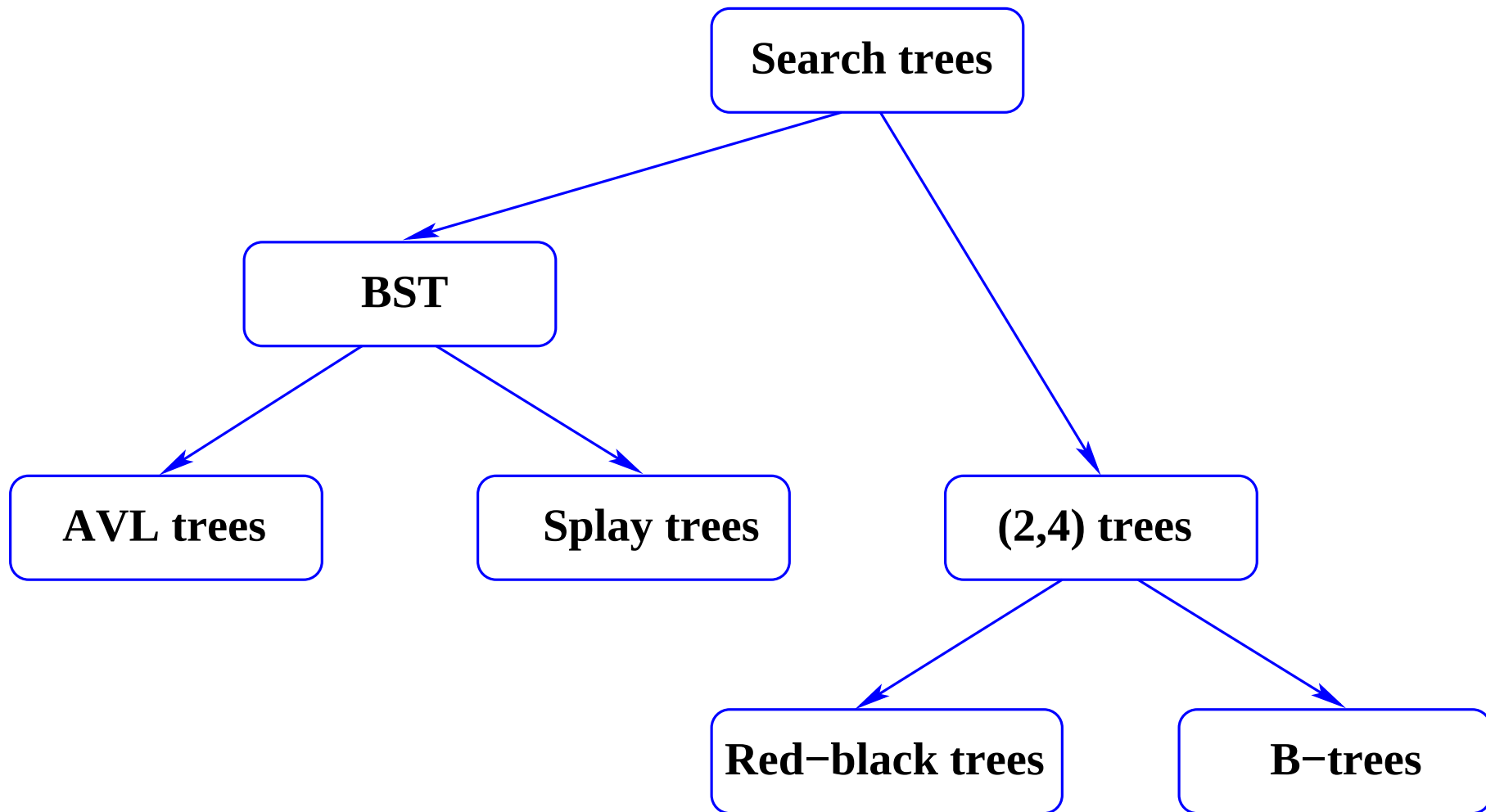


# A complete binary search tree

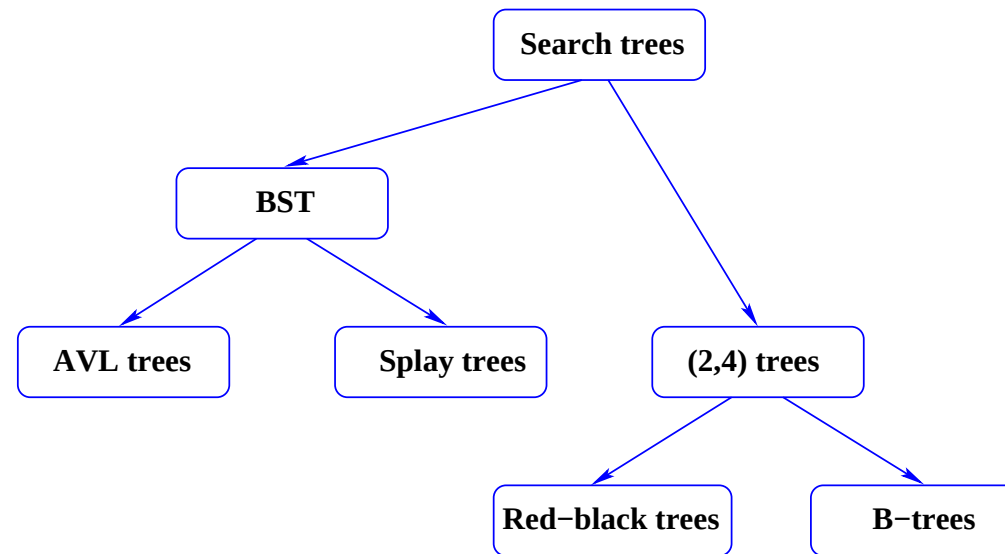


## Search trees—a hierarchy

---



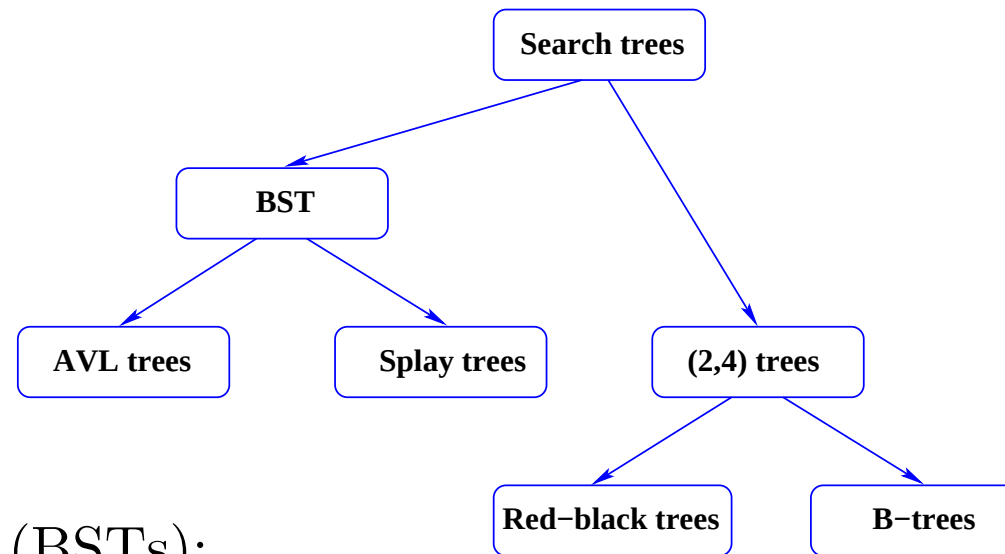
# Search trees



Search trees implement the dictionary abstract data structure (ADT) and have the methods

- `boolean isFull()` and `boolean isEmpty()`
- `node find(k)` as well as `iterator findAll(k)`
- `boolean insert(k, data)` or `boolean insert(n)`
- `node remove(k)`.

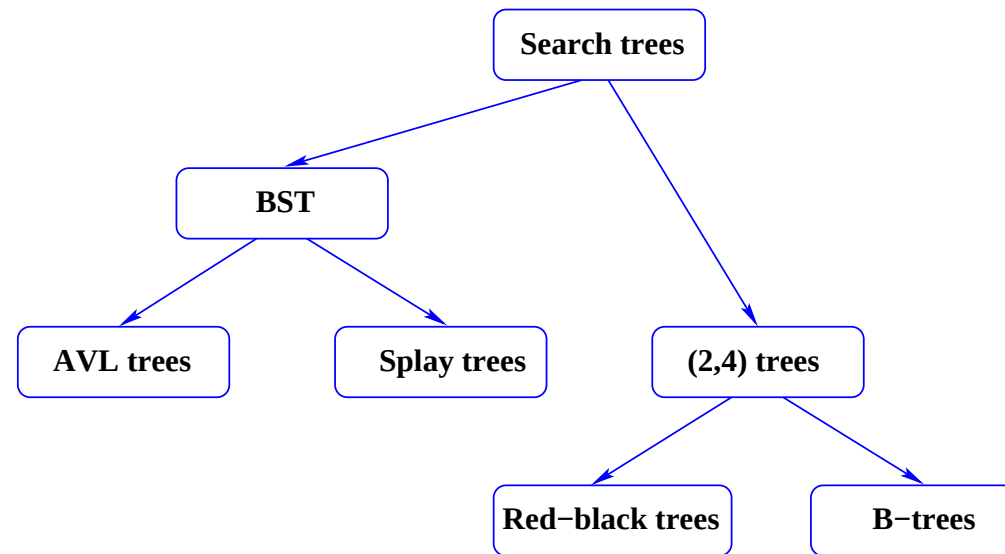
# Binary search trees (BSTs)



Binary search trees (BSTs):

- Unbalanced BSTs may behave badly, e.g. a tree built up from a sorted list presents  $O(n)$  times.
- *AVL* trees are balanced BSTs with  $O(\log n)$  times.
- Splay trees are BSTs that dynamically move the most accessed nodes to the top of the tree. They have an amortized running time: After  $m$  operations the average time to access item  $k$ —which has been accessed  $f(k)$  times—is  $O(\log(m/f(k)))$ .

# Multi-way search trees



Multi-way search trees:

- $2$ - $4$  trees or  $2$ - $3$ - $4$  trees put all the leaf nodes at the same level.
- *Red-black* trees correspond to  $2$ - $4$  trees, use  $O(n)$  space and have  $O(n \log n)$  worst case insertion, deletion and lookup times.
- *B-trees* are multi-way trees that optimize input/output operations for large data bases on disks.

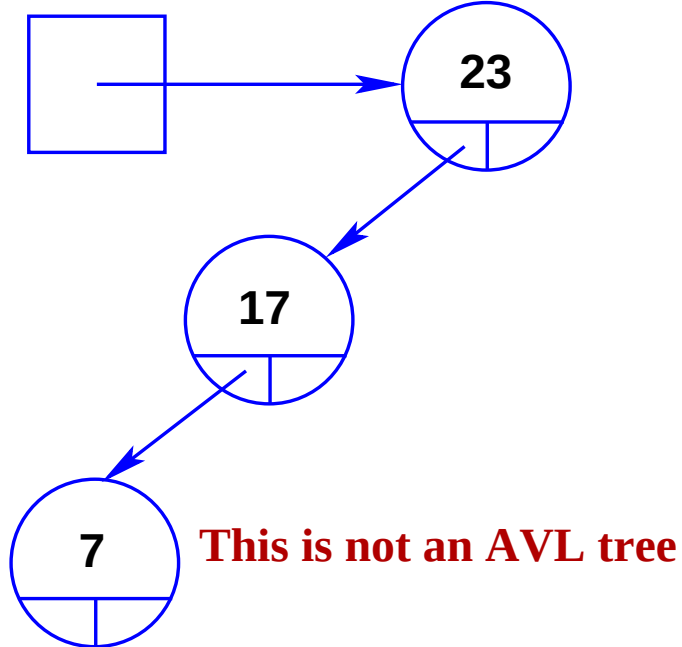


- The most difficult method to program is `delete(k)` which removes the node with key `k` from a BST:
  - Removing node `p` with at least one node that has a `null` child, entails moving the other child pointer up to `p.parent`'s child that points to `p` and so doing discarding `p`.
  - To remove a node `p` with two children: Find its biggest (smallest) child in its left (right) subtree, copy that biggest (smallest) child over the node to be deleted and then remove the node that was copied as usual. The copied node cannot have two children.
- The other methods are pretty elementary.
- Retrieval in a balanced BST takes  $O(\log n)$  time and in an unbalanced BST retrieval can be as bad as  $O(n)$ .
- So, much effort is made to keep BSTs balanced.

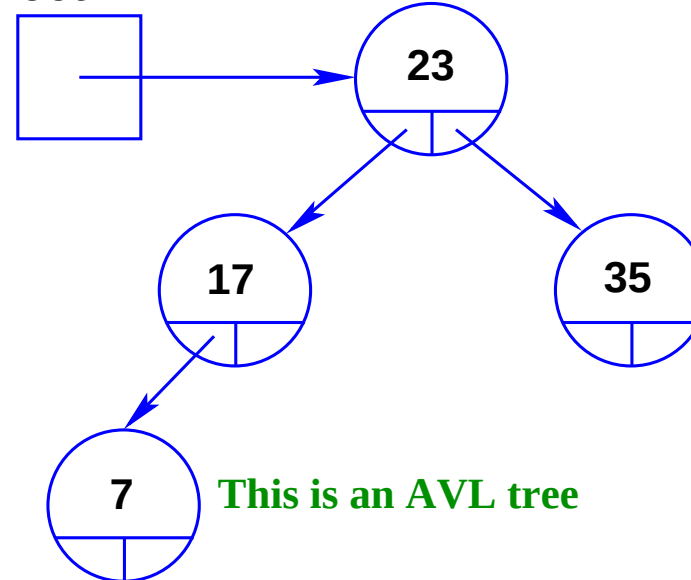
## AVL trees

- The two mathematicians G.M. Adel'son-Vel'skii and Y.M. Landis invented AVL trees in 1962. Georgii Maksimovich Adelson-Velskii and Yevgenii Michailovich Landis.
- An AVL tree has the *height-balanced property* or *AVL property*:  
The children of every internal node  $n$  have heights that differ by at most 1.

root:



root:



- Since AVL trees are balanced the retrieval of nodes is at worst  $\log_2 n$

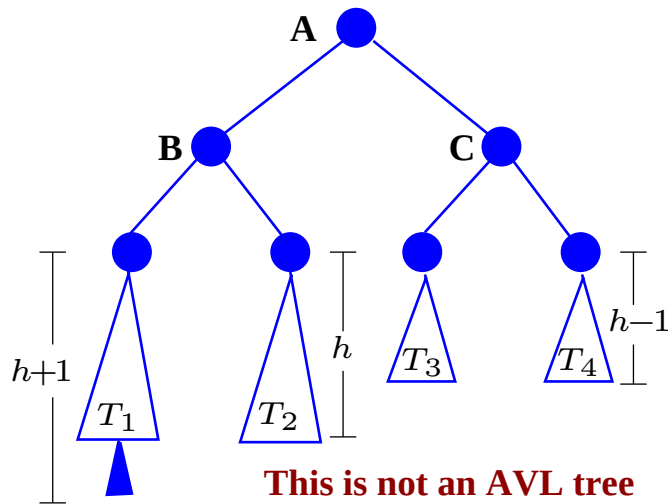
## AVL trees—height of nodes

Consider a complete BST.

| Level   | Nodes<br>on level | Total         |
|---------|-------------------|---------------|
| 1       | 1                 | 1             |
| 2       | 2                 | 3             |
| 3       | 4                 | 7             |
| 4       | 8                 | 15            |
| 5       | 16                | 31            |
| 6       | 32                | 63            |
| ...     | ...               | ...           |
| $h - 1$ | $2^{h-2}$         | $2^{h-1} - 1$ |
| $h$     | $2^{h-1}$         | $2^h - 1$     |

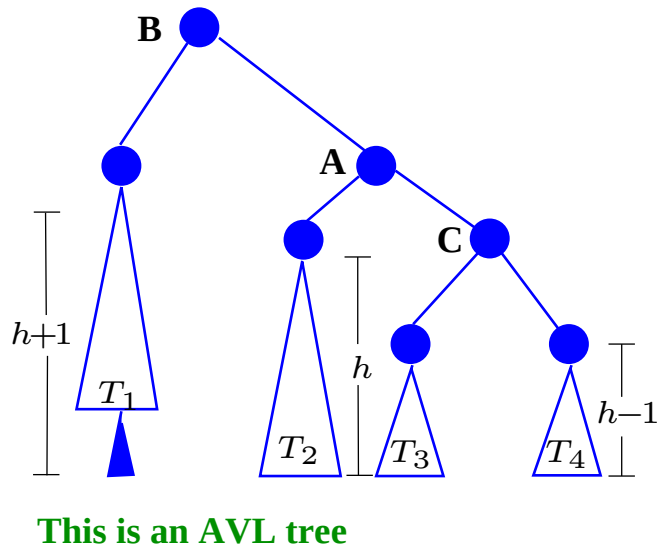
- A complete height- $h$  BST has  $n = 2^h - 1$  nodes, so that  $h = \log_2(n + 1)$ .
- Note that one more than half of its nodes are at level  $h$ .  
Since  $2^h - 1 = 2^{h-1} + [2^{h-1} - 1]$ .
- The maximum height of a complete binary tree is  $h = \log_2(n + 1)$ ,  
i.e. height  $= O(\log n)$ .
- An AVL tree is “almost” complete so its maximum height is  $O(\log n)$ .

## Turning a non-AVL tree into an AVL tree



Node *B* does not violate the AVL property, and neither does node *C*, but node *A* has a child *B* with height  $h+2$  and a child *C* with height  $h$ . The difference in the heights is 2 which violates the AVL property.

After a single rotation, in which *A* becomes *B*'s right child and *T*<sub>2</sub> becomes *A*'s left child; the tree stays a BST and it becomes an AVL tree:

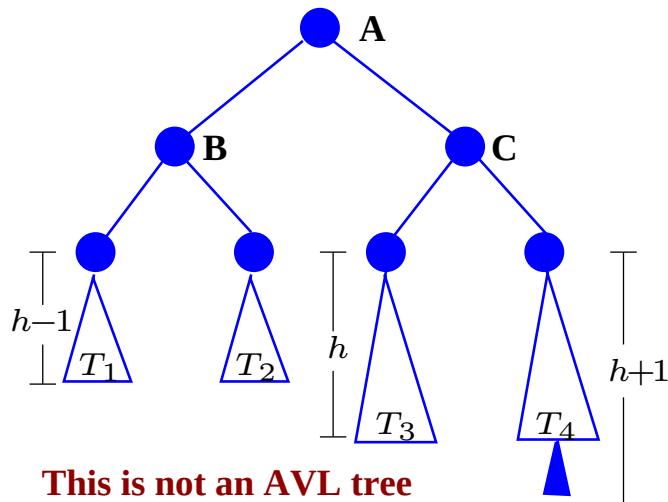


Given only *A*, the code is:

```
B = A.left;  
A.left = B.right;  
B.right = A;
```

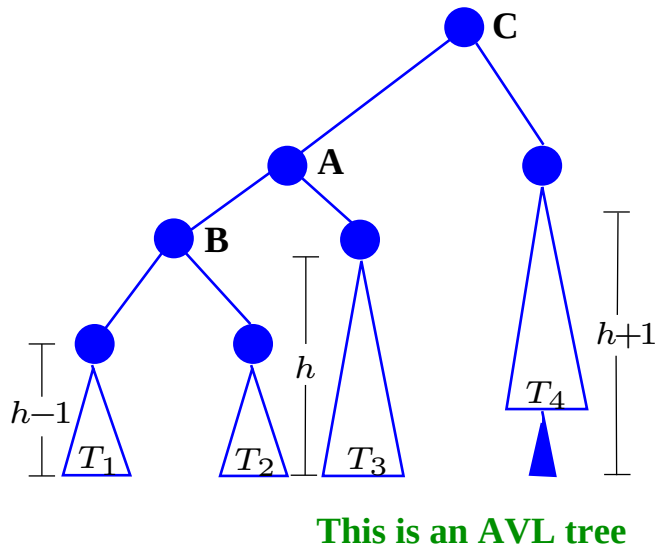
None of the nodes now violates the AVL property.

## Turning a non-AVL tree into an AVL tree—mirror



Node  $B$  does not violate the AVL property, and neither does node  $C$ , but node  $A$  now has a child  $C$  with height  $h + 2$  and a child  $B$  with height  $h$ . The difference in the heights is 2 which violates the AVL property.

After a single rotation,  $A$  becomes  $C$ 's *left* child and  $T_3$  becomes  $A$ 's *right* child; the tree stays a BST and it becomes an AVL tree:

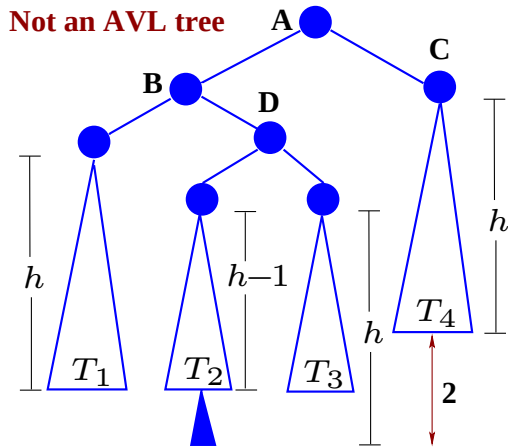


Given only  $A$ , the code is:

```
1 C = A.right;  
2 A.right = C.left;  
3 C.left = A;
```

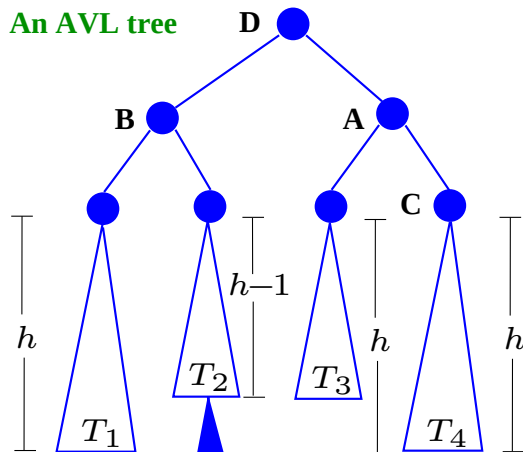
None of the nodes now violates the AVL property.

## AVL trees—double rotations



Node *A* violates the AVL property. The other internal nodes do not. A double rotation is required to turn this into an AVL tree.

After a double rotation, in which *D* becomes the root, the tree stays a BST and becomes an AVL tree:

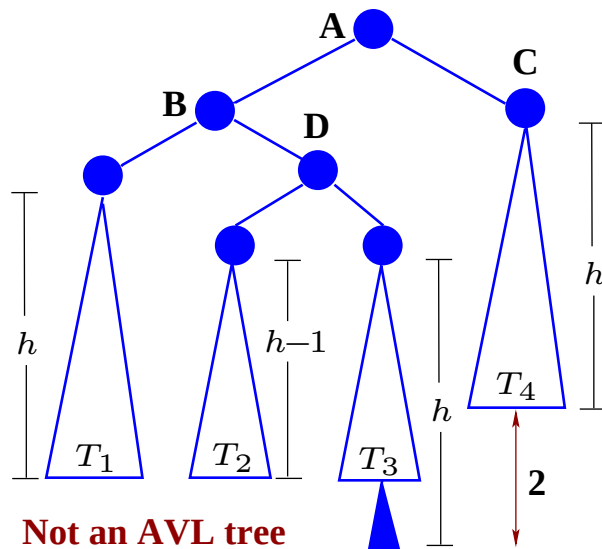


Given only *A*, the code is:

```
1  B = A.left;
2  D = B.right;
3  B.right = D.left;
4  D.left = B;
5  A.left = D.right;
6  D.right = A;
```

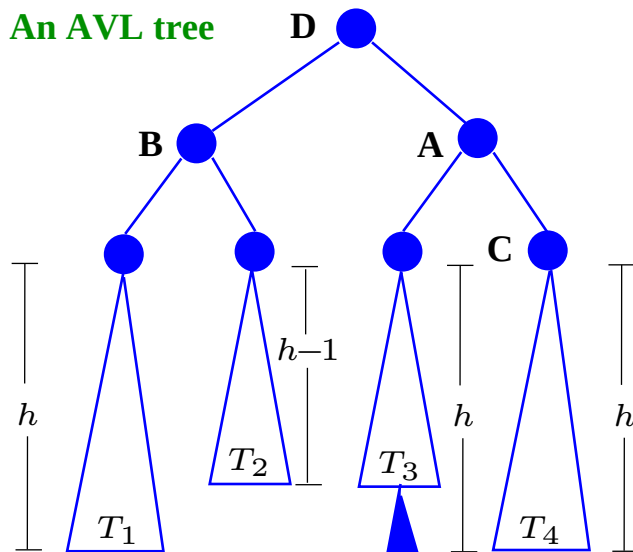
*D* becomes the root and none of the nodes now violates the AVL property.

## AVL trees—double rotations: same code



Insertion at  $T_3$  instead of at  $T_2$  results in: Node  $A$  violates the AVL property. The other internal nodes do not. A double rotation is required to turn this into an AVL tree.

After a double rotation, in which  $D$  becomes the root, the tree stays a BST and becomes an AVL tree:

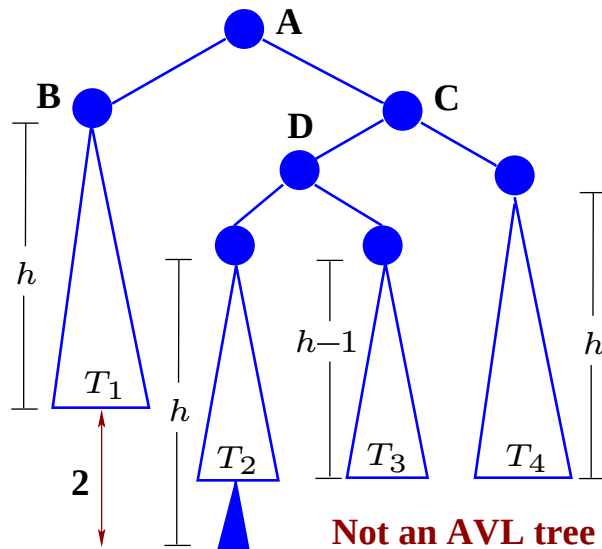


Given only  $A$ , the code is:

```
1  B = A.left;  
2  D = B.right;  
3  B.right = D.left;  
4  D.left = B;  
5  A.left = D.right;  
6  D.right = A;
```

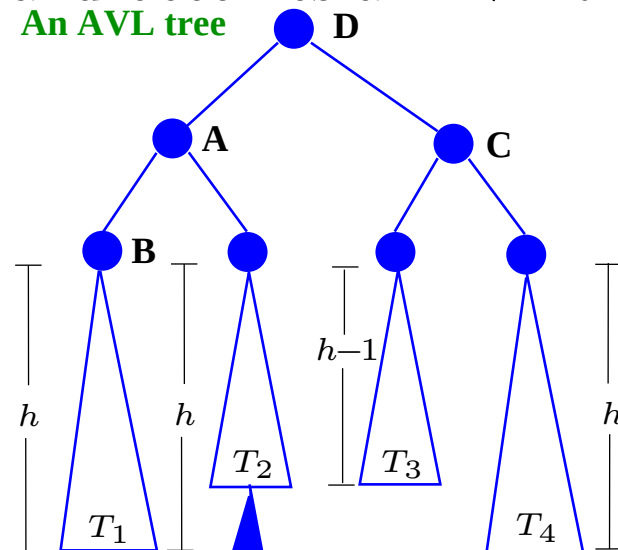
$D$  becomes the root and none of the nodes now violates the AVL property.

## AVL trees—double rotations: mirror



Insertion at  $T_2$  now results in: Node  $A$  violating the AVL property. The other internal nodes do not. A double rotation is required to turn this into an AVL tree.

After a double rotation, in which  $D$  becomes the root, the tree stays a BST and becomes an AVL tree:



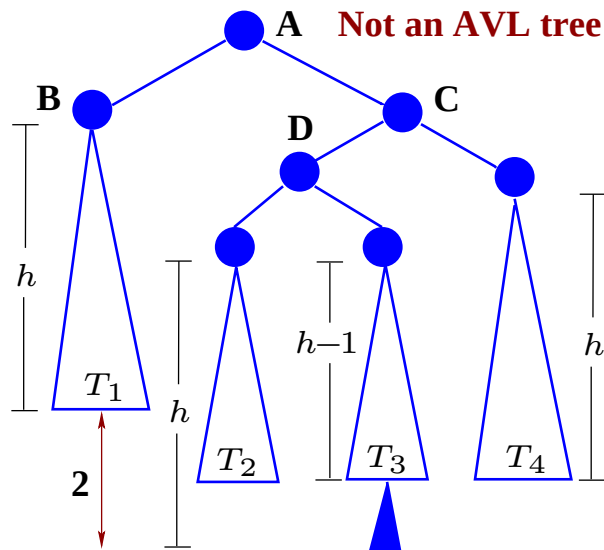
Given only  $A$ , the code is:

```
1  C = A.right;  
2  D = C.left;  
3  A.right = D.left;  
4  D.left = A;  
5  C.left = D.right;  
6  D.right = C;
```

The tree with  $D$  as root is now an AVL tree.

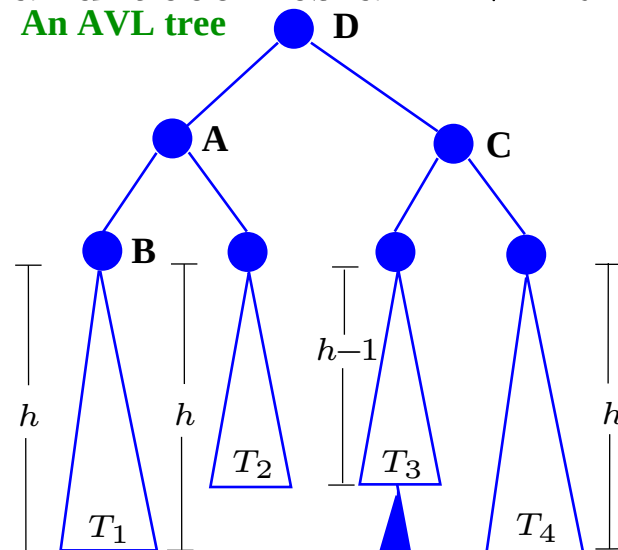


## AVL trees—double rotations: mirror



An insertion at  $T_3$  now results in node  $A$  violating the AVL property. The other internal nodes do not. A double rotation is required to turn this into an AVL tree.

After a double rotation, in which  $D$  becomes the root, the tree stays a BST and becomes an AVL tree:



Given only  $A$ , the code is:

```
1  C = A.right;  
2  D = C.left;  
3  A.right = D.left;  
4  D.left = A;  
5  C.left = D.right;  
6  D.right = C;
```

The tree with  $D$  as root is now an AVL tree.

## Insertion into an AVL tree

- Suppose the new node is called **n**
- **insert** proceeds as usual, but the balance factor **bf** of each traversed node is updated in the appropriate direction:

```
1  if (here.key < (here.parent).key)
2      ++bf;
3  else if (here.key > (here.parent).key)
4      --bf;
```
- Remember the last node with a non-zero balance factor—the critical node.
- Update until a leaf is found for the new node **n**.
- From this position work bottom-up to the critical node readjusting balance factors and perform a rotation if necessary.

## Deletion from an AVL tree

---

- Deletion is more complex.
- The tree can sometimes only be rebalanced after  $O(\log n)$  rotations, where  $n$  is the number of nodes in the tree.
- The worst case is  $O(\log n)$  rotations.

### Summary

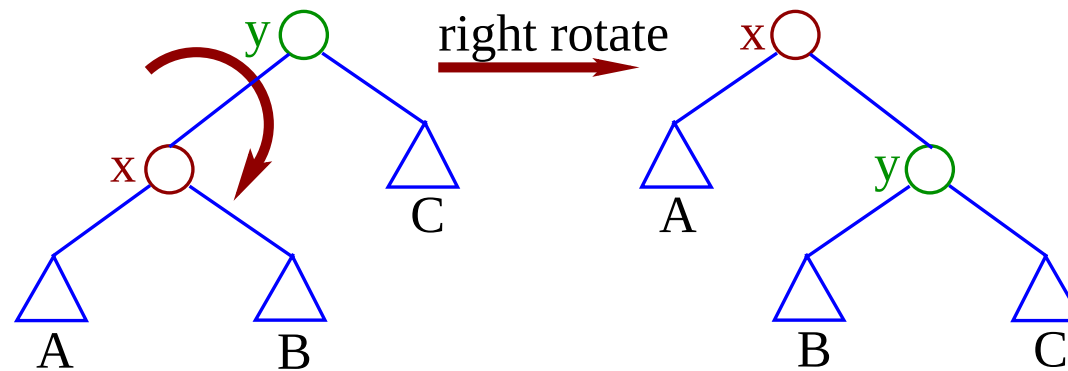
- AVL trees are efficient. It has been shown that they require  $1.45 \times$  comparisons of optimal trees.
- Experiments have shown that AVL trees yield an average search time of  $\log_2 n + 0.25$  comparisons.
- A disadvantage is the extra space, and bookkeeping required for handling the balance factors.

# Splay trees

---

- Splay trees do not force balance.
- After each access a *move-to-root* operation is applied.
- This *splaying* operation is applied at the bottom-most node **n** reached during insertion, deletion or lookup.
- Splaying guarantees  $O(\log n)$  amortized behaviour.
- No balance, height or colour labels are required.
- Splay trees were first described by Sleator and Tarjan in 1983.

## Splay tree rotations—right rotation



Rotation of the edge joining node  $x$  and node  $y$ .

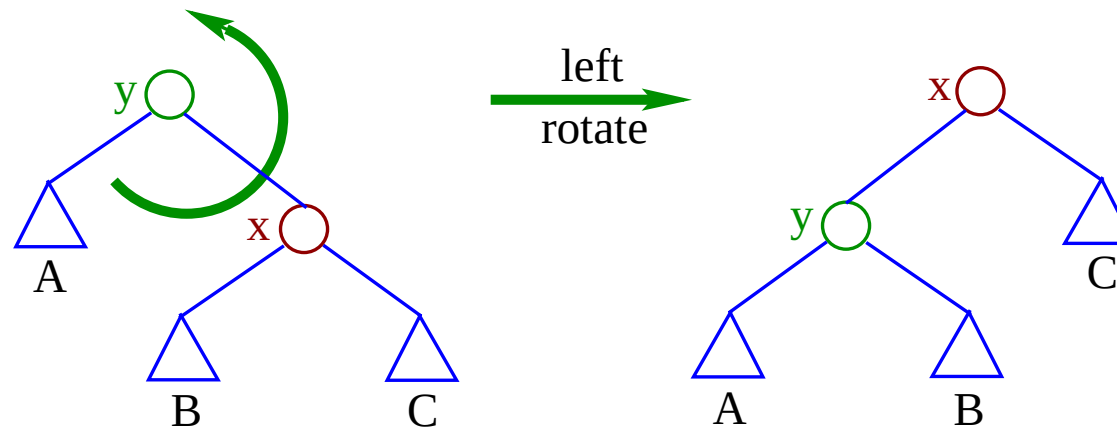
Each triangle represents a subtree.

Note that the subtrees are in the same  $\ell N r$  order before and after a rotation.

Rotation does not affect the ordering of the nodes in the subtrees,  $A$ ,  $B$ , and  $C$ . Assuming the figure on the *left* and pointers to nodes  $x$  and  $y$ , the code to execute a *right* rotation is:

```
1 subtree B = x.right;  
2   x.right = y;  
3   y.left = B;  
4   y.parent.(left or right) = x;  
5   x.parent = y.parent;  
6   y.parent = x;
```

## Splay tree rotations—left rotation

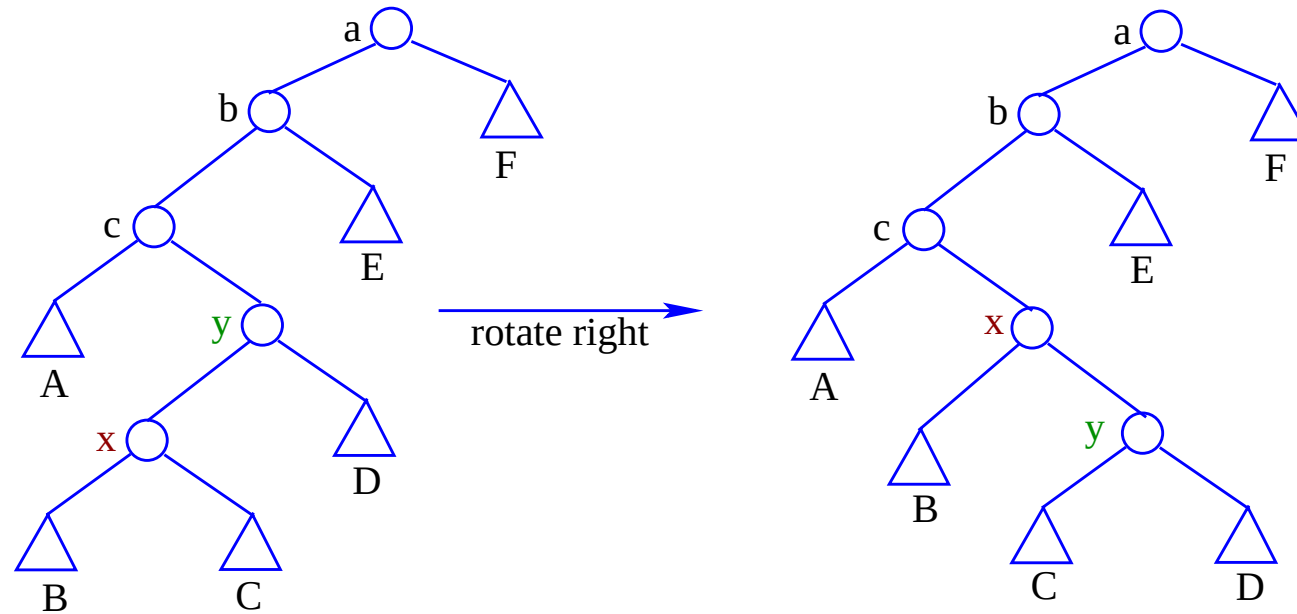


Rotation of the edge joining node  $x$  and node  $y$ . Triangles represent subtrees. Subtrees are in the same  $\ell N r$  order before and after a rotation. Rotation preserves the ordering of the nodes in the subtrees,  $A$ ,  $B$ , and  $C$ . Assuming the figure on the *left* and pointers to nodes  $x$  and  $y$ , the code to execute a *right* rotation is:

```
1 subtree B = x.left;  
2 x.left = y;  
3 y.right = B;  
4 y.parent.(left or right) = x;  
5 x.parent = y.parent;  
6 y.parent = x;
```

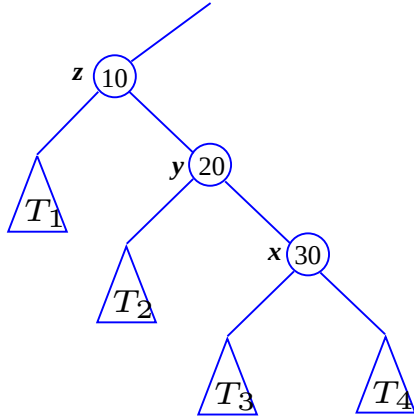
## Splay tree rotations

- A single rotation around  $x - y$  embedded inside the tree.

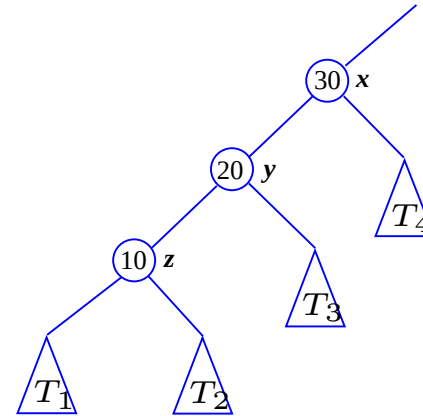


# Splay trees

Before the *zig* operation:



After the *zig* operation:



- Splay trees restructure the tree so that each time a node  $x$  is accessed it is “*moved to the root*” by doing rotations bottom-up along the path of access by one three splaying steps until  $x$  reaches the root.
  1. *zig*, which reduces  $x$ ’s height by one.
  2. *zig-zig*, and
  3. *zig-zag* and their mirrored symmetries.
- *Zig-zig* and *zig-zag* reduce  $x$ ’s height by two, i.e. each splay  $x$  moves 1 or 2 closer to the root.



## The Splaying steps *zig*, *zig-zig* and *zig-zag*

---

- To move  $x$  to the root:

Case 1: *zig*: If  $y = \text{parent}(x)$  is the root, rotate along the edge  $x - y$ .

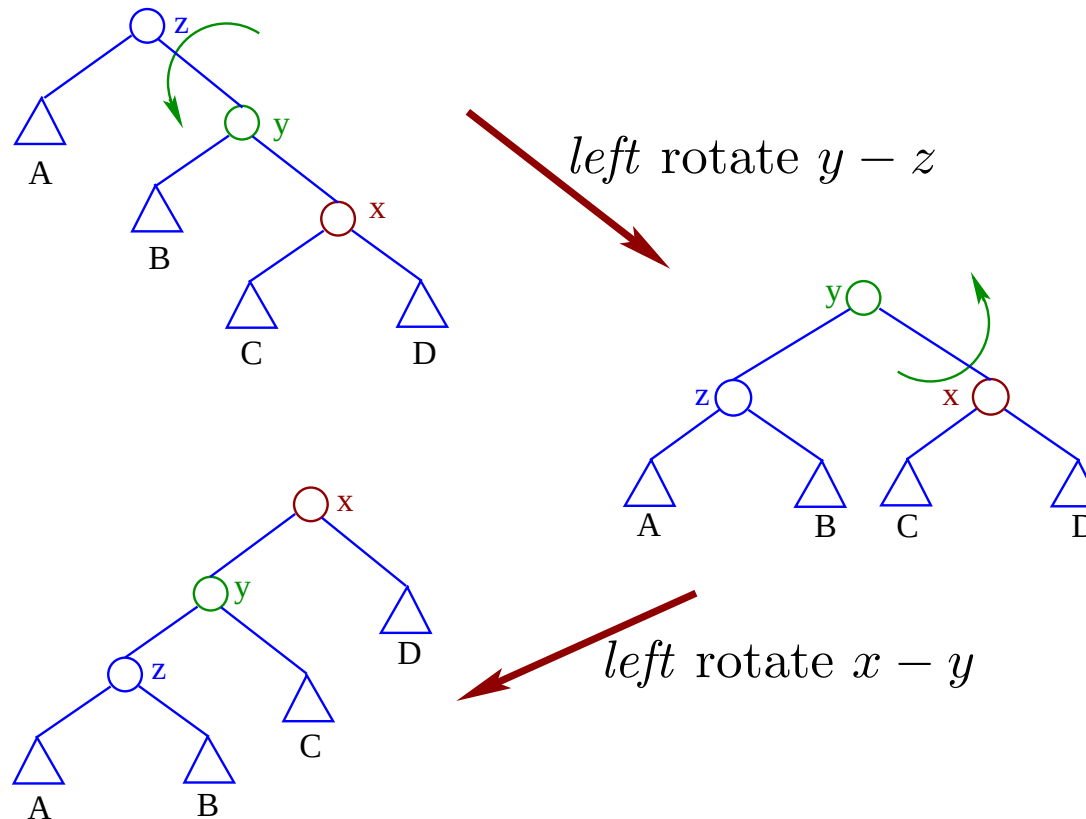
Case 2: *zig-zig*: If  $y = \text{parent}(x)$  is not the root and  $x$  and  $y$  are *both* left children, or *both* right children: let  $z = \text{parent}(y)$ , first rotate the edge  $y - z$  and then rotate the edge  $x - y$ .

Case 3: *zig-zag*: If  $y = \text{parent}(x)$  is not the root and  $x$  is a left child and  $y$  a right child, or vice versa, letting  $z = \text{parent}(y)$ , first rotate the edge  $x - y$  and afterwards rotate  $x - z$ .

- Splaying a node  $x$  at depth  $d$  takes  $\Theta(d)$  time which is proportional to the time that it takes to access  $x$ .
- Note that each splay step preserves the inorder properties of the tree.

## Splay trees—a left-left *zig-zig* step-by-step

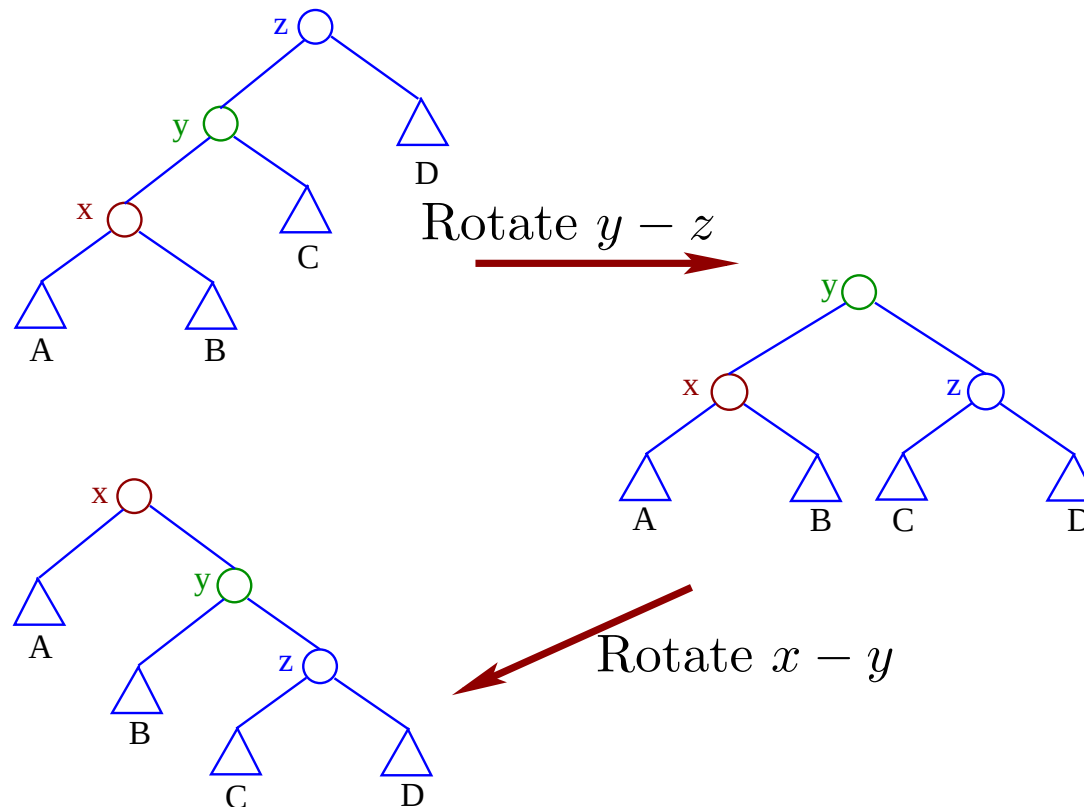
A *zig-zig* is done using two rotations: If  $y = \text{parent}(x)$  is not the root and  $x$  and  $y$  are *both* left<sup>3</sup> children: let  $z = \text{parent}(y)$ , first left rotate the edge  $y - z$  and then *left* rotate the edge  $x - y$ .



<sup>3</sup>Or *both* children are right. See the next slide.

## Splay trees—a right-right *zig-zig* in 2 steps

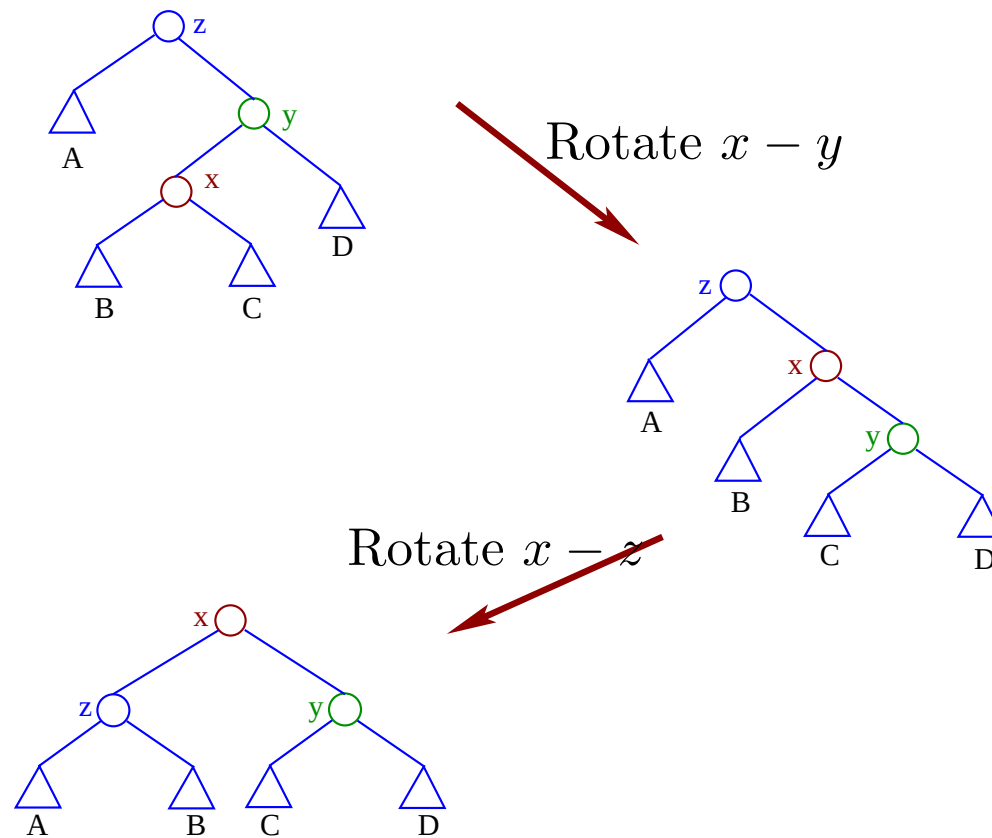
A *zig-zig* is done using two rotations: If  $y = \text{parent}(x)$  is not the root and  $x$  and  $y$  are *both* right<sup>4</sup> children: let  $z = \text{parent}(y)$ , first rotate the edge  $y - z$  and then rotate the edge  $x - y$ .



<sup>4</sup>Or *both* children are left. See the previous slide.

## Splay trees—a left-right *zig-zag* in 2 steps

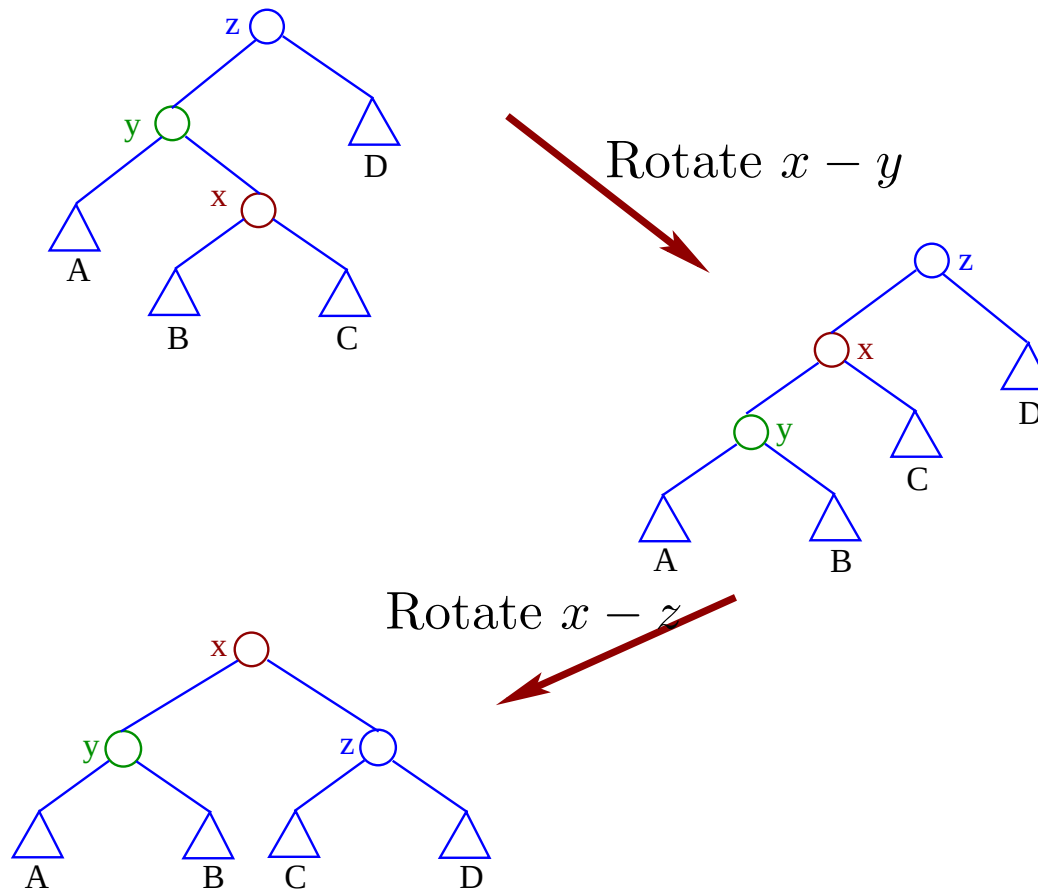
A *zig-zag* employs two rotations: If  $y = \text{parent}(x)$  is not the root and  $x$  is a left child and  $y$  a right child<sup>5</sup> letting  $z = \text{parent}(y)$ , first rotate the edge  $x - y$  and afterwards rotate  $x - z$ .



<sup>5</sup>Or vice versa, see next slide.

## Splay trees—a right-left *zig-zag* in 2 steps

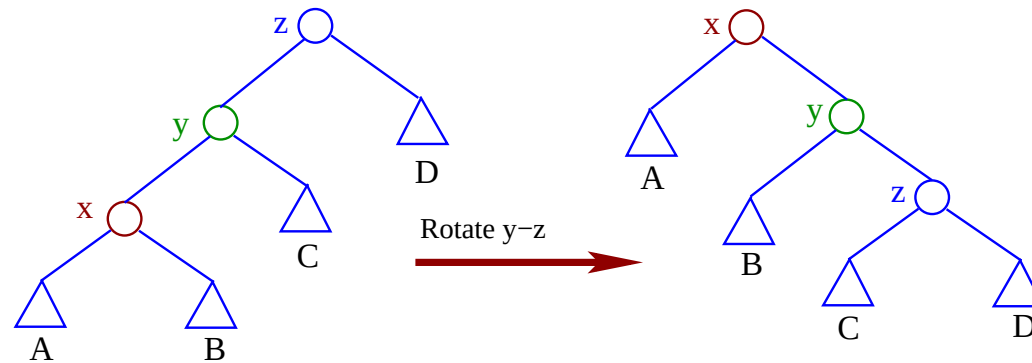
A *zig-zag* is done in two rotations: If  $y = \text{parent}(x)$  is not the root and  $x$  is a right child and  $y$  a left child<sup>6</sup> letting  $z = \text{parent}(y)$ , first rotate the edge  $x - y$  and afterwards rotate  $x - z$ .



<sup>6</sup>Or vice versa, see previous slide.

## Splay trees—a left-left *zig-zig* step-by-step

A direct *zig-zig* is done using two rotations: If  $y = \text{parent}(x)$  is not the root and  $x$  and  $y$  are *both* left<sup>7</sup> children: let  $z = \text{parent}(y)$ , first rotate the edge  $y - z$  around  $y$  and then rotate the edge  $x - y$  around  $x$ .



Assume that we know  $x$ ,  $y$  and  $z$ . Why can we assume this? The code to do a zig-zig:

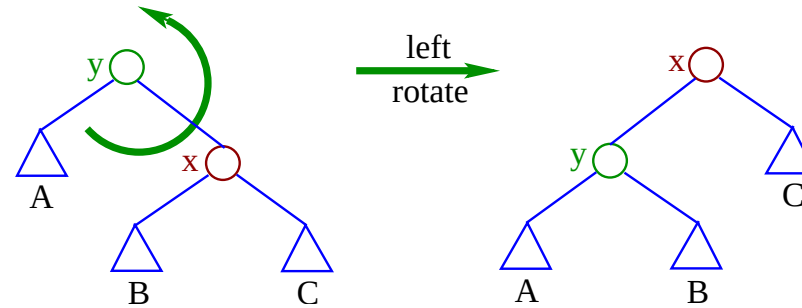
```
1  rightRotate(y, z);  
2  rightRotate(x, y);
```

---

<sup>7</sup>Or *both* children are right. See the next slide.

## Splay trees—code for left rotation

Consider the figure

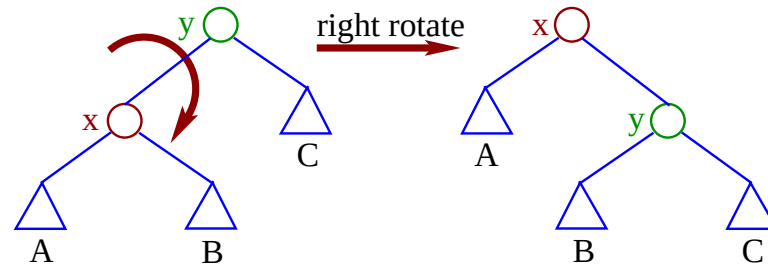


Assume that we know  $x$  and  $y$ . The code for a left rotation:

```
1 rotateLeft(tree x, tree y) {
2   // pre: y == x.parent != null && x != null
3   // && x.key >= y.key
4   tree C = x.left;
5   x.left = y;
6   y.right = C;
7   x.parent = y.parent;
8   y.parent = x;
9   if (x.parent != null)
10      if ((x.parent).key > x.key)
11         (x.parent).left = x;
12      else
13         (x.parent).right = x; }
```

## Splay trees—code for right rotation

Consider the figure



Assume that we know  $x$  and  $y$ . The code for a right rotation:

```
1 rotateRight(tree x, tree y) {
2   // pre: x != null
3   // && y == x.parent != null
4   // && x.key < y.key
5   tree B = x.right
6   x.right = y;
7   y.left = B;
8   x.parent = y.parent;
9   y.parent = x;
10  if ((x.parent).key > x.key)
11    (x.parent).left = x;
12  else
13    (x.parent).right = x; }
```



## Splay trees—code for automatic rotation

Do a *left* rotation when  $x.key \geq (x.parent).key$ , i.e.  $y$  is a *right* child—otherwise do a *right* rotation.

Left rotation:

```
1 rotateLeft(tree x, tree y) {
2 //pre: && x != null
3 // && y == x.parent != null
4 // && x.key >= y.key
5 tree C = x.left;
6   x.left = y;
7   y.right = C;
8   x.parent = y.parent;
9   y.parent = x;
10  if ((x.parent).key > x.key)
11    (x.parent).left = x;
12  else (x.parent).right = x; }
```

Right rotation:

```
1 rotateRight(tree x, tree y) {
2 // pre: x != null
3 // && y == x.parent != null
4 // && x.key < y.key
5 tree B = x.right
6   x.right = y;
7   y.left = B;
8   x.parent = y.parent;
9   y.parent = x;
10  if ((x.parent).key > x.key)
11    (x.parent).left = x;
12  else (x.parent).right = x; }
```

Only one argument is needed: the pivot of the rotation,  $x$ .

$y = x.parent$ ; // assuming that  $x \neq \text{null}$

The temporary nodes **B** or **C** are not needed. Rotation is determined by  $\text{arity}(x)$ , i.e. the *leftness* or *rightness* of  $x$ , given by  $x.key < y.key$ .

## *zig*, *zig-zig* and *zig-zag* with **rotate(x)**

- Splaying a node **x** is a sequence of rotations that moves it to the root.
- Move **x** to the root by repeating:

Case 1: *zig*: If **parent(x)** is the root, **rotate(x)**.

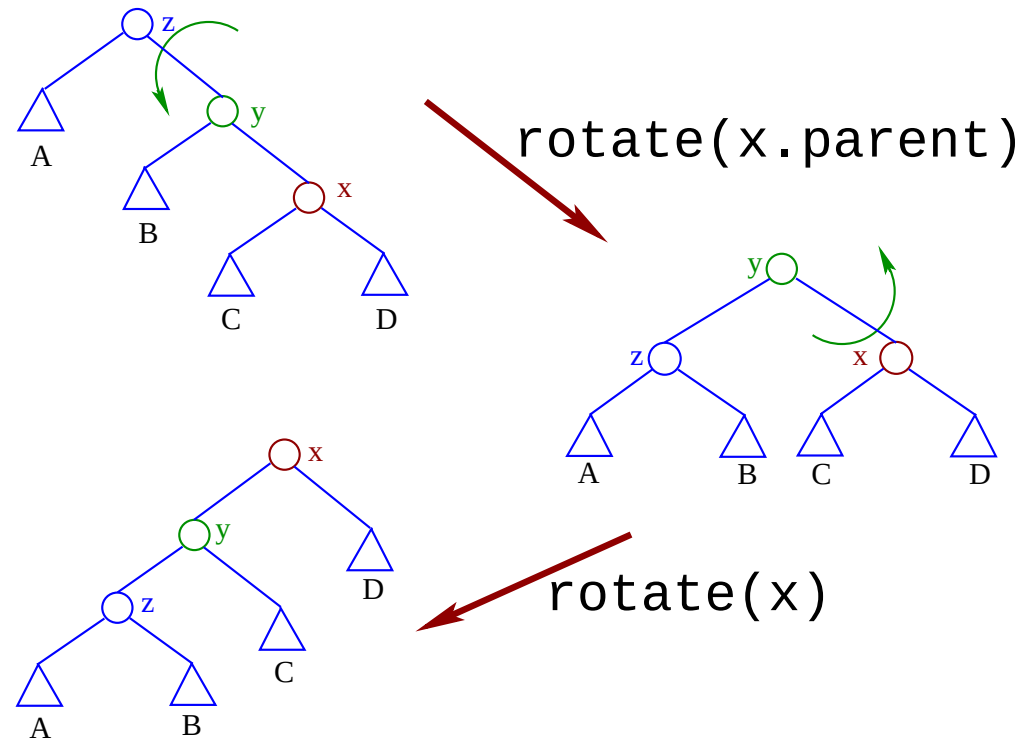
Case 2: *zig-zig*: If **parent(x)** is not the root and **x** and **parent(x)** are both *left* children, or both *right* children: first the **rotate(parent(x))** and then **rotate(x)**.

Case 3: *zig-zag*: If **parent(x)** is not the root and **x** is a left child and **parent(x)** a right child, or vice versa, first **rotate(x)** and afterwards **rotate(x)**.

- Splaying a node **x** at depth  $d$  takes  $\Theta(d)$  time which is proportional to the time that it takes to access **x**.
- Note that each splay step preserves the inorder properties of the tree.

## Splay trees—a left-left *zig-zig* step-by-step

A *zig-zig* is done using two rotations: If  $\text{parent}(x)$  is not the root and  $x$  and  $\text{parent}(x)$  are both *right*<sup>8</sup> children: first  $\text{rotate}(x.\text{parent})$  and then  $\text{rotate}(x)$ .

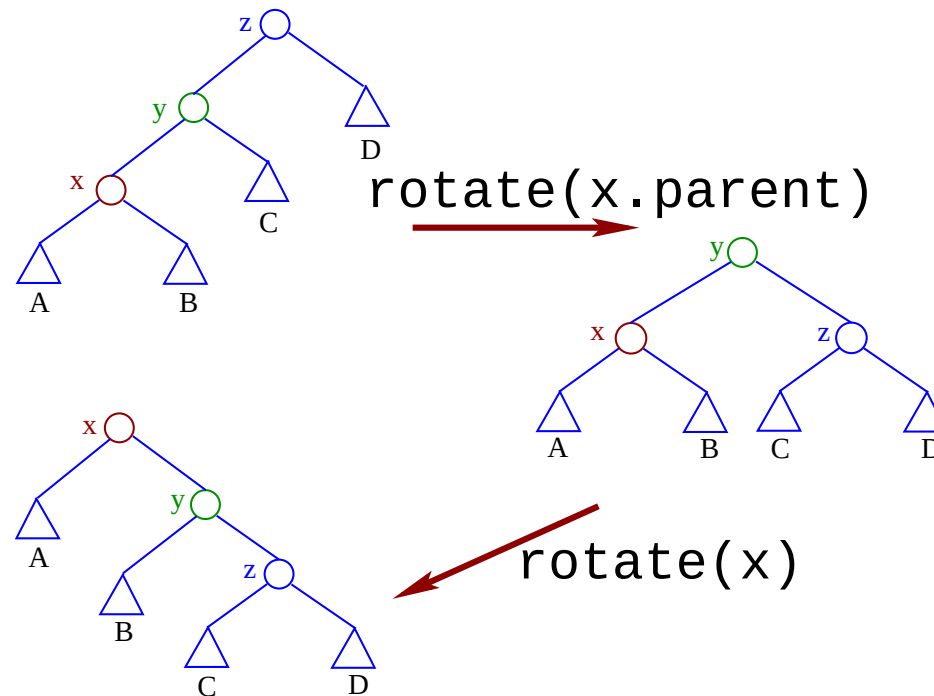


It is called a “left-left” *zig-zig* because both rotations are to the left, i.e. counter-clockwise or anti-clockwise.

<sup>8</sup>Or both are *left* children. See the next slide.

## Splay trees—a right-right *zig-zig* in 2 steps

A *zig-zig* is done using two *clockwise* rotations: If  $\text{parent}(x)$  is not the root and  $x$  and  $\text{parent}(x)$  are both *left*<sup>9</sup> children: first  $\text{rotate}(y)$  and then  $\text{rotate}(x)$  again.

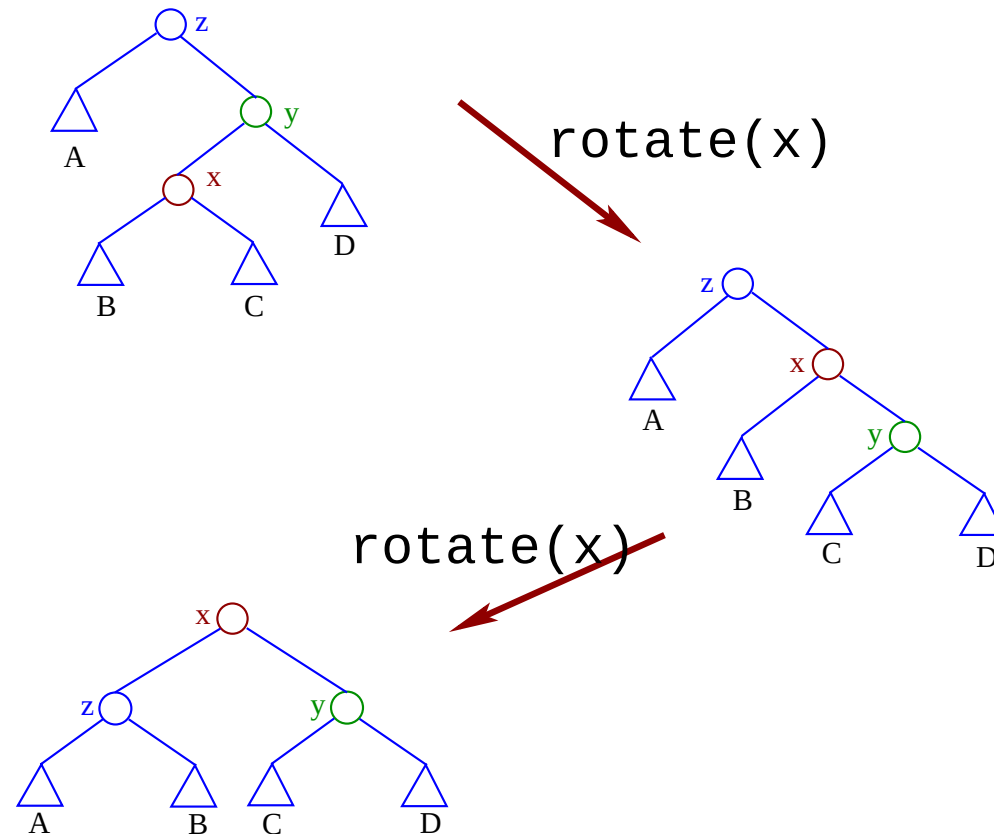


Notice that the code for a “right-right” *zig-zig* is the same as that for a “left-left” one.  $\text{rotate}(x)$  decides the direction.

<sup>9</sup>Or both are *right* children. See the previous slide.

## Splay trees—a left-right *zig-zag* in 2 steps

A *zig-zag* employs two rotations: If  $\text{parent}(x)$  is not the root and  $x$  is a *left* child and  $\text{parent}(x)$  a *right* child<sup>10</sup>: first  $\text{rotate}(x)$  and then  $\text{rotate}(x)$  again.

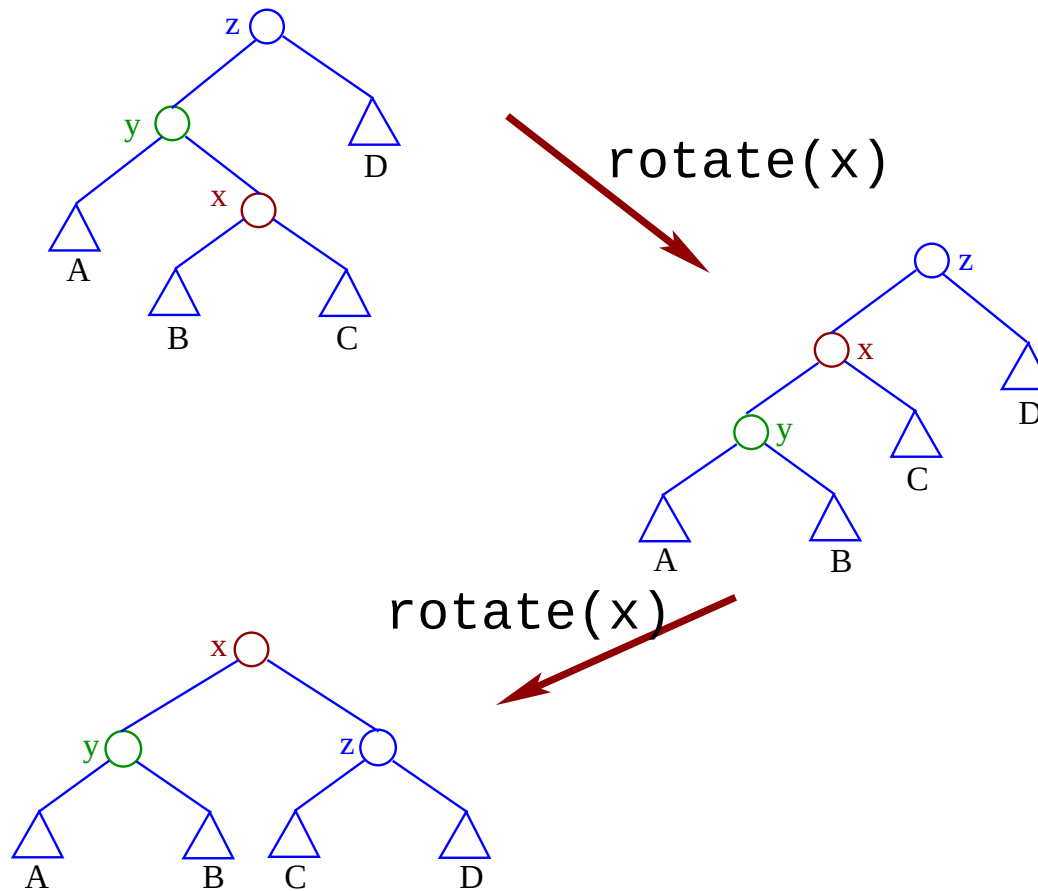


---

<sup>10</sup>Or vice versa, see next slide.

## Splay trees—a right-left *zig-zag* in 2 steps

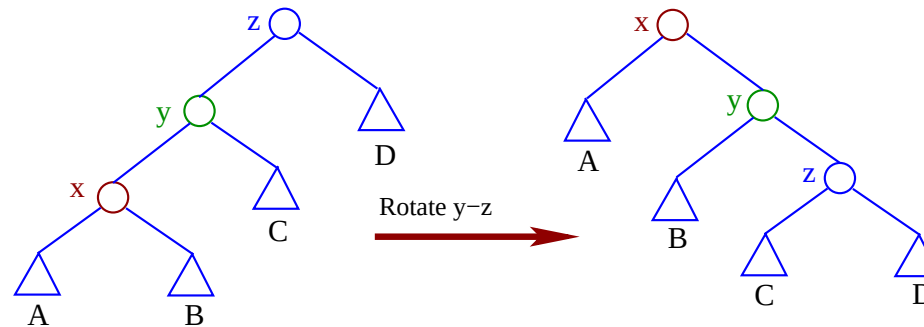
A *zig-zag* is done in two rotations: If  $\text{parent}(x)$  is not the root and  $x$  is a *right* child and  $y$  a *left* child<sup>11</sup> first  $\text{rotate}(x)$  and afterwards  $\text{rotate}(x)$  again.



<sup>11</sup>Or vice versa, see previous slide.

## Splay trees—a left-left *zig-zig* step-by-step

A direct *zig-zig* is done using two rotations: If  $y = \text{parent}(x)$  is not the root and  $x$  and  $y$  are *both* left<sup>12</sup> children: let  $z = \text{parent}(y)$ , first rotate the edge  $y - z$  and then rotate the edge  $x - y$ .



Assume that we know  $x$ ,  $y$  and  $z$ . Why can we assume this? The code to do a zig-zig:

```
1 leftRotate(y, z);  
2 leftRotate(x, y);
```

This becomes

```
1 rotate(x.parent);  
2 rotate(x);
```

---

<sup>12</sup>Or *both* children are right. See the next slide.

## Splay trees—rotation around the pivot

```
1 rotate(tree x) {
2 // pre: x != null
3 tree y = x.parent;
4 if (y != null) // x is not the root
5     if (x.key < y.key) { // right rotation
6         y.left = x.right;
7         x.right = y;
8     }
9     else { // x is right child:
10        left rotation
11        y.right = x.left;
12        x.left = y;
13    }
14    if (y.parent != null)
15        if (y.key < (y.parent).key)
16            (y.parent).left = x;
17        else
18            (y.parent).right = x;
19    x.parent = y.parent;
20    y.parent = x; }
```



## Splay trees

---

- In our pseudo-code we have used the expression `x.parent` instead of `getParent(x)` to clarify our coding.
- Next we tackle the code for `splay(x)` given only the tree node `x`.

## Splay trees—splay(x)—1st version

```
1 void splay(tree x) {
2     while (x.parent != null) { // x is root
3         if ((x.parent).parent == null)
4             rotate(x); // x.parent is root: zig
5         // x.parent has a non-root parent
6         else if (x.key < (x.parent).key) // x is left child
7             if ((x.parent).key < ((x.parent).parent).key)
8                 rotate(x.parent); // left-left: zig-zig
9             else
10                rotate(x); // left-right: zig-zag
11        // x is a right child
12        else if ((x.parent).key < ((x.parent).parent).key)
13            rotate(x); // right-left: zig-zag
14        else { // right-right: zig-zig
15            rotate(x.parent); }
16        rotate(x); } }
```

Take care this has some errors. Can you spot them?

## Splay trees—splay(x)—2nd version

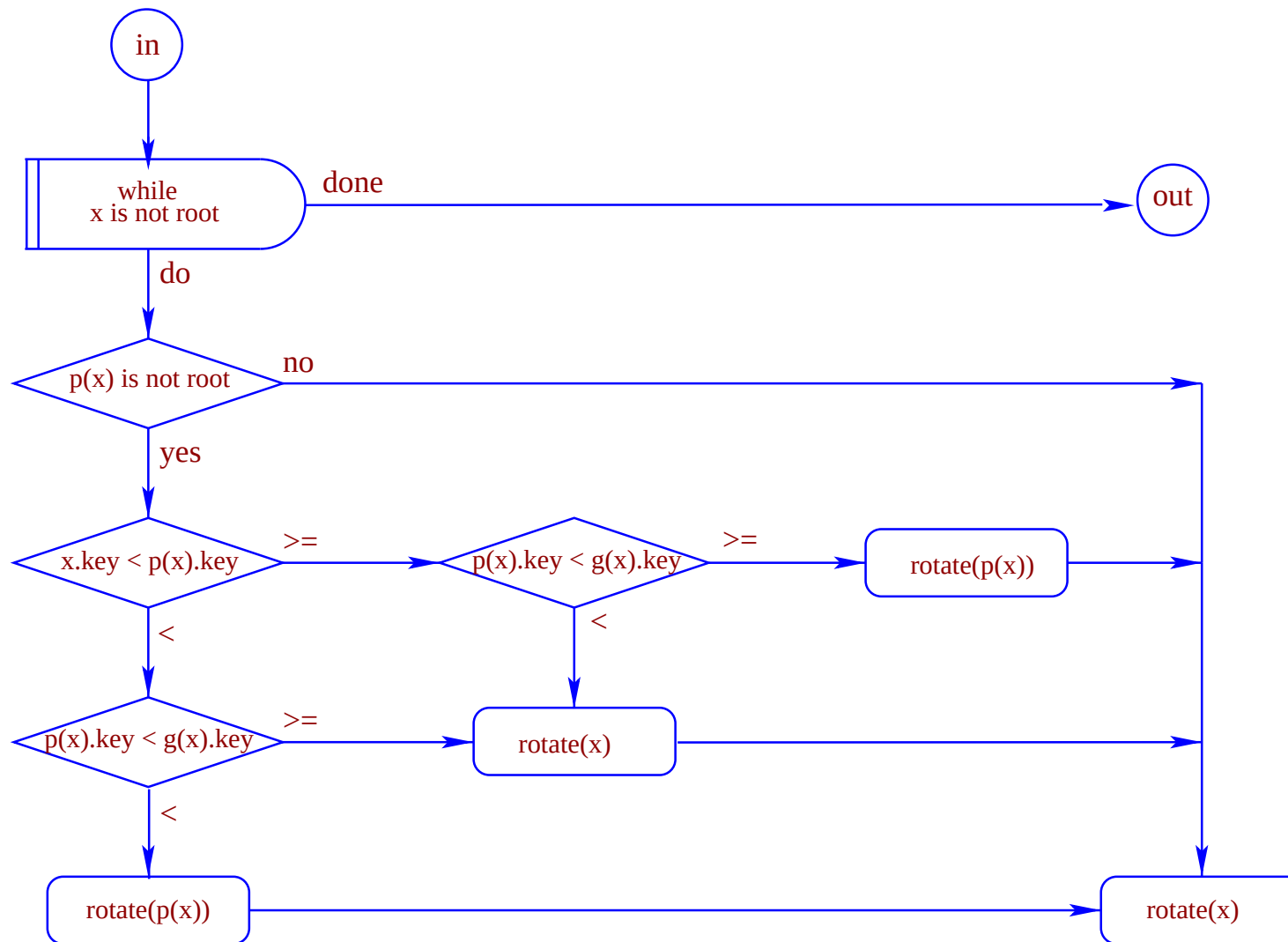
```
1 void splay(tree x) {
2     // repeat while x is not the root node
3     while (x.parent != null) { // y -- x
4         if ((x.parent).parent != null) { // z -- y -- x
5             // x.parent has a non-root parent
6             if (x.key < (x.parent).key)
7                 // x is a left child
8                 if ((x.parent).key < ((x.parent).parent).key)
9                     // x.parent is a left child: left-left zig-zig
10                    rotate(x.parent);
11                else // left-right: zig-zag
12                    rotate(x);
13            else // x is a right child
14                if ((x.parent).key < ((x.parent).parent).key)
15                    // x.parent is a left child: right-left zig-zig
16                    rotate(x.parent);
17                else // right-right: zig-gig
18                    rotate(x);
19            rotate(x); } } }
```

Still error ridden. Can you spot the errors?

## Splay trees—splay(x)—tidier

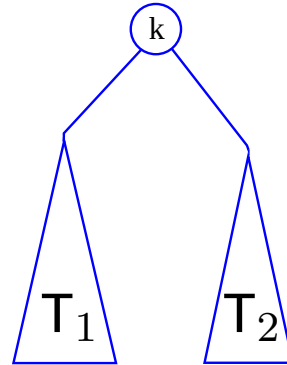
```
1 void splay(tree x) {
2     // repeat while x is not the root node
3     while (x.parent != null) { // y -- x
4         if ((x.parent).parent != null) // z -- y -- x
5             if (x.key < (x.parent).key)
6                 if ((x.parent).key < ((x.parent).parent).key)
7                     rotate(x.parent);
8                 else // left-right: zig-zag
9                     rotate(x);
10            else // right
11                if ((x.parent).key < ((x.parent).parent).key)
12                    rotate(x.parent);
13                else // right-left: zig-gig
14                    rotate(x);
15            rotate(x); } }
```

## Splay trees—**splay(x)**—flow chart



## delete(k) from a *splay tree*

- First `find(k)` and splay it.



- Next replace the root,  $k$ , with the *largest* key in the *left subtree*.  $T_2$  or replace the root with the *smallest* key in the *right subtree*.
- Note that all the keys in  $T_1$  are less than  $k$  and all the keys in  $T_2$  are greater than  $k$ .
- Notice that the *greatest* key in  $T_1$  is a *right* child but has no *right* child: so deleting or removing the *greatest* key from  $T_1$  is simple: move its *left* key to its parent's *right* key.

## Splay trees—complexity

---

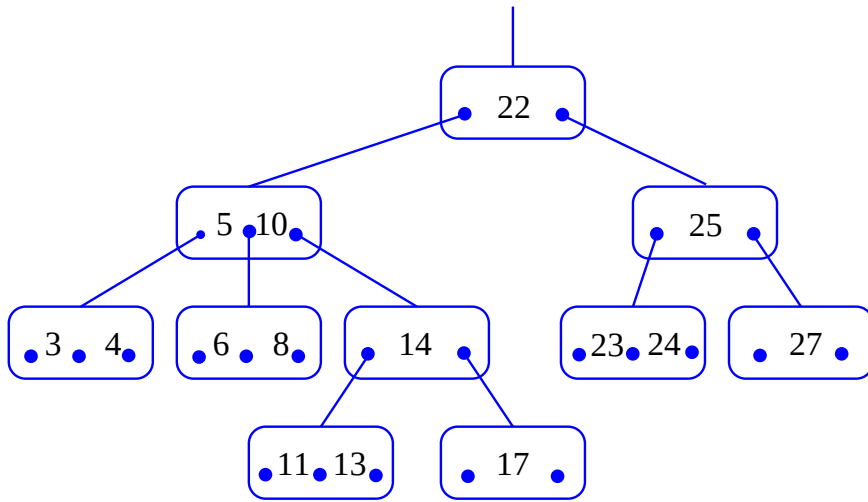
- **Theorem:** Any sequence of  $m$  operations on an initially empty splay tree that never contains more than  $n$  nodes takes  $O(m \log n)$
- Running time of each operation is proportional to time for splaying.
- Define  $rank(v)$  as the  $\log_2 n$ , where  $n$  is the number of nodes in subtree rooted at  $v$ .
- Actual Costs:  $zig = \$1$ ,  $zig-zig = \$2$ ,  $zig-zag = \$2$ .
- Thus, cost for splaying a node at depth  $d = \$d$ .
- Assume that  $rank(v)$  dollars is stored at each node  $v$  of the splay tree
- *Invariant.* For each node  $v \geq \$rank(v)$ .
- **Lemma 1:** Each splay operation requires us to invest at most  $3 \log_2 n + 1$  new dollars, to maintain the invariant.

- See Goodrich and Tamassia for more slides in the SunLab at `../../notes/ds/SplayTrees.pdf` when you are in your home directory.  
More up-to-date versions are on the web site that Goodrich and Tamassia maintain for the book.  
These notes are superficial. Consult the book, pages 440–550 (4th edition).



## Multiway trees

Before discussing  $(2 - 4)$  trees we first consider multiway trees. The figure below is a multiway tree.



An  $n$  entry multiway tree has  $n + 1$  leaves—external nodes. It is proved by induction.

$P[1]$  : A multiway tree with one node has 2 leaves.

$P[k] \rightarrow P[k + 1]$  : Assume  $P[k]$ , i.e.  $T_k$  is a multiway tree with  $k$  entries and has  $k + 1$  leaves. Inserting a node into  $T_k$  at any leaf will replace one leaf and add two, i.e. the new tree  $T_{k+1}$  will have  $k + 2$  leaves. So,  $\forall n P[n]$ .

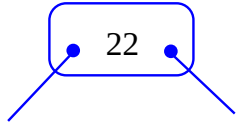
## Red-black trees

---

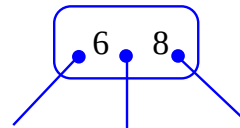
- A  $(2 - 4)$  tree has an update—search, insert and remove—time of  $O(\log n)$ .
- A *red-black tree* has an update time of  $O(\log n)$ , i.e. it needs a constant number of changes to do an update.
- A *red-black tree* is a BST with nodes coloured *red* or *black* using the following rules:
  1. The root is black.
  2. Every external node is black.
  3. The children of a red node are black.
  4. External nodes have the same *black depth*. i.e. the number of black ancestors minus one.
- The following slides shows how to convert nodes from a  $2 - 4$  tree into a *red-black tree*

## Converting a 2 – 4 tree into a red-black tree

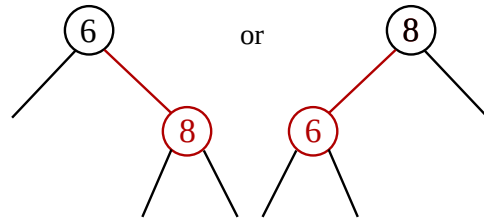
The figures below illustrate the conversion of nodes from a 2 – 4 tree into those for a red-black tree.



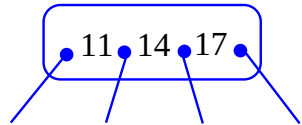
$\Rightarrow$



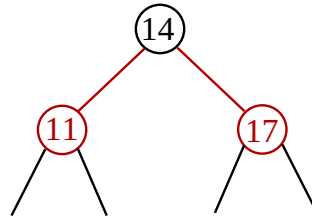
$\Rightarrow$



or

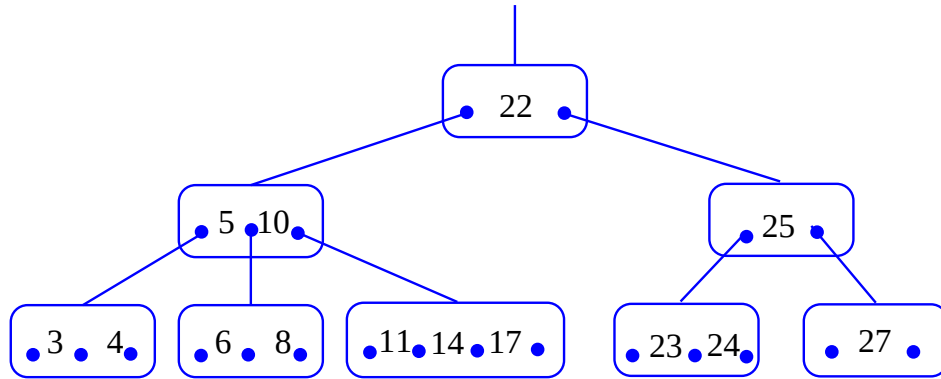


$\Rightarrow$

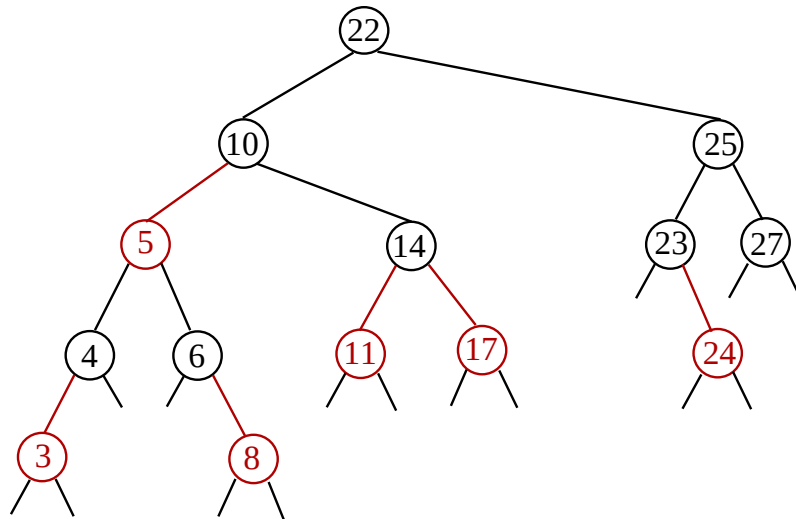


## Converting a 2 – 4 tree into a red-black tree

We convert the following into a *red-black* tree:

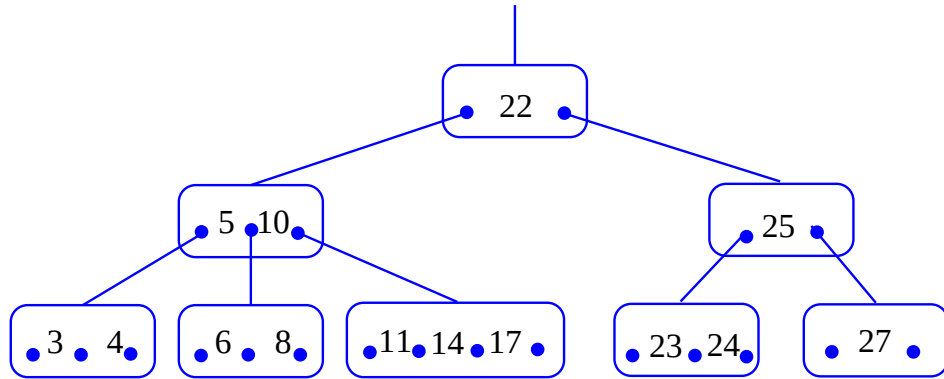


The *red-black* tree follows.

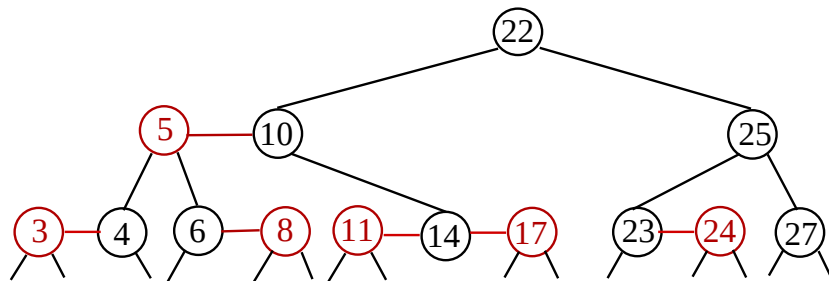


## Converting a 2 – 4 tree into a *vertical-horizontal-red-black* tree

We convert the following into a *red-black* tree:

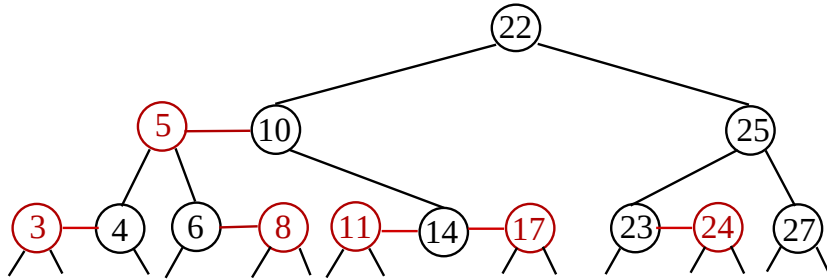


The vertical-horizontal *red-black* tree looks more like the original (2 – 4) tree.

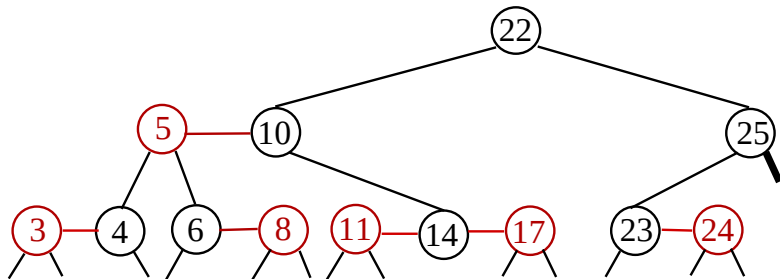


## Deletion from a *red-black* tree

Consider removing 27 from the *red-black* tree below.



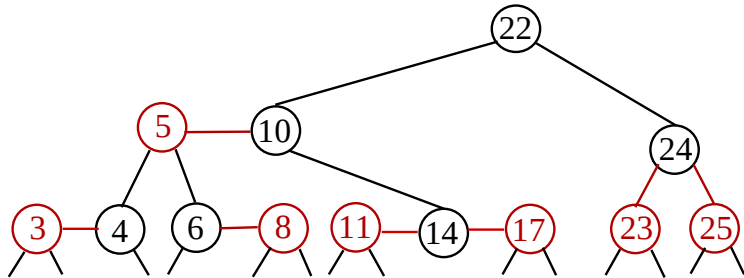
The resultant tree below has lost its black-depth property. Since 25's right child—a *double black*—now has a black depth of 2, it is no longer a *red-black* tree.



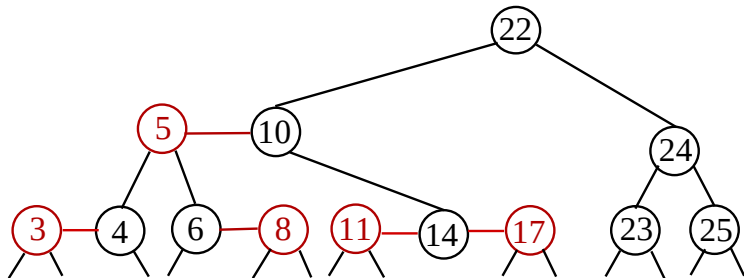
It is easy to repair the *double-black* error by restructuring 23 – 25 – 24 as three separate black 2-nodes with 24 on top and the black-depth property is restored.

## Repairing the *double black* at 25

It is easy to repair the *double-black* error by restructuring 23 – 25 – 24 first into the 4-node 23 – 24 – 25

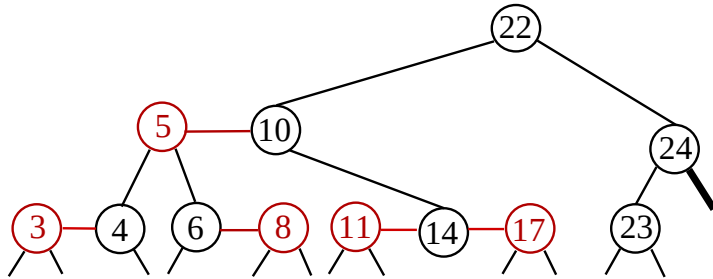


The leaves of 23 and 24 now have the incorrect *black depth* of 2. Subsequently, this 4-node 23 – 24 – 25 is recoloured into three separate black 2-nodes with 24 on top. All the leaves now have a *black depth* of 3.

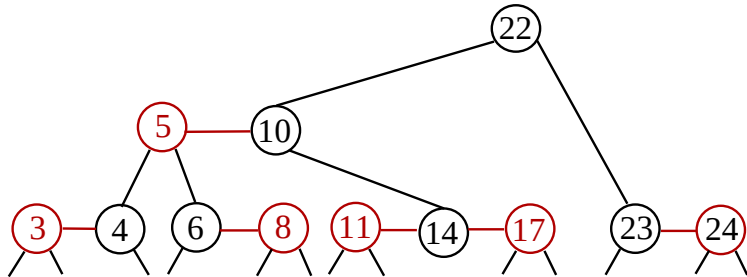


## Non-trivial deletions from this *red-black* tree

Deletion of either 23 or 25 from this *red-black* tree cause a double black to hang from node 24. Let us remove 23. The double black hangs from node 24.



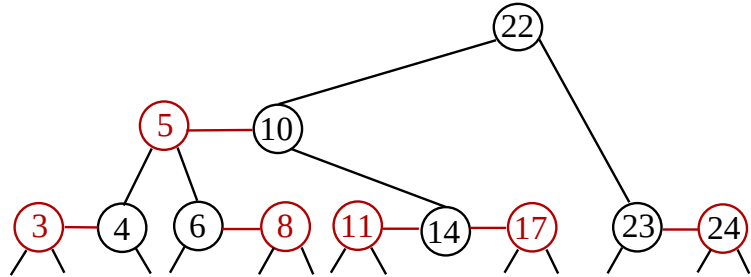
The red-black tree is repaired fixed by first amalgamating 24 and 24 into a 3-node and then the next step becomes obvious.



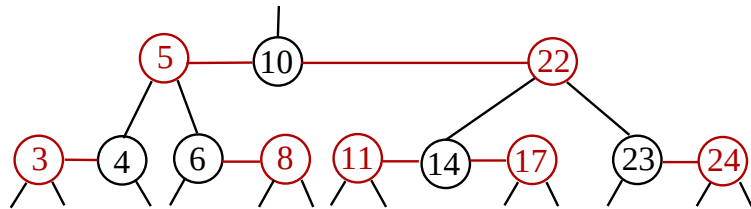


## Easy deletions from this *red-black* tree

Amalgamating 24 and 24 into a 3-node makes the next step seem obvious.



Move the root 22 down to form the 4-node 5 – 10 – 22.



Since they are all red, removing 3, 8, 11, 17 or 24, has no effect on the depths of the leaves of the tree.

- A tree where all the nodes have *at least*  $a$  children and *at most*  $b$  children is a B-tree when  
 $2 \leq a \leq (b + 1)/2$ .  
All the leaves have the same depth.
- The root may have 0 children when the tree is empty.
- Otherwise the root must have at least 2 children.
- If there are less than  $a$  entries, they all lie in the root node, such that the root may have less than  $a$  children.

## The height of an $(a, b)$ tree $T$ with $n$ entries is $\Omega(\log_b n)$ and $O(\log_a n)$

- The height of an  $(a, b)$  tree is bounded from above by  $O(\log_a n)$  and from below by  $\Omega(\log_b n)$ .
- The number of external nodes  $n'$  of  $T$  is at least  $2a^{h-1}$ : If the root has only one entry and the height,  $h = 1$ , then there are 2 leaves. If  $h = 2$  and the root has at least  $a$  children and each child on the second level has a single entry, then there are at least  $2a$  children on the second level. The third level will have at least  $2a^2$  leaves.
- The number of external nodes is *at most*  $b^h$  where  $h$  is the height of the tree.
- An  $n$ -entry mutiway tree has  $n + 1$  leaves, so

$$\begin{aligned} 2a^{h-1} &\leq n + 1 \leq b^h \\ \log_2(2a^{h-1}) &\leq \log_2(n + 1) \leq \log_2(b^h) \\ 1 + (h - 1) \log_2 a &\leq \log_2(n + 1) \leq h \log_2 b \end{aligned}$$

## Upper and lower bounds for the height of an $(a, b)$ tree

---

Since

$$1 + (h - 1) \log_2 a \leq \log_2(n + 1) \leq h \log_2 b$$

it follows that

$$\begin{aligned} (h - 1) &\leq \frac{\log_2(n + 1) - 1}{\log_2 a} \\ h &\leq \frac{\log_2(n + 1) - 1}{\log_2 a} + 1 \\ &\doteq \log_a(n + 1) \\ h &\text{ is } O(\log_a n) \end{aligned}$$

and

$$\begin{aligned} h &\geq \frac{\log_2(n + 1)}{\log_2 b} \\ &= \log_b(n + 1) \\ h &\text{ is } \Omega(\log_b n) \end{aligned}$$

## Searching in an $(a, b)$ tree

---

- Each node of an  $(a, b)$  tree has at least  $a - 1$  entries and at most  $b - 1$  entries.
- Looking for an entry in a *node* using *any* search method will take  $f(b)$  time.
- Searching for an entry in an  $(a, b)$  tree  $T$  will take at worst  $O(f(b) \log_a n)$  time.
- Since  $b$  is constant,  $a$  is also, and the search time is  $O(\log n)$ , which is independent of the specific search method.

## A $B$ -tree of order $d$

---

- A  $B$ -tree of order  $d$  is a  $(\lceil d/2 \rceil, d)$  tree.
- $d$  is chosen such that  $d - 1$  keys and  $d$  references can be stored in a single block of size  $B$ .
- So each time a node is accessed a single disk transfer is done.
- The most transfers that ever need to be done is  $O(\log_a n) = O(\log_{\lceil d/2 \rceil} n)$  and it will occupy  $O(n/B)$  blocks.

## Overflow and underflow in a *B*-tree

---

- During **insertion** *overflow* might occur.
- When the number of entries in a node in which an insertion must be done is already  $d$ , an *overflow* occurs. The node is then split into two halves with  $\lfloor (d+1)/2 \rfloor$  and  $\lceil (d+1)/2 \rceil$  children respectively.
- During **deletion** *underflow* could occur.
- When the number of entries in a node from which a deletion must be done is only  $\lceil d/2 \rceil$ , an underflow occurs and some housekeeping must be done in which the entries of the underfull node is moved to one or more of its siblings.

## Sorting—an $O(n^3)$ method

- Given a list of  $n$  objects  $A[1], A[2], \dots, A[n]$  that can be totally ordered, the problem is to arrange them such that  $A[1] \leq A[2] \leq A[3] \leq \dots \leq A[n]$ .
- Suppose that the first  $i - 1$  elements are in order, i.e.  $A[1] \leq A[2] \leq A[3] \leq \dots \leq A[i - 1]$  insert the  $i$ -th one,  $A[i]$  in its correct position relative to the elements we have already considered. Carry on until  $i = n$ .
- How do we start? Start with  $i = 1$ : the list,  $A[1]$ , of 1 element is already sorted.
- Proceed to  $i = 2$  by inserting  $A[2]$  into its correct position before or after  $A[1]$ . This may entail moving  $A[1]$  down one position to  $A'[2]$  to vacate  $A'[1]$  for  $A[2]$  and, finally, rename the elements giving the sorted list:  $A'[1] \leq A'[2]$ .
- The following step is to insert  $A[3]$  into the sorted list  $A'[1] \leq A'[2]$ .



## Sorting—an $O(n^3)$ method

---

- Insert  $A[3]$  into the sorted list  $A'[1] \leq A'[2]$ .
- First make a copy:  $now \leftarrow A[3]$ , and then compare  $now$  one-by-one with  $A'[1]$  and  $A'[2]$ .
- If  $now \leq A'[j]$ , where  $j \in [1..i-1]$ , where  $i = 3$ , then  $A[i] \equiv now$  has found its place in the list. Starting at  $A'[j]$ , move all the elements in the list down one position and then copy:  $A'[j] \leftarrow now$ .
- Since  $now$  is a true copy of the element that was at  $A[3]$  it is not destroyed when  $A'[2]$  overwrites  $A[3]$ . Throw away the primes ('s).
- At this stage:  $A[1] \leq A[2] \leq A[3]$ .
- Instead of proceeding with  $A[4]$  we rather see how to handle  $A[i]$ .
- In a proof by mathematical induction show that a basis proposition  $\mathcal{P}[0]$  holds, assume that  $\mathcal{P}[k]$  is true and then if  $\mathcal{P}[k+1]$  follows, the proposition  $\mathcal{P}[i]$  holds for all integers  $i$ .

## Sorting—an $O(n^3)$ method

- Assume  $\mathcal{P}[i - 1]$ :  $A[1] \leq A[2] \leq A[3] \leq \dots \leq A[i - 1]$  insert the  $i$ -th element in its correct position in the list.
- Copy:  $now \leftarrow A[i]$ , and then compare  $now$  one-by-one with  $A[j]$  where  $j \in [1..i - 1]$ . Find the first position  $j$  where  $now \leq A[j]$ .
- Move all the elements  $A[j], A[j + 1], \dots, A[i - 1]$  down one position. Copy *from the back to the front* and then place  $now$  at the vacated  $A[j]$ .
- Why must we copy *back-to-front*?
- Now  $A[1] \leq A[2] \leq A[3] \leq \dots \leq A[i]$ .
- This completes the induction: we assumed  $\mathcal{P}[i - 1]$  and showed how to do  $\mathcal{P}[i]$  using the assumption. What was  $\mathcal{P}[0]$ ?
- Carry on until  $i = n$ . The  $n$  element list will then be sorted. We now code the algorithm and analyse it.

## Sorting—an $O(n^3)$ method—the code

- Set up a loop for the variable  $i$ . Start at 2. Why?

```
1  for (i = 2; i <= n; i++) {
```

- Create  $now$  and find  $j \in [1..i - 1]$  for which  $now \leq A[j]$ .

```
1      now = A[i];  
2      for (j = 1; j <= i-1; j++)  
3          if (now <= A[j])  
4              insert(A, i, j); }
```

- The `void` procedure `insert(A, i, j)` inserts `A[i]` into the array `A[1..i]` at `j`, by moving each element down one place and inserting `now` at `A[j]`:

```
1 void insert (int A[], int i, int j) {  
2     int now = A[i], k;  
3     for (k = i; k >= j; k--)  
4         A[k] = A[k-1];  
5     A[j] = now; }
```

## Sorting—an $O(n^3)$ method—the analysis

---

- The length of the **for** (**k** ... loop inside **insert** varies from 1 to  $n - 1$ . So it takes  $O(n)$  time.
- The length of the **for** (**j** ... loop inside the **i** loop executes once, then twice, etc, up to almost  $n$  times. So it also done  $O(n)$  times. The  $O(1)$  stuff inside the **insert** procedure is done at most  $n$  times so it takes  $O(n)$ , thus the **for** (**j** ... loop will consume  $O(n^2)$  time.
- Finally, the outermost loop is itself repeated about  $n$  times, so the complete sort algorithm requires  $O(n^3)$  time to execute.
- This algorithm is a lesson in how *not* to sort.
- There are worse ways to sort. Can you think of sorting method that has exponential time complexity?
- We will next show another bad algorithm that takes  $O(n^2)$  time.

## Sorting—an $O(n^2)$ method

---

- Now we grow our sorted list by placing new elements one-by-one into their *final* position after one sweep of the remainder of the array.
- Given a list of  $n$  objects  $A[1], A[2], \dots, A[n]$  that can be totally ordered, sorting is the problem to arrange them such that  $A[1] \leq A[2] \leq A[3] \leq \dots \leq A[n]$ .
- Suppose that the first  $i - 1$  elements are in their *correct* positions, i.e.  $A[1] \leq A[2] \leq A[3] \leq \dots \leq A[i - 1]$ . Find the  $i$ -th element in the rest of the array so that,  $A[i]$  will be in its final position when it is placed. Carry on until  $i = n$ .
- This is easy to do: Simply find the smallest element in the list  $A[i..n]$  and when it is found swap it with whatever is at  $A[i]$ .
- After the swap  $A[1..i]$  is completely sorted.
- Note this is again a disguised proof by mathematical induction that this sorting method is actually correct.

## Sorting in $O(n^2)$ —the code

- The  $i \in [1..n)$  in the outer loop points to the final position of the smallest element. It sets **smallest** to the  $A[i]$ , and preserves the index. The  $j$ -loop then iterates through all of the elements in  $A[i+1..n]$  to get the smallest element.

```
1  for (i = 1; i <= n-1; i++) {
2      smallest = A[i];
3      indexSmallest = i;
4      for (j = i+1; j <= n; j++) {
5          if (smallest > A[j]){
6              smallest = A[j];
7              indexSmallest = j;} }
8      A[i] = A[indexSmallest];
9      A[indexSmallest] = smallest; }
```

- This time there are two nested loops of similar complexity to those of the previous sorting method. All the other lines are  $O(1)$ , so the algorithm has a complexity of  $O(n^2)$ .
- Although it improves the previous algorithm by an ‘order’ it is not good enough to earn our respect.

## Merge sort—an $O(n \log n)$ algorithm

---

- Given a list of  $to - from + 1$  objects:  
 $A[from], A[from + 1], A[from + 2], \dots, A[to]$  that can be totally ordered, arrange them such that  
 $A[from] \leq A[from + 1] \leq \dots \leq A[to]$ .
- Divide the given list into two almost equal parts:  $A[from..middle - 1]$  and  $A[middle..to]$  and sort each part separately.
- After these sorting each of these two lists, *merge* them into one list  $A[from..to]$ .
- Suppose that sorting  $n$  elements takes  $T(n)$ , then  $T(n) = 2T(n/2) + cn$  where  $cn$  is the cost of merging two lists.
- A one element list is already sorted, so  $T(1) = 0$ . When there are only two elements  $T(2) = 2c$ .
- Solving this, we will find that  $T(n) = O(n \log n)$ .

## Merge sort is an $O(n \log n)$ algorithm

- Suppose that sorting  $n$  elements takes  $T(n)$ , then  $T(n) = 2T(n/2) + cn$  where  $cn$  is the cost of merging two lists.
- Now find a closed form for  $T(n)$  by *telescoping*.

$$\begin{aligned}T(n) &= 2T(n/2) + cn \\&= 2(2T(n/2^2) + cn/2) + cn = 2^2T(n/2^2) + 2cn \\&= 2^3(2T(n/2^3) + cn/2^2) + 2cn = 2^3T(n/2^3) + 3cn \\&= 2^4(2T(n/2^4) + cn/2^3) + 3cn = 2^4T(n/2^4) + 4cn \\&= \dots \\&= 2^iT(n/2^i) + icn\end{aligned}$$

$$\begin{aligned}T(n) &= 2^iT(1) + icn, && \text{when } n = 2^i, \text{ so } i = \log_2 n. \\&= icn, && \text{since } T(1) = 0. \\&= cn \log n\end{aligned}$$

When  $2^i = n$ , i.e.  $i = \log_2 n$ , then  $T(n) = icn$  and  $T(n) = cn \log_2 n$ , i.e.  $T(n)$  is  $O(n \log n)$ .



## mergesort—the $O(n \log n)$ algorithm

- Given a list  $A[from..to]$  Divide it into two almost equal parts at the midpoint  $p$ :  $A[from..p]$  and  $A[p + 1..to]$  and sort each part separately.
- After these sorting each of these two lists, *merge* them into one list  $A[from..to]$ .

```
1 public void mergeSort(int A[], int from, int to) {  
2     int p;  
3     if (from + 1 >= to) {  
4         if (A[from] <= A[to])  
5             return;  
6         swap(A, from, to); }  
7     else {  
8         p = (from+to)/2;  
9         mergeSort (A, from, p);  
10        mergeSort (A, p+1, to);  
11        merge (A, from, p, to); } }
```

- We will now discuss the merging process.

## The merging process—start merging

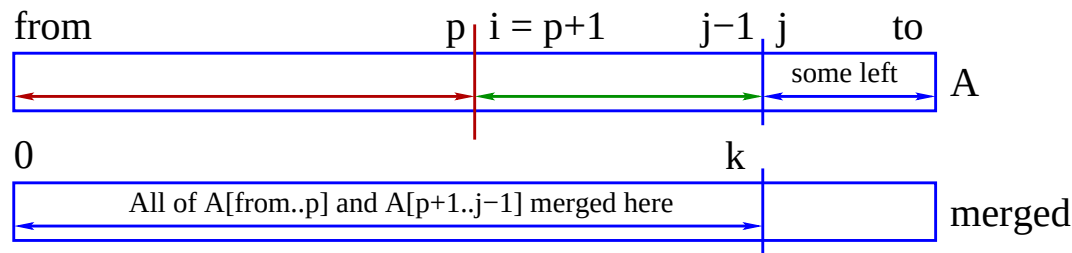
- Put  $i \leftarrow from$ ,  $j \leftarrow to$ , and put  $k \leftarrow 0$ . Compare  $A[i]$  with  $A[j]$  and store the smallest into  $merged[k]$ . Update  $i$  or  $j$ , and  $k$  appropriately.

```
1 public void merge(int A[], int from, int p, int to) {  
2   int merged[] = new int [to - from + 1];  
3   int i = from, j = p+1, k = 0;  
4   while (i <= p && j <= to)  
5     if (A[i] < A[j])  
6       merged[k++] = A[i++];  
7   else  
8     merged[k++] = A[j++]; ...}
```

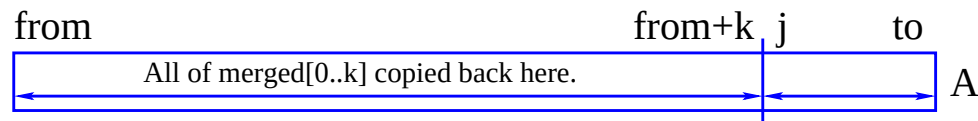
- At some stage either  $i > p$ , or  $j > to$ .
- We will first discuss the case where  $i > p$ , then all of  $A[from..p]$  and only some of  $A[p + 1..to]$  has been copied into  $merged[0..k]$ .
- The tail end of  $A[]$ , namely  $A[k + 1..to]$  is in place and only  $merged[0..k]$  needs to be copied into its right place at  $A[from..from + k]$ .

## Merging— $i$ runs out first, i.e. $i > p$

- At some stage either:  $i > p$ , or  $j > to$ .  
Assume  $i > p$ ,  
then all of  $A[from..p]$  and only some of  
 $A[p + 1..to]$  has been copied into  $merged[0..k]$ .



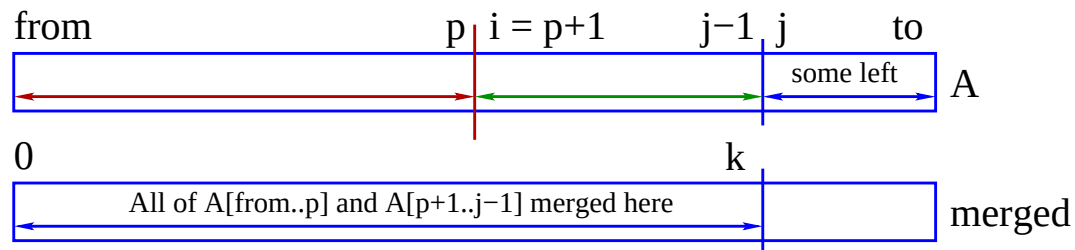
- Notice that, here,  $j$  coincides with  $from + k + 1$ .
- To complete the sorting of  $A[from..to]$ ,  $merged[0..k]$  must be copied back into  $A[from..j - 1]$  since  $A[j..to]$  is already in place.



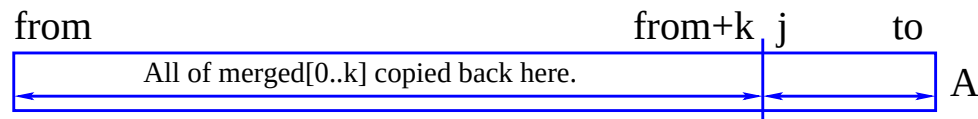
The entire  $A[from..to]$  is now in sorted order.

## Merging—the code when $i$ runs out first

- We will only copy the contents of  $merged[0..k]$  back into  $A[from..j-1]$  since  $A[j..to]$  is already in place.



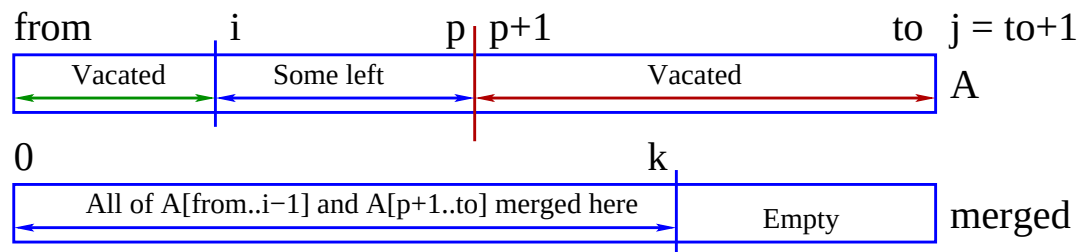
```
1 // Copy merged[] and A[i..p] or A[j..to] back to A[]
2 if (i == p+1) { // rest of A[j..to] already in place
3     r = from; s = 0;
4     while (r < j) // OR while (s <= k)
5         A[r++] = merged[s++]; }
6 else ...
```



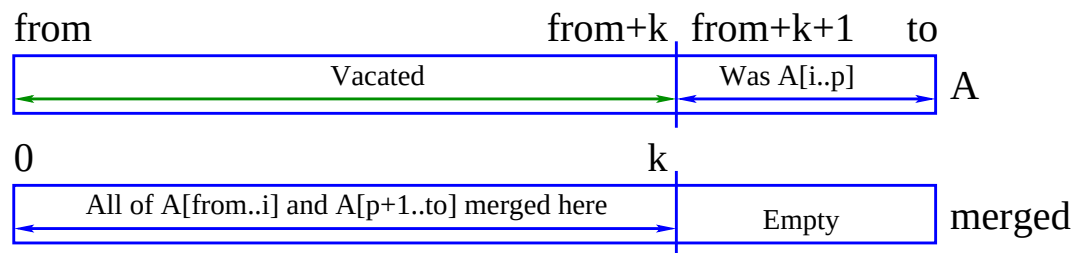
The entire  $A[from..to]$  is now in sorted order.

## Merging— $j$ runs out first

- Put  $i \leftarrow from$ ,  $j \leftarrow to$ , and put  $k \leftarrow 0$ . Compare  $A[i]$  with  $A[j]$  and store the smallest into  $merged[k]$ . Update  $i$  or  $j$ , and  $k$  appropriately.
- At some stage  $i > p$ , or  $j > to$ . Assume  $j > to$ , then all of  $A[p + 1..to]$  and some of  $A[from..p]$  has been copied into  $merged[0..k]$ .



- First copy the contents of  $A[i..p]$  into  $A[k + 1..to]$ :

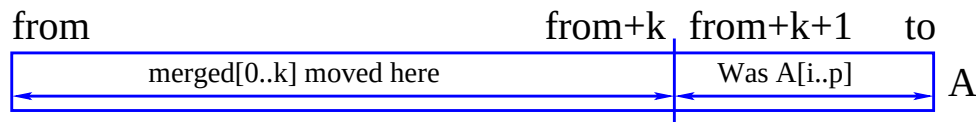


Only  $A[from + k + 1..to]$  is now in its final position.

## Completing the merge when $j > to$

```
1  . . .
2  else { // first park A[i..p] into A[from]k+1..to]
3      r = to; s = p;
4      while (s >= i)
5          A[r--] = A[s--];
6      // move merged[0..k] to A[from..from+k]
7      r = from; s = 0;
8      while (s <= k)
9          A[r++] = merged[s++]; }
```

- To complete the sorting of  $A[from..to]$  the contents of  $merged[0..k]$  must be copied back into  $A[from..from + k]$  since  $A[from + k + 1..to]$  is already in place.



The entire  $A[from..to]$  is now in sorted order.

## mergeSort—the code

```
1 public void mergeSort(int A[], int from, int to) {  
2   // in : A[from..to] array of elements in any order  
3   // int from, and int to, within bounds  
4   //  
5   // out: Array[from..to] in sorted order  
6   int p;  
7   if (from + 1 >= to) {  
8     if (A[from] <= A[to])  
9       return;  
10    swap(A, from, to);  
11    return; }  
12   else {  
13     p = (from+to)/2;  
14     mergeSort (A, from, p);  
15     mergeSort (A, p+1, to);  
16     merge (A, from, p, to);  
17     return; } }
```

# Code for merge

```
1 public void merge(int A[], int from, int p, int to) {
2 // in : A[from..p] array of elements in sorted order
3 // A[p+1..to] array of elements in sorted order
4 // int from, int p, and int to, within bounds
5 // out: returns array A[from..to] in sorted order
6 //
7 int merged[] = new int [to -from + 1], k = 0,
8     i = from, j = p+1, r, s;
9
10 while (i <= p && j <= to)
11     if (A[i] < A[j])
12         merged[k++] = A[i++];
13     else
14         merged[k++] = A[j++];
15 // Copy merged[] and A[i..p] or A[j..to] back to A[]
16 if (i == p+1) { // rest of A[j..to] already in place
17     r = from; s = 0;
18     while (s < k)
19         A[r++] = merged[s++]; }
20 else { // first park A[i..p] into A[..to]
21     r = to; s = p;
22     while (s >= i)
23         A[r--] = A[s--];
24     r = from; s = 0;
25     while (s < k)
26         A[r++] = merged[s++]; } }
```



## Code for swap

The code for **swap**:

```
1 public void swap (int A[], int x, int y) {  
2   int keep = A[x];  
3   A[x] = A[y];  
4   A[y] = keep; }
```

In hindsight we see that the code required to copy *merged*[0..*k* − 1] back into *A*[*from*..*from* + *k* − 1] is duplicated. So **merge** is improved as follows by altering the code of the if statement:

```
1   if (i != p+1) { // first park A[i..p] into A[..to]  
2       r = to; s = p;  
3       while (s >= i)  
4           A[r--] = A[s--]; }  
5   r = from; s = 0;  
6   while (s < k)  
7       A[r++] = merged[s++];
```

Note:

- The copying back of the merged array *merged*[0..*k*] can be replaced with the copying of a few links if the *A*[] is stored as a linked list.

## Quicksort—another $O(n \log n)$ sort

- Quicksort was published in the Computer Journal in 1962 by Tony Hoare. He invented the **case** statement.  
'Partition'  $A[from..to]$  into two parts: first select  $pivot \in A[from..to]$  and save its *index*. Next, move all elements  $< pivot$  to the left of  $pivot$  and all the others to its right. Finally place  $pivot$  at its correct position  $p$  in the final list, then sort  $A[from..p-1]$  and  $A[p+1..to]$  similarly.

```
1 public void quickSort(int A[], int from, int to) {  
2 // in : A[from..to] array of elements in any order  
3 int p;  
4 if (to < 0 || from >= to) return; // nothing to do  
5 if (from - to == 1) // A[from..to] == 2  
6     if (A[from] < A[to]) return;  
7     swap(A, from, to);  
8     return; }  
9 else {  
10     p = partition (A, from, to);  
11     quickSort (A, from, p-1);  
12     quickSort (A, p+1, to); } }
```

## partition—the $O(n)$ heart of quicksort

‘Partition’  $A[from..to]$  into two parts: first select a  $pivot \in A[from..to]$  and save its *index*. Next, move all elements  $< pivot$  to the left of  $pivot$  and all the others to its right. Finally place  $pivot$  at its correct position  $p$  in the final list.

```
1 public int partition(int A[], int from, int to) {  
2     int i, j, pivot = A[from], lower=from+1, upper=to;  
3     while (lower < upper) {  
4         while (A[lower] < pivot && lower < upper)  
5             lower++;  
6         while (pivot <= A[upper] && lower < upper)  
7             upper--;  
8         if (lower < upper) swap(A, lower, upper); }  
9     swap(A, from, lower - 1);  
10    return upper; }
```

## Quicksort—time complexity, $T(n)$

- If *pivot* always partitions the  $A[from..to]$  into two equal parts then  $T(1) = 0$ ,  $T(2) = 1$ , and  $T(n) = 2T(n/2) + O(n)$  implying that  $T(n) = n \log n$ . Of course this hardly ever happens.
- If the one side of *pivot* is nearly always empty then there will be  $O(n)$  partitioning steps taking  $O(n)$  each time, giving  $O(n^2)$ .
- This happens when choosing the *pivot* as  $A[from]$  and when  $A[from..to]$  is almost sorted. Choosing the *pivot* randomly will reduce the likelihood of this happening.
- But  $T(n) = O(n^2)$  in the *worst* case.
- What is the average case?
- Suppose that every  $A[i] \in A[from..to]$  is equally likely to be chosen as the pivot, then the running time of quicksort, if  $i$  is the  $i$ -th smallest element, is  $T(n) = n - 1 + T(i - 1) + T(n - i)$ .

## Quicksort—average running time

- Suppose that every  $A[i] \in A[from..to]$  is equally likely to be chosen as the pivot, then the running time of quicksort, if  $i$  is the  $i$ -th smallest element, is  $T(n) = n - 1 + T(i - 1) + T(n - i)$ .
- To calculate the average, consider:

$$T(n) = n - 1 + T(1 - 1) + T(n - 1)$$

$$T(n) = n - 1 + T(2 - 1) + T(n - 2)$$

$$T(n) = n - 1 + T(3 - 1) + T(n - 3)$$

...

$$T(n) = n - 1 + T(i - 1) + T(n - i)$$

...

$$T(n) = n - 1 + T(n - 1) + T(n - n)$$

Now sum these values and divide by  $n$ .

## Quicksort—average running time

- Given that each element was uniform randomly chosen the average running time is:

$$T(n) = n - 1 + T(1 - 1) + T(n - 1)$$

$$T(n) = n - 1 + T(2 - 1) + T(n - 2)$$

$$T(n) = n - 1 + T(3 - 1) + T(n - 3)$$

...

$$T(n) = n - 1 + T(i - 1) + T(n - i)$$

...

$$T(n) = n - 1 + T(n - 1) + T(n - n)$$

---

$$nT(n) = n(n - 1) + \sum_{i=1}^n (T(i - 1) + T(n - i)), \text{ so that}$$

$$T(n) = n - 1 + \frac{1}{n} \sum_{i=1}^n (T(i - 1) + T(n - i))$$

$$= n - 1 + \frac{1}{n} \sum_{i=1}^n T(i - 1) + \frac{1}{n} \sum_{i=1}^n T(n - i)$$

$$= n - 1 + \frac{2}{n} \sum_{i=0}^{n-1} T(i) \quad \text{A so-called recurrence with full history.}$$

$$= O(n \log n)$$

## Closed form of a recurrence with full history

---

$T(n) = c + \sum_{i=1}^{n-1} T(i)$ , where  $c$  is a constant and  $T(1)$  is given, is called a *recurrence relation with full history*, solved by eliminating the history.

First observe that  $T(n+1) = c + \sum_{i=1}^n T(i)$

Their difference is  $T(n)$ , i.e.  $T(n+1) - T(n) = T(n)$ , giving  $T(n+1) = 2T(n)$ .

Now  $T(2) = T(1) + c$ , so by telescoping,

$$\begin{aligned} T(n+1) &= 2T(n) \\ &= 2^2 T(n-1) \\ &= 2^3 T(n-2) \\ &\quad \dots \\ &= 2^{n-2} T(3) \\ &= 2^{n-1} T(2) \\ &= 2^{n-1} (T(1) + c) \end{aligned}$$

Now consider the recurrence relation with full history

$$T(n) = n - 1 + \sum_{i=0}^{n-1} T(i).$$

## Closed form of $T(n) = n - 1 + \frac{2}{n} \sum_{i=0}^{n-1} T(i)$

---

$$T(n) = n - 1 + \frac{2}{n} \sum_{i=0}^{n-1} T(i), \quad n \geq 2 \quad (1)$$

$$T(n+1) = (n+1) - 1 + \frac{2}{n+1} \sum_{i=0}^n T(i), \quad n \geq 2 \quad (2)$$

Multiply Equation(1) by  $n$  and Equation(2) by  $n+1$  giving:

$$nT(n) = n(n-1) + 2 \sum_{i=0}^{n-1} T(i), \quad n \geq 2 \quad (3)$$

$$(n+1)T(n+1) = n(n+1) + 2 \sum_{i=0}^n T(i), \quad n \geq 2 \quad (4)$$

Subtract Equation (3) from Equation (4):



## Closed form of $T(n) = n - 1 + \frac{2}{n} \sum_{i=0}^{n-1} T(i)$

$$nT(n) = n(n-1) + n \sum_{i=0}^{n-1} T(i), \quad n \geq 2 \quad (3)$$

$$(n+1)T(n+1) = n(n+1) + 2 \sum_{i=1}^n T(i), \quad n \geq 2 \quad (4)$$

Equation (4)–(3):

$$\begin{aligned} (n+1)T(n+1) - nT(n) &= n(n+1) - n(n-1) + 2T(n) \\ &= 2n + 2T(n), \quad n \geq 2 \end{aligned}$$

Giving,

$$\begin{aligned} (n+1)T(n+1) &= (n+2)T(n) + 2n, \\ T(n+1) &= \frac{n+2}{n+1}T(n) + \frac{2n}{n+1}, \quad n \geq 2 \end{aligned}$$

$\frac{2n}{n+1} < 2$ , because  $\frac{n}{n+1} < 1$ ,

$$T(n+1) \leq \frac{n+2}{n+1}T(n) + 2, \quad n \geq 2$$

$$\text{and thus } T(n) \leq 2 + \frac{n+1}{n}T(n-1), \quad n \geq 1$$

## Closed form of $T(n) = n - 1 + \frac{2}{n} \sum_{i=0}^{n-1} T(i)$

Once again we telescope:

$$\begin{aligned} T(n) &\leq 2 + \frac{n+1}{n} T(n-1), \quad n \geq 1 \\ &= 2 + \frac{n+1}{n} \left[ 2 + \frac{n}{n-1} \left[ 2 + \frac{n-1}{n-2} \left[ \cdots \frac{4}{3} \right] \right] \right] \\ &= 2 \left[ 1 + \frac{n+1}{n} + \frac{n+1}{n} \frac{n}{n-1} + \frac{n+1}{n} \frac{n}{n-1} \frac{n-1}{n-2} + \cdots \right. \\ &\quad \left. \cdots + \frac{n+1}{n} \frac{n}{n-1} \frac{n-1}{n-2} \cdots \frac{5}{4} \frac{4}{3} \right] \\ &= 2 \left[ 1 + \frac{n+1}{n} + \frac{n+1}{n-1} + \frac{n+1}{n-2} + \cdots + \frac{n+1}{3} \right] \\ &= 2(n+1) \left[ 1 + \frac{1}{n} + \frac{1}{n-1} + \frac{1}{n-2} + \cdots + \frac{1}{3} \right] \end{aligned}$$

## Closed form of $T(n) = n - 1 + \frac{2}{n} \sum_{i=0}^{n-1} T(i)$

$$\begin{aligned} T(n) &\leq 2 + \frac{n+1}{n} T(n-1), \quad n \geq 1 \\ &= \dots \\ &= 2 \left[ 1 + \frac{n+1}{n} + \frac{n+1}{n-1} + \frac{n+1}{n-2} + \dots + \frac{n+1}{3} \right] \\ &= 2(n+1) \left[ 1 + \frac{1}{n} + \frac{1}{n-1} + \frac{1}{n-2} + \dots + \frac{1}{3} \right] \\ &= 2(n+1) \left[ H(n+1) - \frac{3}{2} \right] \end{aligned}$$

Where  $H(n) = 1 + 1/2 + 1/3 + \dots 1/n$  is the harmonic series,  
 $H(n) = \ln n \gamma + O(1/n)$ , where Euler's constant  $\gamma = 0.57721566 \dots$   
The closed formula for  $T(n)$  is

$$T(n) \leq 2(n+1)(\ln n + \gamma - 1.5) = O(n \log n)$$

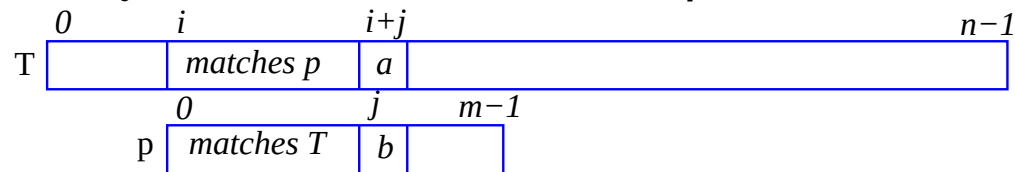
## Pattern matching: $p[0..m-1] \in T[0..n-1]$

---

- A smaller *pattern*,  $p$ , must be found in a larger *text*,  $T$ , and its *position*,  $i \in T$ , must be reported or on failure  $-1$  is returned.
- When programming a matching algorithm, great care must be taken at the boundaries:
  1. If the *pattern* is empty  $p = ""$  then  $i = 0$  should be returned.
  2. If the *text* is empty then failure is always reported,  $i = -1$ .
  3. If  $|p| > |T|$ , i.e.  $p.length() > T.length()$ , then the match fails.
  4. Test that a pattern can match  $T[0..m-1]$ .
  5. Test that a pattern can match  $T[n-m..n-1]$ .
  6. Test that the pattern can match somewhere in  $T[1..n-2]$ , i.e. it does not fall on a boundary.

## Naive, greedy or Brute Force (BF) pattern matching

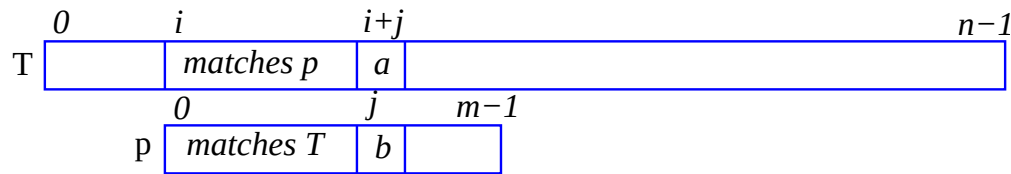
- Find the *pattern*,  $p$ , in the *text*,  $T$ , and return its *position* or the return  $-1$  on failure.
- The *brute force (BF)* method matches characters in  $p$  one-by-one with every character in  $T$  until  $p$  covers a matching part of  $T$ .



Here  $p[0..j-1]$  has matched  $T[i..i+j-1]$  but  $p[j]$  and  $T[i+j]$  do not match.

- Next:  $i++$ ;  $j = 0$ ; and try to match the pattern  $p[0..m-1]$  with the substring  $T[i..i+m-1]$  from the text, until a match is found.
- In the worst case most of  $p$  will match parts of  $T$  and mismatches only occur near the end of  $p$ . In this case at  $O(m)$  matches will occur  $O(n)$  times when the pattern is found near the end of  $T$ .
- Worst case for BF matching is  $O(nm)$ .

## Naive, greedy or Brute Force (BF) pattern matching

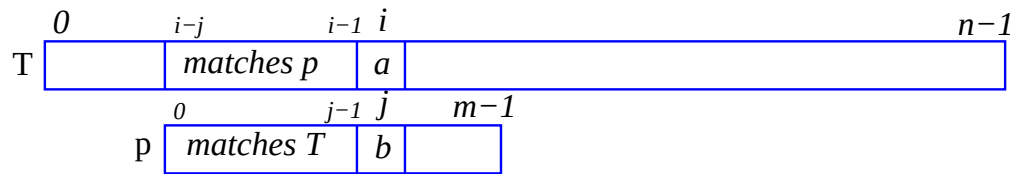


Check if  $p[j] :: T[i+j]$  match. In the figure  
 $p[0..j-1]$  has matched  $T[i..i+j-1]$  but  $p[j] \neq T[i+j]$ .  
Try matching  $p[0]$  and  $T[i+1]$  next.

```
1 int matchBF (String p, String T) {
2   int m = p.length(); if (m == 0) return 0;
3   int n = T.length();
4   int i = 0, found = -1;
5
6   while (found < 0 && i <= n-m) {
7     int j = 0;
8     while (j < m && p[j] == T[i+j])
9       j++;
10    if (j == m)
11      found = i;
12    i++; }
13   return found; }
```

# Naive, greedy or Brute Force (BF) pattern matching

—a different view: match  $p[j] :: T[i]$



Check if  $p[j] :: T[i]$  match. In this figure  
 $p[0..j-1]$  has matched  $T[i-j..i-1]$  but  $p[j] \neq T[i]$ .  
Try  $p[0] :: T[i-j+1]$  next.

```
1 int matchBF (String p, String T) {  
2   int m = p.length(); if (m == 0) return 0;  
3   int n = T.length();  
4   int i = 0, j = 0;  
5   do {  
6     if (p[j] == T[i])  
7       if (j == m - 1) return i - j;  
8       else { i++; j++; }  
9     else {  
10      i = i - j + 1;  
11      j = 0; }  
12   } while (i < n - m);  
13   return -1; }
```

## Naive, greedy or brute Force pattern matching—complexity

---

*Worst case* for BF matching is  $O(nm)$ .

In the *average case* mismatches occur near the middle of pattern, if not earlier, and the pattern is matched near the middle of the text if not sooner. This does not improve the timing by much more than half  $O(m)$  repeated half  $O(n)$  times. The average case is still  $O(nm)$  but the constants are smaller.

Next we discuss three methods that dramatically improve this time complexity by devising ways of avoiding redundant comparisons.

1. *Rabin-Karp* uses hashing to slide  $\mathbf{p}$  over  $\mathbf{T}$ .
2. *Boyer-Moore* determines if and where the character  $\mathbf{T}[\mathbf{i}]$  is present in the pattern  $\mathbf{p}[\mathbf{0} \dots \mathbf{m}-\mathbf{1}]$  to decide where to place  $\mathbf{p}$  next.
3. *Knuth-Morris-Pratt* matches  $\mathbf{p}$  with itself to determine where to move  $\mathbf{p}$  next.



## RK, BM and KMP matching—briefly

Three methods that improve the time complexity:

*Rabin-Karp*-matching compares  $\text{hash}(p) :: \text{hash}(T[i..i+m-1])$ . Using a “rolling” hash economically to calculate  $\text{hash}(T[i+1..i+m])$  from  $\text{hash}(T[i..i+m-1])$ , an  $O(n)$  method ensues.

*Boyer-Moore*-matching preprocesses  $p$  to find the last occurrence of each character. It matches from  $p[m-1]$  backwards through to  $p[0]$ . On non-matching  $p[j]$  and  $T[i]$ , if  $T[i]$  does not occur in  $p$ , then  $p[0]$  is moved to next match  $T[i+1]$ —ignoring characters in  $T[i-j+1..i-1]$ —otherwise  $p$  is moved putting  $p[m-1]$  over  $T[i']$  where  $i' = i + m - \min(j, \text{lastInP}[T[i]]);$ , next, match  $p[m-1] :: T[i']$ .

In the worst case *BM* is likely to scan at most  $n$  characters in  $T$ .

*Knuth-Morris-Pratt*-matching preprocesses  $p$  by first matching  $p$  with itself storing information about where to move  $p[0]$  next when it gets a mismatch at  $T[i]$ — $p[0..j-1] == T[i-j..i-1]$ .

## Rabin-Karp matching (RK)

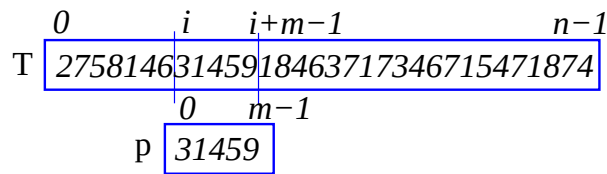
- $|p[0..m-1]| = m$  and  $|T[i..i+m-1]| = m$ .  
If  $\text{hash}(p) == \text{hash}(T[i..i+m-1])$ , then  $p$  and  $T[i..i+m-1]$  are very likely to be equal.
- *Rabin-Karp* compares  $\text{hash}(p)$  with  $\text{hash}(T[i..i+m-1])$ .
- Since Rabin-Karp uses a “rolling” hash to calculate  $\text{hash}(T[i+1..i+m])$  from  $\text{hash}(T[i..i+m-1])$ , an  $O(n)$  method ensues.
- To explain rolling hash, consider patterns of only decimal digits.

|   |                                  |   |       |     |
|---|----------------------------------|---|-------|-----|
|   | 0                                | i | i+m-1 | n-1 |
| T | 27581463145918463717346715471874 |   |       |     |
|   |                                  | 0 | m-1   |     |
| p | 31459                            |   |       |     |

$T = 27581463145918463717346715471874$  First get  $pHash = 31459$ , then one-by-one match it with each  $THash_i$ :

$27581_0, 75814_1, 58146_2, 81463_3, 14631_4, 46314_5, 63145_6, 31459_7$ . Since  $pHash$  matches with  $THash_7$  the position 7 is returned.

# Rabin-Karp—rolling hash for decimal digits



$T = 27581463145918463717346715471874$

$$\begin{aligned} pHash &= 31459 \\ THash_0 &= 27581 \\ THash_1 &= 75814 = (THash_0 - 20000) \times 10 + 4 \\ THash_2 &= 58146 = (THash_1 - 70000) \times 10 + 6 \\ THash_3 &= 81463 = (THash_2 - 50000) \times 10 + 3 \\ THash_4 &= 14631 = (THash_3 - 80000) \times 10 + 1 \\ THash_5 &= 46314 = (THash_4 - 10000) \times 10 + 4 \\ THash_6 &= 63145 = (THash_5 - 40000) \times 10 + 5 \\ THash_7 &= 31459 = (THash_6 - 60000) \times 10 + 9 \\ THash_8 &= 14591 = (THash_7 - 30000) \times 10 + 1 \end{aligned}$$

In general

$$THash_{i+1} = (THash_i - \lfloor THash_i / 10000 \rfloor 10000) \times 10 + (int)T[i + m]$$

and in Java this could be coded as

```
1 THash = (THash - (THash/10000)*10000) * 10
2 + (int)T.charAt(i+m)-(int)'0';
```

## Rabin-Karp—rolling hash for characters

Let  $|T| = m$ , and  $h_i \equiv \text{hash}(T[i..i + m - 1])$ . Given  $h_i$  calculate  $h_{i+1}$ .

Design of a hashing function for Unicode characters is based on the code for digits

1. The contribution of the leftmost character to  $h_i$ :

$$h_{\text{leftmost digit}} = \lfloor h_i / 10^{m-1} \rfloor \times 10^{m-1}$$

2. Deduct it from  $h_i$

$$\text{hash}(T[i + 1..i + m - 1]) = h_i - h_{\text{leftmost digit}}$$

3. Shift this one character left

$$\text{hash}(T[i + 1..i + m - 1]) \times 10$$

4. Add the character  $T[i + m]$ .

$$\begin{aligned} h_{i+1} &= \text{hash}(T[i + 1..i + m]) \\ &= \text{hash}(T[i + 1..i + m - 1]) \times 10 + T[i + m] \end{aligned}$$

## Rabin-Karp—rolling hash for characters

Let  $|T| = m$ , and  $h_i \equiv \text{hash}(T[i..i + m - 1])$ . Given  $h_i$  calculate  $h_{i+1}$ .

The hash function for Unicode characters is based on the code for digits with the base changed from 10 to  $b$ .

1. The contribution of the leftmost character to  $h_i$ :

$$h_{\text{leftmost digit}} = \lfloor h_i / b^{m-1} \rfloor \times b^{m-1}$$

2. Deduct it from  $h_i$

$$\text{hash}(T[i + 1..i + m - 1]) = h_i - h_{\text{leftmost digit}}$$

3. Shift this one character left

$$\text{hash}(T[i + 1..i + m - 1]) \times b$$

4. Add the character  $T[i + m]$ .

$$\begin{aligned} h_{i+1} &= \text{hash}(T[i + 1..i + m]) \\ &= \text{hash}(T[i + 1..i + m - 1]) \times b + T[i + m] \end{aligned}$$

The biggest Unicode character has the value  $2^{16} - 1$ .

Using base  $b = 2^{16}$  makes the above method work. *There is a snag.*

## Rabin-Karp—rolling hash for characters

Basing the hash function for Unicode characters by using the base  $b = 2^{16}$  has some snags:

1. The contribution of the leftmost character to  $h_i$ :

$$h_{\text{leftmost digit}} = \lfloor h_i / b^{m-1} \rfloor \times b^{m-1}$$

The value of  $b^{m-1} = 2^{16(m-1)}$  uses ... bits. So when  $m > \dots$  it will overflow. This is remedied by using modular arithmetic. Every arithmetic operation must be calculated modulo some large prime that is less than  $2^{16}$ .

2. Deduct it from  $h_i$

$$\text{hash}(T[i + 1..i + m - 1]) = h_i - h_{\text{leftmost digit}}$$

*There is a new snag.*

The result may become negative. It must first be made positive.

## Rabin-Karp—rolling hash for characters

---

*There is a new snag.*

The result may become negative. It must first be made positive. This is done by adding `modHash` to the number as many times as needed to make it positive. Suppose that  $h < 0$ , then

3. Shift this one character left

$$\text{hash}(T[i + 1..i + m - 1]) \times b$$

4. Add the character  $T[i + m]$ .

$$\begin{aligned} h_{i+1} &= \text{hash}(T[i + 1..i + m]) \\ &= \text{hash}(T[i + 1..i + m - 1]) \times b + T[i + m] \end{aligned}$$

The biggest Unicode character has the value  $2^{16} - 1$ .

Using base  $b = 2^{16}$  makes the above method work.

# Rabin-Karp—code

---

```
1 public static int modHash = 65521;
2 public static final int base
3     = (Character.MAX_VALUE + 1) % modHash;
4
5 static int firstHash(String s) {
6     int m = s.length();
7     int j, h = 0;
8     for(j = 0; j < m; j++){
9         h = mod(h * base);
10        h = mod(h + s.charAt(j));
11    }
12    return h;
13 }
```



# Rabin-Karp—code

---

```
1 static int hash(String s) {
2     return firstHash(s);
3 }
4
5 static int baseLeftmost (String T) {
6     int baseLeft = 1;
7     int i;
8
9     for (i = 1; i < T.length(); i++)
10         baseLeft = mod(baseLeft*base);
11     return baseLeft;
12 }
13 }
```

# Rabin-Karp—code

---

```
1 static int matchRK(String p, String T){
2     int m = p.length();
3     int n = T.length();
4     if (m > n)
5         return -1;
6     int pHash = hash(p);
7     int THash = hash((String)T.subSequence(0,m));
8     int baseLeft = baseLeftmost(p);
9     int i;
```

# Rabin-Karp—code

```
1 // Get the subsequence=m starting at 0
2
3 for (i = 0; i + m <= n; i++) {
4     if (THash == pHash) {
5         if (stringCompare(
6             (String)T.subSequence(i, i+m), p))
7             return i;
8     }
9     if (i == n-m)
10        return -1;
11    // deduct contribution of left character
12    THash = mod(THash
13        - mod(baseLeft * T.charAt(i)));
14    // Shift left
15    // add character at right
16    THash = mod(mod(THash*base)
17        + T.charAt(i+m)) ;
18 }
19 return -1;
20 }
```

# Rabin-Karp—code

---

```
1 static boolean stringCompare(String p,  
2                               String T) {  
3     int m = p.length();  
4     int n = T.length();  
5  
6     int i;  
7  
8     if (m != n)  
9         return false;  
10    for (i = 0; i < m; i++)  
11        if (p.charAt(i) != T.charAt(i))  
12            return false;  
13    return true;  
14 }
```

# Rabin-Karp—code

---

```
1 static int mod (int top) {  
2   int modulo = top % modHash;  
3  
4   if (modulo < 0)  
5     return modulo  
6       + modHash * (1 + (-modulo/modHash));  
7   else  
8     return modulo;  
9  
10 }
```

## Rabin-Karp matching—rolling hash

- Normally the patterns may get too big to be stored as a unique number. Apply modulo arithmetic when hashing. Use a large prime number `modHash` to `mod` every arithmetic result.
- Calculate the hash roughly as follows:

```
1 int firstHash(String s) {
2   int m = s.length();
3   int j, h=0;
4   for(j = 0, j < m, j++)
5     h = (h+mod(s.charAt(j)*base));
6   return h;
7 }
8
9 int nextHash(char left, char right,
10              int THash) {
11   THash = mod(THash - mod(left*base^(m-1)));
12   return mod(THash*base + right);
13 }
```

# Rabin-Karp matching—pseudo code

```
1 int rabinKarpMatch(String p, String T){
2   int m = p.length();
3   int n = T.length();
4   int pHash = hash(p[0..m-1]);
5   int THash = hash(T[0..m-1]);
6
7   for (i = 0; i < n; i++) {
8     if (THash == pHash)
9       // verify equality of patterns
10      if (T[i..i+m-1] == p)
11        return i;
12     THash = hash(T[i+1..i+m]);
13   }
14   return -1;
15 }
```

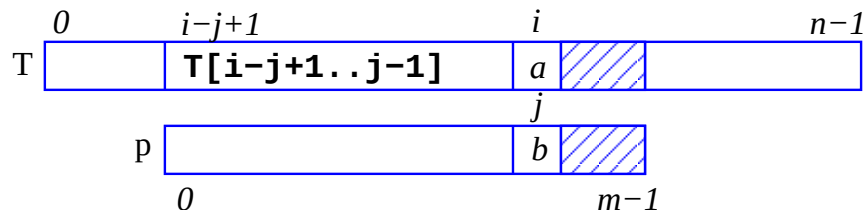
## Rabin-Karp matching (RK)—complexity

---

- Since every character in the *text* file must be used in the hashing formula, at worst all characters need to be scanned, taking  $O(n)$ .
- The pattern must be hashed,  $O(m)$ .
- The worst case time is thus  $O(m + n)$ .
- This is at least  $m$  times faster than the BF method.



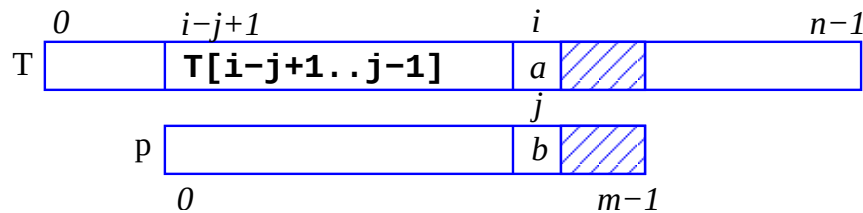
## Boyer-Moore matching (BM)



If the letter  $a$  never appears in  $p$ , the pattern may be moved on, placing  $p[0]$  over  $T[i+1]$  and  $p[m-1]$  over  $T[i+m]$ —*ignoring* the characters in  $T[i-j+1..i-1]$ .

*Boyer-Moore*-matching preprocesses  $p$  to find the last occurrence of each character. It matches from  $p[m-1]$  backwards through to  $p[0]$ . On non-matching  $p[j]$  and  $T[i]$ , if  $T[i]$  does not occur in  $p$ , then  $p[0]$  is moved to next match  $T[i+1]$ —ignoring characters in  $T[i-j+1..i-1]$ —*otherwise*  $p$  is moved putting  $p[m-1]$  over  $T[i']$  where  $i' = i + m - \min(j, 1 + \text{lastInP}(T[i]))$ ;  
next, match  $p[m-1]::T[i']$ . In the worst case *BM* can degenerate to  $O(mn + |\Sigma|)$  but it is likely to scan less than  $n$  characters in  $T$ .

# Boyer-Moore matching (BM)



- For  $p$ , calculate the table `lastInP` and place  $p[0]$  over  $T[0]$ , i.e. set  $i = m-1$  and  $j = m-1$ .
- Compare  $p[j]::T[i]$ , i.e. compare  $p[m-1]::T[m-1]$ . If equal, then  $j--$  and  $i--$ , until a match of  $p$  at  $j == 0$ .
- If  $p[i]::T[i]$  are not equal: Look up the entry in `lastInP(T[i])`. It is  $-1$  if  $T[i] \notin p$ .
- If  $T[i]$  does not appear in the pattern  $p$  then move  $i$  forward:  $i' = i + m$ .
- If the last appearance of  $T[i] \in p$  is at  $k$  then move  $i$  forward:  $i' = i + m - \min(j, 1+k)$ .
- Repeat while  $(i < n)$ .

## Boyer-Moore—getLast(p, lastInP)

```
1 static void getLast(String p, int lastInP[]) {
2 // pre: p[0..m-1] pattern for which
3 // lastInP must be created
4 // post: Boyer-Moore's lastInP_vector
5 //
6
7 int _j;
8 int _s=_256;
9 int _m=_p.length();
10
11 for_(j=0;_j<s;_j++)
12     lastInP[j]=_1;
13 for_(j=0;_j<m;_j++)
14     lastInP[(int)(p.charAt(j))]=_j;
15 }
```

## Boyer-Moore—matchBM(p, T)

Start with the usual tests on the bounds **m** and **n** and set up **lastInP**.

```
1 static int matchBM(String p, String T) {
2 // pre: p[0..m-1] pattern to match
3 // T[0..n-1] text in which to find pattern
4 // post: returns index of first matched position
5
6 int m = p.length();
7 int n = T.length();
8
9     if (m == 0) return 0;
10    else if (n == 0) return -1;
11    else if (m > n) return -1;
12
13 int s = 256;
14 int last[] = new int [s];
15 int i; int j;
16
17    getLast(p, last);
```

## Boyer-Moore—matchBM(p, T)

The code puts  $i$ , so that  $p[0]$  lies at  $T[i-m+1]$ .

```
1  i = j = m - 1;
2  do {
3      if (p.charAt(j)==T.charAt(i))
4          if (j == 0 )
5              return i; // first match
6          else {
7              i--;
8              j--;
9          }
10     else {
11         i = i + m // jump step
12             - Math.min(j, 1+last[T.charAt(i)]);
13         j = m - 1;
14     }
15 } while (i < n );
```

## Boyer-Moore matching—complexity

---

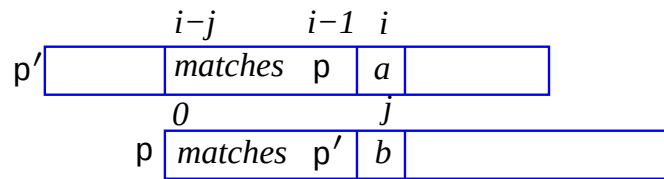
- BM must set up `lastInP[]`.  
This costs  $|\text{lastInP}[]| = |\Sigma|$ .  
Then it starts comparing from the back of `p`.  
Unless `p` is covering itself in `T`,  
or if the  $|p| \ll |T|$  or  $|p| \ll |\Sigma|$ ,  
most comparisons are going to be unequal.

So, on *average*, the complexity is  $O(n + \Sigma) = O(n)$  since  $\Sigma$  is a constant.

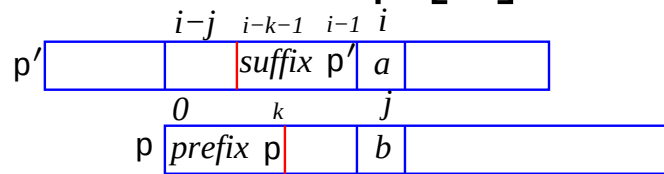
## Boyer-Moore matching—complexity

- Many jumps might be  $m$  long, so the complexity could become  $O(n/m)$ .  
e.g.  $\text{p} = \text{"yyyyyyyyy"}'$ , i.e.  $m$  ‘y’s, and no ‘x’s  
 $\text{T} = \text{"xxxxxxxxxxxxxxxxxxxxxyyyyyyyy"}'$ ,  $|\text{T}| = n$ .
- The *worst case* for BM appears when  
 $\text{p} = \text{"yxxxxxxxxxxx"}'$ , i.e. a ‘y’ and  $m - 1$  ‘x’s and  $\text{T} =$   
 $\text{"xxxxxxxxxxxxxxxxxyxxxxxxxxxxx"}'$ ,  $|\text{T}| = n$ .  
BM degenerates to  $O(mn + |\Sigma|)$
- Generally, BM is likely to scan less than  $n$  characters in  $\text{T}$ .

# Knuth-Morris-Pratt matching—preview



*Knuth-Morris-Pratt-matching* (KMP) preprocesses  $p$  by first matching  $p$  with itself storing information about where to move  $p[0]$  next when it gets a mismatch at  $p'[i]$ : We know  $p[0..j-1] == p'[i-j..j-1]$ .



In the matching piece KMP finds a *prefix* of  $p \equiv p[0..k]$  that matches a *suffix* of  $p' \equiv p'[i-k-1..i-1]$ .

- When there *is a match*, KMP always sets the next value of  $j$  to  $j++$  and steps  $i$  to  $i++$ .
- When there *is no match*,  $j$  becomes  $\text{next}[j] \equiv k$  and if  $j \geq 0$  then  $i$  stays the same and KMP avoids redundant comparisons in  $p'[i-j..i-1]$ , otherwise when  $j < 0$  then  $j = 0$  and  $i++$ .



## KMP matching—examples

Let  $i$  run in  $T$  and  $j$  run through  $p$ , comparing  $p[j]$  with  $T[i]$  and putting  $i++$ ;  $j++$ ; when they are equal.

$T$   $b a n d b a n d a n a o n e b a n d a n a t w o b a n d a n a s$

$p$   $b a n d a n a s$

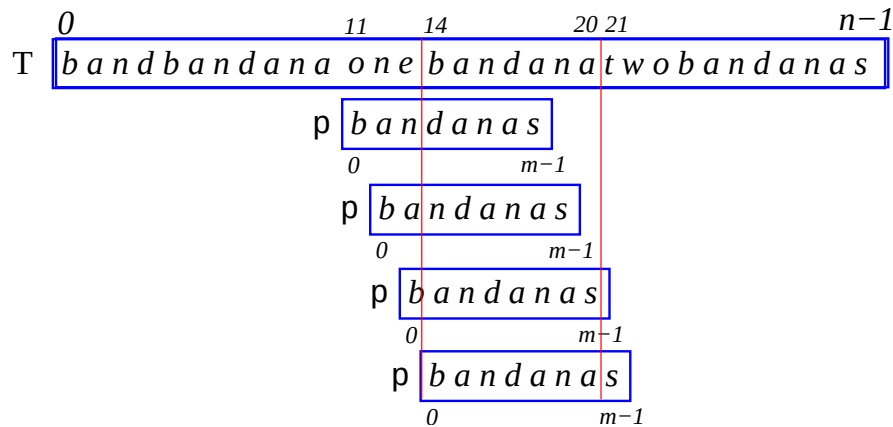
$p[0..3] \equiv T[0..3]$  but  $p[j=4] \equiv 'a' \neq T[i=4] \equiv 'b'$ , Since  $p[0..3] \equiv 'band'$  does not match with any substring of itself, next put  $j=0$ — $i$  stays the same—comparing  $p[0..] :: T[4..]$ . KPM ‘knows’  $p[0] = 'band'$  and safely jumps over it.

$T$   $b a n d b a n d a n a o n e b a n d a n a t w o b a n d a n a s$

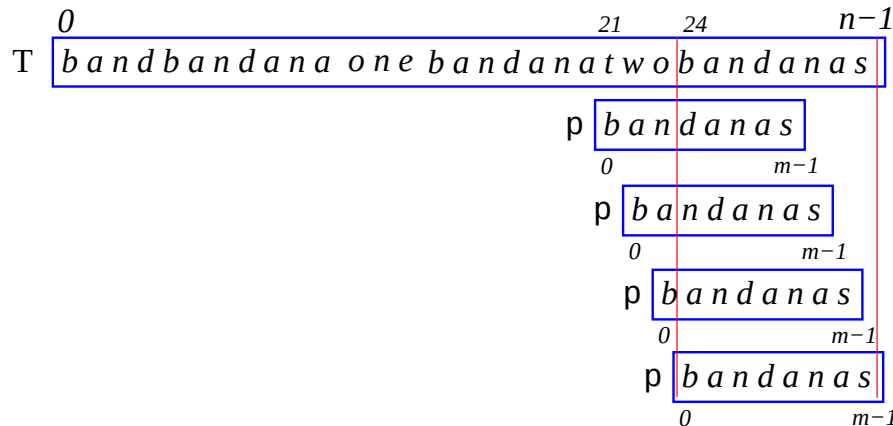
$p$   $b a n d a n a s$

Now  $p[0..6] \equiv 'bandana'$  matches  $T[4..10]$  and  $j$  becomes 0 putting  $p[0..]$  over  $T[11..]$ .

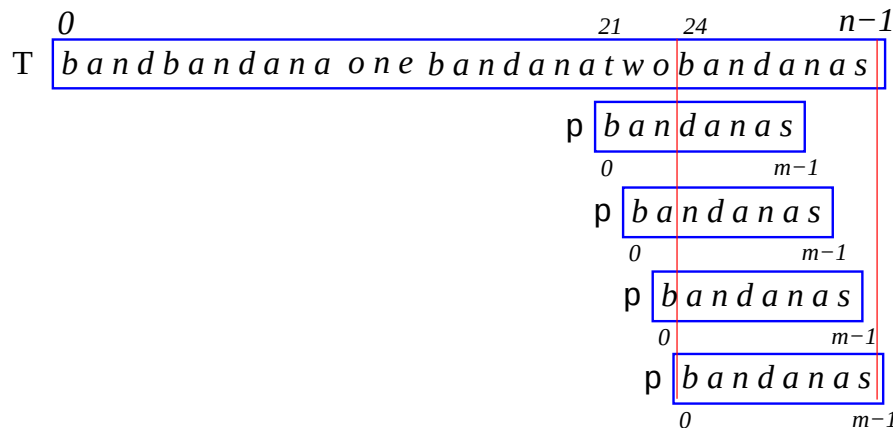
# Knuth-Morris-Pratt matching (KMP)



Subsequently  $j$  hobbles forward one-by-one until **bandana** is matched at  $T[14..20]$ . The mismatch at  $p[7] = 's'$  with  $T[21] = 't'$  causes  $p$  to jump forward to try matching  $p[0..]$  over  $T[21..]$



# Knuth-Morris-Pratt matching (KMP)



When  $p[0]$  mismatches both  $i$  and  $j$  are incremented:

$p[j=0] \equiv 'b'$  mismatches  $T[i=21] \equiv 't'$  and then

$p[j=0] \equiv 'b'$  mismatches  $T[i=22] \equiv 'w'$  and then

$p[j=0] \equiv 'b'$  mismatches  $T[i=23] \equiv 'o'$  and then

$p[j=0] \equiv 'b'$  matches  $T[i=24] \equiv 'b'$ , and eventually the pattern

$p[0..7] \equiv 'bandanas'$  matches  $T[24..31]$ .

The value of  $next[0] = -1$  and all the other values of  $next[j]$  here are  $0$  because 'bandanas' has no suffixes that match prefixes of itself.

The following example has substrings  $p[0..k]$ ,  $k > 0$ , which have prefixes that match their own suffixes.

Consider the pattern **ababbababaa**.

## Knuth-Morris-Pratt matching (KMP)

The pattern **ababbababaa** contains substrings  $p[0..k]$ ,  $k>0$ , with prefixes that match their own suffixes.

| j  | next[j]                                              |
|----|------------------------------------------------------|
| 0  | a -1 -                                               |
| 1  | b 0 a-                                               |
| 2  | a 0 ab-                                              |
| 3  | b 1 aba-: $p[0..0] \equiv a \equiv p[2..2]$          |
| 4  | b 2 abab-: $p[0..1] \equiv ab \equiv p[2..3]$        |
| 5  | a 0 ababb-                                           |
| 6  | b 1 ababba-: $p[0..0] \equiv a \equiv p[5..5]$       |
| 7  | a 2 ababbab-: $p[0..1] \equiv ab \equiv p[5..6]$     |
| 8  | b 3 ababbaba-: $p[0..2] \equiv aba \equiv p[5..7]$   |
| 9  | a 4 ababbabab-: $p[0..3] \equiv abab \equiv p[5..8]$ |
| 10 | a 3 ababbababa-: $p[0..2] \equiv aba \equiv p[7..9]$ |

If the pattern **p=ababbababaa** mismatches the text

T=ababbababababababababababababab at p[j=10]≡‘a’ !=  
T[i=10]≡‘b’, the next[j] value is 2 and j must be set to j =  
next[j]+1≡3, leaving i the same.

## Knuth-Morris-Pratt matching (KMP)

The pattern **abababababa** contains substrings  $p[0..k]$ ,  $k > 0$ , with prefixes that match their own suffixes.

|    |   |    |             |                                          |
|----|---|----|-------------|------------------------------------------|
| 0  | a | -2 | -           |                                          |
| 1  | b | -1 | a-          |                                          |
| 2  | a | -1 | ab-         |                                          |
| 3  | b | 0  | aba-        | $p[0..0] \equiv a \equiv p[2..2]$        |
| 4  | a | 1  | abab-       | $p[0..1] \equiv ab \equiv p[2..3]$       |
| 5  | b | 2  | ababa-      | $p[0..2] \equiv aba \equiv p[2..4]$      |
| 6  | a | 3  | ababab-     | $p[0..3] \equiv abab \equiv p[2..5]$     |
| 7  | b | 4  | abababa-    | $p[0..4] \equiv ababa \equiv p[2..6]$    |
| 8  | a | 5  | abababab-   | $p[0..5] \equiv ababab \equiv p[2..7]$   |
| 9  | b | 6  | ababababa-  | $p[0..6] \equiv abababa \equiv p[2..8]$  |
| 10 | a | 7  | ababababab- | $p[0..7] \equiv abababab \equiv p[2..9]$ |

If the pattern  $p = \text{ababababab}\underline{\text{a}}$  mismatches the text

$T = \text{ababababab}\underline{\text{b}}\text{abababababababababababababab}$  at  $p[j=10] \equiv 'a' \neq T[i=10] \equiv 'b'$ , the  $\text{next}[j]$  value is 7 and  $j$  must be set to  $j = \text{next}[j]+1 \equiv 8$ , leaving  $i$  the same.

## Knuth-Morris-Pratt matching (KMP)

---

- In KMP the text  $T$  is processed forward.
- Since all of  $p[0..j-1]$  has been matched, none of the corresponding characters in  $T[i-j..i-1]$  need ever be matched again.
- Sometimes the character at  $T[i]$  which mismatched with character  $p[j]$  may be repeatedly matched with different characters of the pattern  $p$ .
- The next placement of  $p$  over  $T$  depends entirely on what has already been matched in  $p$ .
- This information is applied to decide which character of  $p$  must next be matched with  $T[i]$ .

## Knuth-Morris-Pratt matching—recap

---

- This information is applied to decide which character of  $p$  must next be matched with  $T[i]$ .
- $k$  is calculated to place  $p[k+1]$  over  $T[i]$  where  $p[0..k]$  is the *greatest* prefix in  $p[0..j-1]$  that matches a suffix  $p[j-k-1..j-1]$ . Note that length  $|p[0..k]| = k + 1 = |p[j-k-1..j-1]|$ .
- We call the table of  $k$ 's `next[]`.

## Knuth-Morris-Pratt matching—recap

---

- The longest prefix-suffix pair is placed over one another first.
- If there is a match:  $i++;$   $j++;$ , else
- If there is a mismatch a next  $j$  that is always smaller than the previous one is calculated using  $j = \text{next}[j] + 1$ .
- Note that  $i$  is only advanced when there is a match or when  $j$  becomes  $0$ .
- We next construct the  $\text{next}[]$  table.
- When there is a mismatch of  $p[j]$  with  $T[i]$  the value of  $\text{next}[j]$  tells us to advance  $j$  to  $\text{next}[j]+1$ .
- Usually  $i$  stays the same but can advance by  $1$ .



## KMP—constructing `next[]`

- When `p[j]` and `T[i]` match, simply increment both `i` and `j`, using `i++; j++;`
- We will now consider mismatches at various values of `j ∈ [0..m-1]`.
- At a mismatch of `p[0]`, simply put `i++; j++;`
- We will construct `next[]` for Manber's example from Udi Manber [Section 6.7, pp. 148–155, *Introduction to Algorithms A Creative Approach*, Addison-Wesley Publishing Co., Reading, MA., 1989]

Note that the **ababbababaa** example is this example in slightly disguised form.

## KMP—constructing next[]

Consider a mismatch at  $p[0]$ . Since no information has been gained, simply  $i++;$   $j++;$ .

$T$   $x \ y \ x \ y \ y \ x \ y \ x \ y \ x \ x$   
 $p$   $\_ \ y \ x \ y \ y \ x \ y \ x \ y \ x \ x$

$T$   $x \ y \ x \ y \ y \ x \ y \ x \ y \ x \ x$   
 $p$   $x \ \_ \ x \ y \ y \ x \ y \ x \ y \ x \ x$

On mismatching  $p[1]$ —above right— $p[0..0]$  is known, but it does not help so  $j=0$  and  $i$  remains the same.

After a mismatch at  $p[2]$ —below left—the substring  $p[0..1]$  is known, but no prefix equals any suffix so again  $j=0$  and  $i$  remains the same.

$T$   $x \ y \ x \ y \ y \ x \ y \ x \ y \ x \ x$   
 $p$   $x \ y \ \_ \ y \ y \ x \ y \ x \ y \ x \ x$

$T$   $x \ y \ x \ y \ y \ x \ y \ x \ y \ x \ x$   
 $p$   $x \ y \ x \ \_ \ y \ x \ y \ x \ y \ x \ x$

The mismatch at  $p[3]$  means that we have covered  $p[0..2] \equiv "xyx"$ . This has a prefix  $x$ - that equals a suffix  $-x$  so now  $j=1$  and  $i$  remains the same.

|         |    |   |   |   |   |   |   |   |   |   |    |
|---------|----|---|---|---|---|---|---|---|---|---|----|
| $j$     | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| $p$     | x  | y | x | y | y | x | y | x | y | x | x  |
| next[j] | -1 | 0 | 0 | 1 | . | . | . | . | . | . | .  |

## KMP—constructing next[]

When mismatching at  $p[4]$ ,  $p[0..3] \equiv \text{"xyxy"}$  It has a prefix  $\text{xy-}$  that equals a suffix  $\text{-xy}$ , so now  $j=2$  and  $i$  remains the same.

T  $\boxed{\text{x y x y}} \boxed{\text{y x y x y x x}}$   
 p  $\boxed{\text{x y x y}} \boxed{\text{- x y x y x x}}$

T  $\boxed{\text{x y x y y}} \boxed{\text{x y x y x x}}$   
 p  $\boxed{\text{x y x y y}} \boxed{\text{x y x y x x}}$

On mismatching  $p[5]$ —above right— $p[0..4] \equiv \text{"xyxyy"}$ , has no prefix-suffix pair, so  $j=0$  and  $i$  static.

After a mismatch at  $p[6]$ —below left—the substring  $p[0..5] \equiv \text{"xyxyyx"}$  has the prefix-suffix pair  $\text{x-}$ ,  $\text{-x}$  and  $j=1$  and  $i$  is static.

T  $\boxed{\text{x y x y y x}} \boxed{\text{y x y x x}}$   
 p  $\boxed{\text{x y x y y x}} \boxed{\text{- x y x x}}$

T  $\boxed{\text{x y x y y x y}} \boxed{\text{x y x x}}$   
 p  $\boxed{\text{x y x y y x y}} \boxed{\text{- y x x}}$

The mismatch at  $p[7]$  means that we have covered  $p[0..6] \equiv \text{"xyxyyyxy"}$ . This has a prefix  $\text{xy-}$  that equals a suffix  $\text{-xy}$  so now  $j=2$  and  $i$  remains the same.

|         |    |   |   |   |   |   |   |   |   |   |    |
|---------|----|---|---|---|---|---|---|---|---|---|----|
| j       | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| p       | x  | y | x | y | y | x | y | x | y | x | x  |
| next[j] | -1 | 0 | 0 | 1 | 2 | 0 | 1 | 2 | . | . | .  |

## KMP—constructing next[]

When mismatching at  $p[8]$ ,  $p[0..7] \equiv \text{"xyxyyxxyx"}$  It has a prefix  $\text{xyx-}$  that equals a suffix  $\text{-xyx}$ , so now  $j=3$  and  $i$  remains the same.

T 

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| x | y | x | y | y | x | y | x | y | x | x |
|---|---|---|---|---|---|---|---|---|---|---|

T 

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| x | y | x | y | y | x | y | x | y | x | x |
|---|---|---|---|---|---|---|---|---|---|---|

If  $p[9] \neq T[9]$ —above right— $p[0..8] \equiv \text{"xyxyyxxyxy"}$ , This has a prefix  $\text{xyxy-}$  that equals a suffix  $\text{-xyxy}$ , so now  $j=4$  and  $i$  remains the same.

After the mismatch at  $p[10]$ —below—the substring

$p[0..9] \equiv \text{"xyxyyxxyxxy"}$  has the prefix-suffix pair  $\text{xyx-}$ ,  $\text{-xyx}$ , so  $j=3$  and  $i$  is static.

T 

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| x | y | x | y | y | x | y | x | y | x | x |
|---|---|---|---|---|---|---|---|---|---|---|

p 

|   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|
| x | y | x | y | y | x | y | x | y | x | - |
|---|---|---|---|---|---|---|---|---|---|---|

The table has now been completed.

|         |    |   |   |   |   |   |   |   |   |   |    |
|---------|----|---|---|---|---|---|---|---|---|---|----|
| j       | 0  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| p       | x  | y | x | y | y | x | y | x | y | x | x  |
| next[j] | -1 | 0 | 0 | 1 | 2 | 0 | 1 | 2 | 3 | 4 | 3  |

## KMP—buildNext

Pseudocode for buildNext.

```
1 void buildNext(String p, int [] next) {  
2   int m = p.length();  
3   int i, j;  
4  
5   next[0] = -2;  
6   next[1] = -1;  
7   for (i = 2; i < m; i++) {  
8     j = next[i-1]+1;  
9     while (j >=0 && p[i-1] != p[j]){  
10      j = next[j]+1; }  
11     next[i] = j; }  
12   for (j = 0; j < m; j++)  
13     next[j]++; }
```

## KMP—KMPmatch

Pseudocode for buildNext

```
1 int matchKMP(String p, String T) {  
2     int m = p.length();  
3     int n = T.length();  
4  
5     if (m == 0) return 0;  
6     else if (n == 0) return -1;  
7     else if (m > n) return -1;  
8  
9     int i, j, next[] = new int [m+1];  
10  
11     buildNext(p, next);  
12     ...
```

## KMP—KMPmatch

```
1  ...
2  buildNext(p, next);
3  i = 0;
4  j = 0;
5  while (i < n) {
6      if (p[j] == T[i]){
7          i++;
8          j++; }
9      else {
10         j = next[j];
11         if (j < 0) {
12             j = 0;
13             i++; } }
14     if (j == m)
15         return i - m; }
16 return -1; }
```

## Knuth-Morris-Pratt—complexity

---

- Calculating `buildNext[]` takes  $O(m)$ . Matching the pattern using the `next[]` table takes  $O(n)$ . KMP runs in  $O(m + n)$ .
- One character in `T[i]` may be matched with many characters from the pattern `p`. How many?
- When there is a mismatch with `T[i]` another character from `p` is matched if `next[j] >= 0`.
- Suppose the first mismatch involved `p[j]`, since each evaluation of `next[j]` leads us to a smaller index  $j \in [0..j-1]$ , so we can only  $j$  times. However, to reach `p[j]` we must have gone forward  $j$  times without any backtracking.
- If we assign the costs of backtracking to the forward moves then we at most double the number of forward moves.
- There are only  $n$  forward moves so the number of comparisons is  $O(n)$ .



## Preprocess the *text* $T$ —not the *pattern* $p$

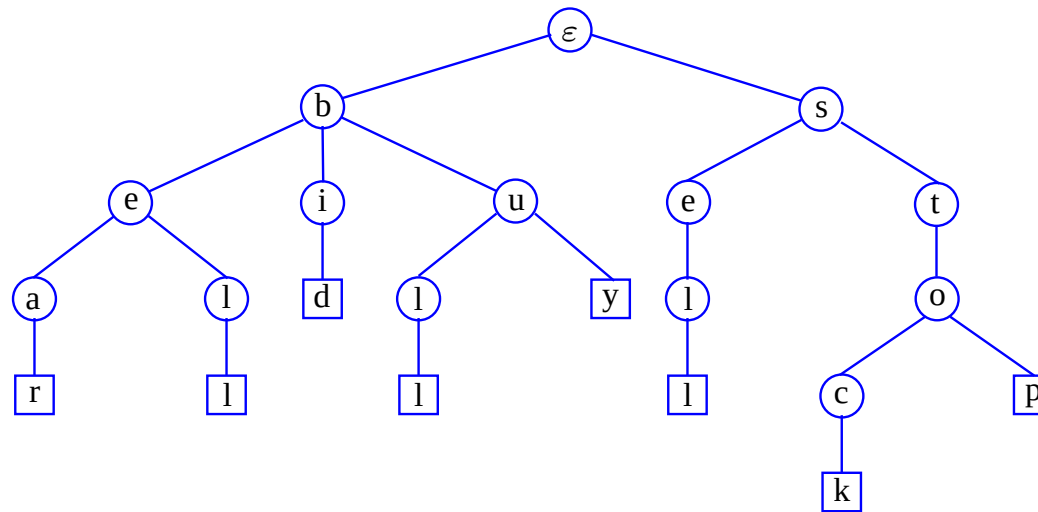
- In Rabin-Karp, Boyer-Moore, and Knuth-Morris-Pratt pattern matching the pattern  $p$  is *preprocessed* to speed up searching for its occurrence in a text  $T$ .
- If many searches  $p \in T$  are done, such as searching for the presence of a word  $p$  in a dictionary  $T$ , then *preprocessing* the text  $T$  may help to speed up queries, especially if  $T$  is large or when many different queries  $p$  are made.
- A *trie* is a very fast, dense structure that allows searches in time proportion to the *pattern* size, i.e.  $O(m)$ .

## Standard tries

The standard trie for a set of strings  $T$  is an ordered tree such that:

- Each node but the root is labelled with a character.
- The children of a node are alphabetically ordered.
- The paths from the leaves to the root yield the strings of  $T$ .
- e.g. standard trie for the set of strings

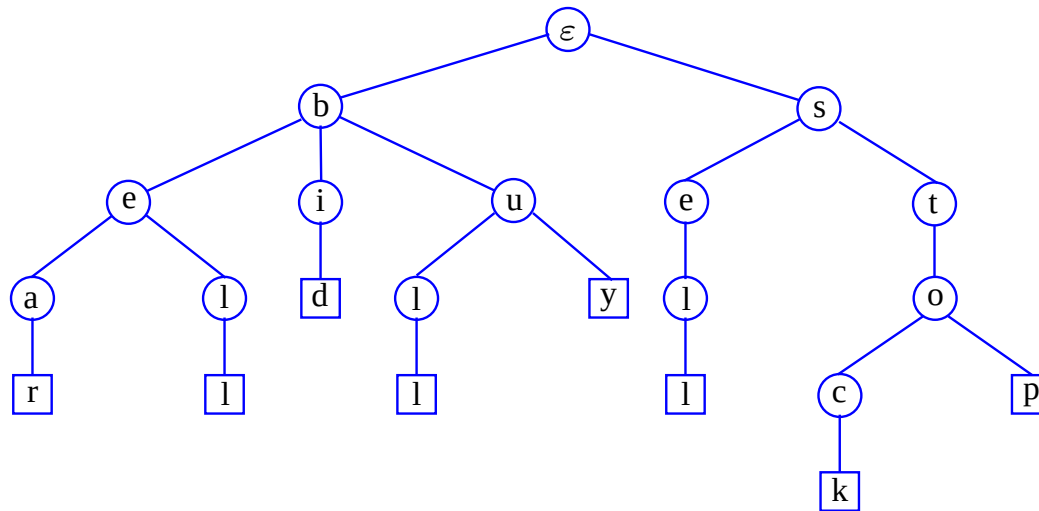
$T = \{bear, bell, bid, bull, buy, sell, stock, stop\}$



## Analysis of standard tries

A standard trie uses  $O(n)$  space and supports searches, insertions and deletions in time  $O(md)$ , where:

- $n$  total size of the strings in  $\mathsf{T}$ , and  $m$  size of the string parameter of the operation, while  $d = |\Sigma|$  size of the alphabet.
- Using binary search at each node for the next letter, searches, insertions and deletions may be done in  $O(m \log d)$  time.
- By using hashing or direct lookup this is further reduced to  $O(m)$ .



# Word matching with a trie

|   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| s | e | e |   | a |   | b | e | a | r | ?  |    | s  | e  | l  | l  |    | s  | t  | o  | c  | k  | !  |    |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| s  | e  | e  |    | a  |    | b  | u  | l  | l  | ?  |    | b  | u  | y  |    | s  | t  | o  | c  | k  | !  |    |
| 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 |

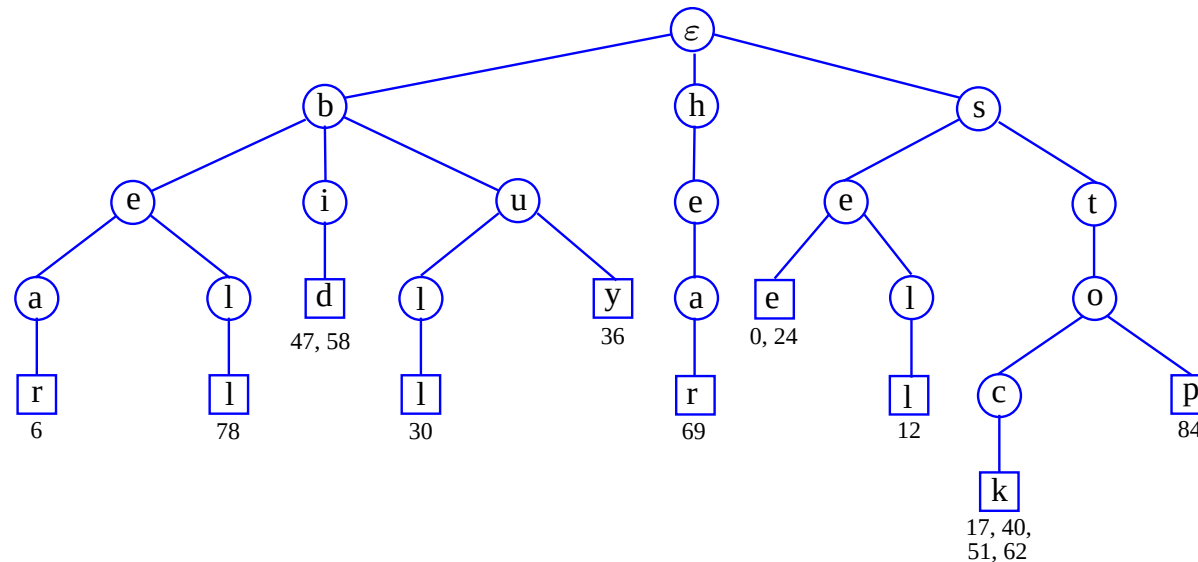
  

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| b  | i  | d  |    | s  | t  | o  | c  | k  | !  |    | b  | i  | d  |    | s  | t  | o  | c  | k  | !  |    |
| 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 |

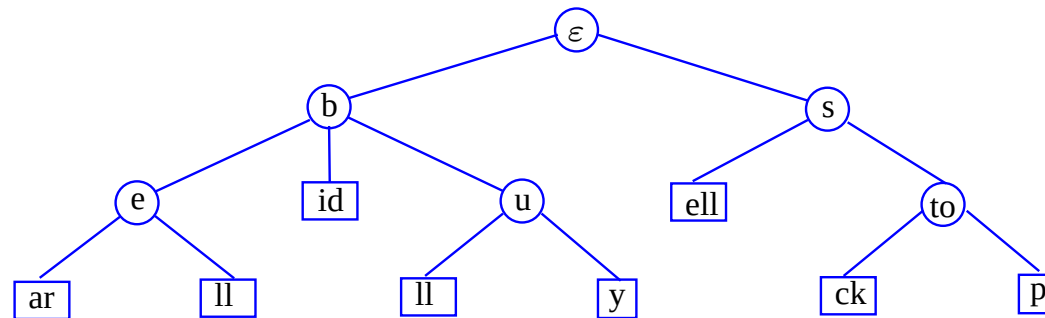
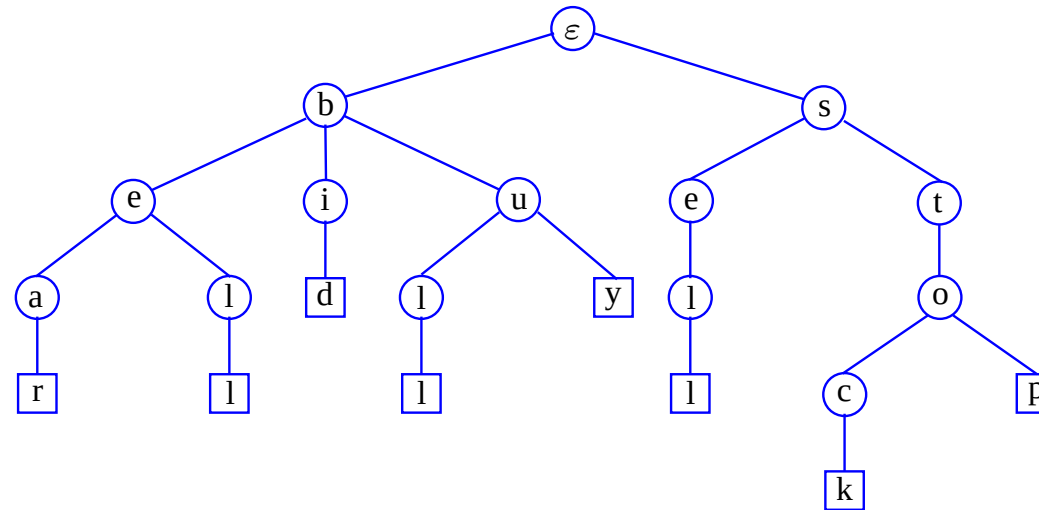
|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |  |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|--|
| h  | e  | a  | r  |    | t  | h  | e  |    | b  | e  | l  | l  | ?  |    | s  | t  | o  | p  | !  |  |
| 69 | 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 |  |

- We insert some of the words of the text into a trie.
- Each leaf stores the occurrences of the associated word in the text.



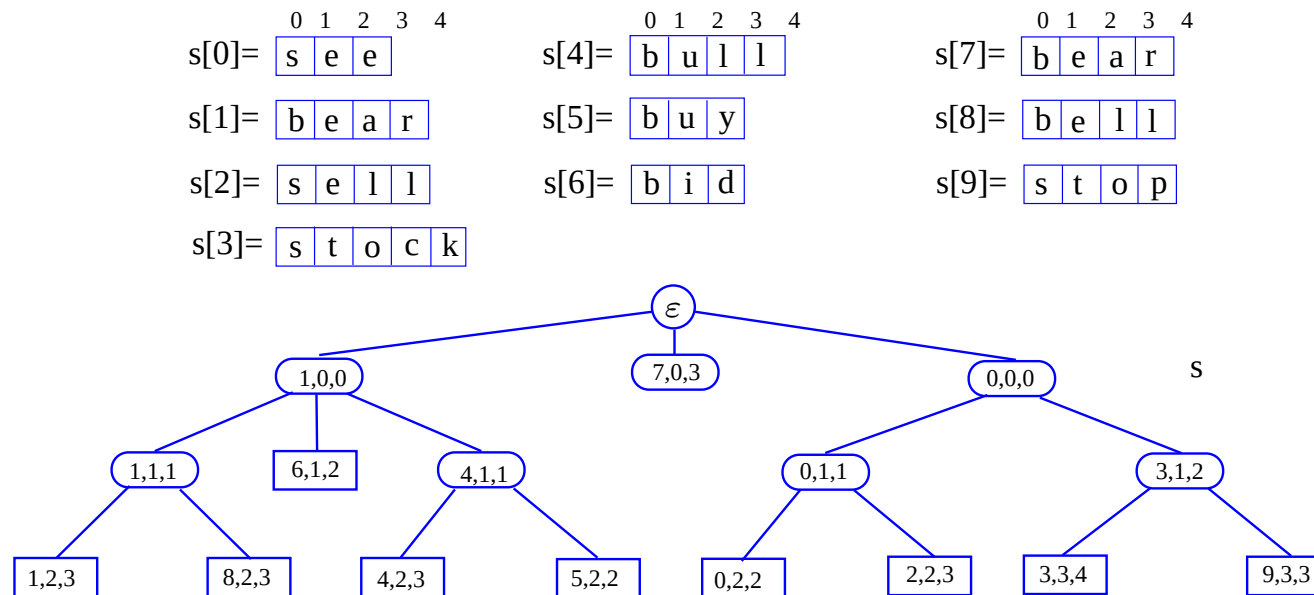
# Compressed tries

- A compressed trie has internal nodes of degree at least two.
- It is obtained from a standard trie by compressing chains of *redundant* nodes.



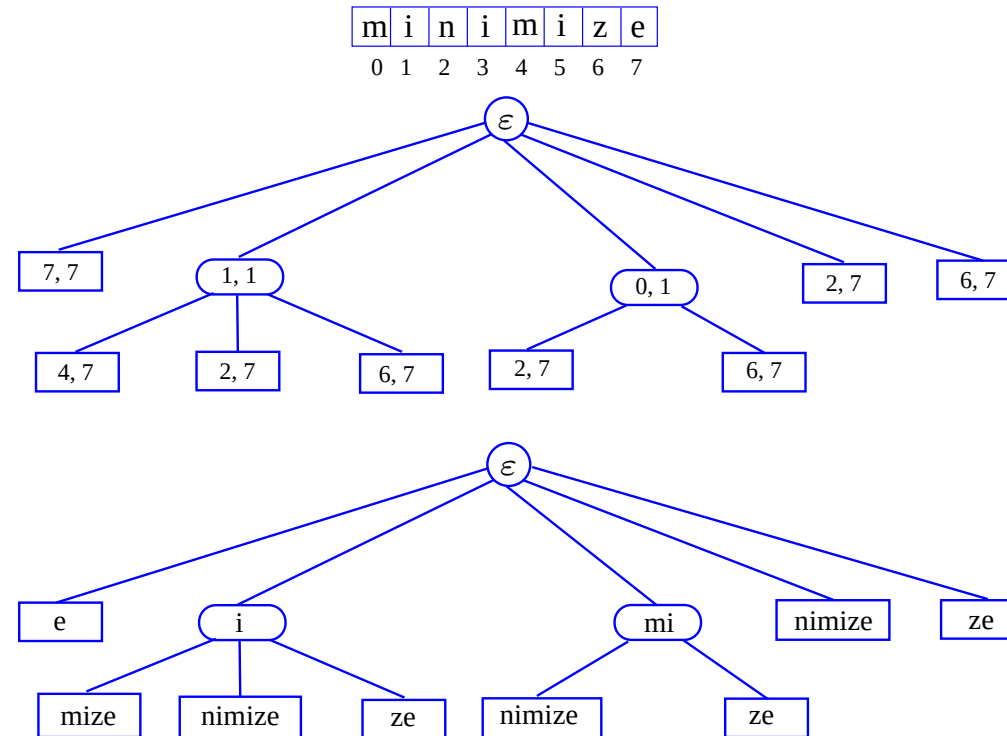
# Compact Representation

- Compact representation of a compressed trie for an array of strings:
- Stores at the nodes ranges of indices instead of substrings
- Uses  $O(s)$  space, where  $s$  is the number of strings in the array
- Serves as an auxiliary index structure



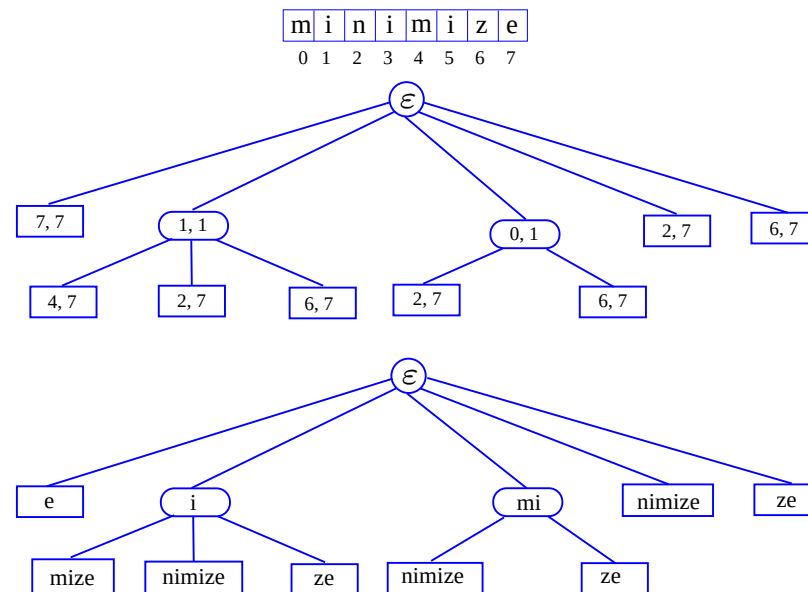
# Suffix trie

- The suffix trie of a string  $T$  is the compressed trie of all the suffixes of  $T$ .



# Analysis of suffix tries

- Compact representation of the suffix trie for a string  $T$  of size  $n$  from an alphabet of size  $d$ .
- Uses  $O(n)$  space.
- Supports arbitrary pattern matching queries in  $T$  in  $O(dm)$  time, where  $m$  is the size of the pattern.
- Can be constructed in  $O(n)$  time.





## Text compression with Huffman's code

---

- Given a text  $T$  reproduce it in a lossless way using less memory.

$T$  is a piece of text in which we count the characters to find their frequencies.

- Huffman scheme counts the number of times characters appear in  $T$ .
- Pseudo code for getting the frequencies of the characters  $T$  in follows:

## Text compression with Huffman's code

---

| Ch |   | a | b | c | d | e | f | h | i | k | n | o | r | s | t | u | v |
|----|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| F  | 9 | 5 | 1 | 3 | 7 | 3 | 1 | 1 | 1 | 1 | 4 | 1 | 5 | 1 | 2 | 1 | 1 |

- By combining pairs of these frequencies, with the lowest first, and then adding their combinations to the frequency list, and removing the originals a binary tree is produced from which an optimal code can be written down.
- The compressed code may then rapidly be produced from the binary tree.

## Huffman code—adapted from GT page 566.

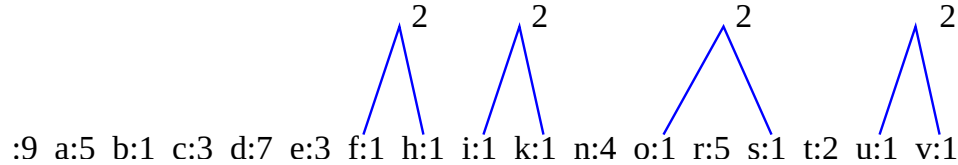
Note that the text I have used differs slightly from that of GT page 566.

|       |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Char. |   | a | b | c | d | e | f | h | i | k | n | o | r | s | t | u | v |
| Freq. | 9 | 5 | 1 | 3 | 7 | 3 | 1 | 1 | 1 | 1 | 4 | 1 | 5 | 1 | 2 | 1 | 1 |

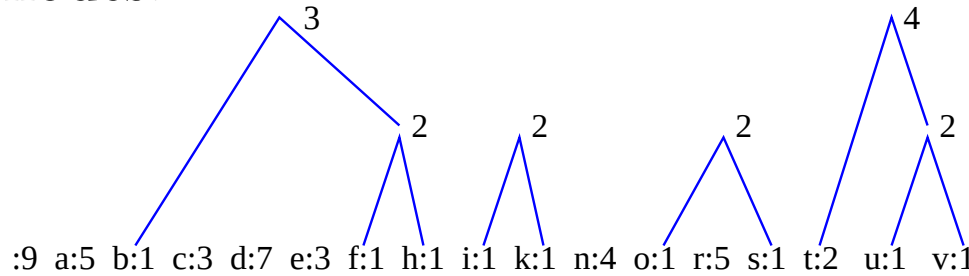
- In order to find the lowest frequency characters first, an obvious method is to: sort the characters by frequency; combine the top pair; remove the top two characters but insert their combination back into the list at its correct position.
- The very first pair with the lowest frequency could appear at the lowest node in the final tree.
- Subsequent combinations fill the tree from the lowest level.
- The last combination is hooked into the root of the tree.

## Huffman coding—combining the lower nodes

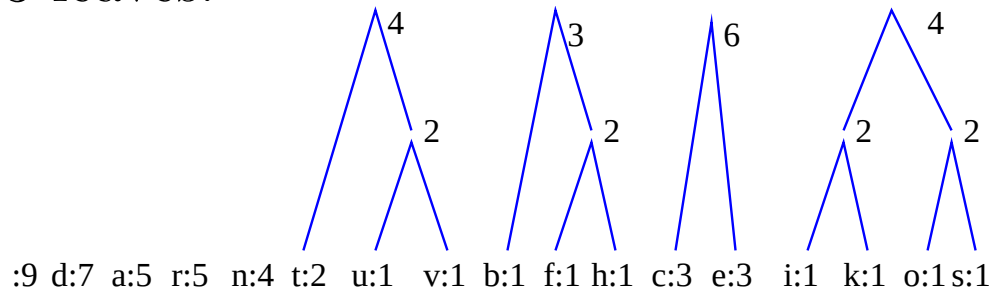
First, combine nodes that are pairs of 1s into 2-nodes.



Then combine the last 1-node with a 2 node, and start combining pairs of 2 nodes.

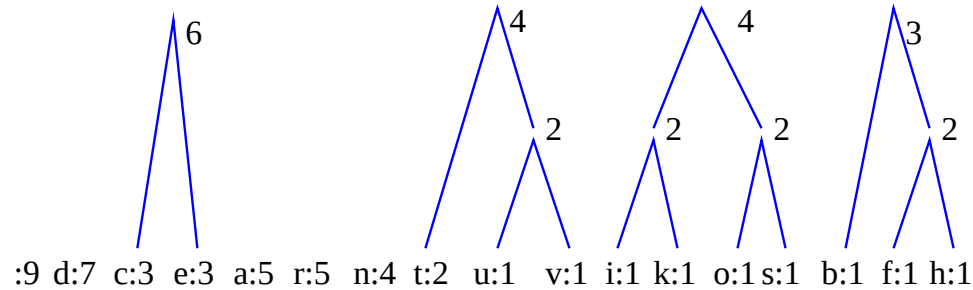


Reorder the list and combine the remaining 2 nodes. Combine the pair of 3-leaves.

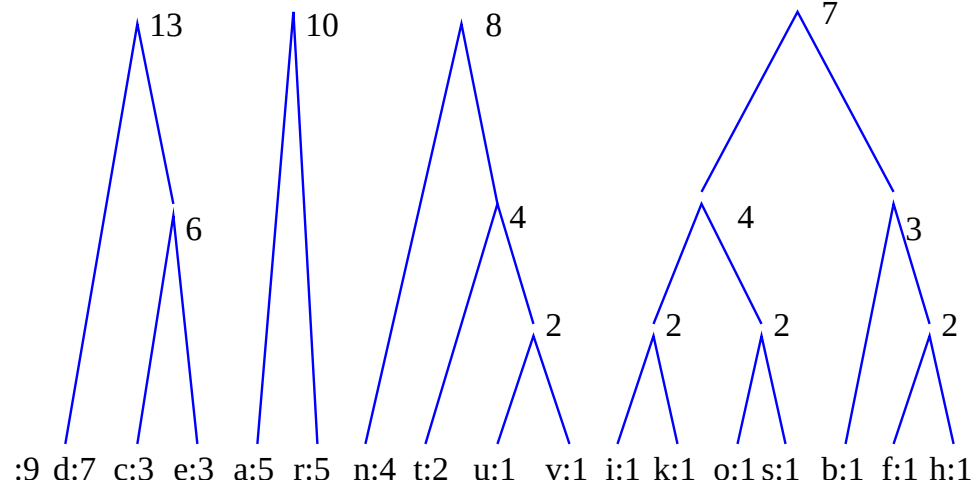


# Huffman coding—reorder and combine

Simply reorder the list.



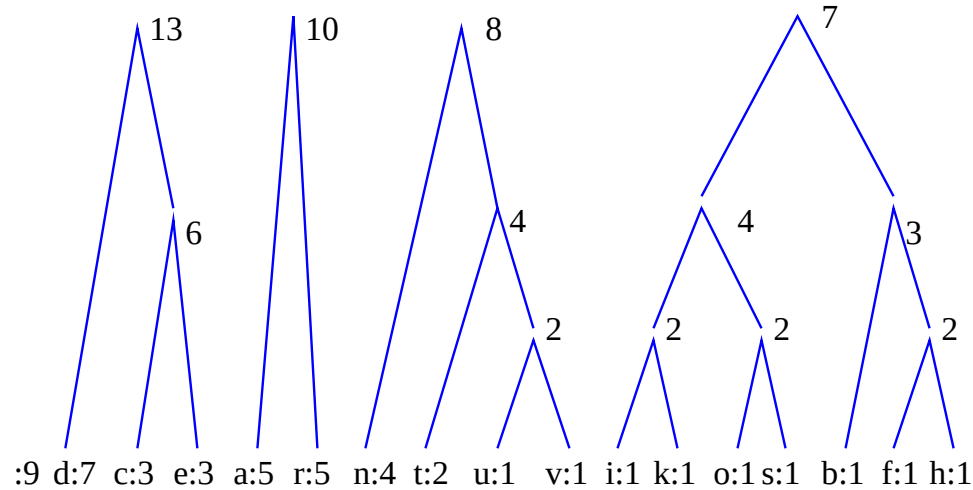
Combine lowest nodes to the next level.



Note that there are only five nodes left to combine.

## Huffman coding—combine the 1s

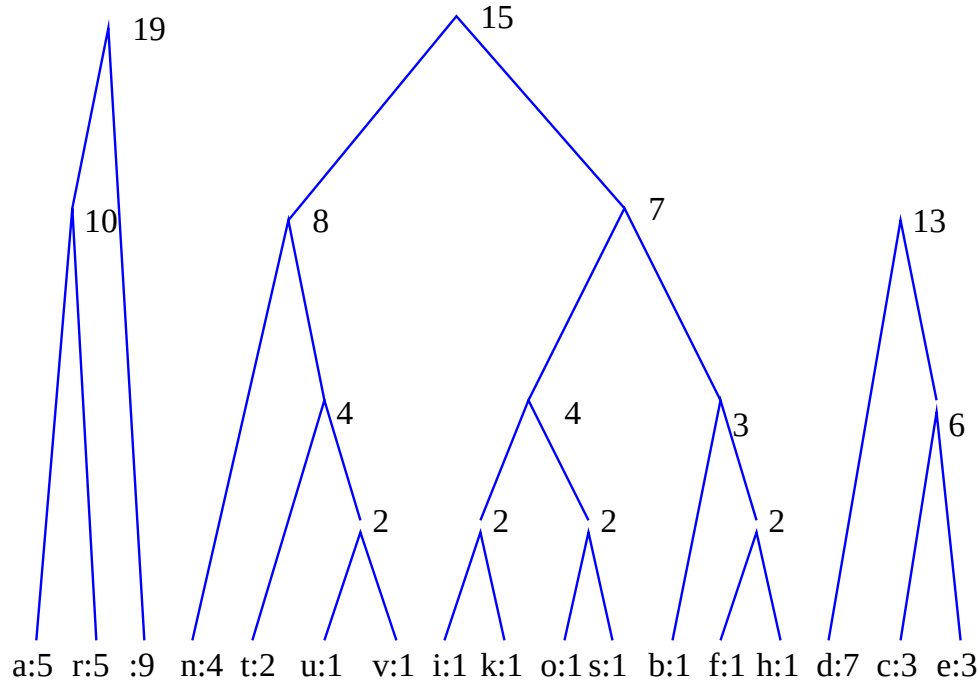
After combining 7 + 8 and 9 + 10, we get:



Next we will rearrange the nodes—high frequencies to the left.

# Huffman coding—rearrange the forest of trees from highest to lowest

The nodes rearranged—high counts to the left, lowering to the right.

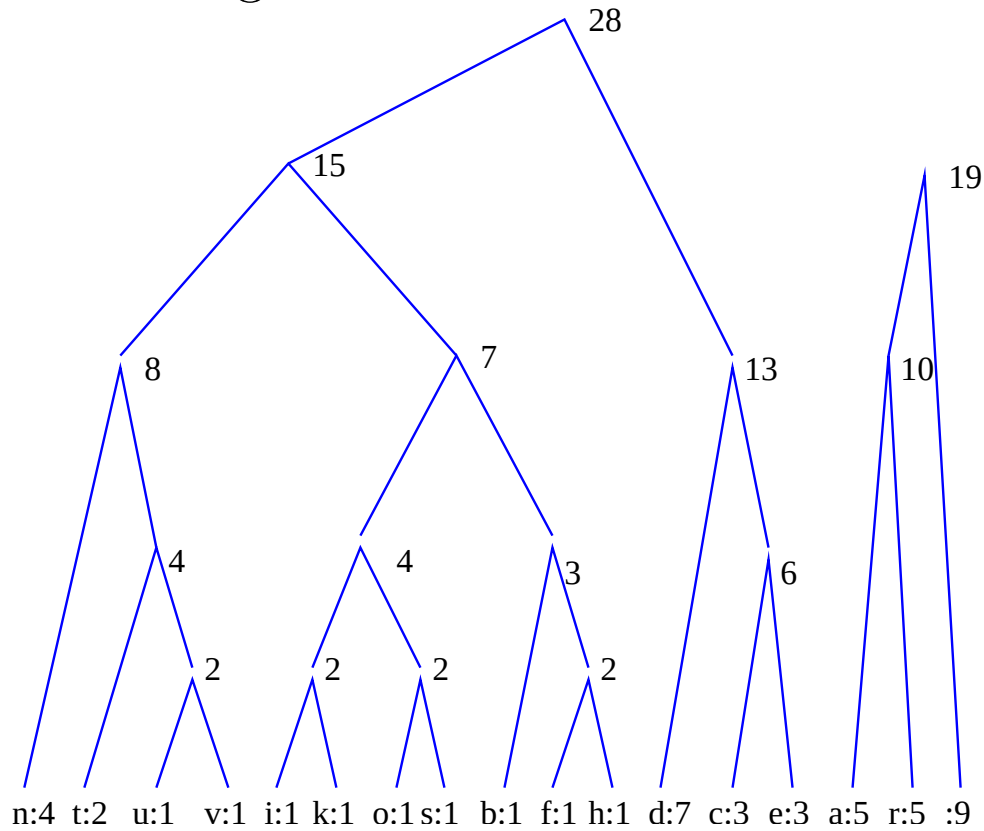


There are only three nodes left.

We will combine the lowest two, resort and again combine the lowest two to create a single rooted tree from the forest.

## Huffman coding—combine and sort

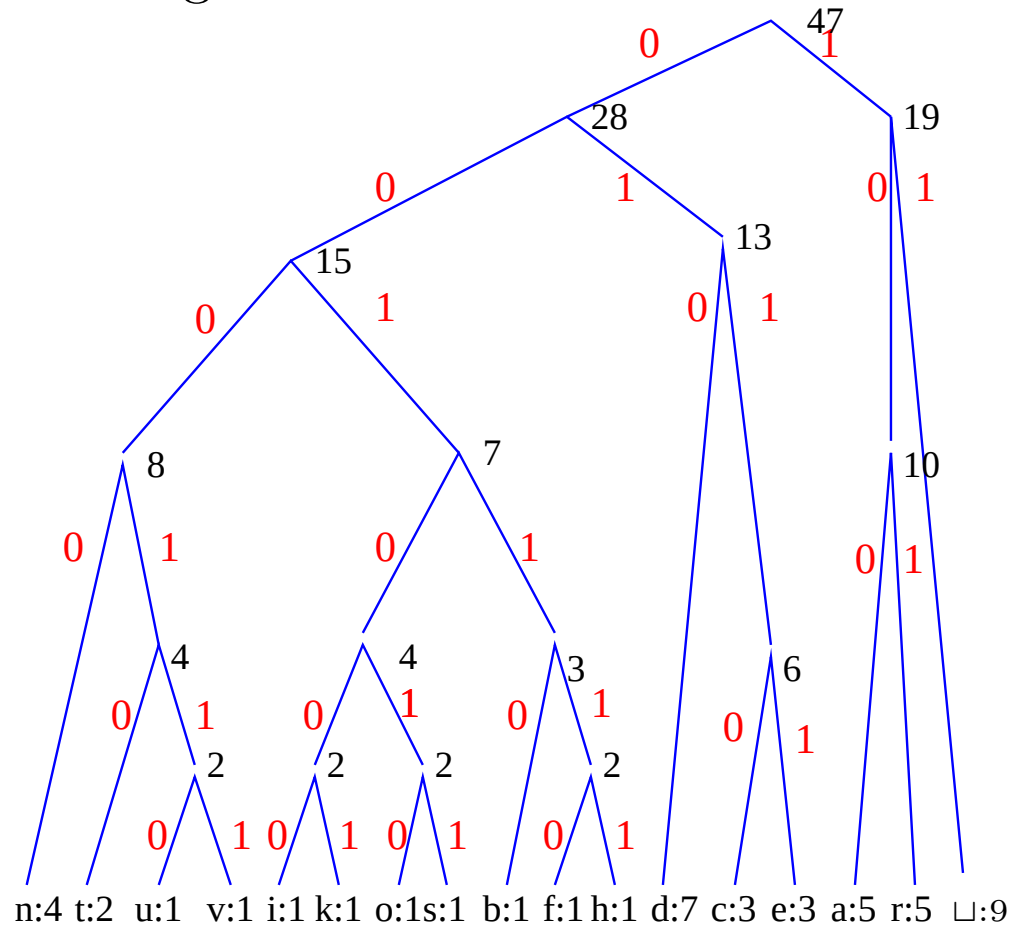
Now combine the 13-tree and the 15-tree into a 28-tree, and resort the remaining forest of two trees.





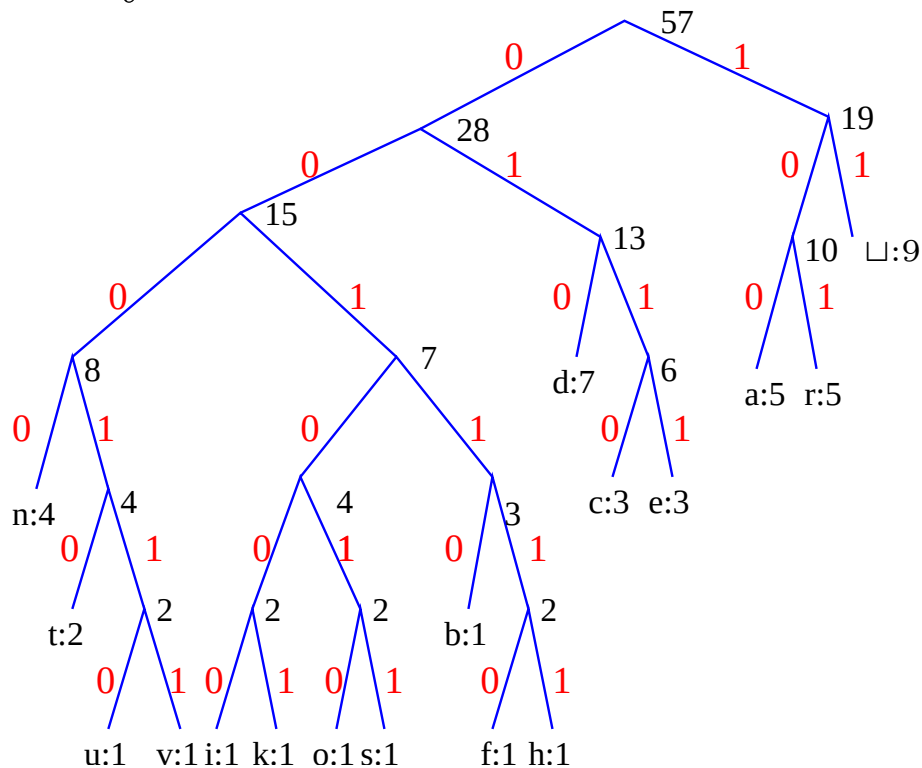
## Huffman code—finally combine and label

Combine the last two trees and label each left branch of the tree with a 0 and each right branch with a 1.



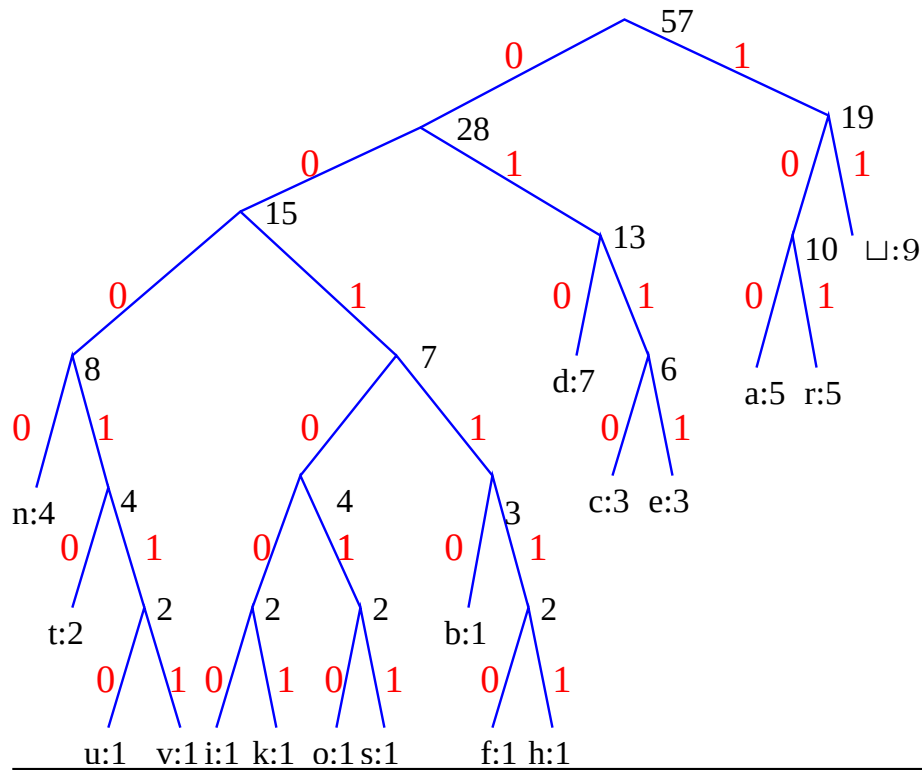
## Huffman code—finally combine and label

We rearrange the tree such that the levels of each character can be seen more easily.



From the figure it follows that the number of digits needed to represent each character is the same as its depth in the Huffman tree.

# Huffman code in tabular form



|       |    |     |       |      |     |      |        |        |        |
|-------|----|-----|-------|------|-----|------|--------|--------|--------|
| Char. |    | a   | b     | c    | d   | e    | f      | h      | i      |
| Freq. | 9  | 5   | 1     | 3    | 7   | 3    | 1      | 1      | 1      |
| Code  | 11 | 100 | 00110 | 0110 | 010 | 0111 | 001110 | 001111 | 001000 |

|       |        |      |        |     |        |       |        |        |
|-------|--------|------|--------|-----|--------|-------|--------|--------|
| Char. | k      | n    | o      | r   | s      | t     | u      | v      |
| Freq. | 1      | 4    | 1      | 5   | 1      | 2     | 1      | 1      |
| Code  | 001001 | 0000 | 001010 | 101 | 001011 | 00010 | 000110 | 000111 |

"a fast runner need never be afraid of the dark" coded is:

1001100111 0100001011 0001011101 0001100000 0000011110  
 1110000011 1011101011 0000011100 0111011110 1110011001  
 1111100001 1101011000 0100001011 0010100011 1011000100

0111101111 1010100101 001001

## Huffman code for T

|       |    |     |       |      |     |      |        |        |        |
|-------|----|-----|-------|------|-----|------|--------|--------|--------|
| Char. | □  | a   | b     | c    | d   | e    | f      | h      | i      |
| Freq. | 9  | 5   | 1     | 3    | 7   | 3    | 1      | 1      | 1      |
| Code  | 11 | 100 | 00110 | 0110 | 010 | 0111 | 001110 | 001111 | 001000 |

|       |        |      |        |     |        |       |        |        |
|-------|--------|------|--------|-----|--------|-------|--------|--------|
| Char. | k      | n    | o      | r   | s      | t     | u      | v      |
| Freq. | 1      | 4    | 1      | 5   | 1      | 2     | 1      | 1      |
| Code  | 001001 | 0000 | 001010 | 101 | 001011 | 00010 | 000110 | 000111 |

"a fast runner need never be afraid of the dark" coded is:

1001100111 0100001011 0001011101 0001100000 0000011110  
1110000011 1011101011 0000011100 0111011110 1110011001  
1111100001 1101011000 0100001011 0010100011 1011000100

0111101111 1010100101 001001

The Huffman code has 176 bits. Since  $|T| = 46$  and it has an alphabet of 16 characters a fixed length code of 4 bits per character is needed to encode this string, i.e.  $46 \times 4 = 184$ .

This is *not* the optimal Huffman code for this string. Why not?

## Huffman code for T

|       |    |     |       |      |     |      |        |        |        |
|-------|----|-----|-------|------|-----|------|--------|--------|--------|
| Char. | □  | a   | b     | c    | d   | e    | f      | h      | i      |
| Freq. | 9  | 5   | 1     | 3    | 7   | 3    | 1      | 1      | 1      |
| Code  | 11 | 100 | 00110 | 0110 | 010 | 0111 | 001110 | 001111 | 001000 |

|       |        |      |        |     |        |       |        |        |
|-------|--------|------|--------|-----|--------|-------|--------|--------|
| Char. | k      | n    | o      | r   | s      | t     | u      | v      |
| Freq. | 1      | 4    | 1      | 5   | 1      | 2     | 1      | 1      |
| Code  | 001001 | 0000 | 001010 | 101 | 001011 | 00010 | 000110 | 000111 |

"a fast runner need never be afraid of the dark" coded is:

1001100111 0100001011 0001011101 0001100000 0000011110  
1110000011 1011101011 0000011100 0111011110 1110011001  
1111100001 1101011000 0100001011 0010100011 1011000100

0111101111 1010100101 001001

The Huffman code has 176 bits. Since  $|T| = 46$  and it has an alphabet of less than 16 characters a fixed length code of 4 bits per character is needed to encode this string, i.e.  $46 \times 4 = 184$ .

This is *not* the optimal Huffman code for this string. Why not?

Since our code caters for some of characters that have different counts from T, e.g. **b**, **c**:3, **d**:7, **e**:3, and **f**:1, than this string has, the code cannot be optimal. H.W. Calculate the optimal Huffman code for T.

## Huffman with sorting and insertion into the sorted list

---

- Sorting the  $m$  characters of the text according to their frequencies may be done in  $O(m \log m)$  time.
- The position of each insertion is found in  $O(\log m)$  time. How? But there are this does not really help because insertions entail moving the rest of the list down—on average  $m/2$  such items are shifted. So this part takes at worst  $O(m)$  time for each of  $m$  entries, i.e. it takes  $O(m^2)$  time.
- So the Huffman tree is built in  $O(m \log m) + m^2$  which yields  $O(m^2)$  time using the sort-and-insert method.
- This time can be improved.

## Huffman code using a heap

---

- Instead of repeated sorting and insertion into the sorted list, a heap should rather be used.
- The  $\langle \text{character(s)}, \text{frequency} \rangle$  entries are placed into a heap with the entries with the lowest frequency appearing on the top of the heap.
- The top two entries are removed from the heap and combined into a new entry.
- The new entry is then inserted into the heap at its appropriate position.
- Building the heap *bottom-up* takes  $O(m)$  time. Doing  $m$  insertions into a heap takes  $O(m \log m)$  time.
- The total cost of constructing the Huffman tree is thus  $m \log m$ .

## Huffman code explained

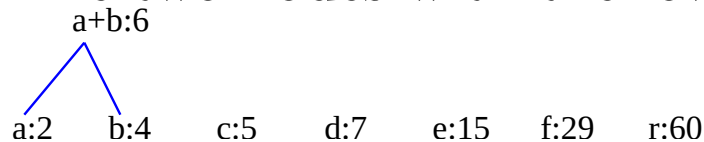
Consider the set  $\{a, b, c, d, e, f, r\}$  with the frequencies and probabilities for each letter:

| Character   | a     | b     | c     | d     | e     | f     | r     | All   |
|-------------|-------|-------|-------|-------|-------|-------|-------|-------|
| Frequency   | 2     | 4     | 5     | 7     | 15    | 29    | 60    | 122   |
| Probability | 0.016 | 0.033 | 0.041 | 0.057 | 0.123 | 0.238 | 0.492 | 1.000 |

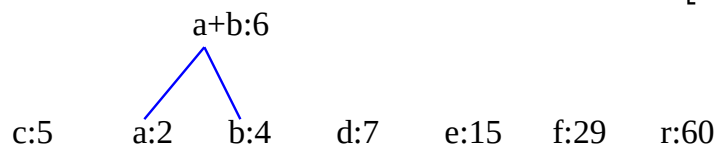
The total of the frequencies is 122.

The letters are sorted according to counts or probability—smallest first. The probability is frequency divided by the total frequency.

The two nodes with the lowest counts are combined:



Insert the combined node  $[a+b : 6]$  after node  $[c : 5]$  and before node  $[d : 7]$ .



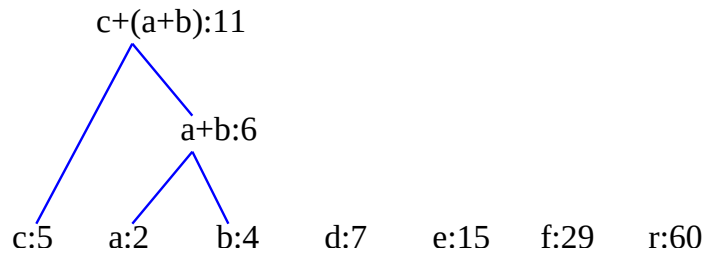


# Huffman coding

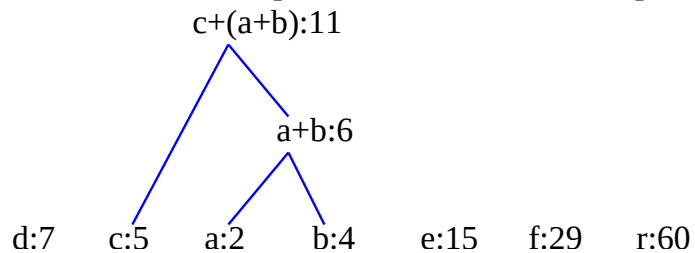
Starting with



form node  $[c+(a+b) : 11]$  by combining combine nodes  $[c : 5]$  and node  $[a+b : 6]$ :

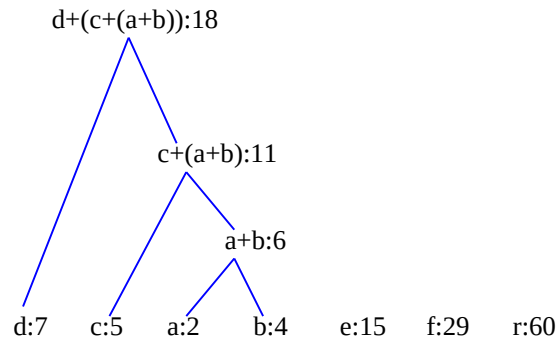


Put node  $[c+(a+b) : 11]$  in its correct place:

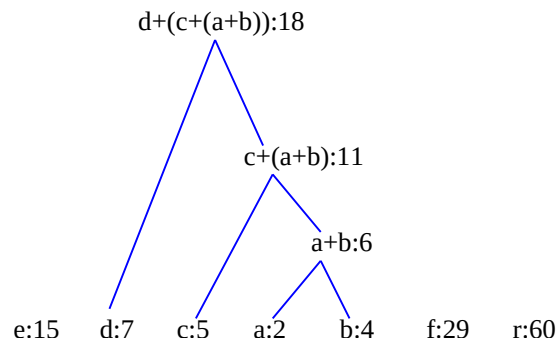


# Huffman coding

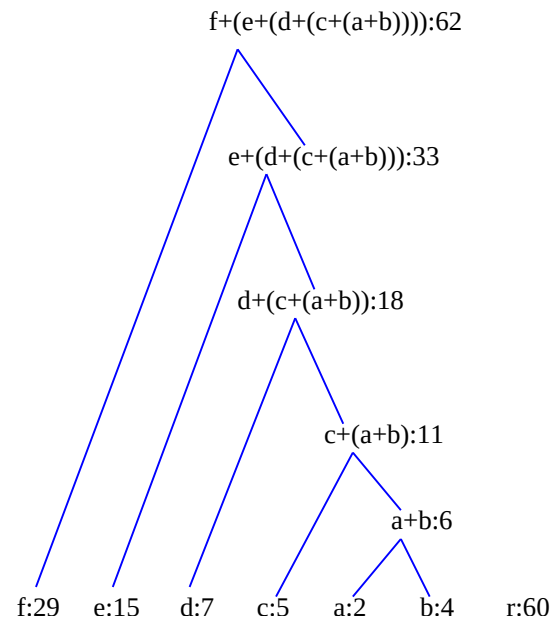
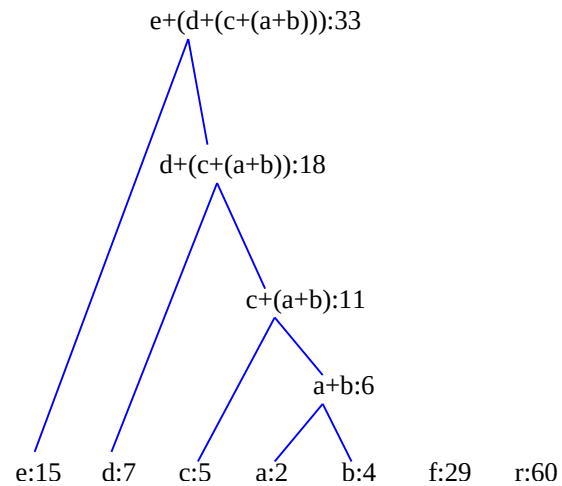
Form node  $[d+(c+(a+b)) : 18]$  by combining combine nodes  $[d : 7]$  and node  $[c+(a+b) : 11]$ :



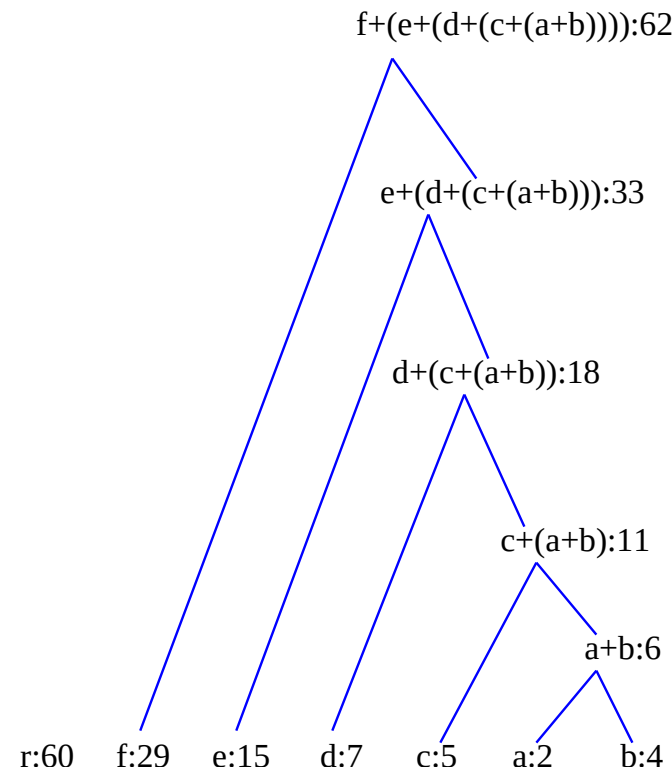
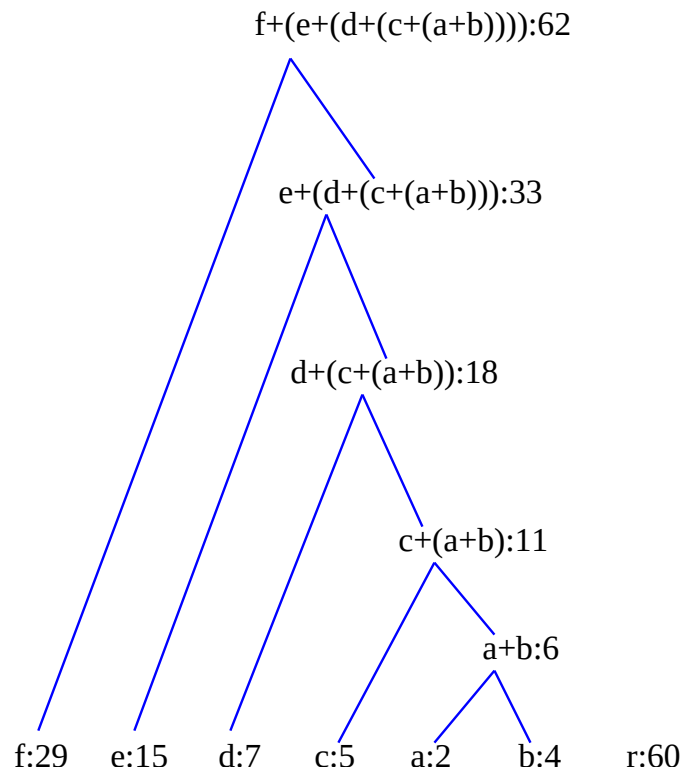
Put node  $[d+(c+(a+b)) : 18]$  in its correct place:



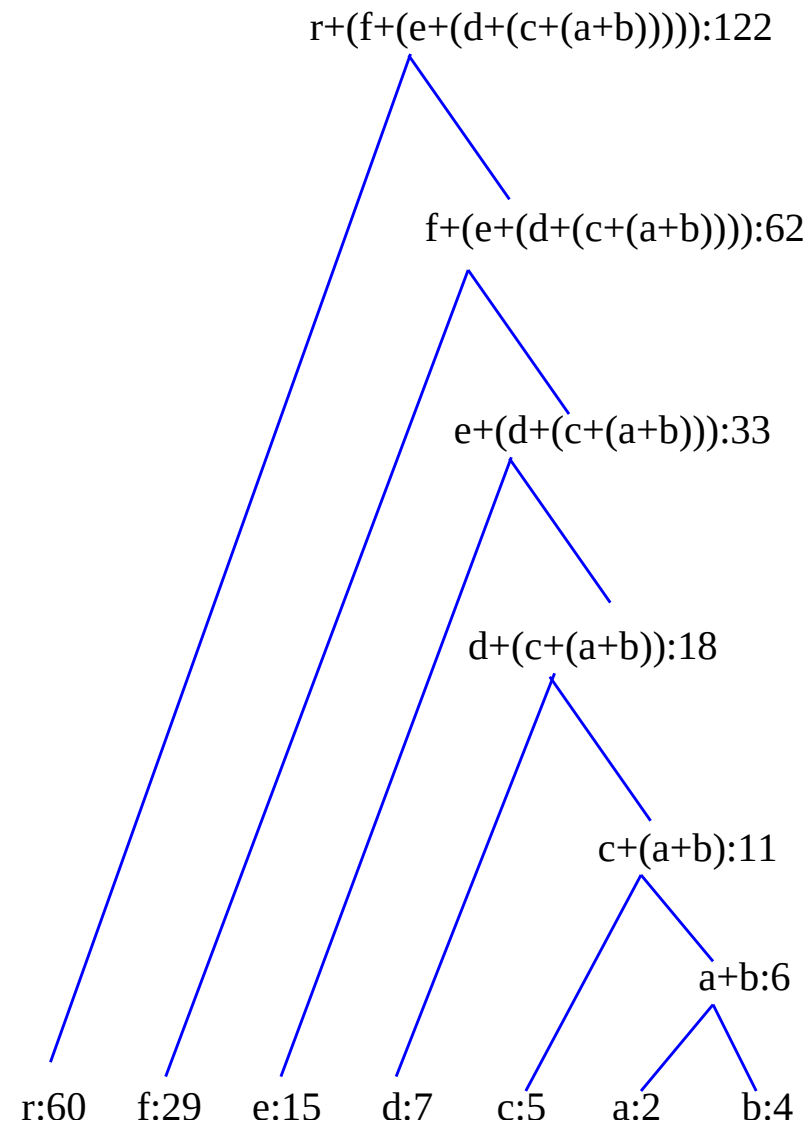
# Huffman coding—repeat steps



# Huffman coding—repeat steps

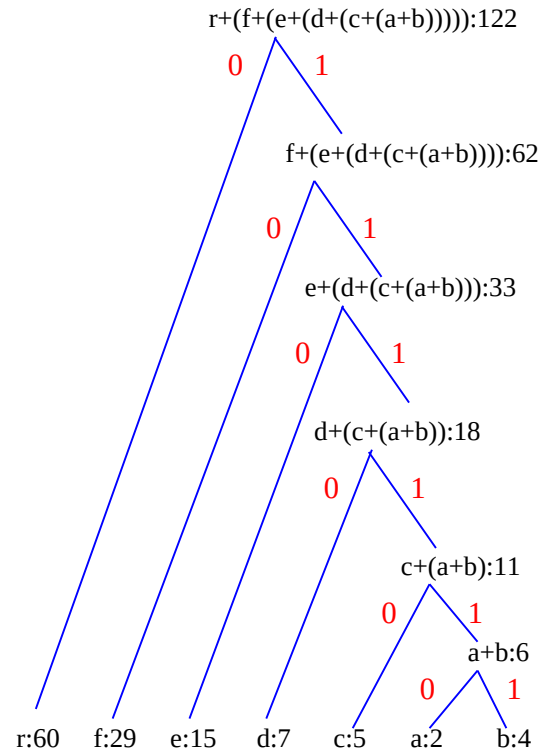


# Huffman coding—last combination



# Huffman coding—enter coding

Tag each left leg of the tree with a **0** and each right leg with a **1**.



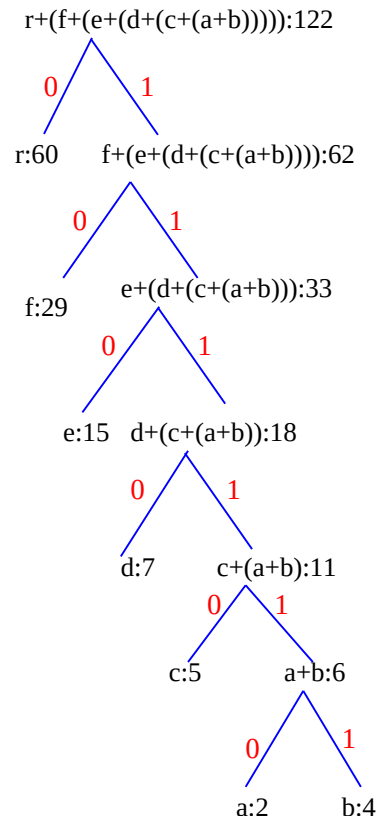
Read off the codes for each letter in the tree:

| Character   | a      | b      | c     | d     | e     | f     | r     |
|-------------|--------|--------|-------|-------|-------|-------|-------|
| Frequency   | 2      | 4      | 5     | 7     | 15    | 29    | 60    |
| Probability | 0.016  | 0.033  | 0.041 | 0.057 | 0.123 | 0.238 | 0.492 |
| Code        | 111110 | 111111 | 11110 | 1110  | 110   | 10    | 0     |

What does **1111110111110111101101110** represent?

# Huffman coding—enter coding

A Huffman tree from which the levels of each character may be seen.



The code in tabular form.

| Character   | a      | b      | c     | d     | e     | f     | r     |
|-------------|--------|--------|-------|-------|-------|-------|-------|
| Frequency   | 2      | 4      | 5     | 7     | 15    | 29    | 60    |
| Probability | 0.016  | 0.033  | 0.041 | 0.057 | 0.123 | 0.238 | 0.492 |
| Code        | 111110 | 111111 | 11110 | 1110  | 110   | 10    | 0     |

## Divide-and-conquer methods

- In general, divide-and-conquer methods *divide* a problem with a set of  $N$  inputs  $X_i$  where  $i \in [0..N)$  into  $k$  equal or almost equal parts.
- Each of these parts is then in turn divide-and-conquer-ed.
- Finally the results are combined.

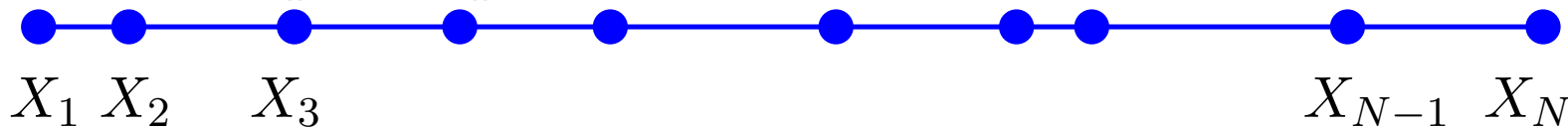
```
1 def divide_and_conquer(X, low, high):
2     if high - low < small:
3         quickly resolve problem for small  $N$  in  $O(1)$  time
4         return result
5     else:
6         divide  $X$  into smaller parts  $P_1, P_2, \dots, P_k, k \geq 1$ 
7         return combine(divide_and_conquer( $P_1$ ),
8                           divide_and_conquer( $P_2$ ),
9                           ...,
10                          divide_and_conquer( $P_k$ ))
```

- The resulting timing depends largely on the time taken by **combine**.
- If  $T(\text{combine}) = O(N)$ ,  
$$T(N) = kT(N/k) + O(N) = O(N \log_k N)$$



## An $O(N^2)$ method for 1-D closest pair

Given a set of  $N$  points in a straight line—use the  $X$ -axis for simplicity—find the closest pair of points.

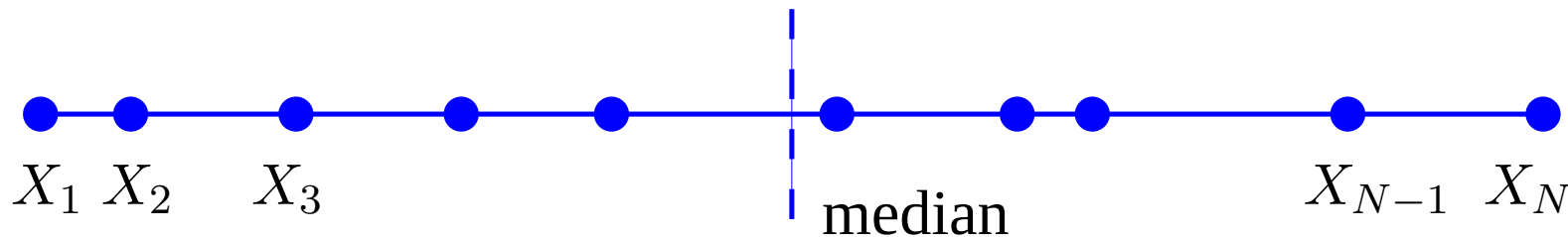


- The problem can be solved brute force by running through all the pairs in  $O(N^2)$  time as follows

```
1  shortestDistance = |X[1] - X[2]|
2  indexJ = 1
3  indexK = 2
4  for j in [1..N-1]:
5      for k in [j+1..N]:
6          difference = |X[j] - X[k]|
7          if difference < shortestDistance:
8              shortestDistance = difference
9              indexJ = j
10             indexK = k
```

## An $O(N \log N)$ divide-and-conquer method for 1-D closest pair

Given a set of  $N$  points on the  $X$ -axis for find the closest pair of points.



The problem can be solved in  $O(N \log N)$  time using divide-and-conquer.

```
1 def closestPairs(X, low, high):
2     if high - low == 2: return |X[high] - X[low]|
3     elif high == low: return 1.79768e+308
4     else:
5         m = median(X, low, high)
6         L = closestPairs(X, low, m)
7         R = closestPairs(X, m+1, high)
8          $\delta = \min(L, R)$ 
9         straddle = minimum (X, m- $\delta$ , m+ $\delta$ )
10    return min(L, R, straddle)
```

It takes time  $T(N) = 2T(N/2) + O(N)$ , i.e.,  
 $T(N) = O(N \log N)$ .

## The longest maximum subvector problem

In a list or vector  $X$  of  $N$  integers find the maximum sum of any contiguous subvector.<sup>13</sup>

|     |     |    |    |    |   |     |    |   |     |
|-----|-----|----|----|----|---|-----|----|---|-----|
| -66 | -15 | 87 | 91 | 32 | 5 | -47 | 47 | 4 | -77 |
| 0   | 1   | 2  | 3  | 4  | 5 | 6   | 7  | 8 | 9   |

In the list  $X$  above the subvector  $X[2..8]$  has the maximum sum 219.

Clearly if all the numbers are negative then the maximum contiguous subvector (MCS) is simply the largest number. In this case we define the MCS as 0.

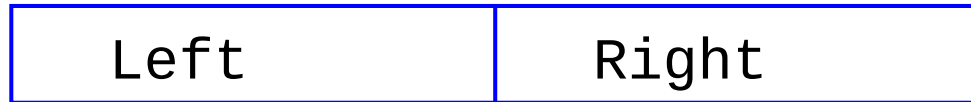
We know how to solve this problem in  $O(N)$  time but we are now interested in using divide-and-conquer to solve it in the slower time of  $O(N \log N)$

---

<sup>13</sup>This example comes from pages 77–81, “Programming Pearls”, Jon Bentley, ACM Press and Addison-Wesley, New York, 2000.

## A divide-and-conquer method for MCS

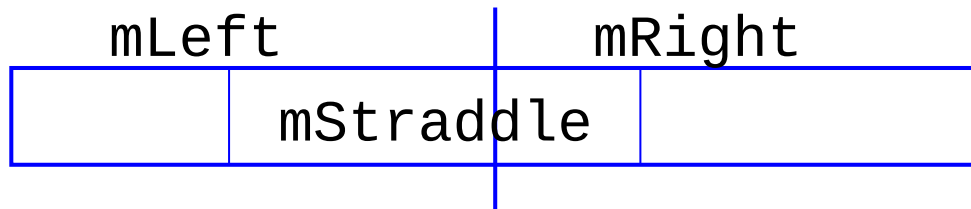
First, we divide the given array  $X[ ]$  into two parts, **Left** and **Right**:



and then find the MCS **mLeft** in **Left** and the MCS **mRight** in **Right**.



There is still a snag—the MCS might straddle both **Left** and **Right**.



The MCS is the subvector that gives the biggest of these three values.

The final algorithm is (1)  $\text{middle} = N/2$ ,

(2) Get MCS, **mLeft** of  $X[\text{low}..\text{middle}]$ , using

$\text{mLeft} = \text{MCS}(X, \text{low}, \text{middle})$ , and get the MCS, **mRight** of  $X[\text{middle}+1..N]$ , using  $\text{mRight} = \text{MCS}(X, \text{middle}+1, N)$ .

More difficult is getting the straddling MCS, **mStraddle**.

## Calculating the straddling MCS

The idea of the straddling MCS is first to calculate the MCS, called ‘maxsofarLeft’ that ends at  $X[\text{middle}]$  but starts somewhere in Left—running backwards, and also calculating the MCS called ‘maxsofarRight’ that starts at  $X[\text{middle}+1]$  and ends somewhere in Right. The sum of these two values is mStraddle.

```
1 int maxStraddle(low, high)
2     middle = (low + high) / 2
3     sum = 0; maxsofarLeft = 0
4     for i in [low..middle]
5         sum += X[middle - i + 1]
6         if sum > maxsofarLeft
7             maxsofarLeft = sum
8     sum = 0; maxsofarRight = 0
9     for i in [middle+1..high]
10        sum += X[i]
11        if sum > maxsofarRight
12            maxsofarRight = sum
13    return maxsofarLeft + maxsofarRight
```

## A divide-and-conquer method for MCS

```
1 int MCS(low, high)
2     if low > high
3         return 0
4     if low == high
5         return max(0, X[low])
6     middle = (low + high) / 2
7     mLeft = MCS(X, low, middle)
8     mRight = MCS(X, middle+1, high)
9     mStraddle = maxStraddle (X, low, high)
10    return max (mLeft, mRight, mStraddle)
```

Suppose ‘MCS’ takes time  $T(N)$  for an input of  $N$ . Then ‘mLeft’ takes time  $T(N/2)$  and ‘mRight’ takes time  $T(N/2)$ . ‘maxStraddle( )’ takes  $O(N)$  because, at worst, it scans the entire input once. We conclude that

$$\begin{aligned} T(N) &= 2T(N/2) + O(N) \\ &= O(N \log N) \end{aligned}$$

## Divide-and-conquer methods— $O(N \log N)$

---

Suppose the divide-and-conquer method has time complexity  $T(N)$  when it has an input of size  $N$ .

Divide-and-conquer methods entail:

- (1) *dividing* the problem into equal parts,
- (2) *solving each part* of the problem separately, and
- (3) *joining* the results somehow.

1. *Dividing* the problem in two, has the low cost of  $O(1)$ . All that needs to be done is  $N = N/2$ .
2. *Solving each half* of the problem separately costs  $2T(N/2)$ .
3. *Joining the two results* is the critical part. Try to make this depend *linearly* on  $N$ . If it is sublinear all the better. Usually this part entails looking at every input once. In mergesort the two sorted halves are merged in time  $cN$ , making the cost of merge sort,  $T(N) = 2T(N/2) + cN$ .
4. In MCS straddle passes through all  $N$  elements once, giving the same formula.

## Divide-and-conquer methods— $O(N \log N)$ —I

---

Solving the recursive equation for merge sort and MCS

$$T(N) = 2T(N/2) + cN.$$

is quite easy.

$$\begin{aligned} T(N) &= 2T(N/2) + cN \\ &= 2[2T(N/4) + cN/2] + cN \\ &= 2^2T(N/2^2) + cN + cN \\ &= 2^2[2T(N/2^3) + cN/2^2] + cN + cN \\ &= 2^3T(N/2^3) + cN + cN + cN \\ &= 2^3T(N/2^3) + 3cN \\ &\vdots \\ &= 2^kT(N/2^k) + kcN \end{aligned}$$

Continue until  $N/2^k = 1$ .



## Divide-and-conquer methods— $O(N \log N)$ —II

---

$$\begin{aligned} T(N) &= 2T(N/2) + cN \\ &\vdots \\ &= 2^k T(N/2^k) + kcN \end{aligned}$$

Continue until  $N/2^k = 1$ .

Since there is nothing to do when the input is of size 1, it follows that  $T(1) = 0$ , so we set  $N/2^k = 1$  giving  $N = 2^k$ . This happens when  $k = \log_2 N$  and

$$\begin{aligned} T(N) &= T(1) + kcN \\ &= kcN \\ &= cN \log_2 N \\ &= O(N \log N) \end{aligned}$$

## An $O(n)$ method for MCS( $X$ , low, high)

This method starts at  $X[\text{low}]$  and scans through all the elements up to  $X[\text{high}]$  keeping book of the maximum contiguous subvector found so far.

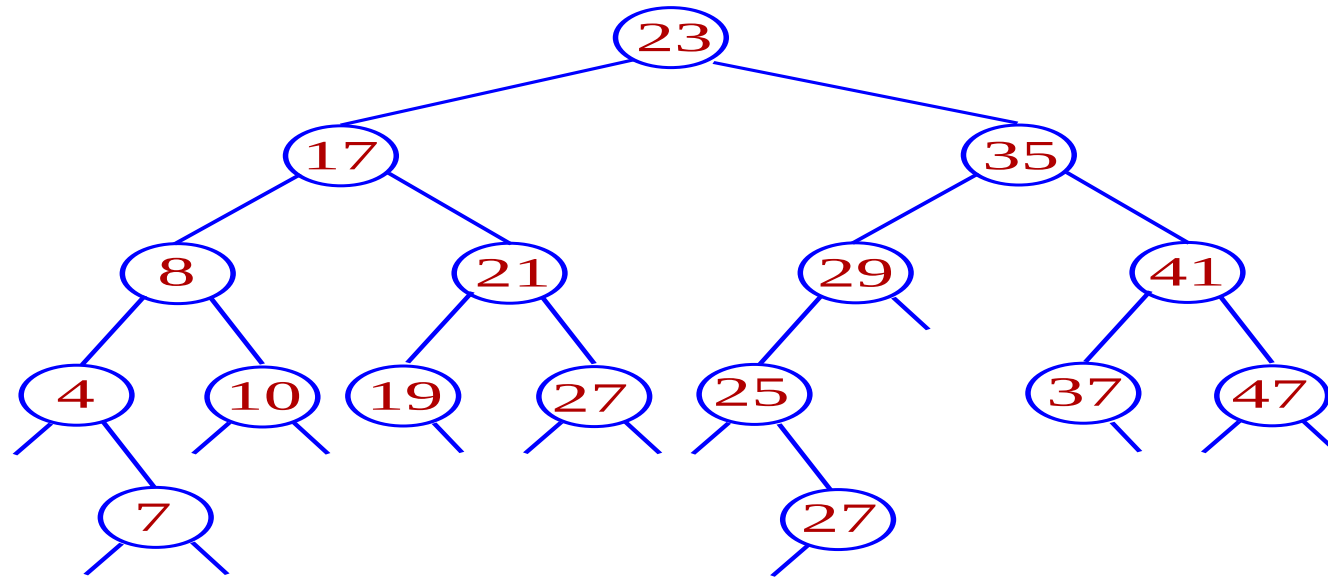


The maximum sum in the first  $i$  elements is either the maximum in the first  $i - 1$  elements, called 'maxSoFar' or that of the subvector that ends in  $X[i]$ , called 'maxToHere'.

'maxToHere' contains the biggest sum *ending* at  $X[i - 1]$ . Considering item  $X[i]$  take the biggest of  $\text{maxToHere} + X[i]$  and 0.

```
1 int MCS(low, high)
2     maxSoFar = 0.0
3     maxToHere = 0.0
4     for i in [low, high]
5         maxToHere = max(maxToHere + X[i], 0.0)
6         maxSoFar = max(maxSoFar, maxToHere)
7     return maxSoFar
```

## Traversal of trees



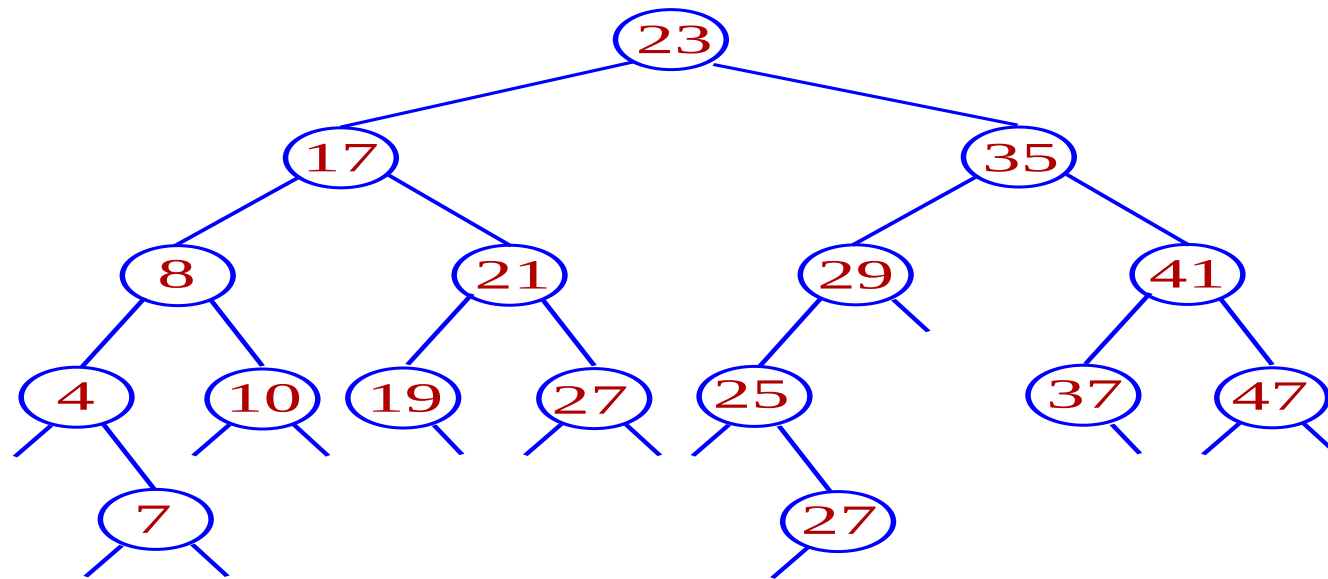
Searching for a node, given its key, in a *balanced* binary search tree (BST), tends to be logarithmic— $O(\log n)$ .

**L** indicates a move to the **Left** subtree. **R** moves to the **Right** subtree and **N** denotes a visit to the **Node** itself.

The **LNR**—in-order—traversal of the above tree is:

4, 7, 8, 10, 17, 19, 21, 22, 23, 25, 27, 29, 35, 37, 41, 47.

## Traversal of trees



The 6 standard orders of traversal when searching for a node in such trees are: **NLR**—pre-order, **LNR**—in-order, **LRN**—end-order or post-order, **NRL**, **RNL** and **RLN**.

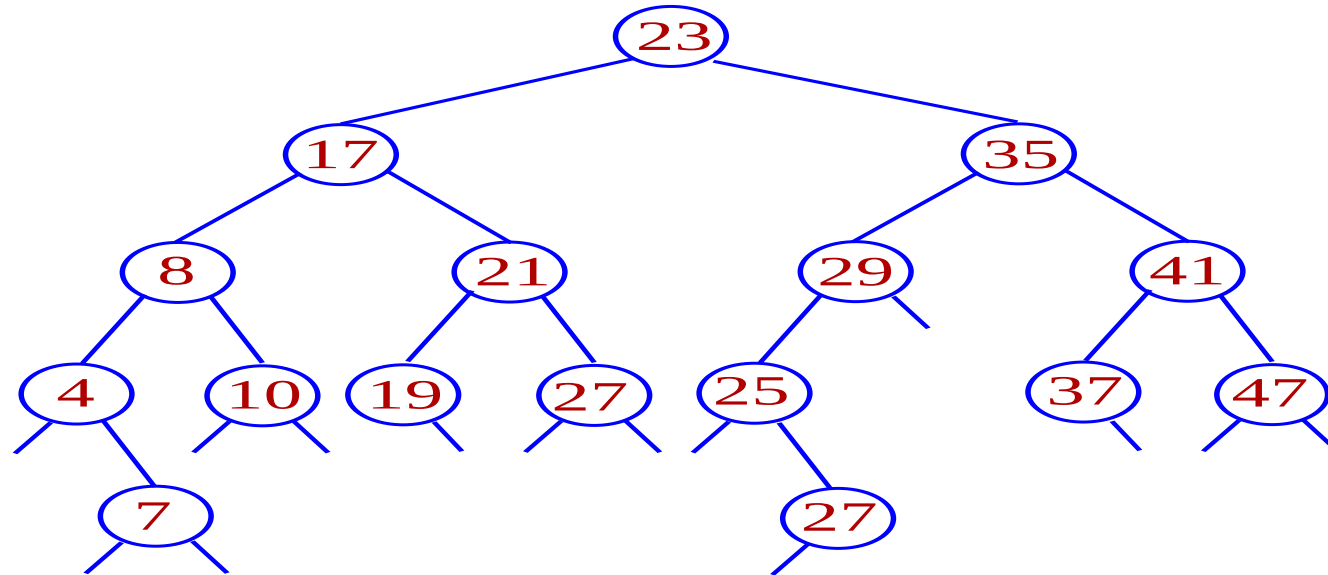
The **LNR**—in-order—traversal of the above tree is:

4, 7, 8, 10, 17, 19, 21, 22, 23, 25, 27, 29, 35, 37, 41, 47.

Which order will visit all the nodes in the opposite order?

## Traversal of trees—LRN

The following tree *is* a BST.



The **LNR**—in-order traversal of the above tree is:

4, 7, 8, 10, 17, 19, 21, 22, 23, 25, 27, 29, 35, 37, 41, 47.

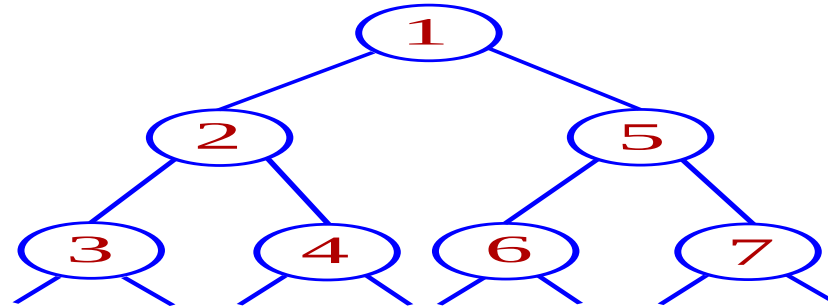
Which ordering traverses the keys from the largest to the smallest?

What order does **LRN** yield?

What order does **NLR** yield?

## Tree traversal—examples using a non BST

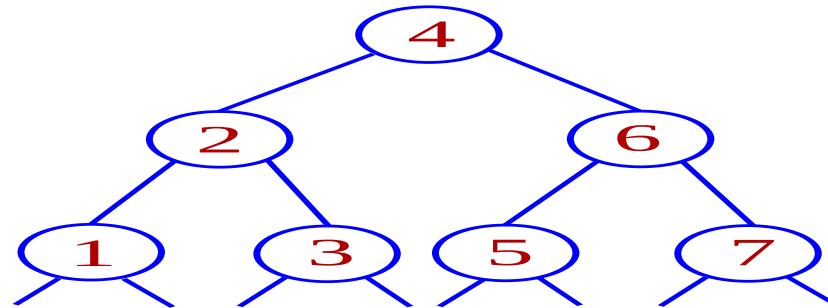
The following tree *is not* a BST.



- The **LNR**—in-order traversal of the above tree is:  
3, 2, 4, 1, 6, 5, 7.
- The **RNL**—RL-in-order traversal of the above tree:  
7, 5, 6, 1, 4, 2, 3.
- The **NLR**—pre-order traversal of the above tree:  
1, 2, 3, 4, 5, 6, 7.
- The **RLN**—RL-end-order or post-order traversal of the above tree is:  
7, 6, 5, 1, 4, 3, 2.

## Traversal of trees—examples using a BST

This tree *is* a BST.



- The **LNR**—in-order traversal of the above tree is:  
1, 2, 3, 4, 5, 6, 7.
- The **RNL**—RL-in-order traversal of the above tree:  
7, 6, 5, 4, 3, 2, 1.
- The **NLR**—pre-order traversal of the above tree is:  
4, 2, 1, 3, 6, 5, 7.
- The **LRN**—end-order or post-order traversal of the above tree is:  
1, 3, 2, 5, 7, 6, 4.

## Tree traversal—breadth-first and depth-first

- Another approach is to order the nodes according methods that lump the children of a node together.

```
TRAVERSETREE
```

```
insert the root node into list.
```

```
do
```

```
    visit and remove the head of list:
```

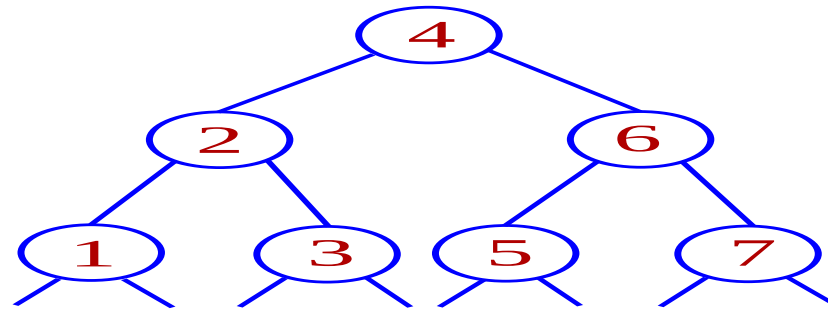
```
    visit by inserting head's children in the list.
```

```
    repeat until the list is depleted.
```

- The list uses a
  1. **stack** for **depth-first** traversal.
  2. **queue** for **breadth-first** order.



## Tree traversal—depth-first



### DEPTHFIRSTTREETRAVERSAL

*push* the root node onto stack.

**do**

*visit* and *remove* the top of the stack:

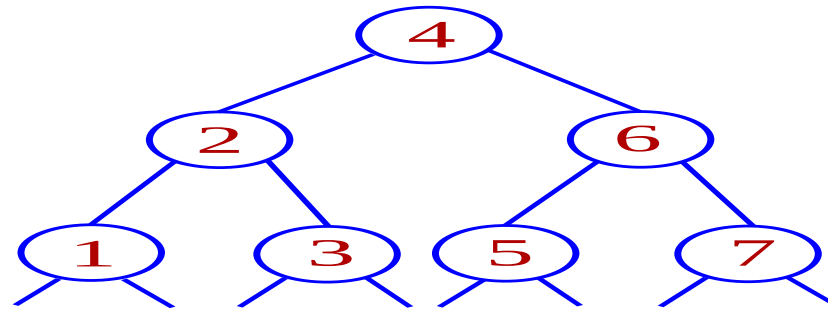
*visit* by pushing top's children onto the stack.

**repeat** until the stack is empty.

The **depth-first** traversal of the above tree:

4, 2, 1, 3, 6, 5, 7.

## Tree traversal—breadth-first



Using a **queue** instead of a *stack* turns the algorithm into **breadth-first traversal**.

BREADTHFIRSTTREETRAVERSAL

*add* the root node into the queue.

**do**

*visit* and *remove* the front of the queue:

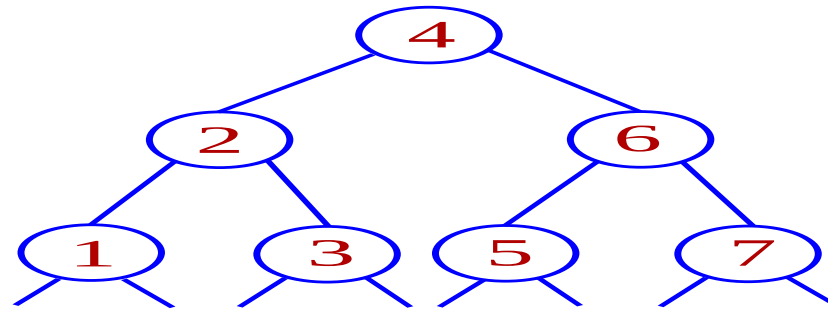
*visit* by adding front's children into the queue.

**repeat** until the queue is empty.

The **breadth-first** traversal of the above tree:

4, 2, 6, 1, 3, 5, 7.

## Tree traversal—depth-first recursively



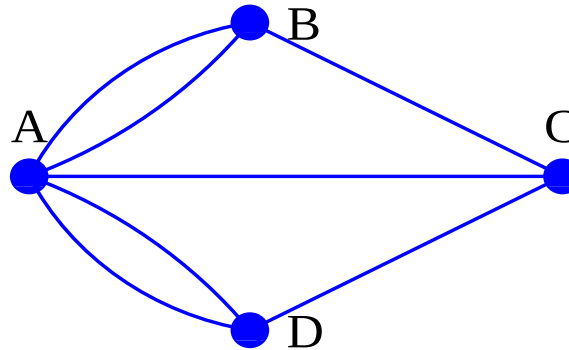
DEPTHFIRSTTRAVERSAL

```
void DFS (tree  $T$ , node  $v$ ) {  
    mark the node  $v$ .  
    call prelude( $v$ );  
    for all edges  $\{(v, z) | z \in \text{childOf}(v)\}$   
        if ( $z$  is unmarked) DFS ( $T$ ,  $z$ );  
        call postlude( $v, z$ );}
```

The **depth-first** traversal of the above tree:  
4, 2, 1, 3, 6, 5, 7.

## Graphs—edges, nodes, degree

A **graph**  $G = (V, E)$  is a set  $V$  of **vertices** or **nodes** and a set  $E$  of **edges**. Each edge connects two vertices. A **self-loop** connects a node to itself.



The edges represent the bridges and the nodes represent land in Euler's Königsberg bridges problem:

Is there a path where each bridge is traversed exactly once?

Edges may be **directed** or **undirected**.

Each vertex has a **degree**,  $d(v)$  is number of edges incident to  $v$ .

$d(A) = 5$  and  $d(B) = 3$ .

In a directed graph the **outdegree** is the number of edges that leave the node, and **indegree** is the number of vertices that enter the node.

## Graphs—path, cycle, reachable, tree

---

A **path** from  $n_1$  to  $n_k$  is a sequence of nodes  $n_1, n_2, \dots, n_k$  that are connected by edges

$$(n_1, n_2), (n_2, n_3), \dots, (n_{k-1}, n_k).$$

In a **simple path** each node appears only *once* and  $u$  is **reachable** from  $v$  if there is a path from  $u$  to  $v$ .

A **circuit** or **cycle** is a path whose first and last vertices are the same.

The **undirected form** of a directed graph  $G = (V, E)$  is the same graph without directions on the edges.

A graph in its undirected form is called **connected** if there is a path from any node to any other node.

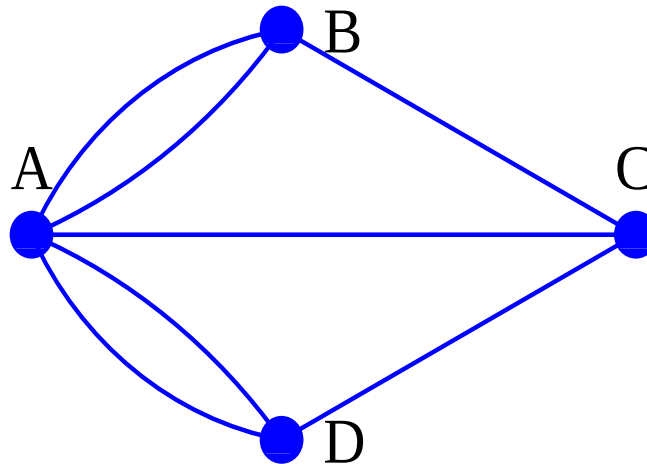
A **forest** is graph that does not contain a cycle.

A **tree** is a connected forest and a **rooted tree** is a directed tree with one distinguished node called the **root** such that all its edges point away from the root.

## Graphs—Eulerian graphs

Euler's Königsberg bridges problem:

Which graphs can be traversed with a circuit that visits each edge exactly once?



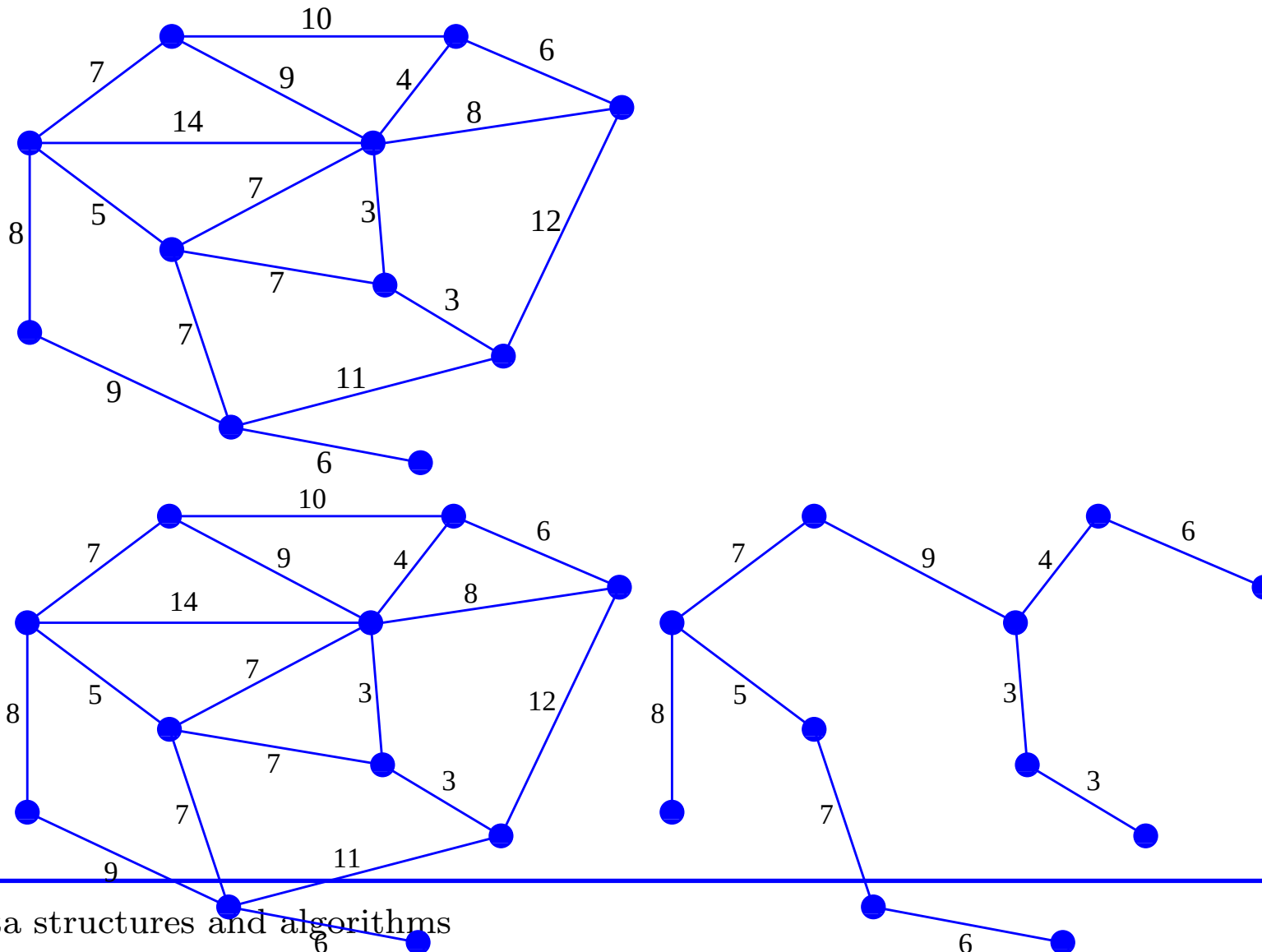
In an **Eulerian graph** the degree of EVERY NODE MUST BE EVEN.

Each vertex has a **degree**,  $d(v)$  is number of edges incident to  $v$ .  $d(A) = 5$ ,  $d(B) = 3$ ,  $d(C) = 3$ ,  $d(D) = 3$ .

So the Euler's Königsberg bridges problem is **not** Eulerian.

# Graphs—spanning tree, subgraph

A **subgraph** of a graph  $G = (V, E)$  is a graph  $H = (U, F)$  such that  $U \subseteq V$  and  $F \subseteq E$ .

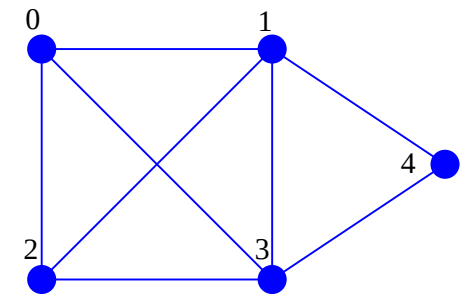
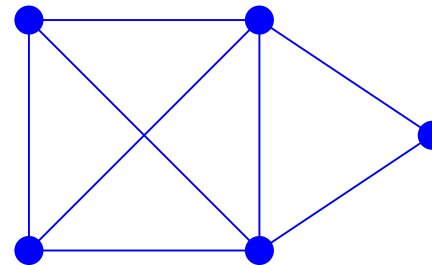
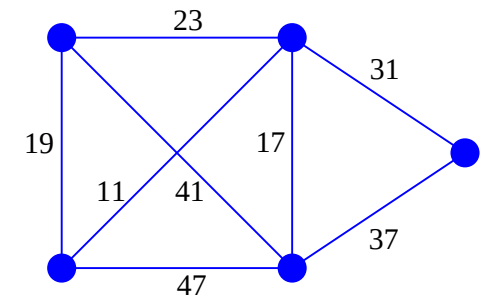
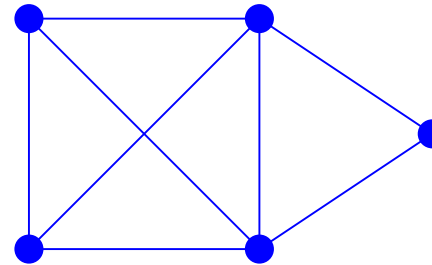
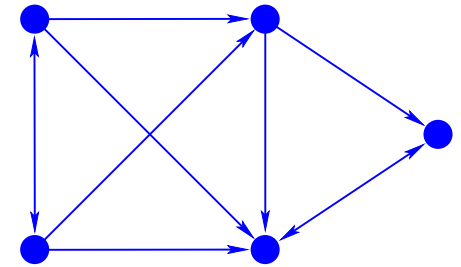
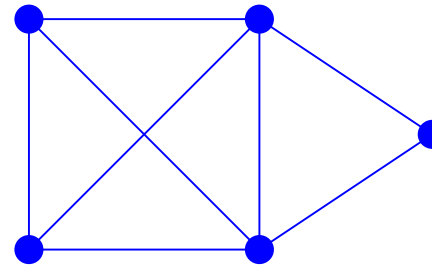


# Graphs—types I

A graph  $G$  may be  
*undirected* or *directed*,

*unweighted* or *weighted*,

*unlabelled* or *labelled*,



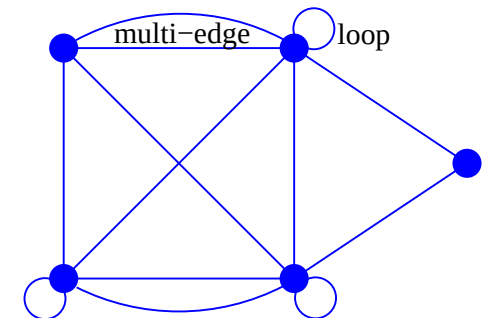
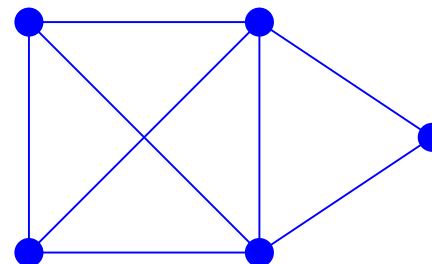
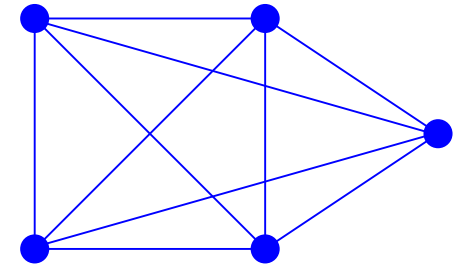
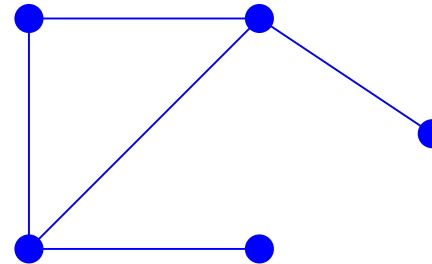
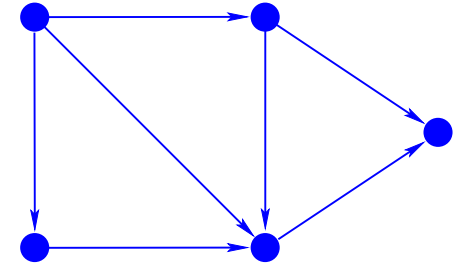
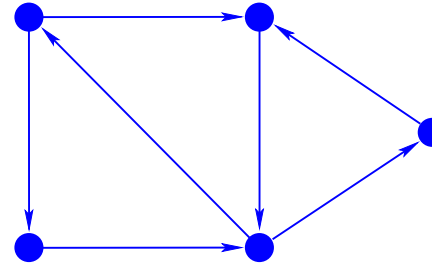


## Graphs—types II

A graph  $G$  may be  
*cyclic* or *acyclic*, i.e. a DAG or directed, acyclic graph,

*sparse* or *dense*,

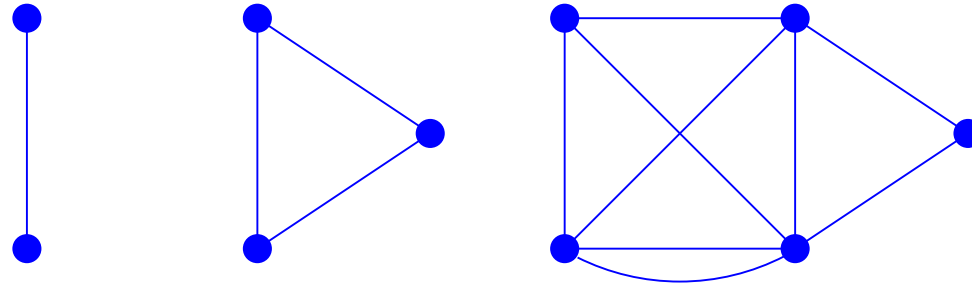
*simple* or *non-simple*—with multi-edges and loops,



## Graphs—types III

---

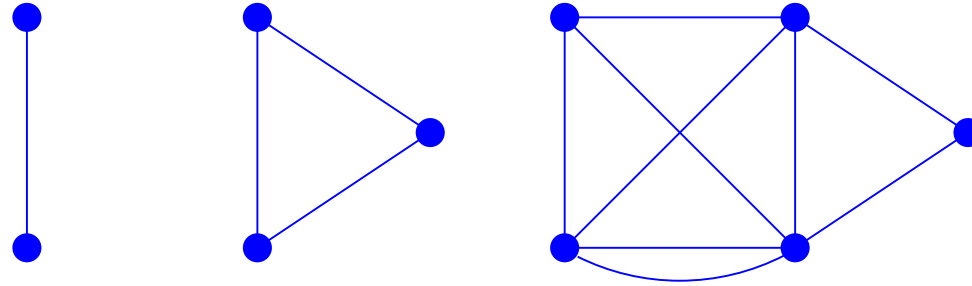
A graph  $G$  may be *unconnected* or *connected* .



A directed graph  $G$  may be *cyclic* or *acyclic*.

## Graphs—types IV

---



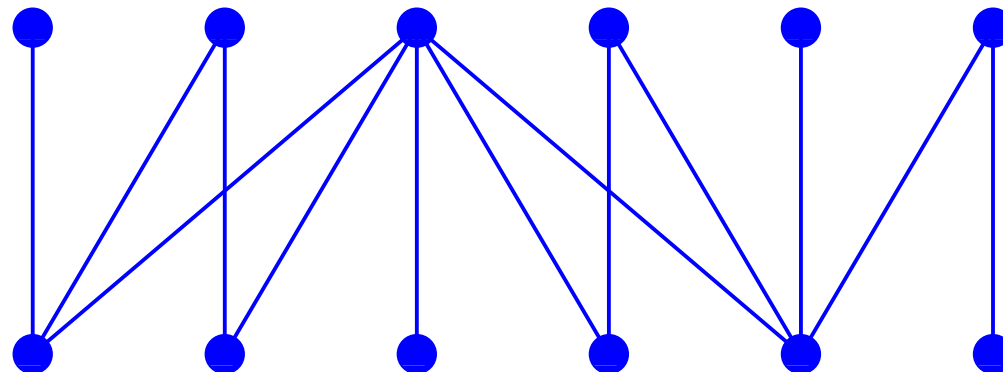
## Graphs—spanning tree, subgraph

---

A **spanning forest** of an undirected graph  $G$  is a subgraph of  $G$  that is a forest and that contains all the vertices of  $G$ .

A **vertex-induced subgraph** of a graph  $G = (V, E)$  is a subgraph  $H = (U, F)$  such that  $U \subseteq V$  and  $F$  consists of all the edges of  $E$  both of whose vertices belong to  $U$ .

A **bipartite graph** is a graph whose nodes can be divided into two sets such that all edges connect nodes from one set to nodes in the other set.

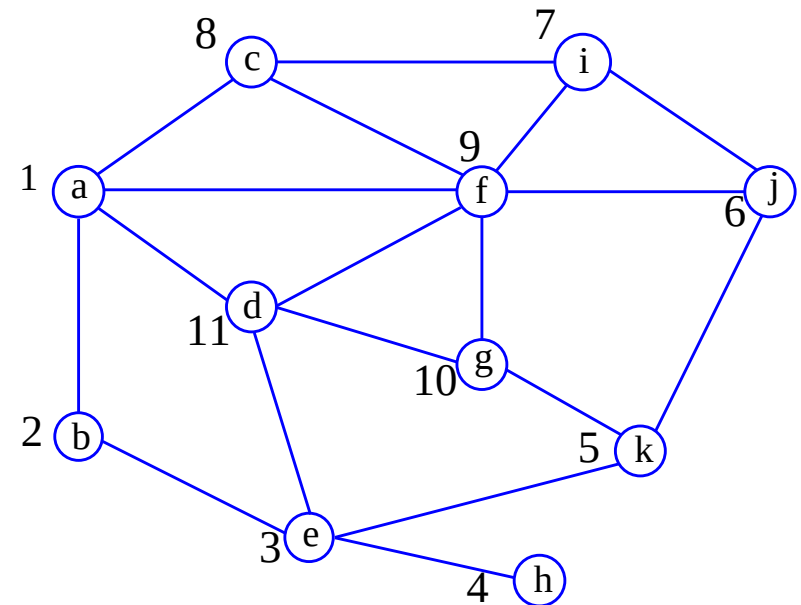


## Graph traversal—depth-first

The idea is to begin by visiting some start or **root** node, marking it and then calling DFS using each neighbour one-by-one as the root.

**Depth-first search** (DFS) in an undirected simple graph starts at a node we call the **root**.

```
void DFS (graph  $G$ , node  $v$ ) {  
    mark the node  $v$ .  
    call  $prelude(v)$ ;  
    for all edges  $\{(v, z) | z \in$   
        neighbourOf( $v$ ) $\}$   
        if ( $z$  is unmarked)  
            DFS ( $G$ ,  $z$ );  
    call  $postlude(v, z)$ ;  
}
```



Every node is visited once.

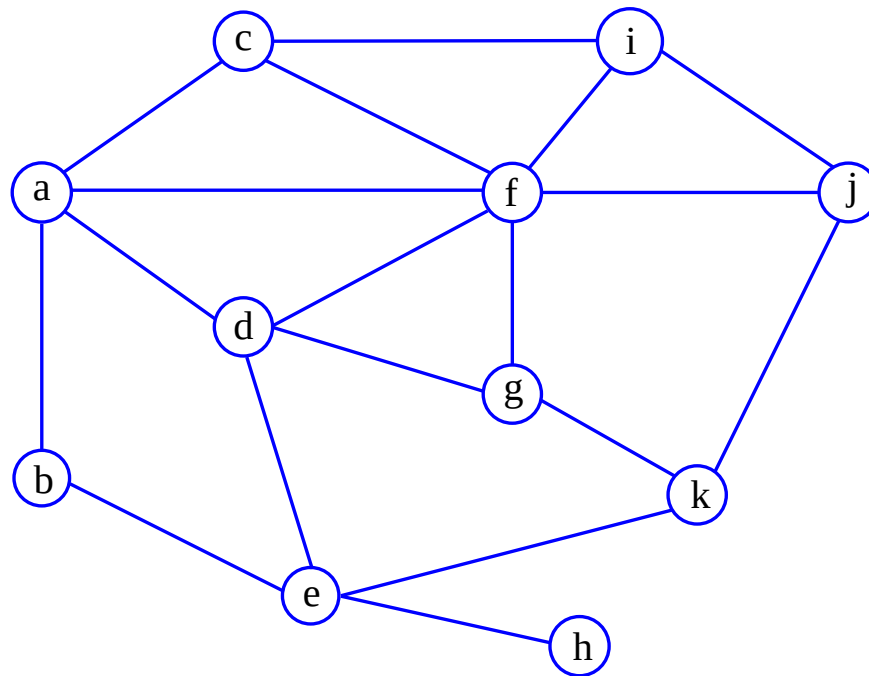
## Graph traversal—depth-first iteratively

Here, again, the idea is to visit every node once.

**Depth-first search** (DFS) in an undirected simple graph starts at a node we call the **root**.

Mark the node visited and consider all its neighbours by *stacking* them.

Continue by visiting the top node in the stack, marking it, stacking its neighbours if they have not been marked. Continue until the stack is empty.



## Graph traversal—depth-first

**Depth-first search** (DFS) in an undirected simple graph starts at a node we call the **root**.

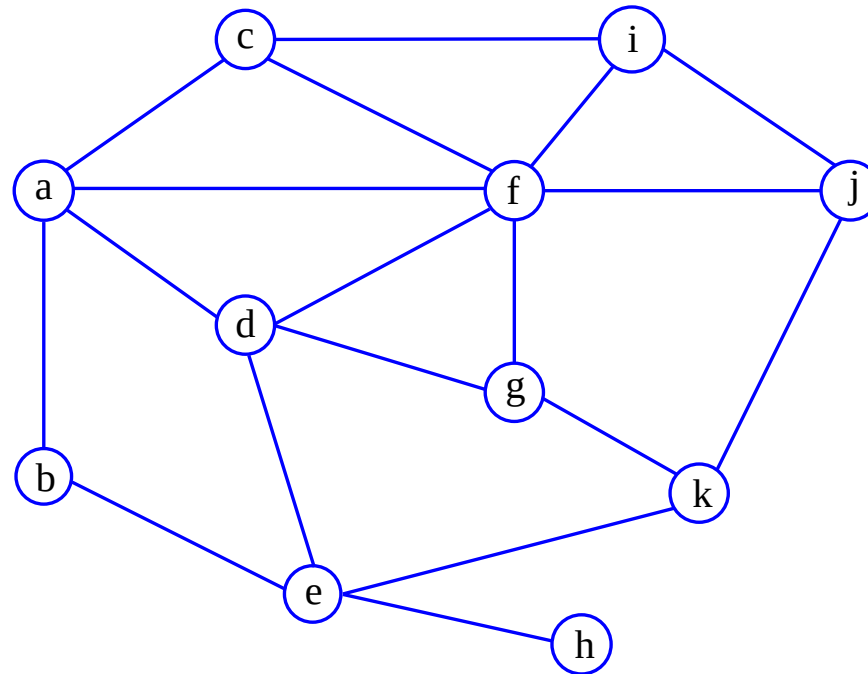
*push* the root node onto stack.

**do**

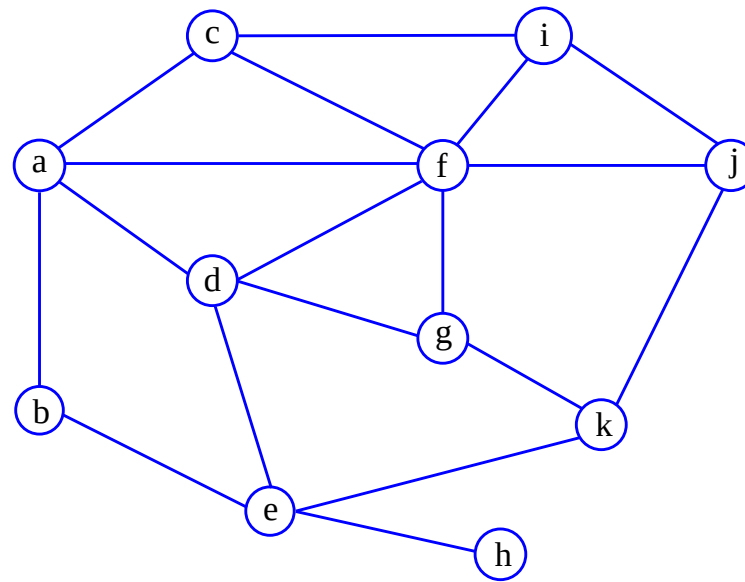
*visit, mark* and *remove* the top of the stack:

*visit* the node by working with it and by pushing top's unmarked neighbours onto the stack.

**repeat** until the stack is empty.



## Graph traversal—depth-first



Some depth-first traversals of this graph are:

**a, b, e, h, k, j, i, f, c, d, g** and

**a, d, f, i, j, k, g, e, h, c, b.**

Are these paths both depth-first traversals?



## Digraph—depth-first traversal

**Depth-first search** (DFS) in an directed simple graph starts a node we call the **root**.

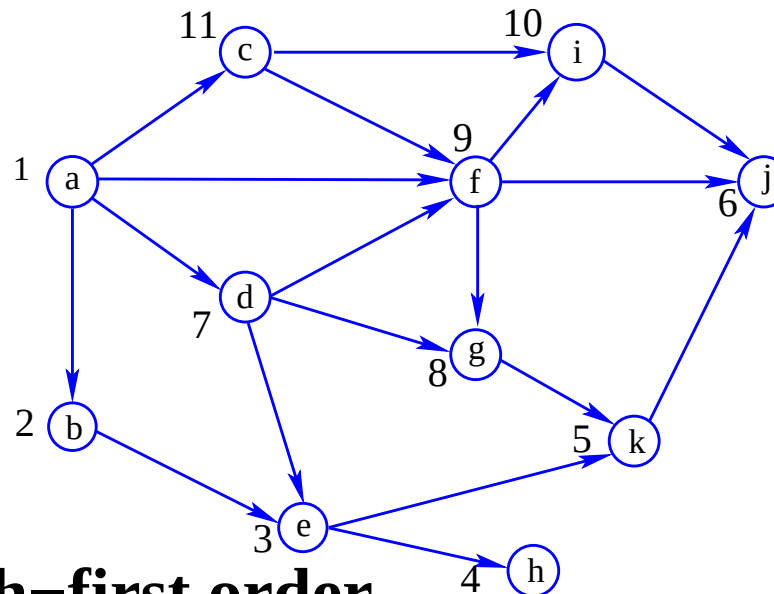
*push* the root node onto stack.

**do**

*visit, mark* and *remove* the top of the stack:

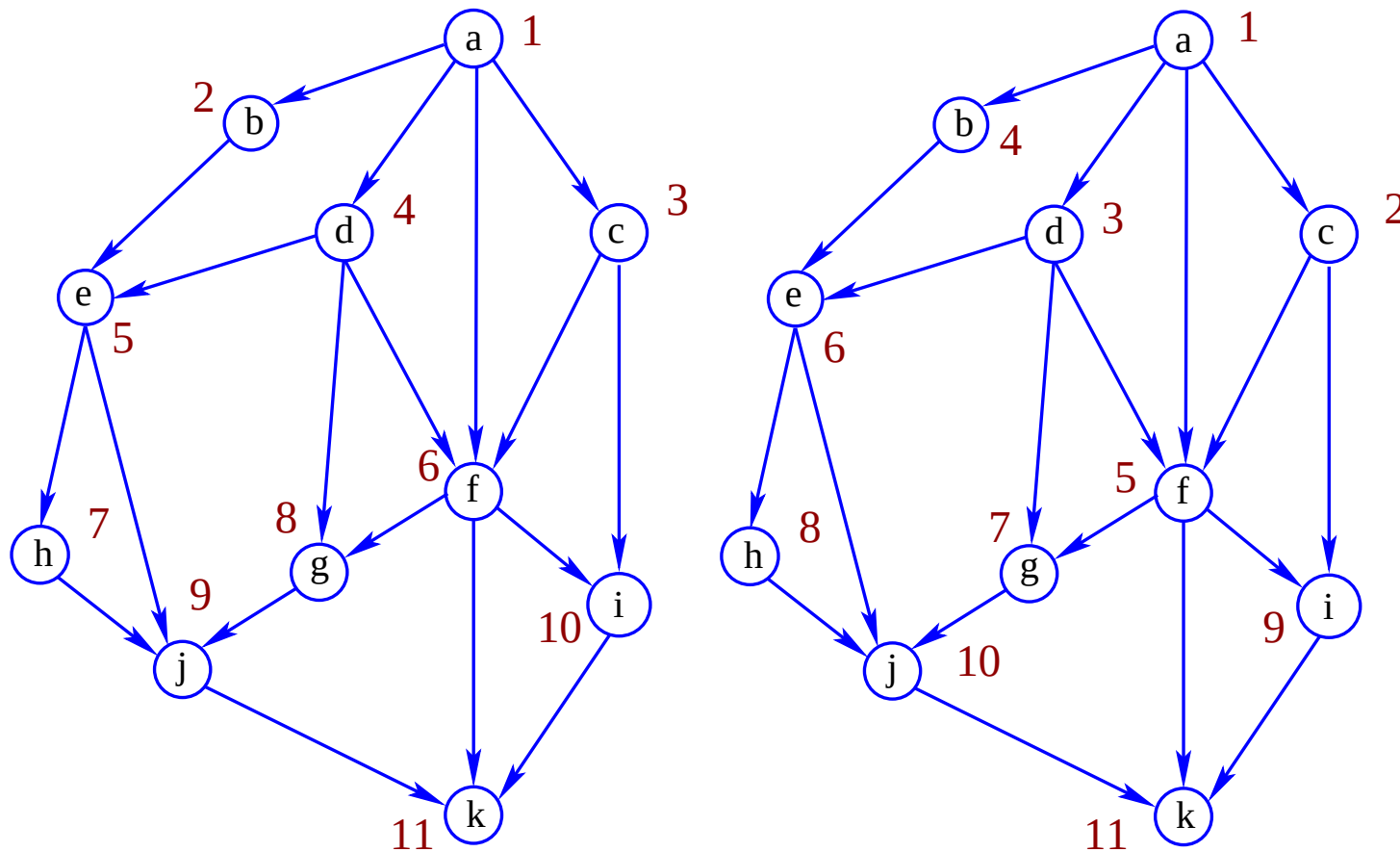
*visit* the node by working with it and by pushing top's unmarked reachable neighbours onto the stack.

**repeat** until the stack is empty.



**Depth-first order**

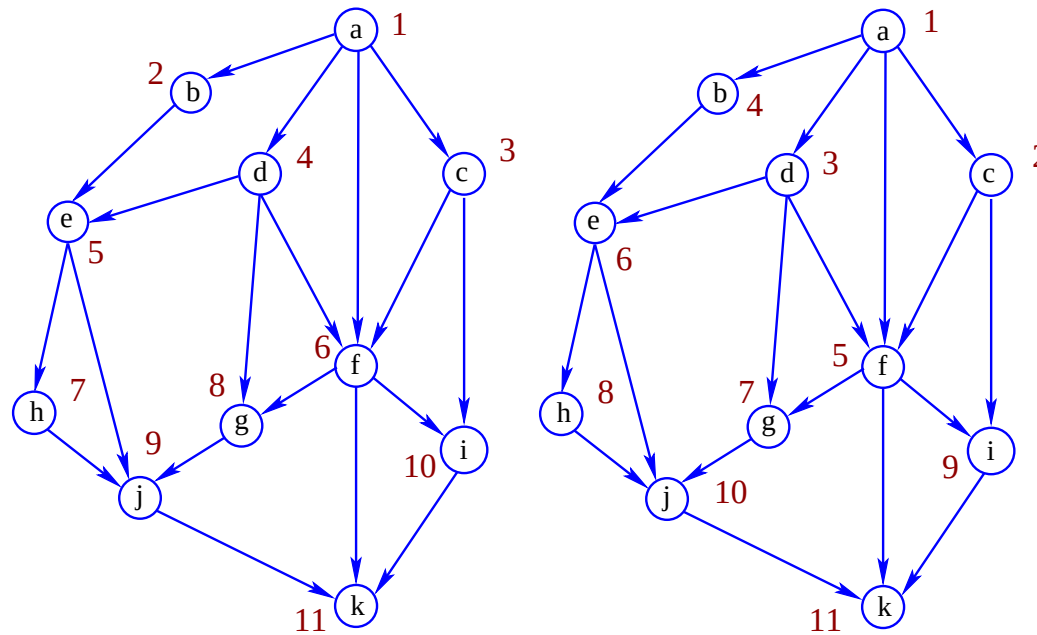
## Directed acyclic graph—Topological sorting



Let  $\vec{G}$  be a digraph with  $n$  nodes. The edges above represent **precedence**, ' $\prec$ ', such that  $a \prec b, b \prec e, a \prec d, d \prec e, d \prec f, a \prec f, a \prec c, c \prec f, \dots$

The figures show two different topological orderings of the same graph.

# Directed acyclic graph—Topological sorting



Let  $\vec{G}$  be a digraph with  $n$  nodes. The edges above represent **precedence**, ' $\prec$ ', such that  $a \prec b, b \prec e, a \prec d, d \prec e, d \prec f, a \prec f, a \prec c, c \prec f, \dots$

A **topological ordering** of  $\vec{G}$  is an ordering  $v_1, \dots, v_n$  of the vertices of  $\vec{G}$  such that for every edge  $(v_i, v_j)$  of  $\vec{G}$ ,  $i < j$ , i.e. any directed path in  $\vec{G}$  traverses nodes in precedence order.

A directed graph  $\vec{G}$  has a topological ordering if and only if it is acyclic.

## Directed acyclic graph—Topological sorting

```
1 void topologicalSort (graph G) {  
2   // pre: acyclic digraph  
3   // post: topological ordering  
4   S = initially empty stack;  
5   for all u in G.nodes() do  
6     if (inDegree(u) == 0)  
7       S.push(u);  
8   i = 1;  
9   while !S.isEmpty() do  
10    u = S.pop();  
11    i++;  
12    for all outgoingEdge(u,w) do  
13      inDegree(w)--;  
14      if (incounter(w) == 0)  
15        S.push(w); }
```

## Shortest paths

---

Let  $G = (V, E)$  be a weighted graph where  $V$  is a set of nodes or vertices,  $E$  is a set of weighted nodes.

Each edge may be labelled with a *length* or *weight*.

The length between two nodes  $v_i$  and  $v_{i+1}$  is indicated by  $\text{length}(v_i, v_{i+1})$ .

The *length* of a path

$$P = ((v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k))$$

in the graph is defined as

$$\text{length}(P) = \sum_{i=0}^{k-1} \text{length}(v_i, v_{i+1})$$

The *distance*,  $d(v, u)$  from vertex  $v$  to  $u$  in a graph  $G$  is the shortest path length from  $v$  to  $u$ , if it exists.

## Shortest paths

---

When no path exists from  $v$  to  $u$  we write

$$d(v, u) = \infty.$$

Negative weights are possible but can lead to unforeseen problems.  
We will regard the distance as a positive metric.

## Single-source shortest path

---

Dijkstra's algorithm for an undirected simple graph finds the shortest path from a node  $v$  to every other node in the graph.

Each vertex  $w \in G$  is marked with a label  $w.SP$  which is the shortest path  $(v, w)$  attained so far.

Initially  $v.SP = 0$  and  $w.SP = \infty$ , for each  $w \neq v$ .

We use a heap  $H$  to store all the nodes according to their shortest-path distance from  $v$ . Initially  $v$  is at the top of the heap  $H$ .

Now consider node  $w$  at the top of the heap and its nearest neighbour is entered onto the heap.

## Dijkstra's algorithm for shortest paths

```
1 void shortestPaths (graph G, node v) {  
2   // pre: G = (V,E) is a weighted directed graph, v is the source node.  
3   // post: for each w, w.SP is the shortest path (v,w).  
4   i = 0;  
5   for w in G do  
6     w.mark = false;  
7     w.SP = infty;  
8     Heap[++i]=w;  
9   v.mark = true; v.SP = 0; Heap[0]=v;  
10  // Enter all nodes w into a heap H according to w.SP;  
11  while w in Heap && w.mark == false do  
12    get w with minimal w.SP off heap;  
13    w.mark = true;  
14    for all edges (w,z) && z.mark == false do  
15      if (w.SP + length(w,z) < z.SP)  
16        z.SP = w.SP + length(w,z);  
17      update z in Heap; }
```



## Dijkstra's algorithm for shortest paths

---

Updating the length of a path **z.SP** takes  $O(\log m)$  comparisons, where  $m$  is the size of the heap.

There are  $|V|$  deletions from the heap and at most  $|E|$  updates.

So there are  $O(|E| \log |V|)$  comparisons in the heap, giving a running time of  $O((|E| \log |V|) \log |V|)$

Putting  $|V| = n$  and  $|E| = m$ , the running time is  $O((m \log n) \log n)$

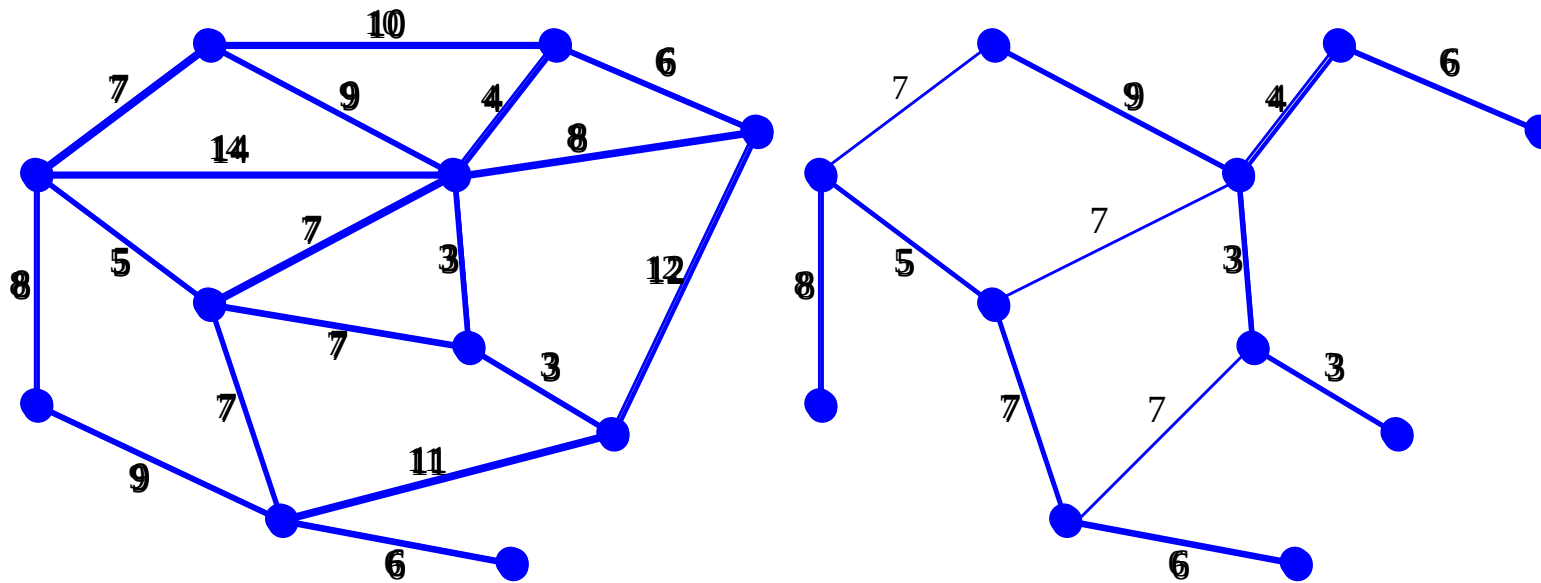
## Minimal spanning trees (MST)

---

- A *forest* is a loop-free graph.
- A *tree* is a connected forest.
- A subgraph *spans* a graph if it contains all the vertices of the graph.

## Minimal spanning trees (MST)

Recall that a spanning tree is a tree that connects all the vertices  $V$  of a graph  $G = (V, E)$  using edges from  $E$ .



Several of 7 edges can be switched, so that the MST need not be unique.

A minimal spanning tree (MST) uses edges optimally such that the sum of their weights is a minimum.

The total cost is  $8 + 5 + 7 + 6 + 7 + 9 + 4 + 6 + 3 + 3 = 58$ .

The MST is unique when all the weights are different.

There will be  $n-1$  edges in the MST for a graph with  $n$  nodes.

# Minimal spanning trees (MST)

---

The three algorithms we will consider are called

- Otakar Borůvka's algorithm (1926)  
Used to set up electrical network in Moravia.
- Joseph Kruskal algorithm (1956) and the
- Prim-Jarník algorithm (1930, 1957)

There are some others too.

## Borůvka's algorithm for MSTs

```
1 //Pre: Connected weighted graph  $G = (V, E)$ ,  $n = |E|$ 
2 //Post: Minimal spanning tree in  $T$ 
3  $T = (V, \emptyset)$  // all the vertices of  $G$  but no edges
4 while number of edges in  $T < n - 1$  do
5      $E = \{\}$  // an empty set of edges
6     for each component  $C$  in  $T$  do
7          $S = \{\}$  // an empty set of edges
8         for each vertex  $v$  in  $C$  do
9             add minimal edge  $(v, u)$  where  $u \notin C$  to  $S$ 
10        add the cheapest edge in  $S$  to  $E$ 
11    add edges  $E$  of to  $T$ .
```

If  $v = |V|$  is the number of vertices in the graph  $G$ . Since there are  $e = |E|$  edges the outer loop of Borůvka's algorithm clearly takes  $O(e)$  steps. The inside loops take at most  $v/2, v/2^2, v/2^3$ , etc., steps, i.e.,  $\log v$  steps, so the algorithm runs in time  $O(e \log v)$ .

## Borůvka's algorithm for MSTs

An alternate statement of Borůvka's algorithm for MSTs is

```
1  Given  $G = (V, E)$ 
2   $T =$  graph consisting of  $V$  with no edges
3  while  $T$  has  $< n-1$  edges do
4      for each connected component  $C$  of  $T$  do
5           $e =$  minimum cost edge  $(v, u)$  so that  $v$  in  $C$  and  $u$  not in  $C$ 
6           $T = T$  union  $\{e\}$ 
```

## The union-find algorithm

---

The union-find algorithm deals with adding edges to a set of  $v$  vertices or nodes and then finding out whether two nodes are connected.

- **union** command connects two nodes  
e.g. `union(u, v)` links node  $u$  to node  $v$  if they are not already connected.
- **find** or **isConnected** determines if two nodes are linked. e.g.  
`isConnected(u, v)` is a predicate that checks if node  $v$  is linked to node  $u$ .

# The union-find algorithm

Consider the disconnected graph with no edges and eight nodes  
Before any operations



`isConnected(a, f)  $\Rightarrow$  false`

`union(b, f)`



`isConnected(a, f)  $\Rightarrow$  false`

`union(a, b)`



`isConnected(a, f)  $\Rightarrow$  false`

`union(a, e)`



`isConnected(a, f)  $\Rightarrow$  true`



## The union-find algorithm

`isConnected(u, v)` is an *equivalence relation* since it is:

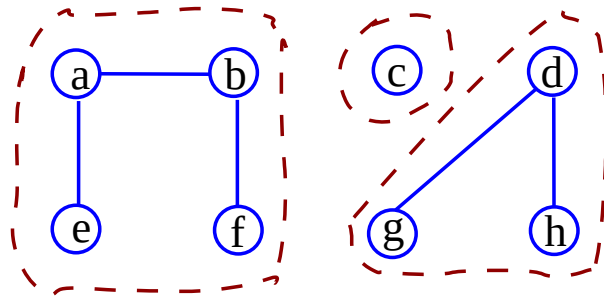
reflexive: `isConnected(u, u)`,

symmetric: `isConnected(u, v)  $\equiv$  isConnected(v, u)`, and

transitive: if `isConnected(u, v)` and `isConnected(v, r)` then `isConnected(u, r)`.

Equivalence relations divide sets into equivalence classes.

The graph depicted by



has three equivalence classes  $\{a, b, e, f\}$ ,  $\{c\}$ ,  $\{d, g, h\}$ .

These classes consist of connected components.

## A Java API for a union-find data type

A data structure for union-find where the size of the graph/set  $N$  can be very large and the number of operations  $M$  can be very large.

The `isConnected` and `union` commands can appear in any order.

```
1 public class UF
2 UF(int N) // initialize union-find data structure with N objects [0..N - 1]
3 void union(int u, int v) // add connection between u and v
4 boolean isConnected(int u, int v) // u and v in same component?
5 int find(int u) // component identifier for u ∈ [0..N - 1]
6 int count() // number of components
```

The nodes `p` and `q` are connected if and only if `id[p]` equals `id[q]`, thus and all connected components have the same `id[.]` value.

The code for `find`

```
1 public int find(int p) {
2     return id[p]; }
```

## Code for **union** and **isConnected**

The code for **union**

```
1 public void union(int p) {  
2   int pID = find(p),  
3     qID = find(q);  
4   if (pID == qID) return;  
5  
6   for (int i = 0; i < id.length; i++)  
7     if (id[i] == pID) id[i] = qID;  
8   count--; }
```

## Kruskal's algorithm for MSTs

- 1 Repeat the following step as many times as possible:
- 2     Among the edges of  $G$  not yet chosen,
- 3     choose the shortest edge which does not form any loops,
- 4     with those edges already chosen.

Clearly the set of edges eventually chosen forms a spanning tree of  $G$ , and in fact it forms a minimal spanning tree.<sup>14</sup>

---

<sup>14</sup>Joseph B. Kruskal, Jr. *On the shortest spanning subtree of a graph and the traveling salesman problem*, Proceedings of the American Mathematical Society, Vol. 7, No. 1 (Feb., 1956), pp. 48–50.

## Longest maximum subvector problem solved in 5 ways

---

In a list or vector  $X$  of  $N$  integers find the maximum sum of any contiguous subvector.<sup>15</sup>

|     |     |    |    |    |   |     |    |   |     |
|-----|-----|----|----|----|---|-----|----|---|-----|
| -66 | -15 | 87 | 91 | 32 | 5 | -47 | 47 | 4 | -77 |
| 0   | 1   | 2  | 3  | 4  | 5 | 6   | 7  | 8 | 9   |

In the list  $X$  above the subvector  $X[2..8]$  has the maximum sum 219.

Clearly if all the numbers are negative then the maximum contiguous subvector (MCS) is simply the largest number. In this case we define the MCS as 0.

---

<sup>15</sup>This example comes from pages 77–81, “Programming Pearls”, Jon Bentley, ACM Press and Addison-Wesley, New York, 2000.

## An $O(n^3)$ method for MCS

In our first attempt we simply calculate *all* possible subvectors  $X[\text{low}..\text{high}]$  and find the greatest.

```
1 int max_n^3 (x)
2     n = len(x)
3     maxsofar = 0
4     for low in [0,n)
5         for high in [low,n)
6             sum = 0
7             for r in [low,high)
8                 sum += x[r]
9                 if (sum > maxsofar)
10                     maxsofar = sum
11 return maxsofar
```

Clearly three nested loops yield an  $O(N^3)$  method.

There is a simple way to reduce this to only two loops by preserving partial sums.

## An $O(n^2)$ method for MCS—1

In our second attempt again calculate all the subvectors  $X[\text{low}..\text{high}]$  but use partial sums to determine them and find the greatest.

```
1 int max_n^2(x)
2     n = len(x)
3     maxsofar = 0
4     for low in [0,n)
5         sum = 0
6         for high in [low,n)
7             sum += x[r]
8             if (sum > maxsofar)
9                 maxsofar = sum
10    return maxsofar
```

There are now only two nested loops clearly making it an  $O(N^2)$  method.  
There is another way of using only two loops.

## Another $O(n^2)$ method for MCS—2

In our third attempt we first calculate the cumulative sums for for each possible range of numbers subvectors  $X[0..i]$  where where  $i \in [0..N]$

```
1 int max_n^2(x)
2     n = len(x)
3     sumTo[0] = 0
4     for i in [0,n)
5         sumTo[i] = sumTo[i-1] + X[i]
6     maxsofar = 0
7     for low in [0,n)
8         for high in [low,n)
9             sum = sumTo[high] - sumTo[low-1]
10            // sum of all elements in X[low..high)
11            if (sum > maxsofar)
12                maxsofar = sum
13 return maxsofar
```

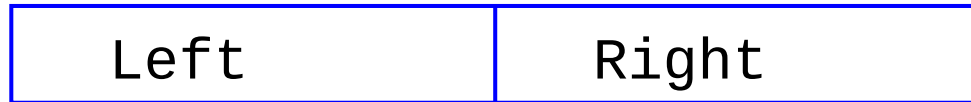
With two nested loops, this is still  $O(N^2)$ .

The next method is subquadratic.



## A divide-and-conquer method for MCS

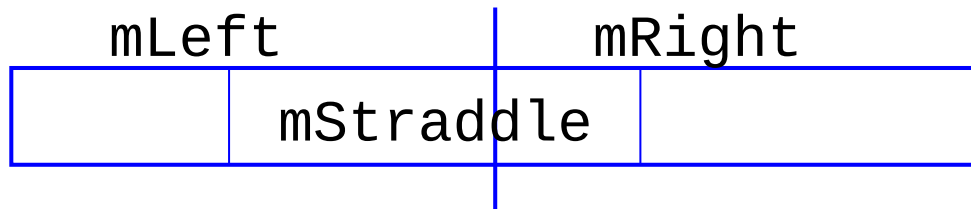
First, we divide the given array  $X[ ]$  into two parts, **Left** and **Right**:



and then find the MCS **mLeft** in **Left** and the MCS **mRight** in **Right**.



There is still a snag—the MCS might straddle both **Left** and **Right**.



The MCS is the subvector that gives the biggest of these three values.

The final algorithm is (1)  $\text{middle} = N/2$ ,

(2) Get MCS, **mLeft** of  $X[\text{low}..\text{middle}]$ , using

$\text{mLeft} = \text{MCS}(X, \text{low}, \text{middle})$ , and get the MCS, **mRight** of  $X[\text{middle}+1..N]$ , using  $\text{mRight} = \text{MCS}(X, \text{middle}+1, N)$ .

More difficult is getting the straddling MCS, **mStraddle**.

## Calculating the straddling MCS

The idea of the straddling MCS is first to calculate the MCS, called ‘maxsofarLeft’ that ends at  $X[\text{middle}]$  but starts somewhere in **Left**—running backwards, and also calculating the MCS called ‘maxsofarRight’ that starts at  $X[\text{middle}+1]$  and ends somewhere in **Right**. The sum of these two values is **mStraddle**.

```
1 int maxStraddle(low, high)
2     middle = (low + high) / 2
3     sum = 0; maxsofarLeft = 0
4     for i in [low..middle]
5         sum += X[middle - i + 1]
6         if sum > maxsofarLeft
7             maxsofarLeft = sum
8     sum = 0; maxsofarRight = 0
9     for i in [middle+1..high]
10        sum += X[i]
11        if sum > maxsofarRight
12            maxsofarRight = sum
13    return maxsofarLeft + maxsofarRight
```

## A divide-and-conquer method for MCS

```
1 int MCS(low, high)
2     if low > high
3         return 0
4     if low == high
5         return max(0, X[low])
6     middle = (low + high) / 2
7     mLeft = MCS(X, low, middle)
8     mRight = MCS(X, middle+1, high)
9     mStraddle = maxStraddle (X, low, high)
10    return max (mLeft, mRight, mStraddle)
```

Suppose ‘MCS’ takes time  $T(N)$  for an input of  $N$ . Then ‘mLeft’ takes time  $T(N/2)$  and ‘mRight’ takes time  $T(N/2)$ . ‘maxStraddle()’ takes  $O(N)$  because, at worst, it scans the entire input once. We conclude that

$$\begin{aligned} T(N) &= 2T(N/2) + O(N) \\ &= O(N \log N) \end{aligned}$$

## Divide-and-conquer methods— $O(N \log N)$

Suppose the divide-and-conquer method has time complexity  $T(N)$  when it has an input of size  $N$ .

Divide-and-conquer methods entail:

- (1) *dividing* the problem into equal parts,
- (2) *solving each part* of the problem separately, and
- (3) *joining* the results somehow.

1. *Dividing* the problem in two, has the low cost of  $O(1)$ . All that needs to be done is  $N = N/2$ .
2. *Solving each half* of the problem separately costs  $2T(N/2)$ .
3. *Joining the two results* is the critical part. Try to make this depend *linearly* on  $N$ . If it is sublinear all the better. Usually this part entails looking at every input once. In mergesort the two sorted halves are merged in time  $cN$ , making the cost of merge sort,  $T(N) = 2T(N/2) + cN$ .
4. In MCS straddle passes through all  $N$  elements once, giving the same formula.

## Divide-and-conquer methods— $O(N \log N)$ —I

---

Solving the recursive equation for merge sort and MCS

$$T(N) = 2T(N/2) + cN.$$

is quite easy.

$$\begin{aligned} T(N) &= 2T(N/2) + cN \\ &= 2[2T(N/4) + cN/2] + cN \\ &= 2^2T(N/2^2) + cN + cN \\ &= 2^2[2T(N/2^3) + cN/2^2] + cN + cN \\ &= 2^3T(N/2^3) + cN + cN + cN \\ &= 2^3T(N/2^3) + 3cN \\ &\vdots \\ &= 2^kT(N/2^k) + kcN \end{aligned}$$

Continue until  $N/2^k = 1$ .

$$\begin{aligned} T(N) &= 2T(N/2) + cN \\ &\vdots \\ &= 2^k T(N/2^k) + kcN \end{aligned}$$

Continue until  $N/2^k = 1$ .

Since there is nothing to do when the input is of size 1, it follows that  $T(1) = 0$ , so we set  $N/2^k = 1$  giving  $N = 2^k$ . This happens when  $k = \log_2 N$  and

$$\begin{aligned} T(N) &= T(1) + kcN \\ &= kcN \\ &= cN \log_2 N \\ &= O(N \log N) \end{aligned}$$

## An $O(n)$ method for MCS( $X$ , low, high)

This method starts at  $X[\text{low}]$  and scans through all the elements up to  $X[\text{high}]$  keeping book of the maximum contiguous subvector found so far.



The maximum sum in the first  $i$  elements is either the maximum in the first  $i - 1$  elements, called ‘maxSoFar’ or that of the subvector that ends in  $X[i]$ , called ‘maxToHere’.

‘maxToHere’ contains the biggest sum *ending* at  $X[i - 1]$ . Considering item  $X[i]$  take the biggest of  $\text{maxToHere} + X[i]$  and 0.

```
1 int MCS(low, high)
2     maxSoFar = 0.0
3     maxToHere = 0.0
4     for i in [low, high]
5         maxToHere = max(maxToHere + X[i], 0.0)
6         maxSoFar = max(maxSoFar, maxToHere)
7     return maxSoFar
```

```
String T = "The cat sat on the mat.";
int n = T.length(), i;

for(i=0; i <=n-5; i++) {
    // get T[0..0+i-1], length = i.
    System.out.println(T.subSequence(0,i));
    // get T[i..i+5], length 6.
    System.out.println(T.substring(i,i+5));
}
```



## Java's `toString()` method

- The `String` representation of `Object` is a convenient way to print it.
- Java provides for a `toString` method in any `class`.
- Suppose that your `Node` consists of three fields defined by

```
1  long key;  
2  String data;  
3  Node next;
```

This is typical of a node in a linked list.

- A simple node `Node` defined by  
`Node N;`  
may be instantiated by the statement  
`N = new Node(123L, "one_two_three");`  
`String toString()`
-

### **COS 211 Examination—the date to be announced.**

In 2015 the first semester examinations started on 18 May 2015, and the CSC211 examination was held on 4th June 2015, i.e., Thursday of the third week of the first semester examinations.

It is possible that the time table is changed because it is being planned by new administrators.

Note: I do not take any responsibility for informing you of venue or timetable changes.

## Useful mathematical facts—Logarithms

We write  $\log$  for  $\log_2$ , and  $\ln$  for  $\log_e$ —some authors use  $\lg$  for  $\log_2$

$$\text{If } x = b^n, \text{ then } \log_b x = n.$$

$$\log_b 1 = 0, \log_b b = 1.$$

$$\log x \times y = \log x + \log y,$$

$$\log x/y = \log x - \log y,$$

$$\log_b x^n = \log_b x + \log_b x + \dots + \log_b x (n \text{ times})$$

$$= n \log_b x, \text{ therefore } \log_b b^n = n, \text{ and}$$

$$\log_b x = \frac{\log_{10} x}{\log_{10} b} = \frac{\log_e x}{\log_e b} = \frac{\log_2 x}{\log_2 b} = \frac{\log x}{\log b} = \Theta(\log x),$$

$$\log_2 x = \frac{\log_{10} x}{\log_{10} 2} \approx \frac{\log_{10} x}{0.30102000} \approx \frac{\log_{10} x}{0.3},$$

$$\log_{10} x \approx 0.3 \times \log_2 x.$$

$$\text{Since } \log_b y \cdot \log_b x = \log_b x \cdot \log_b y,$$

$$\log_b x^{\log_b y} = \log_b y^{\log_b x},$$

$$x^{\log_b y} = y^{\log_b x}.$$

### Summing logarithms

$$\begin{aligned}\sum_{r=1}^n \lfloor \log_2 r \rfloor &= (n+1) \lfloor \log_2 n \rfloor - 2^{\lfloor \log_2 n \rfloor + 1} + 2 \\ &= \Theta(n \log n).\end{aligned}$$

### Sum of functions bounded by integral

If  $f(x)$  is a monotonically increasing continuous function,

$$\sum_{r=1}^n f(r) \leq \int_1^{n+1} f(x) dx.$$

## Useful mathematical facts—Bits and digits

---

The formula,  $\log_2 x = \frac{\log_{10} x}{\log_{10} 2} \approx \frac{\log_{10} x}{0.30102000} \approx \frac{\log_{10} x}{0.3}$ , shows how to calculate the number of bits  $x$  requires given  $\log_{10} x$ —its number of digits.

The formula  $\log_{10} x \approx 0.3 \times \log_2 x$  shows us how to get the number of digits represented by a given number of bits— $\log_2 x$ —of a number  $x$ .

e.g. The largest unsigned 24-bit number is  $2^{24} - 1$ ,

$$0.3 \log_2(2^{24} - 1) \approx 0.3 \times 24 = 7.2 \text{ digits.}$$

So 24 bits accurately represent 7 digits.

On the other hand, by the same argument a 23 bit number can only represent numbers of  $\lfloor 6.9 \rfloor = 6$  digits accurately.

This is where the *hidden 1 bit* of the 23-bit mantissa of the IEEE 754 Floating Point Standard is used to turn a 23 mantissa into a 24 bit mantissa by adding a hidden bit that is always 1 excepting when the number is zero.

## Useful mathematical facts—Asymptotic analysis

---

Since we are making approximations of the time that algorithms take, we consider non-negative increasing functions.

**Big- $O$  notation**— $T(n) = O(g(n))$ —“ $T(n)$  is of order  $g(n)$ ”

$\iff$  there is a constant  $c$  such that  $T(n) \leq cg(n)$ , when  $n$  is big enough.  
 $T(n)$  is asymptotically bounded from *above* by  $g(n)$ .

**Omega  $\Omega$  notation**— $T(n) = \Omega(g(n))$ —“ $T(n)$  is at least of order  $g(n)$ ”

$\iff$  there is a constant  $c$  such that  $T(n) \geq cg(n)$ , when  $n$  is big enough.  
 $T(n)$  is asymptotically bounded from *below* by  $g(n)$ .

**Theta  $\Theta$  notation**— $T(n) = \Theta(g(n))$ —“ $T(n)$  is of order exactly  $g(n)$ ”

$\iff$  both  $T(n) = O(g(n))$  and  $T(n) = \Omega(g(n))$  hold.  
 $T(n)$  is asymptotically *tightly* bounded by  $g(n)$ .

## Useful mathematical facts—Asymptotic analysis, examples

---

e.g. Let  $T(n) = 64n^3 + 32n^2 + 16n + 8$

$T(n)$  is  $O(n^3)$ ,  $O(n^4)$ ,  $\dots O(n^{100})$ ,  $\dots$ ,  $\Omega(n^3)$ ,  $\Omega(n^2)$ ,  $\Omega(n)$ , and  $\Theta(n^3)$ .

$T(n)$  is not  $O(n)$ ,  $\Omega(n^4)$ ,  $\Theta(n)$ ,  $\Theta(n^2)$ .

Although  $T(n)$  is  $O(n^4)$  it is not  $\Omega(n^4)$ . So it is not  $\Theta(n^4)$ .

### Small $o$ notation

We write  $f(n) = o(g(n))$  when  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$ .

It means that the function  $f(n)$  is an order smaller than  $g(n)$ .

This implies that if  $g(n) = o(T(n))$  then  $O(T(n) + g(n)) = O(T(n))$ ,

e.g.  $O(n^n + n^2 + n \log n + n + 1000000000) = O(n^n)$ .



### Asymptotically equal.

We write  $f(n) \sim g(n)$ —“ $f(n)$  is asymptotically equal to  $g(n)$ ”  
 $\iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 1$ .

**L'Hospital's Rule** (1696)—was discovered by **Jacob Bernoulli** (1654–1705) but named after **Guilliaume L'Hospital** (1661–1704).

For functions  $f$  and  $g$ ,  
if both  $\lim_{x \rightarrow a} f(x) = 0$  and  $\lim_{x \rightarrow a} g(x) = 0$ ,  
or both  $\lim_{x \rightarrow a} f(x) = \pm\infty$  and  $\lim_{x \rightarrow a} g(x) = \pm\infty$ ,  
and  $\lim_{x \rightarrow a} \frac{f'(x)}{g'(x)}$  exists,  
then  $\lim_{x \rightarrow a} \frac{f(x)}{g(x)} = \lim_{x \rightarrow a} \frac{f'(x)}{g'(x)}$ .

## Useful mathematical facts— $n!$

---

How do  $n!$  and  $n^n$  compare?

We have  $n! = 1.2.3 \dots n < n.n.n \dots n = n^n$ , such that  $n!$  is clearly bounded from above by  $n^n$ , i.e.  $n! = O(n^n)$  and  $n^n = \Omega(n!)$ .

**James Stirling** (1692–1770) gave the following approximation

$\lim_{n \rightarrow \infty} \frac{n!}{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n} = 1$ , or  $n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n (1 + O(1/n))$ , i.e.

$$\begin{aligned} \ln n! &= \ln \left[ \sqrt{2\pi n} \left(\frac{n}{e}\right)^n (1 + O(1/n)) \right] \\ &= \ln \sqrt{2\pi} + 0.5 \ln n + n \ln n - n \ln e + \varepsilon \\ &= \Theta(n \ln n) \end{aligned}$$

**R.W. Gosper**, in 1975, gives a better approximation of  $n!$  which yields reasonable values of  $n!$  for all  $n$  with a small relative error,  $\sqrt{2n + \frac{1}{3}\pi} \left(\frac{n}{e}\right)^n$

### Summation

$$\sum_{r=1}^n r = \frac{n(n+1)}{2} = \Theta(n^2),$$

$$\sum_{r=1}^n r^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3),$$

$$\sum_{r=1}^n r^3 = \frac{n^2(n+1)^2}{4} = \Theta(n^4),$$

$$\sum_{r=1}^n r^k = \Theta(n^{k+1}).$$

$$\sum_{r=0}^n (b-1)b^r = b^{n+1} - 1,$$

e.g.  $b = 10$ ,  $n = 7$ ,  $\text{sum} = 99999999 = 10^8 - 1 \approx 10^8$ .

$$\sum_{r=-1}^n (b-1)b^r,$$

e.g.  $b = 10$ ,  $n = 7$ ,  $\text{sum} = 99999999.9 = 10^8 - 0.1 \approx 10^8$ .

$$\sum_{r=0}^n 2^{-r} = 1.111\dots 11_2 = 2 - 2^{-n},$$

$$\sum_{r=0}^n 2^r = 111\dots 11_2 = 2^{n+1} - 1,$$

$$\sum_{r=1}^n r2^r = 2 + (n-1)2^{n+1},$$

$$\sum_{r=1}^n r2^{-r} < 2.$$

## Preparation for COS 211 Examination

---

- Understand and be able to describe and replicate parts of all practicals done during the semester, such as finding the  $k$ -element in a list, anagrams present in a text, noisy Huffman, Wayner encryption, .
- Study the class notes and other handouts carefully.
- Understand tutorials, quizzes, homework.
- Be prepared for repeat questions out of old exam papers and class test papers—please note that I do not supply these. Beware that when old questions are repeated they invariably have changes.

## Preparation for COS 211 Examination

---

- Although this course is based on all your preceding courses in Computer Science, concentrate on this semester's work.
- Make a special effort to study and understand the chapter on analysis of algorithms—Chapter 4 in Goodrich and Tamassia 4th or 5th edition, or Chapter 3 of the 3rd edition.
- When answering questions in the examination read the questions properly before answering. Students very often answer questions that they imagine have been asked.

## Memo: shuffling

---

- Understand how to shuffle a list of  $N$  items efficiently in  $O(N)$  time, and be able to program it: Given a *description* of the best way to shuffle  $N$  items and *derive its time complexity*, and be able to *program it*.
- Read the question properly before answering it. STUDENTS VERY OFTEN ANSWER QUESTIONS THAT THEY IMAGINE HAVE BEEN ASKED. For example this question was recently asked:

“Shuffling the  $N$  numbers from 1 to  $N$  can be done in  $\Theta(N)$  time by first initializing each number in an array of  $N$  integers, `int d[] = new int [N];` to `d[0]=1, d[1]=2, ..., d[N-1]=N`, and then swapping each number in order with another number randomly selected from the array. Give pseudo code for ANY OTHER algorithm to shuffle the numbers in the array `d[]`. (8) Give the time complexity of your code without deriving it. (2) [10]

Do you understand what is being asked?

## Memo: binary and linear search

---

- Linear search—be able to give or describe the algorithm and its timing. Suppose 85% of the search takes place in the first 5% of the list, what is the average search time? Can it be fine tuned?
- Binary search—be able to give or describe algorithm and timing: e.g. How long does it take to find an element *not* in the list? Derive the *average search time* of binary search in a list with  $2^N$  items.

## Memo: linked lists

---

Suppose you have linked list composed of nodes defined as

```
1 class Node {  
2   long key;  
3   node next; }
```

- Define a linked list with a dummy “head” node.
- Count the items in a linked list.
- Remove the last item from a linked list.
- Delete the item with **key = 10L** from a linked list.
- Reverse the items in a linked list.
- Copy one linked list into another.
- Merge two sorted linked lists into one sorted linked list.
- Print the items in two separate sorted linked list in sorted order.



## Memo: binary search trees (BST)

---

- Build a BST filled with the numbers  $\{1, 2, 3, \dots, 2^N - 1\}$  that is fully balanced.
- Draw such a tree for  $N = 4$ , then delete the numbers 1, 15, 8, 7 from this tree in the given order. Redraw the whole tree after each deletion.
- What is the height of a balanced BST with  $2^N - 1$  nodes assuming that the root is at level one?
- How many nodes are there in a tree with  $K$  levels, where the root is at level one?
- In a fully balanced BST with 10 levels—the root is at level one—give the fraction of nodes in the bottom level in exact numbers.
- Write code to declare a BST class.

## Memo: heaps / priority queues

---

Heaps / priority queues.

- Be able to explain the time complexity of building a heap top-down and building a heap bottom-up and code them.
- Explain sorting with heap and explain its space and time complexity and be able to program sorting using a heap.
- What algorithms do you know that use heaps to speed them up? Give a list. (Not a CSC 211 question.)
- How would you find the largest and 2nd largest element in a list using a heap? Give an efficient way that does not use a heap.
- Find  $k$ th smallest element using a heap—explain how to do this without a heap. How much time does each of your methods take?
- Be able to code all of the heap methods and code a **heap** class.

## Memo for examination—four $O(N \log N)$ sorting methods

---

- Quicksort: partition, timing was dealt with in detail in class notes.
- Merge sort. Timing. Find the  $k$ -th smallest/largest element in a list without sorting—hint: use partition, or use a priority list built bottom-up.
- Balanced-binary-search-tree sort. Build the BBST in time  $O(N \log N)$  and then traverse the tree in  $\ell\mathbf{Nr}$  order in time  $2N$ . Explain why the time is  $2N$ .
- Heap sort. Build the heap bottom up in time  $O(N)$ , then swap elements  $A[1]$  and  $A[n]$ , reduce size of tree by one and **heapify(1)** until there is only one element left.
- Explain radix sort. It is so fast it should always be used—why is it not always used?

## Memo for examination

---

- Boyer-Moore, Knuth-Morris-Pratt, Rabin-Karp versus brute force. Know all their timings. Know details of algorithms. Be able to fill in gaps in given programs.
- Huffman encoding and decoding. How does Vayner encryption—based on Huffman coding—work.
- Speed of Huffman using heap versus not using a heap.
- Be able to describe efficient spelling checkers given constraints such as:  
“Using a preprocessed dictionary the checking algorithm must independent of the size of the dictionary.” or “The program may not use a method that relies on hashing.” or “The dictionary and data must be processed at the same time.”

## Memo for examination

---

- Longest common subsequence. What is dynamic programming? What other methods have you seen that use dynamic programming.
- Graphs: definitions. Depth-first search (DFS) for trees. Breadth-first search (BFS) for trees. DFS and BFS for graph traversal. Dijkstra's shortest path algorithm,
- For graphs also be able to explain the datastructures to store them: edge list, node list, adjacency list, etc.
- Topological sorting.
- Be able to demonstrate all algorithms, given a small set of data.

## Memo: divide-and-conquer algorithms

---

- Understand and be able to derive and explain  $T(N) = 2T(N/2) + cN = O(N \log N)$ —the time taken by divide-and-conquer algorithms.
- Be able to code the maximum contiguous sum problem using algorithms that take  $O(N^3)$ ,  $O(N^2)$ ,  $O(N \log N)$  and  $O(N)$ .
- Be able to code and explain the nearest point algorithm in 1-D
- Find the maximum element in an array using a divide-and-conquer algorithm.

# Turning exponential algorithms into polynomial time algorithms

- Consider the code to calculate the n-th fibonacci number

```
1 def F(n):  
2     if n == 0: return 0  
3     if n == 1: return 1  
4     return F(n-1) + F(n-2)
```

- Explain why it is exponential.
- Alter the code to speed it up.
- Give code to calculate the n-th Fibonacci number without using a large array.
- Explain why this recursive code for factorial does not work for large values of n.

```
1 def factorial(n):  
2     if n < 2: return 1  
3     return n * factorial(n-1)
```

- Alter the code so that it does not show this behaviour.

## Memo: exponential algorithms

---

- Give and explain an  $O(2^n)$  algorithm.
- What is  $\log_b$  of any number given as a base  $b$  number in terms of its digits?
- What is the biggest unsigned/signed number that can be represented using 16, 32, 64 or 128 bits— give the answer without using a calculator.
- How long will the Towers of Hanoi run using 64 disks. Estimate without using a calculator. Be able to derive the time complexity  $T(N) = 2^N - 1$  for the Towers of Hanoi.
- Given code for hanoi, derive the number of calls to the recursive routine.
- Understand and be able to derive and explain  $T(N) = 2T(N/2) + cN = O(N \log N)$ —the time taken by divide and conquer algorithms.



## Some other algorithms

---

- Multiply matrices
- Do  $x^n$  by turning the  $n$  into a binary number.
- Find sums and products of polynomials.

# Another approach to programming

---

## Dynamic programming

- Matrix chain products
- The knapsack algorithm.
- Note we will look at the algorithm for the longest common subsequence problem next semester but you must understand the idea subproblem optimality.